

RUST · SYSTEMS PROGRAMMING

Coding Interviews for Cracked Rustaceans

A Zero-Cost Abstraction Obsession Anatomy

Arpan Pathak

Contents

<i>Preface</i>	6
<i>The code</i>	9

Part I: Sequences and Maps

Part page.....	13
1. Two Sum.....	14
2. Valid Parentheses	19
3. Min Stack.....	24
4. Longest Substring Without Repeating Characters	29
5. Merge Intervals.....	33
6. Top K Frequent Elements	39

Part II: Trees and Graphs

Part page.....	43
7. Binary Tree	44
8. Binary Search Tree.....	50
9. Binary Search on a Rotated Array	56
10. Trie	60
11. Topological Sort	65

Part III: Recursion and Dynamic Programming

Part page.....	72
12. Coin Change	73
13. Subsets.....	77

Part IV: Linked Structures

Part page.....	81
14. Reverse a Linked List.....	82
15. Singly Linked List.....	86
16. Doubly Linked List	91

Part V: Caches

Part page.....	99
17. LRU Cache.....	100
18. LRU Cache, Array-Backed.....	106

Part VI: Concurrency and the Operating System

Part page.	113
19. Workload State Machine	114
20. Token Bucket Rate Limiter	120
21. Worker Pool	125
22. Mutex Poisoning	131
23. Counting Islands	138
24. Syscall Overhead	145
25. Shadowing and Mutation.	151

Part VII: Files, Directories, and Arguments

Part page.	157
26. Reading and Writing Files.	158
27. Paths and Directory Traversal	163
28. File Operations and Errors	168
29. Command-Line Arguments	172
66. Paginating a File with a Byte Cursor.	178

Part VIII: Coordination and Parallelism

Part page.	184
30. A Bounded Buffer	185
31. Parallel Sum.	195

Part IX: Sequences, Revisited

Part page.	201
32. Running Median	202
33. Reversing a String In Place.	208

Part X: Reliability

Part page.	213
34. Idempotent Operations	214

Part XI: Types and Pointers

Part page.	220
35. Types That Reject Invalid Values	221
36. Smart Pointers and Interior Mutability.	230

Part XII: Recursion, Memory, and Caches

Part page.	238
--------------------	-----

37. Fibonacci Three Ways	239
38. Four List Layouts and the Recursive Drop	245
39. A Bump Allocator	258
40. Measuring Two LRU Layouts	265
41. A Sharded Concurrent Cache	278
42. Consistent Hashing	285

Part XIII: Threads and Synchronization

Part page	293
43. Threads, Send, and Sync	294
44. A Spin Lock	302
45. A Counting Semaphore	310
46. A Closable Bounded Queue	317
47. A Lock-Free Ring Buffer	324

Part XIV: Performance and the Operating System

Part page	333
48. Cache Locality and False Sharing	334
49. Amdahl's Law and Deadlock Detection	343
50. Paging and Round-Robin Scheduling	350

Part XV: Networking and HTTP

Part page	357
51. IPv4 Addresses and TCP Windows	358
52. A Thread-per-Connection Echo Server	365
53. An epoll Echo Server	371
54. Parsing HTTP/1.1 Requests	383
55. A Small HTTP Server	395

Part XVI: Async Rust and Cloud Clients

Part page	404
56. A Minimal Async Runtime	405
57. Retry with Backoff and Jitter	416
58. An HTTP Client with Retries and Caching	425

Part XVII: Interview Drills

Part page	435
59. Interview Drills	436

Part XVIII: Graphs and Lists, Revisited

Part page.	448
60. Graph Traversal: BFS and DFS	449
61. Dijkstra's Shortest Paths	457
62. Merge K Sorted Lists	463
65. Kruskal's Minimum Spanning Tree	480

Part XIX: Two Pointers, Revisited

Part page.	486
63. Three Sum	487

Part XX: Thread Pools

Part page.	492
64. Thread Pool - Workers of the Pool Assemble	493

Closing

Part page.	508
References	18
Index	512

Preface

What you don't use, you don't pay for. What you do use, you couldn't hand code any better.

Bjarne Stroustrup, *Foundations of C++*, ECOOP 2012

About the book

It can be very hard to use Rust for coding interviews if you don't have guidance. I felt the need of having a book, being a senior engineer, who can teach the language and its idiomatic patterns effortlessly with code that you can reproduce under pressure.

This book prepares you for those interviews and solidifies your intuition.

The code is in `rust-interview-lab`, at <https://github.com/arpnanpathak/cracking-the-systems-programming-interview>.

What you need to know

The book assumes that you can read Rust and that you know what `Vec`, `HashMap`, `Option` and `Result` are. Every structure the text uses is built in the text. Every cost claim is derived on the page or measured by a program printed beside it.

How the chapters are built

Problem Statement. The input, the output, and the boundary cases.

Designing a Solution. The mechanism the solution rests on.

Implementation. The file the chapter reads, followed by a reading of its decisions.

Intuition. The state of the program after each step on a concrete input.

Time and Space Complexity. The cost of each operation, with the condition stated in the same row when a bound depends on an assumption.

Limitations. The cases the code does not handle and the resources it does not bound.

A chapter that measures rather than traces prints the program that produced the figure.

How the book is organised

Chapters 1 to 13 cover arrays, strings, trees, graphs, recursion and search. Chapters 14 to 18 cover linked structures and caches. Chapters 19 to 25 cover concurrency and the operating system. Chapters 26 to 29 cover files, directories and command-line arguments. Chapters 30 and 31 cover coordination and parallelism. Chapters 32 and 33 cover a running median and an in-place reversal. Chapter 34 covers idempotent operations.

Chapters 35 to 59 cover the rest of the repository: types and smart pointers (35 and 36), recursion, memory layout, and caches (37 to 42), threads and synchronization primitives (43 to 47), performance and the operating system (48 to 50), networking and HTTP (51 to 55), an async runtime, retries, and an HTTP client (56 to 58), and the short interview drills (59). Chapters 60 to 62 add breadth-first and depth-first search, Dijkstra's shortest paths, and merging k sorted lists, and chapter 63 adds Three Sum. Chapter 64 builds a thread pool in four versions.

The repository

The code is in `rust-interview-lab`:

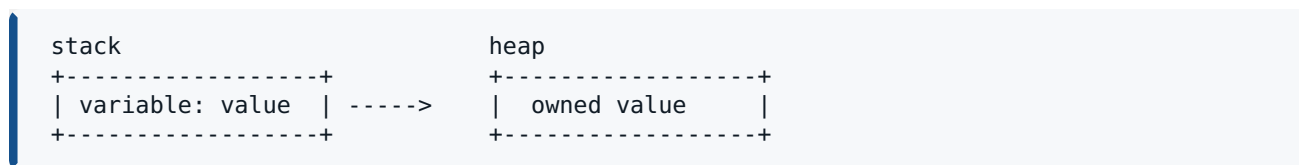
<https://github.com/arpanpathak/cracking-the-systems-programming-interview>

```
cd rust-interview-lab
cargo test                # library and program tests
cargo run --bin count_islands
cargo run --release --bin syscall_overhead
cargo run --bin mutex_poisoning
cargo run --bin shadowing
cargo run --bin read_write_file
cargo run --bin list_directory
cargo run --bin bounded_buffer
cargo run --release --bin parallel_sum
```

The listings reproduce the files as the repository holds them. Chapter 18 prints a test with an assertion added, and says so.

Conventions

A trace is a table with one row per step. Where a structure needs a picture rather than a table, boxes inside a frame are values on the stack, and boxes to the right of the frame joined by an arrow are values on the heap. A solid arrow is a pointer that owns what it points at; a dashed arrow is a pointer that does not.



Cost tables use the usual notation. A bound that holds only under an assumption states the assumption in the same row:

Resource	Cost	Condition
Time	$O(n)$ expected	the hash function distributes keys evenly
Space	$O(n)$	up to n entries

A measured figure names the machine that produced it.

What this book does not cover

It is not a Rust tutorial, an ecosystem survey, or a catalogue of interview questions. The operator and the cluster work are in the repository and in the study guides beside it.

Acknowledgments

The claims about the Rust language and its standard library rest on the documentation the Rust Project Developers maintain: the *Rust Book*, the *Rust Reference*, the *Rustonomicon*, the standard library reference, and the API Guidelines. The algorithms are attributed in each chapter and listed with their sources in the References.

Arpan Pathak

The Code

Layout

The repository holds one Cargo package. The tree below lists the files this book reads; the package also contains concurrency, asynchronous, networking and benchmarking code that lies outside the scope of these chapters.

```
rust-interview-lab/
├── Cargo.toml
├── README.md
├── BENCHMARKS.md
├── src
│   ├── lib.rs                                the crate root and its re-exports
│   └── problems
│       ├── mod.rs                            the module list
│       ├── backtracking.rs                   Chapter 13
│       ├── binary_search.rs                  Chapter 9
│       ├── binary_tree.rs                    Chapter 7
│       ├── dp.rs                             Chapter 12
│       ├── graph_topology.rs                 Chapter 11
│       ├── linked_list.rs                    Chapter 14
│       ├── lru_cache.rs                      Chapter 17
│       ├── merge_intervals.rs                Chapter 5
│       ├── min_stack.rs                      Chapter 3
│       ├── rate_limiter.rs                   Chapter 20
│       ├── sliding_window.rs                 Chapter 4
│       ├── state_machine.rs                  Chapter 19
│       ├── top_k_frequent.rs                  Chapter 6
│       ├── trie.rs                           Chapter 10
│       ├── two_sum.rs                         Chapter 1
│       ├── valid_parentheses.rs              Chapter 2
│       └── worker_pool.rs                     Chapter 21
├── bin
│   ├── append_to_file_open_options.rs        Chapter 26
│   ├── bounded_buffer.rs                     Chapter 30
│   ├── bst_clean.rs                           Chapter 8
│   ├── command_line_args.rs                  Chapter 29
│   ├── copy_rename_delete_file.rs             Chapter 28
│   ├── count_islands.rs                       Chapter 23
│   ├── file_error_handling.rs                 Chapter 28
│   ├── idempotent_operation.rs                Chapter 34
│   ├── idempotent_operation_with_error_propagation.rs Chapter 34
│   ├── list_directory.rs                      Chapter 27
│   ├── ll.rs                                 Chapter 16
│   ├── median_finder.rs                       Chapter 32
│   ├── mutex_poisoning.rs                     Chapter 22
│   ├── parallel_sum.rs                        Chapter 31
│   ├── path_buff.rs                           Chapter 27
│   ├── read_write_file.rs                     Chapter 26
│   ├── readfile_line_by_line.rs               Chapter 26
│   ├── recursive_directory_walk.rs             Chapter 27
│   ├── reverse_string.rs                       Chapter 33
│   ├── shadowing.rs                           Chapter 25
│   ├── singly_linked_list.rs                  Chapter 15
│   └── syscall_overhead.rs                     Chapter 24
└── benchmarking_examples/                    the measured structures and the benchmark
```

One file in `src/problems/` is empty, `hash_map.rs`, and nothing in the crate refers to it. Chapter 18 reads `lru_cache_easy.rs`, the array-backed cache that keeps its entries in a `Vec` and links them by index.

Modules and programs

Seventeen of the files above are library modules. Each holds one problem, one type, or one pair of a type and its policy, and each ends with a `#[cfg(test)] mod tests`, so the list printed by `cargo test` reads as an index of this book.

Twenty-two are programs. Files under `src/bin/` are separate crates that link the library, and each has a `main` function that runs without arguments. Most of them also carry the tests for the code they contain, so `cargo test` exercises the programs as well as the library.

What the files have in common

Four conventions run through the code. They are stated once here so that the chapters need not repeat them.

Enumerations instead of flags. Where a value has a fixed set of states, the code uses an `enum` and a `match` with no fall-through arm, so that adding a state is a compile error at every place that must handle it. `WorkloadState` in Chapter 19 and `ApiError` beside it are examples.

Option and Result instead of sentinels. Absence is `Option` and failure is `Result`, so the type states which. Chapter 12 keeps a sentinel inside the dynamic program's table and translates it at the boundary, so a caller never sees it.

Ownership written out. The linked structures in Chapters 14 to 16 give three different answers to the question of who owns the next node, namely `Box`, `Rc` with `Weak`, and a vector of indices, and each answer is visible in the type rather than in a comment.

Tests beside the code. A unit test that reaches a private item is a licence to change that item. Each module keeps its own tests in a `#[cfg(test)] mod tests`, so the list printed by `cargo test` reads as an index of the book.

Files read by chapters 35 to 59

File	Chapter
<code>src/problems/adt_idioms.rs</code>	35
<code>src/problems/smart_pointers.rs</code>	36
<code>src/bin/cs_fib.rs</code>	37
<code>benchmarking_examples/lists/, list_box.rs, list_enum.rs, list_drop.rs, benchmark.rs</code>	38
<code>src/problems/bump_allocator.rs</code>	39
<code>benchmarking_examples/cache/, benchmark.rs</code>	40
<code>src/problems/sharded_cache.rs</code>	41
<code>src/problems/consistent_hash.rs</code>	42
<code>src/problems/threads.rs</code>	43
<code>src/problems/spin_lock.rs</code>	44
<code>src/problems/semaphore.rs</code>	45
<code>src/problems/bounded_queue.rs</code>	46
<code>src/problems/ring_buffer.rs</code>	47
<code>src/bin/cs_locality.rs, src/bin/concurrency_false_sharing.rs</code>	48
<code>src/bin/concurrency_amdahl.rs, src/bin/concurrency_deadlock.rs</code>	49
<code>src/bin/os_paging.rs, src/bin/os_scheduler.rs</code>	50
<code>src/bin/net_ipv4.rs, src/bin/net_window.rs</code>	51
<code>src/bin/tcp_echo_server.rs</code>	52
<code>src/bin/epoll_echo.rs</code>	53
<code>src/problems/http_request.rs</code>	54
<code>src/bin/http_server.rs</code>	55
<code>src/problems/async_mini.rs, src/bin/async_demo.rs</code>	56
<code>src/problems/retry.rs</code>	57
<code>src/bin/fun_network_call.rs, src/bin/request_and_tokio.rs</code>	58

File	Chapter
<code>src/problems/drills.rs</code>	59
<code>src/bin/lru_cache_arena.rs</code>	40
<code>src/problems/graph_bfs.rs</code> , <code>src/problems/graph_dfs.rs</code>	60
<code>src/problems/graph_dijkstra.rs</code>	61
<code>src/bin/merge_k_sorted_lists_divide.rs</code>	62
<code>src/problems/three_sum.rs</code>	63
<code>src/bin/false_sharing.rs</code>	48
<code>src/bin/thread_pool.rs</code> , <code>src/problems/thread_pool_v2.rs</code> , <code>thread_pool_v3.rs</code> , <code>thread_pool_v4.rs</code>	64

Part I: Sequences and Maps

Two Sum

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/two_sum.rs`. Test it with `cargo test two_sum`.

Problem Statement

Given a slice of integers and a target value, return the indices of two positions whose values sum to the target, or report that no such pair exists. The inputs this function handles contain at most one such pair. The two positions may hold the same value provided the positions differ.

Comparing every element with every element after it costs $n(n - 1) / 2$ comparisons for a slice of length n , which is $O(n^2)$. One million elements is about five hundred billion comparisons.

Designing a Solution

For an element x and a target t , the value that completes the pair is $t - x$. It follows from x and t alone, so finding it requires no search.

The direct method asks whether $t - x$ occurs anywhere in the remainder of the slice, and every answer costs a scan. Recording each element as it is passed replaces the scan with a hash-table lookup: when the cursor reaches the second element of a pair, the first element is already in the table.

x	the element under the cursor
$t - x$	the only value that would complete a pair with x

The table maps a value to the position where it was found, because the answer is a pair of indices. It is keyed by the element's own value rather than by the

complement, because one element can be the second half of one pair and the first half of another.

Implementation

```

/// Two Sum: find the pair of indices whose values add up to a target.
///
/// Use a `HashMap` for one pass: O(n) time, O(n) space.

use std::collections::HashMap;

pub fn two_sum(nums: &[i32], target: i32) -> Option<(usize, usize)> {
    let mut seen = HashMap::with_capacity(nums.len());
    for (idx, &num) in nums.iter().enumerate() {
        match seen.get(&(target - num)) {
            Some(&prev) => return Some((prev, idx)),
            None => {
                seen.insert(num, idx);
            }
        }
    }
    None
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn finds_pair() {
        let nums = [2, 7, 11, 15];
        assert_eq!(two_sum(&nums, 9), Some((0, 1)));
    }

    #[test]
    fn returns_none_when_missing() {
        assert_eq!(two_sum(&[1, 2, 3], 100), None);
    }
}

```

The return type is `Option<(usize, usize)>`. Absence is part of the answer, and a caller cannot read index zero as a failure because the indices exist only inside `Some`. Both are `usize`, which is what `enumerate` produces and what indexes a slice.

`HashMap::with_capacity(nums.len())` bounds the number of entries the loop can insert. Without the hint the table rehashes as it grows, and the hint removes that work.

`nums.iter()` yields `&i32`, `enumerate` pairs each reference with its position, and the pattern `(idx, &num)` deconstructs that pair, copying the integer out of the reference.

`seen.get(&(target - num))` computes the complement and looks it up in one expression; `get` takes a reference to the key. The borrow that `get` returns ends with the `match`, which is what allows the `None` arm to insert into the same map.

`Some(&prev)` copies the stored index out of the returned reference. `Some(prev)` would move a `usize` out of a map the function does not own.

Intuition

The first trace uses the slice from the test suite. The pair completes on the second element.

```
nums = [2, 7, 11, 15]      target = 9

step  idx  num  complement  9-num  present in the map?  map after the step
1      0    2      7          no          {2: 0}
2      1    7      2          yes, at index 0    return Some((0, 1))
```

The second trace has two positions holding the same value. The positions differ, which the answer requires; the values are equal, which the input permits.

```
nums = [3, 3]              target = 6

step  idx  num  complement  6-num  present in the map?  map after the step
1      0    3      3          no          {3: 0}
2      1    3      3          yes, at index 0    return Some((0, 1))
```

With `nums = [1, 2, 3]` and `target = 100`, every complement is negative, no lookup succeeds, and the loop ends with three entries in the map. The map is released when the function returns, so the memory used follows the input rather than whether a pair exists.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$ expected	the hash function distributes keys evenly
Space	$O(n)$	one entry per element, when every element is distinct

The time bound is expected rather than guaranteed. A hash table hashes the key, selects a bucket from part of the hash, and probes from there. Evenly distributed keys give a small constant number of probes per opera-

tion. Keys that collide systematically turn every operation in a colliding group into a scan, and n insertions cost $O(n^2)$ in total.

The standard library's default hasher is SipHash-1-3 with a key chosen at random for each map, so which keys collide depends on a seed the program cannot observe. A program that installs its own hasher, as programs that hash integers do for speed, gives up that property, and the expected bound becomes a claim its input can break.

Space is $O(n)$ because the table holds one entry per distinct value visited, and an input with no pair keeps every element distinct. For a slice of integers the table is larger than the input: a million `i32` values occupy four megabytes, while a map of a million entries stores a key, a value, and the table's bookkeeping for each.

Limitations

The subtraction `target - num` can overflow. `two_sum(&[i32::MIN], 0)` evaluates `0 - i32::MIN`, and no `i32` holds that value. A debug build panics with an arithmetic overflow. A release build wraps, the wrapped value matches no element, and the function returns `None`. The two profiles therefore disagree on the same input, and a program tested in debug and shipped in release can change behaviour on a value read from a file or a socket. `checked_sub` reports the case instead:

```
let complement = match target.checked_sub(num) {
    Some(value) => value,
    None => {
        seen.insert(num, idx);
        continue;
    }
};
```

The element that has no complement is still recorded, because a later element may need to find it.

`insert` overwrites the index of a repeated value, so the map keeps the most recent position. A pair completed by the second occurrence returns the index the first occurrence recorded, which is the answer the function promises. A caller that needs the pair with the smallest first index, or every pair, requires information this function discards.

The function returns as soon as it reaches the second element of any pair, so on an input with several valid pairs the answer depends on the order of the elements. Reporting every pair requires the loop to continue and the map to hold every index for each value. Reporting the pair with the earliest first index requires `seen.entry(num).or_insert(idx)` in place of `insert`.

The element type is `i32` and the index type is `usize`. The algorithm depends on neither choice, but the signature fixes both, so a slice of `i64`, or of a type that is not `Copy`, cannot be passed without changing the declaration.

References

- Standard library, `HashMap`, on the default hasher and its random seed.
- Standard library, `HashMap::with_capacity`.
- Standard library, `i32::checked_sub`, and `HashMap::entry` for the variant that keeps the earliest index.
- The Rust Reference, [Arithmetic overflow](#), on the difference between debug and release builds.

Valid Parentheses

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/valid_parentheses.rs`. Test it with `cargo test valid_parentheses`.

Problem Statement

Given a string built from the six delimiter characters `()[]{}` , decide whether every opening delimiter is closed by its own kind, with the pairs properly nested. Any other character makes the string invalid.

Designing a Solution

Two conditions must hold. A closing delimiter may appear only when the delimiter it must match is the most recent unmatched opening, and nothing may remain unmatched at the end of the scan.

A counter of openings and closings satisfies the second condition and not the first. In `([)]` every kind of delimiter appears in matching quantity, so a counter reports balance, and the string is invalid because `)` closes `(` while `[` is still open. Correctness depends on the order of the unmatched openings, and an ordered sequence of pending openings is a stack.

The stack holds the opening delimiters whose counterpart has not been seen, with the most recently opened on top. That entry is the delimiter a closing character must match. An opening delimiter is pushed. A closing delimiter pops the top and compares its kind. A closing delimiter that arrives at an empty stack has nothing to close, and the string is invalid at that point. When the scan

finishes the stack must be empty, because anything left in it is an opening delimiter with no counterpart.

The scan is a pushdown automaton, the weakest machine that recognises a language with matched nesting. A finite automaton has no stack and cannot decide the question, so no regular expression can match arbitrarily deep nesting.

Implementation

```
/// Valid Parentheses: classic stack problem with idiomatic pattern matching.
///
pub fn is_valid(s: &str) -> bool {
    use std::collections::HashMap;

    let mut stack = Vec::with_capacity(s.len());

    // Maps each closing delimiter to the opening delimiter it must match.
    let expected = HashMap::from([(')', '('), (')', '{'), (']', '[')]);

    for ch in s.chars() {
        match ch {
            '(' | '[' | '{' => stack.push(ch),
            // Every other character, including one outside the six delimiters,
            // must close the top of the stack.
            _ => {
                if stack.pop().as_ref() != expected.get(&ch) {
                    return false;
                }
            }
        }
    }

    stack.is_empty()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn valid_cases() {
        assert!(is_valid("()"));
        assert!(is_valid("()[{}]"));
        assert!(is_valid("{[()]}" ));
    }

    #[test]
    fn invalid_cases() {
        assert!(!is_valid("[" ));
        assert!(!is_valid("([)]"));
        assert!(!is_valid("(" ));
        assert!(!is_valid(")"));
        assert!(!is_valid("((" ));
    }
}
```

The capacity hint is `s.len()`, the byte length of the string. A character count never exceeds a byte count, so the hint bounds the number of pushes and the

vector does not grow during the scan. For text outside ASCII the hint over-allocates.

`expected` maps each closing delimiter to the opening delimiter it must match. The pairing sits in one structure rather than in one `match` arm per delimiter, so adding a pair to the grammar extends the array rather than the `match`.

The loop has two arms. The first pushes every opening delimiter. The second is the catch-all arm the compiler requires, and it performs the comparison for every other character:

```
- => {
  if stack.pop().as_ref() != expected.get(&ch) {
    return false;
  }
}
```

`stack.pop()` returns `Option<char>`, and `.as_ref()` turns it into `Option<&char>` for comparison with `expected.get(&ch)`, which has the same type. The guard returns `false` whenever the two options differ, which covers a mismatched delimiter and a closing delimiter on an empty stack. The next section states the case it does not cover.

`stack.is_empty()` at the end rejects a string of opening delimiters alone. Without it, `"("` would be reported as valid.

Intuition

input: `{[()]}`

char	stack before	pop result	stack after	value returned
{	[]	-	[{]	
[	[{]	-	[{[]	
(	[{[]	-	[{[()]	
)	[{[()]	Some(' (')	[{[]	-
]	[{[]	Some(' [')	[{	
}	[{	Some(' {')	[]	
end				true

input: `([])`

char	stack before	pop result	stack after	value returned
(	[]	-	[(]	
[	[(]	-	[([]	-
)	[([]	Some(' [')	[(]	-
]	[(]	Some(' (')	[]	false

The delimiters appear in matching quantities and in the wrong nesting order, so the scan reports the failure at the final character.

```
input: )
```

char	stack before	pop result	stack after	value returned
)	[]	None	[]	false

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$	one push or one pop per character, each constant
Space	$O(n)$	for a string of opening delimiters only
Space on typical input	$O(d)$	d is the greatest nesting depth reached

The worst case is a string such as "((((", where the stack grows with the input. The typical case is smaller, because the stack holds only the openings that have not been closed, which is the nesting depth at the cursor rather than the length of the string.

The scan is iterative, so nesting depth is bounded by available memory rather than by the thread's stack. A recursive formulation would exhaust the stack on a deeply nested string, which is a denial-of-service condition in a program that validates a document supplied by a user.

Limitations

A character outside the alphabet is accepted when the stack is empty. The catch-all arm compares two `Option<&char>` values. For a character that is not one of the six delimiters, `expected.get(&ch)` is `None`. If the stack is also empty, `stack.pop().as_ref()` is `None`, the two are equal, and the scan continues instead of rejecting the character. `is_valid("abc")` and `is_valid("( )x")` therefore return `true`. The case does not arise when the stack is non-empty, because a popped `Some(...)` never equals `None`, so the same character is rejected after an opening delimiter. No test in the file covers an input of this kind.

The function returns `bool`, so it cannot report where the failure occurred. A parser that reports "unmatched (at offset 12" needs the position. A version returning `Result<(), usize>` carries it with one change: iterate with `enumerate` and return the index in place of `false`.

The capacity hint is measured in bytes while the stack stores characters. For a string of three-byte characters the allocation is three times larger than the characters can justify, and the vector's length still never exceeds the character count.

The stack element type is `char`, one Unicode scalar value per entry. The algorithm does not require that. A version that reports positions would push the index of the opening character, and the comparisons would be between indices.

References

- Standard library, `Vec::pop`, on the `Option` it returns.
- Standard library, `str::chars`, on iterating scalar values rather than bytes.
- Michael Sipser, *Introduction to the Theory of Computation*, 3rd edition, Cengage Learning, 2012, Chapter 2, on pushdown automata and the limits of finite automata.

Min Stack

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/min_stack.rs`. Test it with `cargo test min_stack`.

Problem Statement

Implement a stack with four operations: `push`, `pop`, `top`, and `get_min`. Each runs in $O(1)$ time. The first three are the standard stack operations. `get_min` returns the smallest value currently on the stack.

A `Vec<i32>` together with a variable holding the current minimum does not satisfy the bound. The variable is correct while values are pushed. A `pop` that removes the current minimum leaves the previous minimum unavailable, because the variable was overwritten when the smaller value was pushed.

Designing a Solution

Store the minimum at every depth. A second vector, the same length as the first, records at index `i` the smallest value among `values[0..=i]`.

values	3	1	4	1	5
mins	3	1	1	1	1
	^	^			
			the smallest of 3, 1		
			the smallest of 3		

The invariant is `mins[i] == min(values[0..=i])`. Two consequences cover the implementation. `get_min` returns the last entry of `mins`, because the last index covers the whole stack. `pop` removes the last entry from both vectors, and the invariant still holds for every remaining index, so nothing is recomputed.

`push` appends to `mins` either the incoming value, when it is smaller than the current minimum, or the current minimum otherwise. Both are available in constant time, which is where the bound comes from.

Implementation

```
/// Min Stack: O(1) push, pop, top, and get_min.
///
/// We keep a parallel `mins` stack. Each push stores the current minimum at that
/// point in history, so pop is symmetric.

#[derive(Debug, Default)]
pub struct MinStack {
    values: Vec<i32>,
    mins: Vec<i32>,
}

impl MinStack {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn push(&mut self, val: i32) {
        match self.mins.last() {
            Some(&min) if min <= val => self.mins.push(min),
            _ => self.mins.push(val),
        }
        self.values.push(val);
    }

    pub fn pop(&mut self) -> Option<i32> {
        self.mins.pop();
        self.values.pop()
    }

    pub fn top(&self) -> Option<i32> {
        self.values.last().copied()
    }

    pub fn get_min(&self) -> Option<i32> {
        self.mins.last().copied()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn stack_tracks_minimum() {
        let mut stack = MinStack::new();
        stack.push(-2);
        stack.push(0);
        stack.push(-3);
        assert_eq!(stack.get_min(), Some(-3));
        assert_eq!(stack.pop(), Some(-3));
        assert_eq!(stack.top(), Some(0));
        assert_eq!(stack.get_min(), Some(-2));
    }

    #[test]
    fn handles_duplicates() {
```

```

    let mut stack = MinStack::new();
    stack.push(1);
    stack.push(1);
    stack.pop();
    assert_eq!(stack.get_min(), Some(1));
}

```

`MinStack::new` returns `Self::default()`, so the constructor and the `Default` implementation cannot disagree.

`push` matches on the last entry of `mins` with a guard. `Some(&min)` if `min <= val` handles a non-empty stack whose incoming value is not smaller, and repeats the previous minimum. The underscore arm covers the empty stack and the strictly smaller value; in both, the value pushed is `val`.

The comparison is `<=` rather than `<`. With `<` the arms remain correct for the minimum, because equal values give the same answer. The choice matters for the variant that stores counts, where equality with the current minimum increments the count instead of appending an entry.

`pop` removes the last entry of each vector and returns the value from the first. Both vectors lose one element per call, so the equal-length invariant holds by construction, and the discarded entry of `mins` is never read.

`top` and `get_min` use `copied`, which turns `Option<&i32>` into `Option<i32>`. Without it the returned reference would borrow the stack and prevent a push while the result is held.

Intuition

call	values	mins	returns
push(-2)	[-2]	[-2]	-
push(0)	[-2, 0]	[-2, -2]	-
push(-3)	[-2, 0, -3]	[-2, -2, -3]	-
get_min()	[-2, 0, -3]	[-2, -2, -3]	Some(-3)
pop()	[-2, 0]	[-2, -2]	Some(-3)
top()	[-2, 0]	[-2, -2]	Some(0)
get_min()	[-2, 0]	[-2, -2]	Some(-2)
pop()	[-2]	[-2]	Some(0)
pop()	[]	[]	Some(-2)
get_min()	[]	[]	None

The trace exercises the case a single tracked minimum would fail. After `-3` is popped, `get_min` reports `-2`, which was the second smallest value. A single tracked minimum would report `-3`, a value no longer on the stack.

The second test pushes two equal values and pops one. The minimum after the pop is still that value, because the surviving entry records it.

Time and Space Complexity

Operation	Time	Space
push	$O(1)$ amortised	one <code>i32</code> in each vector
pop	$O(1)$	releases one <code>i32</code> from each vector
top	$O(1)$	none
get_min	$O(1)$	none

Push is amortised rather than constant because a `Vec` occasionally reallocates and moves its elements. The capacity doubles, so n pushes perform $O(n)$ total movement.

The structure stores two integers per element to hold one. A stack of one million entries occupies eight megabytes instead of four. The alternative under `Limitations` stores a pair per run of equal minima, which uses less space when the minimum repeats and no more when it does not.

Limitations

The memory overhead is proportional to the number of elements, not to the number of distinct minimum values. An increasing sequence changes the minimum on every push and records a distinct value each time. A sequence whose minimum never changes records the same value repeatedly, and the second vector is redundant. A design that stores `(value, repeat_count)` pairs, appending only when the minimum strictly decreases and incrementing the count when it repeats, holds fewer entries on inputs with repeated minima and never more than one entry per element. That variant is not implemented here.

`pop` on an empty stack returns `None` and changes nothing. `top` and `get_min` also return `None` on an empty stack, so an empty stack reports no minimum rather than a minimum of zero.

The structure has no `len` and no `is_empty`, and both fields are private, so the depth is not observable outside the module. A caller can push, pop,

read the top and read the minimum, and nothing else. An application that needs the depth has to track it separately.

The value type is `i32`. Making the structure generic over a type `T`: `Ord` requires a type parameter on the struct, the implementation, and both vectors; the comparison in `push` is the only operation that needs the bound. The second vector stores cloned values, which would also require `Clone`.

`push` appends to `mins` before `values`. If the second append could fail, the vectors would differ in length and the invariant would break. Rust aborts the process on allocation failure, so the case is unreachable through this function. The invariant holds because of the order of two adjacent statements, which the compiler does not check.

References

- Standard library, `Vec::last`.
- Standard library, `Option::copied`, on the conversion from a borrowed to an owned value.
- Standard library, `Vec::push`, on the amortised cost of growth.

Longest Substring Without Repeating Characters

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/sliding_window.rs`. Test it with `cargo test sliding_window`.

Problem Statement

Given a text, return the length of the longest contiguous substring that contains no repeated character. The answer is a length, not the substring, so the implementation can discard the contents of the window and keep only its start.

Designing a Solution

Maintain a window `[start, end)` over the characters and a map from each character to the last index where it was seen. Extend the window by one character at a time. If the new character was last seen at an index at or after `start`, the window must begin after that index.

```
"pwwkew"
```

end	char	window	last_seen	longest
0	p	[p]	{p:0}	1
1	w	[pw]	{p:0, w:1}	2
2	w	[w]	start = 1 + 1 = 2	2
3	k	[wk]	{w:2, k:3}	2
4	e	[wke]	{e:4}	3
5	w	[ew]	start = 2 + 1 = 3	3

The line that carries the algorithm is `start = start.max(previous + 1)`. The `max` is required, not defensive. A character may have been seen, left the

window, and been seen again; its recorded index is then before `start`, and assigning `previous + 1` unconditionally would move the window backwards and count characters that were already counted. `start` is monotonic, and that monotonicity is what makes the scan linear.

Implementation

```
///! Longest substring without repeating characters.
///!
///! Sliding window with `HashMap<char, usize>`; the window start is monotonic.

use std::collections::HashMap;

pub fn length_of_longest_substring(s: &str) -> usize {
    let mut last_seen = HashMap::with_capacity(s.len());

    let (mut start, mut longest) = (0, 0);

    for (end, ch) in s.chars().enumerate() {
        if let Some(&previous) = last_seen.get(&ch) {
            start = start.max(previous + 1);
        }
        longest = longest.max(end - start + 1);
        last_seen.insert(ch, end);
    }

    longest
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn examples() {
        assert_eq!(length_of_longest_substring("abcabcbb"), 3);
        assert_eq!(length_of_longest_substring("bbbbbb"), 1);
        assert_eq!(length_of_longest_substring("pwwkew"), 3);
        assert_eq!(length_of_longest_substring(""), 0);
    }
}
```

`for (end, ch) in s.chars().enumerate()` iterates over Unicode scalar values, not bytes. `end` is therefore a character index, and `end - start + 1` is a length in characters. A byte-oriented version would report a different number for any text outside ASCII.

`if let Some(&previous) = last_seen.get(&ch)` copies the index out of the map, so the borrow ends at the end of the `if let` and the `insert` below is allowed.

`start` and `longest` carry no type annotation. Both are inferred as `usize`, the type `s.chars().enumerate()` produces for `end` and the type `last_seen`'s capacity hint expects; the compiler resolves the binding from its first use rather than from the literal `0` alone.

Intuition

A case where the repeat has left the window, which is the case the `max` exists for:

```
"abba"
```

end	char	previous	start before	start after	window	longest
0	a	none	0	0	[a]	1
1	b	none	0	0	[ab]	2
2	b	Some(1)	0	2	[b]	2
3	a	Some(0)	2	2	[ba]	2

At end = 3 the recorded index of 'a' is 0, which is before the window start of 2. ``start.max(0 + 1)`` keeps start at 2. Assigning `0 + 1` instead would set start to 1 and count "abb" as a window.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$	one step per character; start never decreases
Space	$O(\min(n, k))$	k distinct characters occur in the text

Limitations

The map is sized by a byte count. `HashMap::with_capacity(s.len())` uses the byte length as the capacity hint, which over-allocates for multi-byte text. The hint is safe because a byte count bounds a character count.

The answer is a length, and the substring is discarded. A caller that needs the text of the longest window must record `start` alongside `longest` and slice the original at the end. That change is two lines and is not in the file.

Non-ASCII text is measured in characters, not in grapheme clusters. A combining sequence such as `e` followed by a combining acute accent is two scalar values, so the function counts it as two characters. A user-facing length would need grapheme segmentation, which the standard library does not provide.

The map keeps an entry per distinct character ever seen. Entries for characters that have left the window are not removed, so the space bound is over the whole text rather than over the current window. An implementa-

tion that removes entries on eviction would bound memory by the window instead. For `char` keys the map cannot hold more than 1,112,064 entries, the number of Unicode scalar values.

References

- Standard library, `str::chars`.
- Standard library, `Iterator::enumerate`.
- Standard library, `Ord::max`.

Merge Intervals

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/merge_intervals.rs`. Test it with `cargo test merge_intervals`.

Problem Statement

Given a list of closed intervals `[start, end]`, return the list of maximal intervals covering exactly the same points. Overlapping intervals are merged, and so are intervals that only touch: `(1, 4)` and `(4, 5)` become `(1, 5)`, because `4` belongs to both. Half-open intervals would not merge.

Designing a Solution

Sorting by start removes the need to compare intervals with each other. After the sort, an interval can only overlap the most recent one in the output, so each interval costs one comparison.

input	(1, 3) (2, 6) (8, 10) (15, 18)	already sorted here
interval	merged so far	decision
(1, 3)	[(1, 3)]	the first interval is always pushed
(2, 6)	[(1, 6)]	$2 \leq 3$, so extend the end to $\max(3, 6)$
(8, 10)	[(1, 6), (8, 10)]	$8 > 6$, a gap, so push
(15, 18)	[..., (15, 18)]	$15 > 10$, a gap, so push

The end of the merged interval is the `max` of the two ends. With `(1, 10)` followed by `(2, 3)`, assigning the incoming end would shrink the interval.

Implementation

Three sweeps follow, and they differ only in what they own. The first consumes the vector it is given, the second borrows it, and the third merges inside the vector and allocates no output at all.

```

/// Merge Intervals: sort by start, then merge overlapping intervals in one pass.
///
/// The `match out.last_mut()` pattern is intentionally clean: no deep nesting,
/// no confusing boolean flags.

#[derive(Debug, PartialEq, Clone, Copy)]
pub struct Interval {
    start: i32,
    end: i32
}

pub fn merge_intervals(mut intervals: Vec<Interval>) -> Vec<Interval> {
    if intervals.is_empty() {
        return intervals;
    }

    intervals.sort_unstable_by_key(|interval| interval.start);

    let mut merged: Vec<Interval> = Vec::with_capacity(intervals.len());
    for Interval { start, end } in intervals {
        match merged.last_mut() {
            Some(previous) if start <= previous.end => {
                previous.end = previous.end.max(end);
            }
            _ => merged.push(Interval{start, end}),
        }
    }
    merged
}

pub fn merge_intervals2(mut intervals: Vec<(i32, i32)>) -> Vec<(i32, i32)> {
    if intervals.is_empty() {
        return intervals;
    }

    intervals.sort_unstable_by_key(|(start, _)| *start);

    let mut merged: Vec<(i32, i32)> = Vec::with_capacity(intervals.len());

    for &(start, end) in &intervals {
        match merged.last_mut() {
            Some(previous) if start <= previous.1 => {
                previous.1 = previous.1.max(end);
            }
            _ => merged.push((start, end)),
        }
    }
    merged
}

/// In place: `last` is the last merged interval at the front of the vector.
/// O(1) extra space.
pub fn merge_intervals_in_place(intervals: &mut Vec<Interval>) {
    if intervals.is_empty() {
        return;
    }

    intervals.sort_unstable_by_key(|interval| interval.start);

```

```

let mut last = 0;
for i in 1..intervals.len() {
    match (intervals[last], intervals[i]) {
        (previous, current) if current.start <= previous.end => {
            intervals[last].end = previous.end.max(current.end);
        }
        (_, current) => {
            last += 1;
            intervals[last] = current;
        }
    }
}
intervals.truncate(last + 1);
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_overlapping_and_adjacent() {
        let input = [(1, 3), (2, 6), (8, 10), (15, 18)].map(|(start, end)| Interval { start, end });
        let expected = [(1, 6), (8, 10), (15, 18)].map(|(start, end)| Interval { start, end });
        assert_eq!(merge_intervals(input.into()), expected);
    }

    #[test]
    fn handles_empty_and_disjoint() {
        assert_eq!(merge_intervals(vec![], vec![]));
        let input = [(1, 2), (3, 4)].map(|(start, end)| Interval { start, end });
        let expected = [(1, 2), (3, 4)].map(|(start, end)| Interval { start, end });
        assert_eq!(merge_intervals(input.into()), expected);
    }

    #[test]
    fn merges_in_place() {
        let mut intervals: Vec<Interval> = [(8, 10), (1, 3), (15, 18), (2, 6)]
            .map(|(start, end)| Interval { start, end })
            .into();
        merge_intervals_in_place(&mut intervals);
        let expected = [(1, 6), (8, 10), (15, 18)].map(|(start, end)| Interval { start, end });
        assert_eq!(intervals, expected);

        let mut empty = vec![];
        merge_intervals_in_place(&mut empty);
        assert!(empty.is_empty());
    }
}

```

`sort_unstable_by_key` sorts by the start value and does not preserve the relative order of intervals with equal starts. Intervals with equal starts are merged into one, so their order cannot matter.

Destructuring in the loop header is what keeps the body short. `for Interval { start, end } in intervals` consumes the vector and moves each interval out; `for &(start, end) in &intervals`, in the older tuple-based `merge_intervals2`, copies each pair out of

a vector the caller keeps. Both `Interval` and `(i32, i32)` derive `Copy`, so either loop compiles, and the choice says only whether the input survives the call.

A named struct earns its place here beyond taste. `previous.1 = previous.1.max(end)` and `previous.end = previous.end.max(end)` compile to the same instructions, and only the second one says which field is being extended.

`match merged.last_mut()` with a guard keeps the three cases flat: an overlap, an empty output, and a gap. The last two share an arm because they share an action.

The variant that merges in place

Both sweeps above allocate a second vector, and the output can never be longer than the input, so the allocation is avoidable. Writing the merged intervals over the front of the same vector and truncating at the end brings the extra space down to $O(1)$.

```
/// In place: `last` is the last merged interval at the front of the vector.
///  $O(1)$  extra space.
pub fn merge_intervals_in_place(intervals: &mut Vec<Interval>) {
    if intervals.is_empty() {
        return;
    }

    intervals.sort_unstable_by_key(|interval| interval.start);

    let mut last = 0;
    for i in 1..intervals.len() {
        match (intervals[last], intervals[i]) {
            (previous, current) if current.start <= previous.end => {
                intervals[last].end = previous.end.max(current.end);
            }
            (_, current) => {
                last += 1;
                intervals[last] = current;
            }
        }
    }
    intervals.truncate(last + 1);
}
```

Two indices walk the vector. `i` reads, `last` marks the end of the merged prefix, and `last` never overtakes `i`, so the write at `intervals[last] = current` can only land on a slot that has already been read. That is the invariant the whole sweep rests on, and it is the reason nothing is lost when the prefix and the remainder share one allocation.

`match (intervals[last], intervals[i])` copies both intervals out before the arms run. The copy is what makes the borrow checker accept the assignment inside the first arm: `intervals[last].end = previous.end.max(current.end)`

needs a mutable borrow of the vector, which would clash with a borrow still held by the scrutinee. Copying two structs of eight bytes is cheaper than the borrow-splitting the alternative would require, and `Interval` derives `Copy` for exactly this.

`truncate(last + 1)` drops the tail that the sweep has already absorbed. The vector keeps its capacity, so the merged result sits in the allocation the caller passed in, and the function returns nothing.

Intuition

```
input: [(15, 18), (2, 3), (1, 10), (1, 10)]
sorted by start: [(1, 10), (1, 10), (2, 3), (15, 18)]

interval  merged before  overlap?      merged after
(1, 10)    []             output is empty [(1, 10)]
(1, 10)    [(1, 10)]      1 <= 10         [(1, 10)]          end stays max(10, 10)
(2, 3)     [(1, 10)]      2 <= 10         [(1, 10)]          end stays max(10, 3)
(15, 18)   [(1, 10)]      15 > 10         [(1, 10), (15, 18)]
```

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n \log n)$	the sort dominates; the sweep is $O(n)$
Space	$O(n)$	the output holds at most one interval per input

The input vector is taken by value, so its allocation is available to the function; the first two versions still allocate a second vector for the output. `merge_intervals_in_place` does not, and its space cost is the two indices, which is $O(1)$. All three are dominated by the sort, so the in-place version buys memory rather than time.

Limitations

The second function is unreferenced. `merge_intervals2` is not exported from `lib.rs`, and no test in the file calls it. Its only difference from `merge_intervals` is the loop, which borrows the vector instead of consuming it. An unused `pub fn` in a library is not a compiler warning, so the two functions coexist without a signal that one of them is unused.

There is no test for the borrow-based variant. Even if the function is kept, nothing checks that it agrees with the other one. The assertion `assert_eq!(merge_intervals2(v.clone()), merge_intervals(v))` would cover it.

The element type is `i32`. The end of a merged interval is `previous.1.max(end)`, so the function performs no arithmetic and cannot overflow. The bound matters only in that a caller with larger times cannot use the function.

References

- Standard library, `slice::sort_unstable_by_key`.
- Standard library, `Vec::last_mut`.
- Standard library, `Ord::max`.

Top K Frequent Elements

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/top_k_frequent.rs`. Test it with `cargo test top_k_frequent`.

Problem Statement

Given a slice of integers and a number `k`, return the `k` values that occur most often. The result order is defined only if the function states how ties are broken. This implementation keys the heap on `(count, value)`, compared lexicographically, so a larger value wins a tie.

Designing a Solution

Two passes. The first counts occurrences in a hash map. The second moves every `(count, value)` pair into a `BinaryHeap` and pops the largest keys.

`BinaryHeap` in Rust is a max-heap, so the pairs come out in descending count and, for equal counts, in descending value. The alternative, a min-heap holding at most `k` entries, costs $O(m \log k)$ time and $O(k)$ space and is not implemented here. The cost table states what the simpler choice costs.

Implementation

```
/// Top K Frequent Elements.
///
/// Count occurrences with a HashMap, then use a max-heap (`BinaryHeap`) to pull
/// out the top k elements.  $O(n \log n)$  worst case; acceptable and readable.

use std::collections::{BinaryHeap, HashMap};
```

```

pub fn top_k_frequent(nums: &[i32], k: usize) -> Vec<i32> {
    let mut counts: HashMap<i32, usize> = HashMap::with_capacity(nums.len());
    for &num in nums.iter() {
        *counts.entry(num).or_insert(0) += 1;
    }

    // BinaryHeap is a max-heap in Rust, so (count, num) keeps highest counts first.
    let mut heap: BinaryHeap<(usize, i32)> = counts
        .into_iter()
        .map(|(num, count)| (count, num))
        .collect();

    let mut result = Vec::with_capacity(k);
    while result.len() < k {
        match heap.pop() {
            Some((_, num)) => result.push(num),
            None => break,
        }
    }
    result
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn returns_top_two() {
        let nums = [1, 1, 1, 2, 2, 3];
        let mut result = top_k_frequent(&nums, 2);
        result.sort_unstable();
        assert_eq!(result, vec![1, 2]);
    }

    #[test]
    fn k_larger_than_unique_values_returns_all() {
        let result = top_k_frequent(&[1, 2, 3], 10);
        assert_eq!(result.len(), 3);
    }
}

```

`*counts.entry(num).or_insert(0) += 1` looks up the count or inserts zero, then increments through the returned `&mut usize`. The map is searched once per element rather than twice.

`counts.into_iter()` consumes the map and yields owned pairs, so the heap is built without borrowing the map. `map(|(num, count)| (count, num))` reverses each pair so that the count is the first component of the key, which is the tie-break rule.

`while result.len() < k` with `None => break` handles a `k` larger than the number of distinct values: the heap empties, the loop stops, and the result is shorter than `k`.

Intuition

```

nums = [1, 1, 1, 2, 2, 3]      k = 2

pass 1: counts = {1: 3, 2: 2, 3: 1}

pass 2: heap after collecting the pairs (count, value):
        pop order is (3, 1), (2, 2), (1, 3)

pop 1 -> push value 1      result = [1]
pop 2 -> push value 2      result = [1, 2]      length is k, stop

The heap still holds (1, 3), which is not popped.

```

Time and Space Complexity

Re-source	Cost	Condition
Time	$O(n + m \log m)$ expected	m distinct values; the first pass is $O(n)$, the heap operations are $O(m \log m)$
Space	$O(m)$	the map and the heap each hold one entry per distinct value

The heap holds one entry per distinct value rather than k entries. For a slice of a million elements with half a million distinct values and $k = 10$, the program builds a heap of half a million entries to return ten. A min-heap bounded at k entries, which discards each element smaller than its root, costs $O(m \log k)$ time and $O(k)$ space. The file does not implement it.

Limitations

The heap is unbounded in the number of distinct values. Every distinct value counted becomes an entry, so the structure holds m entries at its largest, where m is the number of distinct values in the input.

The result order is fixed by the heap comparison and nothing else. The pair ordering makes it deterministic, and the first test sorts the result before asserting, so the file never pins the order. A caller that prints the result directly sees descending count and then descending value.

A `k` of zero returns an empty vector. `Vec::with_capacity(0)` allocates nothing and the loop does not run, so the answer is empty rather than a panic. The file has no test for it.

The tie-break rule is not documented. A reader has to derive it from `(count, num)` and from the fact that `BinaryHeap` is a max-heap. Two sentences in the module documentation would make it part of the contract rather than a consequence of the implementation.

References

- Standard library, `BinaryHeap`, which documents that the heap is a max-heap.
- Standard library, `cmp::Reverse`, the wrapper that turns a max-heap into a min-heap.
- Standard library, `HashMap::entry`.

Part II: Trees and Graphs

Binary Tree

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/binary_tree.rs`. Test it with `cargo test binary_tree`.

Problem Statement

Represent a binary tree of integers, compute its depth, mirror it in place, and produce its three depth-first traversals.

Designing a Solution

The type is an algebraic data type with two variants:

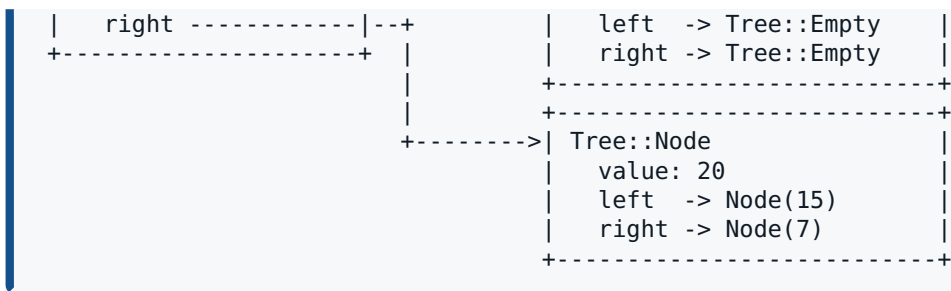
```
Tree = Empty
      | Node { value: i32, left: Box<Tree>, right: Box<Tree> }
```

Two properties follow from this representation rather than from nullable pointers.

A node always has both children. There is no state in which the left child is absent and the right one is present, because the `Node` variant has no optional fields. Every function that matches on `Tree::Node` binds `left` and `right` without testing them.

The recursion is in the type. A `Node` owns its children, and each child is boxed, so the size of a `Tree` value does not depend on how many nodes exist:

stack	heap
+-----+	+-----+
Tree::Node	Tree::Node
value: 3	value: 9
left -----+	



Each arrow is one eight-byte ownership pointer. The `Empty` variant is a discriminant with no payload, so it occupies no heap space.

Implementation

```

///! Binary tree as an algebraic data type.
///!
///! ```text
///! Tree = Empty
///! | Node { value, left: Box<Tree>, right: Box<Tree> }
///! ```

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Tree {
    Empty,
    Node {
        value: i32,
        left: Box<Tree>,
        right: Box<Tree>,
    },
}

impl Tree {
    pub fn max_depth(&self) -> usize {
        match self {
            Tree::Empty => 0,
            Tree::Node { left, right, .. } => 1 + left.max_depth().max(right.max_depth(
)),
        }
    }

    pub fn invert(&mut self) {
        match self {
            Tree::Empty => {},
            Tree::Node { left, right, .. } => {
                left.invert();
                right.invert();
                std::mem::swap(left, right);
            }
        }
    }

    pub fn preorder(&self) -> Vec<i32> {
        let mut out = Vec::new();
        self.collect_preorder(&mut out);
        out
    }

    pub fn inorder(&self) -> Vec<i32> {
        let mut out = Vec::new();
        self.collect_inorder(&mut out);
        out
    }
}

```

```

}

pub fn postorder(&self) -> Vec<i32> {
    let mut out = Vec::new();
    self.collect_postorder(&mut out);
    out
}

fn collect_preorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            out.push(*value);
            left.collect_preorder(out);
            right.collect_preorder(out);
        }
    }
}

fn collect_inorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            left.collect_inorder(out);
            out.push(*value);
            right.collect_inorder(out);
        }
    }
}

fn collect_postorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            left.collect_postorder(out);
            right.collect_postorder(out);
            out.push(*value);
        }
    }
}
}

#[cfg(test)]
mod tests {
    use super::*;

    fn leaf(value: i32) -> Tree {
        Tree::Node {
            value,
            left: Box::new(Tree::Empty),
            right: Box::new(Tree::Empty),
        }
    }

    fn node(value: i32, left: Tree, right: Tree) -> Tree {
        Tree::Node {
            value,
            left: Box::new(left),
            right: Box::new(right),
        }
    }

    fn sample() -> Tree {
        node(3, leaf(9), node(20, leaf(15), leaf(7)))
    }
}

#[test]

```

```

fn max_depth_works() {
    assert_eq!(Tree::Empty.max_depth(), 0);
    assert_eq!(leaf(1).max_depth(), 1);
    assert_eq!(sample().max_depth(), 3);
}

#[test]
fn invert_swaps_subtrees() {
    let mut tree = sample();
    tree.invert();

    match &tree {
        Tree::Node { left, right, .. } => {
            match left.as_ref() {
                Tree::Node { value, .. } => assert_eq!(*value, 20),
                _ => panic!("expected left node"),
            }
            match right.as_ref() {
                Tree::Node { value, .. } => assert_eq!(*value, 9),
                _ => panic!("expected right node"),
            }
        }
        _ => panic!("expected node"),
    }
}

#[test]
fn traversals_work() {
    let tree = sample();
    assert_eq!(tree.preorder(), vec![3, 9, 20, 15, 7]);
    assert_eq!(tree.inorder(), vec![9, 3, 15, 20, 7]);
    assert_eq!(tree.postorder(), vec![9, 15, 7, 20, 3]);
}

```

`max_depth` uses `..` in the pattern to ignore `value`, and computes `1 + left.max_depth().max(right.max_depth())` without a binding for the child depths. `Ord::max` on two `usize` values is a single instruction.

`invert` swaps the two children after recursing into both. `std::mem::swap(left, right)` exchanges the two `Box` values, which swaps two pointers per node and moves no subtree. Swapping before recursing would also be correct; the two subtrees are independent.

The three collection helpers take `&mut Vec<i32>`. Returning a new vector from each recursive call would copy the whole output at every level of the tree, which turns a linear traversal into a quadratic one.

The test helpers `leaf` and `node` exist because building a tree means writing `Box::new` at every level, and the constructors keep the shape of the sample tree readable. They are in the test module, so they are not part of the library's API.

Intuition

sample tree

3

9

20

15

7

preorder

value, left, right

3 9 20 15 7

inorder

left, value, right

9 3 15 20 7

postorder

left, right, value

9 15 7 20 3

max_depth

1 + max(depth of 9, depth of the 20 subtree)

depth of 9 = 1

depth of 20 = 1 + max(1, 1) = 2

result = 1 + max(1, 2) = 3

invert

left and right exchanged at every node

3

20

9

7

15

Time and Space Complexity

Operation	Time	Space
max_depth	$O(n)$	$O(h)$ stack frames
invert	$O(n)$	$O(h)$ stack frames, no allocation
each traversal	$O(n)$	$O(n)$ for the output, $O(h)$ stack frames

h is the height of the tree: $O(\log n)$ when the tree is balanced and $O(n)$ when it is a chain.

Limitations

A deep tree exhausts the stack. All five methods recurse, so each uses one stack frame per level. A left-leaning tree of a few hundred thousand nodes overflowing the default thread stack aborts the process, because a stack overflow is not a panic that unwinds. The traversals cannot be reformulated iteratively without an explicit work list, and `#[derive(Clone)]` has the same problem: the derived `clone` recurses as well.

The derived destructor recurses too. Dropping a deep tree follows the boxes, one frame per level, for the same reason. Freeing the memory is therefore as unsafe for a deep tree as cloning it.

The value type is fixed at `i32`. Every method works for any type, and making the declaration generic is one change to the `enum` and one to each signature, but the file as written is not generic.

`Tree` has no `PartialOrd`, no iteration, and no `len`. A caller that needs the node count writes a recursive helper or a loop with a work list.

The test for `invert` inspects two values and not the shape. It checks that the roots of the two subtrees were exchanged, which catches an `invert` that does nothing and not one that swaps the children of the root only. Asserting `tree.preorder()` against the mirrored sequence would test the whole tree in one line.

References

- The Rust Book, [Defining an enum](#).
- The Rust Book, [Enabling recursive types with boxes](#).
- Standard library, `std::mem::swap`.

Binary Search Tree

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/bst_clean.rs`. Run it with `cargo run --bin bst_clean`.

Problem Statement

Implement a binary search tree with four operations: `insert`, `contains`, `remove`, and `range`, which returns the values in a closed interval. Removal has a case the others do not: a node with two children cannot be unlinked without a replacement that preserves the ordering.

Designing a Solution

The ordering invariant defines the structure:

```
for every node:  every value in the left subtree < the node's value
                  every value in the right subtree > the node's value
```

A lookup follows one path from the root and discards a subtree at each comparison. An in-order traversal reads the values in ascending order, which is the usual way to test that the invariant holds.

Removal has three shapes:

```
delete a leaf           the link becomes Empty

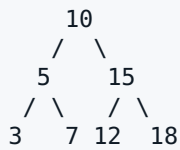
delete a node with one child the child takes its place

delete a node with two children  the smallest value of the right subtree takes its
                                  place; it is the next value in order, so the
                                  ordering holds on both sides

example: remove 10 from
          10
```



the right subtree is 15, whose smallest value is 15, so 15 takes the place of 10



remove 5 -> its only child 7 moves up
 remove 10 -> 12, the successor, moves up

Implementation

```

use std::cmp::Ordering;

type TreeLink<T> = Box<BST<T>>;

#[derive(Debug, Clone, Default)]
pub enum BST<T> {
    #[default]
    Empty,
    Node {
        value: T,
        left: TreeLink<T>,
        right: TreeLink<T>,
    },
}

impl<T: Ord> BST<T> {
    pub fn new() -> Self {
        Self::Empty
    }

    pub fn insert(&mut self, val: T) {
        match self {
            Self::Empty => {
                *self = Self::Node {
                    value: val,
                    left: Box::new(Self::Empty),
                    right: Box::new(Self::Empty),
                };
            }
            Self::Node { value, left, right } => match val.cmp(value) {
                Ordering::Less => left.insert(val),
                Ordering::Greater => right.insert(val),
                Ordering::Equal => {},
            },
        }
    }

    pub fn contains(&self, val: &T) -> bool {
        match self {
            Self::Empty => false,
            Self::Node { value, left, right } => match val.cmp(value) {
                Ordering::Less => left.contains(val),
                Ordering::Greater => right.contains(val),
                Ordering::Equal => true,
            },
        }
    }
}

```

```

}

pub fn remove(&mut self, val: &T) -> bool {
    match self {
        Self::Empty => false,
        Self::Node { value, left, right } => match val.cmp(value) {
            Ordering::Less => left.remove(val),
            Ordering::Greater => right.remove(val),
            Ordering::Equal => {
                if left.is_empty() && right.is_empty() {
                    *self = Self::Empty;
                    return true;
                }
                if left.is_empty() {
                    *self = *std::mem::take(right);
                    return true;
                }
                if right.is_empty() {
                    *self = *std::mem::take(left);
                    return true;
                }
                let min_val = right.remove_min().unwrap();
                *value = min_val;
                true
            }
        },
    }
}

fn remove_min(&mut self) -> Option<T> {
    match self {
        Self::Empty => None,
        Self::Node { left, .. } if left.is_empty() => {
            let node = std::mem::take(self);
            if let Self::Node { value, right, .. } = node {
                *self = *right;
                Some(value)
            } else {
                unreachable!()
            }
        }
        Self::Node { left, .. } => left.remove_min(),
    }
}

pub fn range(&self, low: &T, high: &T) -> Vec<&T> {
    let mut result = Vec::new();
    self.range_helper(low, high, &mut result);
    result
}

// Explicit lifetime 'a ties the references in the vector to the lifetime of &self.
fn range_helper<'a>(&'a self, low: &T, high: &T, acc: &mut Vec<&'a T>) {
    match self {
        Self::Empty => {}
        Self::Node { value, left, right } => {
            if value < low {
                right.range_helper(low, high, acc);
            } else if value > high {
                left.range_helper(low, high, acc);
            } else {
                left.range_helper(low, high, acc);
                acc.push(value);
                right.range_helper(low, high, acc);
            }
        }
    }
}

```

```

    }

    fn is_empty(&self) -> bool {
        matches!(self, Self::Empty)
    }
}

fn main() {
    let mut bst = BST::new();
    bst.insert(10);
    bst.insert(5);
    bst.insert(15);
    bst.insert(3);
    bst.insert(7);
    bst.insert(12);
    bst.insert(18);

    println!("Contains 7: {}", bst.contains(&7));
    println!("Contains 12: {}", bst.contains(&12));

    let range = bst.range(&4, &16);
    println!("Values in [4, 16]: {:?}", range);

    println!("Remove 5: {}", bst.remove(&5));
    println!("Remove 10: {}", bst.remove(&10));
    println!("Remove 99: {}", bst.remove(&99));

    println!("After removals: {:#?}", bst);
}

```

The type is its own node. `BST<T>` is either `Empty` or a node with two boxed subtrees, so `Box<BST<T>>` is the link type and there is no separate wrapper struct. The `Default` derive with `#[default]` on `Empty` gives `BST::default()` as an empty tree.

`match val.cmp(value)` produces one arm per comparison result. Using `cmp` rather than a pair of comparisons decides the ordering once per node, and the `Ordering::Equal` arm is explicit at every call site.

`remove` handles the two-children case by moving the smallest value out of the right subtree into the node being deleted. `right.remove_min()` returns that value, and the `unwrap` is safe because the branch is reached only when the right subtree is non-empty. Values move; links stay.

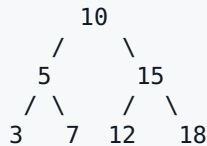
`std::mem::take(right)` replaces the right subtree with `Empty` and returns the old one. `*self = *std::mem::take(right)` then moves the subtree into the node, which detaches it from its old position in one assignment.

`range_helper` prunes: a subtree entirely below `low` is searched on its right only, and one entirely above `high` on its left only. The lifetime parameter `'a` ties the references in `acc` to the tree that `self` borrows, which is what allows the returned vector to outlive the call.

Intuition

bst_clean output, with the tree built by main:

```
insert 10, 5, 15, 3, 7, 12, 18
```



```
in order      : 3 5 7 10 12 15 18
```

```
range (4, 16) : 5 7 10 12 15
```

```

remove(&5)  -> true    5 has one child, so 7 moves up
remove(&10) -> true    10 has two children, so 12 moves up
remove(&99) -> false   the descent ends at Empty
  
```

Time and Space Complexity

Operation	Time	Space
insert	$O(h)$	$O(h)$ stack frames
contains	$O(h)$	$O(h)$ stack frames
remove	$O(h)$	$O(h)$ stack frames, plus one descent to the successor
range	$O(h + k)$	$O(k)$ for the output, $O(h)$ stack frames

h is the height. The tree does not balance, so h is $O(\log n)$ only when the input arrives in an order that produces a balanced shape, and $O(n)$ when it arrives sorted.

Limitations

The file has no tests. `src/bin/bst_clean.rs` contains no `#[cfg(test)]` module, and `main` prints values without asserting anything, so a break in the ordering invariant appears as terminal output rather than as a failing test. The behaviour to pin down is the ordering invariant and the three removal shapes; a test that inserts a known sequence, removes a leaf, a one-child node and a two-child node, and asserts the in-order traversal after each step covers all of it.

The tree does not balance. Inserting 1 to 1000 in ascending order produces a chain of height 1000, and every operation becomes linear. A pro-

duction ordered collection uses a red-black tree or a B-tree; the standard library's `BTreeMap` is the one to reach for.

Removal by successor does not balance either. Repeated deletions under this rule can leave a tree taller than the input would suggest.

`remove_min` contains an `unreachable!()` arm. The branch is genuinely unreachable: the guard `left.is_empty()` and the pattern `Self::Node` together guarantee that the taken node is a node. The arm exists because the compiler cannot see that, and `unreachable!()` records the guarantee, but it is a panic path in a function whose caller does not expect one.

`remove` panics if the invariant is broken. The two-child branch calls `right.remove_min().unwrap()`, and `remove_min` returns `None` only for an empty subtree. The `unwrap` cannot fail while the tree is ordered; it is a second panic path.

The value type needs `Ord`. A caller with a type that is only `PartialOrd`, such as a floating-point number, cannot use the tree at all, because `insert` matches on `Ordering` and `f64` does not implement `Ord`.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Chapter 12, "Binary Search Trees".
- Standard library, `std::cmp::Ordering`.
- Standard library, `std::mem::take`.
- Standard library, `BTreeMap`.

Binary Search on a Rotated Array

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/binary_search.rs`. Test it with `cargo test binary_search`.

Problem Statement

A sorted slice of distinct values has been rotated by an unknown amount. Find the index of a target value in $O(\log n)$, or report that it is absent.

```
[4, 5, 6, 7, 0, 1, 2]    a rotation of [0, 1, 2, 4, 5, 6, 7]
   sorted         sorted
```

Designing a Solution

The loop keeps a half-open range `[low, high)` and the invariant

the target, if it is present, lies in `nums[low..high]`.

In a rotated array, at least one half of any range is sorted. Testing `nums[low] <= nums[mid]` says which half. If the left half is sorted, one range test decides whether the target can be in it; if it cannot, the target is in the right half. That test is one comparison, which is what keeps the loop logarithmic.

```
nums = [4, 5, 6, 7, 0, 1, 2]    target = 0

low  high  mid  nums[mid]  left sorted?  decision
0    7     3    7         yes, 4 <= 7   target not in [4, 7) -> low = 4
4    7     5    1         no, 0 <= 1    target not in (1, 2] -> high = 5
4    5     4    0         found                      -> Some(4)
```


Implementation

```

/// Binary search on a rotated sorted array.
///
/// Classic  $O(\log n)$  interview problem. The code uses explicit range bounds and
/// sorted-half checks; no recursion or nested `if` chains are required.

pub fn search_rotated(nums: &[i32], target: i32) -> Option<usize> {
    let mut low = 0usize;
    let mut high = nums.len();

    while low < high {
        let mid = low + (high - low) / 2;
        if nums[mid] == target {
            return Some(mid);
        }

        // 6,7, 1,2,3,4,5
        // start = 0, end = 6, 0 + (6-0)/2 = 3

        // 4,5,6,1,2,3
        let left_is_sorted = nums[low] <= nums[mid];
        if left_is_sorted {
            if nums[low] <= target && target < nums[mid] {
                high = mid;
            } else {
                low = mid + 1;
            }
        } else if nums[mid] < target && target <= nums[high - 1] {
            low = mid + 1;
        } else {
            high = mid;
        }
    }

    None
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn finds_target_in_rotated_array() {
        let nums = [4, 5, 6, 7, 0, 1, 2];
        assert_eq!(search_rotated(&nums, 0), Some(4));
        assert_eq!(search_rotated(&nums, 3), None);
        assert_eq!(search_rotated(&nums, 5), Some(1));
    }

    #[test]
    fn handles_unrotated_and_single() {
        assert_eq!(search_rotated(&[1, 2, 3, 4], 3), Some(2));
        assert_eq!(search_rotated(&[1], 1), Some(0));
        assert_eq!(search_rotated(&[1], 2), None);
    }
}

```

`high` starts at `nums.len()`, not at `nums.len() - 1`, because the range is half-open. That choice removes the special case for an empty slice: `low == high == 0` and the loop does not run.

`let mid = low + (high - low) / 2` avoids the overflow that `(low + high) / 2` would have in a debug build for a slice near the maximum length. The midpoint satisfies `low <= mid < high`, so `nums[mid]` and `nums[high - 1]` are both in bounds.

`nums[low] <= target && target < nums[mid]` is a half-open interval test: the lower bound is inclusive and the upper bound exclusive, matching the convention used by the loop range.

Intuition

```
nums = [4, 5, 6, 7, 0, 1, 2]      target = 3, absent

low  high  mid  nums[mid]  left sorted?  test                                next
0    7     3    7          yes, 4 <= 7    4 <= 3? no                          low = 4
4    7     5    1          no, 0 <= 1    1 < 3 and 3 <= nums[6]=2? no        high = 5
4    5     4    0          found? no      left sorted? 0 <= 0 yes
                                   0 <= 3 and 3 < 0? no
5    5     -    -          low == high, loop ends
                                   None
```

Time and Space Complexity

Resource	Cost
Time	$O(\log n)$, the range halves on every iteration
Space	$O(1)$, three indices and no recursion

Limitations

Repeated values break the sorted-half test. With duplicates, `nums[low] <= nums[mid]` can hold for both halves, so the code can discard the half holding the target. On `[1, 1, 1, 0, 1]` the search for `0` fails. The function is correct only for strictly increasing inputs, which the problem statement guarantees and the documentation does not mention.

The returned index is not the first or the last match. With duplicates, any index holding the target satisfies the problem as stated. A caller that needs a boundary needs a different function.

The slice must be a rotation of one sorted array, not two ascending runs. The runs have to be the tail and the head of a single sorted sequence, which is what makes the range tests meaningful.

There is no test for a two-element rotation. `[2, 1]` is the smallest input where `nums[low] == nums[mid]` is false and the sorted-half test has to choose correctly.

References

- Standard library, `slice::binary_search`, the unrotated case.
- Standard library, `usize`, whose arithmetic the midpoint formula is written to avoid overflowing.

Trie

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/trie.rs`. Test it with `cargo test trie`.

Problem Statement

Store a set of strings so that two questions are cheap: whether an exact string is present, and whether any stored string begins with a given prefix. A hash map answers the first in constant time and the second only by scanning every key, which is the reason for the structure.

Designing a Solution

Each node holds a map from a character to a child node, plus a flag saying whether the path that ends at this node spells a stored string. A string is present when walking its characters from the root succeeds and the final node has the flag set.

```
words: "gpu", "gpucloud"

      (root)
      |
      g
      |
      p
      |
      u    <- terminal: "gpu" is a word
      |
      c
      |
      l
      |
      o
      |
```

```

u
|
d  <- terminal: "gpucloud" is a word

```

The flag is what distinguishes a stored word from a prefix of a longer word. Without it, "gp" and "gpu" would be indistinguishable, and `search("gp")` would report `true`.

Implementation

```

///! Trie / prefix tree.
///!
///! Relevant to NVIDIA-style SDK/CLI/codegen work: autocomplete, filtering GPU
///! SKUs, command completion, and prefix matching.

use std::collections::HashMap;

#[derive(Default)]
pub struct Trie {
    root: Node,
}

#[derive(Default)]
struct Node {
    children: HashMap<char, Node>,
    terminal: bool,
}

impl Trie {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn insert(&mut self, word: &str) {
        let mut node = &mut self.root;
        for ch in word.chars() {
            node = node.children.entry(ch).or_default();
        }
        node.terminal = true;
    }

    pub fn search(&self, word: &str) -> bool {
        self.find_node(word).is_some_and(|node| node.terminal)
    }

    pub fn starts_with(&self, prefix: &str) -> bool {
        self.find_node(prefix).is_some()
    }

    fn find_node(&self, word: &str) -> Option<&Node> {
        let mut node = &self.root;
        for ch in word.chars() {
            node = node.children.get(&ch)?;
        }
        Some(node)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

```

```

#[test]
fn trie_insert_search_and_prefix() {
    let mut trie = Trie::new();
    trie.insert("gpu");
    trie.insert("gpucloud");

    assert!(trie.search("gpu"));
    assert!(trie.search("gpucloud"));
    assert!(!trie.search("gp"));
    assert!(!trie.search("gpucloudapi"));

    assert!(trie.starts_with("gp"));
    assert!(trie.starts_with("gpucloud"));
    assert!(!trie.starts_with("h100"));
}
}

```

`insert` walks with `&mut` and creates children on demand: `node.children.entry(ch).or_default()` inserts an empty node the first time the character is seen and returns a mutable reference to the existing one otherwise. The walk ends with a mutable reference to the final node, and one assignment sets the flag.

`find_node` is the read-only counterpart, and it uses `?` to leave the loop early: `node.children.get(&ch)?` returns `None` from the function when the character is absent. That one expression replaces a conditional and a `return`.

`is_some_and(|node| node.terminal)` combines two questions into one expression: does the path exist, and is its end a word. Without the second part, `search` would answer the same question as `starts_with`.

Intuition

```

insert("gpu")
  node = root
  'g' -> root.children has no 'g', so it is created; node = that node
  'p' -> created; node = that node
  'u' -> created; node = that node
  node.terminal = true

insert("gpucloud")
  'g', 'p', 'u' all exist, so the walk reuses the nodes built above
  'c', 'l', 'o', 'u', 'd' are created
  node.terminal = true

search("gp")           the walk succeeds, the final node's terminal flag is false
                       -> false
starts_with("gp")      the walk succeeds -> true
search("gpucloudapi")
                       'a' is absent after "gpucloud" -> the walk returns None
                       -> false

```

Time and Space Complexity

Operation	Time	Space
insert	$O(k)$ expected	one node per character not already present
search	$O(k)$ expected	none
starts_with	$O(k)$ expected	none

k counts characters, because `word.chars()` iterates over Unicode scalar values.

Limitations

One node per distinct character position. The structure stores a `HashMap` per node, and a `HashMap` is larger than the characters it holds. A set of strings with no shared prefixes occupies more memory than the same strings in a vector. A radix tree compresses chains of single-child nodes and recovers most of that space; the file does not implement one.

There is no deletion. Removing a word means clearing a terminal flag and then pruning the nodes that are no longer on any path to a word. Neither operation is present, so the only way to empty the structure is to drop it.

There is no query that returns the words under a prefix. `starts_with` answers a boolean, and a caller that wants the completions must write its own traversal. The structure supports the query; the API does not expose it.

The node type is not public, and neither is the root. A caller cannot inspect the tree's shape, count its nodes, or measure its memory from outside the module. For a teaching structure that is a defensible choice, and it means a memory question can only be answered by reading the source.

References

- Edward Fredkin, "Trie memory", *Communications of the ACM* 3(9), 1960, pages 490–499.

- Donald E. Knuth, *The Art of Computer Programming*, Volume 3, 2nd edition, Addison-Wesley, 1998, Section 6.3, "Digital Searching".
- Standard library, `HashMap::entry`.
- Standard library, `Option::is_some_and`, stabilised in Rust 1.70.0.

Topological Sort

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/problems/graph_topology.rs`, tested with `cargo test graph_topology`, and `src/bin/dependency_resolution.rs`, run with `cargo run --bin dependency_resolution`.

Problem Statement

Given a directed graph, produce an ordering of its vertices in which every edge points forward. If the graph contains a cycle, no such ordering exists and the function reports that instead.

```
edges:  0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3

  0 -----> 1
  |         |
  v         v
  2 -----> 3

valid orders: [0, 1, 2, 3], [0, 2, 1, 3]
```

Designing a Solution

Kahn's algorithm. A vertex can be placed as soon as every vertex that must precede it has been placed, so counting the number of unplaced predecessors, the in-degree, is enough to decide when a vertex becomes available.

```
in-degree of each vertex: 0:0   1:1   2:1   3:2
ready queue: [0]

take 0 -> order [0]           in-degree 1:0, 2:0       queue [1, 2]
take 1 -> order [0, 1]       in-degree 3:1             queue [2]
take 2 -> order [0, 1, 2]    in-degree 3:0             queue [3]
take 3 -> order [0, 1, 2, 3]                                queue []
```

Four vertices were placed out of four, so the graph is acyclic. If the loop ends with fewer vertices placed than the graph holds, the unplaced vertices are exactly those on or after a cycle: each vertex of a cycle waits for another vertex of the same cycle, so none of them reaches in-degree zero. The cycle test is therefore one comparison at the end, with no separate traversal.

Implementation

```
/// Topological sort (Kahn's algorithm).
///
/// Common in NVIDIA-style scheduling questions: prerequisite graphs, pipeline
/// stages, dependency ordering of GPU jobs.

use std::collections::VecDeque;

pub fn topological_sort(num_nodes: usize, edges: &[(usize, usize)]) ->
Option<Vec<usize>> {
    let mut graph = vec![Vec::new(); num_nodes];
    let mut indegree = vec![0; num_nodes];

    for &(from, to) in edges {
        graph[from].push(to);
        indegree[to] += 1;
    }

    let mut queue: VecDeque<usize> = (0..num_nodes).filter(|&n| indegree[n] == 0).collect();
    let mut order = Vec::with_capacity(num_nodes);

    while let Some(node) = queue.pop_front() {
        order.push(node);

        for &next in &graph[node] {
            indegree[next] -= 1;
            if indegree[next] == 0 {
                queue.push_back(next);
            }
        }
    }

    (order.len() == num_nodes).then_some(order)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn returns_order_for_dag() {
        // 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3
        let edges = [(0, 1), (0, 2), (1, 3), (2, 3)];
        let order = topological_sort(4, &edges).expect("DAG has an order");
        assert_eq!(order.len(), 4);
        assert!(
            order.iter().position(|&n| n == 0).unwrap()
                < order.iter().position(|&n| n == 1).unwrap()
        );
        assert!(
            order.iter().position(|&n| n == 1).unwrap()
                < order.iter().position(|&n| n == 3).unwrap()
        );
    }
}
```

```

    }

    #[test]
    fn detects_cycle() {
        // 0 -> 1 -> 0 is a cycle.
        let edges = [(0, 1), (1, 0)];
        assert_eq!(topological_sort(2, &edges), None);
    }

    #[test]
    fn disconnected_nodes_are_included() {
        let edges = [(0, 1)];
        let order = topological_sort(3, &edges).expect("DAG has an order");
        assert_eq!(order.len(), 3);
    }
}

```

`vec![Vec::new(); num_nodes]` builds the adjacency list with one vector per vertex. The inner vectors grow as edges are added, so the total allocation is proportional to the edge count.

`queue` is seeded with every vertex whose in-degree is already zero. A `VecDeque` is used because vertices arrive at the back and leave from the front: `pop_front` and `push_back` are both constant time, and removing from the front of a `Vec` is linear.

`indegree[next] -= 1` runs once per edge leaving `node`, so the total work of the inner loop across the whole algorithm is the edge count. That is what makes the cost $O(V + E)$.

`(order.len() == num_nodes).then_some(order)` consumes `order` only when the comparison is true. If the graph has a cycle, the partial order is dropped and the caller receives `None`, so no caller can schedule a partial order by accident.

The variant that names vertices with strings

Dependency graphs rarely arrive as integers. A build system is handed package names, a course catalogue is handed course codes, and a scheduler is handed job identifiers. Kahn's algorithm does not change, but every vector indexed by a vertex becomes a hash map keyed by a name, and that substitution has a price worth seeing in full.

Source file: `src/bin/dependency_resolution.rs`. Run it with `cargo run --bin dependency_resolution`.

```

use std::collections::{HashMap, VecDeque};

#[derive(Clone)]
struct Course {
    name: String,
    depends_on: Vec<String>,
}

```

```

fn topo_sort(courses: &[Course]) -> Vec<String> {
    let mut adj: HashMap<String, Vec<String>> = HashMap::new();
    let mut indegree: HashMap<String, usize> = HashMap::new();

    for Course { name, depends_on } in courses {
        indegree.insert(name.clone(), depends_on.len());
        adj.entry(name.clone()).or_default();
        for dep in depends_on {
            adj.entry(dep.clone()).or_default().push(name.clone());
            indegree.entry(dep.clone()).or_insert(0);
        }
    }

    let mut queue: VecDeque<_> = indegree
        .iter()
        .filter(|&(_, deg)| *deg == 0)
        .map(|(name, _)| name.clone())
        .collect();

    let mut order = Vec::new();
    while let Some(node) = queue.pop_front() {
        for next in &adj[&node] {
            let d = indegree.get_mut(next).unwrap();
            *d -= 1;
            if *d == 0 {
                queue.push_back(next.clone());
            }
        }
        order.push(node);
    }

    if order.len() == indegree.len() { order } else { Vec::new() }
}

fn main() {
    let courses = vec![
        Course { name: "algorithms".into(), depends_on: vec!["datastructures".into()] },
        Course { name: "compilers".into(), depends_on: vec!["algorithms".into(), "os".into()] },
        Course { name: "os".into(), depends_on: vec!["datastructures".into()] },
    ];

    let coned2 = courses.clone();

    println!("{:?}", topo_sort(&courses));
}

```

The input is declared in the direction a person would write it. A `Course` names what it depends on, so `depends_on.len()` is already the in-degree of that course and can be inserted directly, where the index version increments a counter once per edge. The edges themselves point the other way: `adj.entry(dep).or_default().push(name)` records that finishing `dep` unblocks `name`, which is the direction the traversal needs.

Three calls in the build loop exist only to make sure no vertex is missing. `adj.entry(name).or_default()` gives every course an adjacency entry, so the later `&adj[&node]` cannot panic on a course with no dependents. `indegree.entry(dep).or_insert(0)` registers a name that appears only as a

dependency and never as a `Course` of its own. In the example that name is `datastructs`, and without the `or_insert` it would never appear in the in-degree map, never enter the queue, and nothing would start. `or_insert` rather than `insert` is what keeps it from overwriting a count already computed for a course that happens to be listed later.

The completeness test at the end compares against `indegree.len()` rather than `courses.len()`, and the difference is exactly those dependency-only names. The map holds every vertex the graph mentions; the slice holds only the ones somebody declared.

What the rewrite costs is visible in the number of `clone` calls. Every name is cloned into the adjacency map, again into the in-degree map, again into the queue, and a `String` clone is a heap allocation and a copy. A resolver that runs on a large graph interns the names once, into a `HashMap<String, usize>` built at the edge of the system, and then runs the index-based version printed above, where a vertex is a `usize` and a lookup is a bounds check.

Intuition

```
num_nodes = 4, edges = [(0, 1), (0, 2), (1, 3), (2, 3)]

adjacency: 0 -> [1, 2]    1 -> [3]    2 -> [3]    3 -> []
indegree: [0, 1, 1, 2]
```

step	queue	pop	order	in-degree after	queue after
1	[0]	0	[0]	1:0, 2:0	[1, 2]
2	[1, 2]	1	[0, 1]	3:1	[2]
3	[2]	2	[0, 1, 2]	3:0	[3]
4	[3]	3	[0, 1, 2, 3]	-	[]

`order.len() == 4 == num_nodes`, so the result is `Some(order)`.

A cycle: `num_nodes = 2, edges = [(0, 1), (1, 0)]`

```
indegree: [1, 1]
the queue is seeded empty, the loop body never runs, order has length 0,
and 0 != 2, so the result is None.
```

Time and Space Complexity

Re-source	Cost
Time	$O(V + E)$, each vertex enters the queue once and each edge is relaxed once

Re-source	Cost
Space	$O(V + E)$, the adjacency list and the two auxiliary vectors

The string-keyed variant has the same asymptotic cost and a much larger constant. Each of the $O(V + E)$ steps hashes a `String` rather than indexing a vector, and the names are cloned into the maps and the queue, so the allocation count grows with the graph instead of staying at two vectors.

Limitations

The order changes between runs. The initial queue is built by iterating `indegree`, and a `HashMap` iterates in an order that depends on a seed randomised for each process. A graph with a single root, like the one in `main`, hides this, because there is only one vertex to start from. Adding a second independent course makes it visible:

```
["datastructs", "ethics", "algorithms", "os", "compilers"]
["datastructs", "ethics", "algorithms", "os", "compilers"]
["ethics", "datastructs", "algorithms", "os", "compilers"]
["datastructs", "ethics", "algorithms", "os", "compilers"]
["ethics", "datastructs", "algorithms", "os", "compilers"]
```

Every one of those is a correct topological order, and a build system that printed a different plan on each invocation would still be trusted less for it. Collecting the roots into a vector and sorting them before the traversal makes the output reproducible, at a cost of $O(V \log V)$.

A cycle and an empty graph give the same answer. `topo_sort` returns `Vec::new()` when the order is short, and an input with no courses also returns an empty vector, so the caller cannot tell a refused graph from an empty one. The index-based version says it properly with `Option`, and `(order.len() == indegree.len()).then_some(order)` is the same change here.

The unused clone in main. `let coned2 = courses.clone();` duplicates the whole course list and is never read, which the compiler reports as an unused variable. Removing it removes an allocation per course.

An edge naming a vertex outside `0..num_nodes` panics. `graph[from].push(to)` and `indegree[to] += 1` index directly, so `topological_sort(2, &[(5, 0)])` panics with an index-out-of-bounds whose

message names a vector index rather than the offending edge. A caller that reads a graph from a file must validate the vertex numbers first.

None does not say which cycle was found. A caller that wants to report the cycle must run a depth-first search over the unplaced vertices. The information is available inside the algorithm, since the unplaced vertices are exactly the ones the loop never reached, but the return type discards it.

The choice among valid orders is unspecified. For the diamond graph both `[0, 1, 2, 3]` and `[0, 2, 1, 3]` are correct, and which one is produced depends on the order of the edges in the input and on the vertex numbering. The tests assert only the properties that must hold, and a caller must not depend on a particular order.

Duplicate edges are tolerated but not detected. Two copies of `(0, 1)` add two to the in-degree of vertex 1, and the same edge is relaxed twice, so the counts stay consistent and the result is unaffected. Nothing reports the duplication.

References

- Arthur B. Kahn, "Topological sorting of large networks", *Communications of the ACM* 5(11), 1962, pages 558–562.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 20.4, "Topological sort".
- Standard library, `VecDeque`.
- Standard library, `bool::then_some`.

Part III: Recursion and Dynamic Programming

Coin Change

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/dp.rs`. Test it with `cargo test dp`.

I call it my billion-dollar mistake. It was the invention of the null reference in 1965.

C. A. R. Hoare, QCon London, 2009

Problem Statement

Given a list of coin denominations and a target amount, return the smallest number of coins whose values sum exactly to the amount, or report that no combination exists. Each denomination may be used any number of times.

Designing a Solution

A greedy rule does not work. Taking the largest coin that fits fails for `coins = [1, 3, 4]` and `amount = 6`: the greedy choice of 4 leaves 2, which needs two more coins, for a total of three, while `3 + 3` needs two.

The recurrence is over amounts. Let `dp[t]` be the smallest number of coins that sums to `t`. The last coin used is some denomination `c`, and everything before it must sum to `t - c` in the smallest possible number of coins:

```
dp[0] = 0
dp[t] = min over c of ( dp[t - c] + 1 )    for each c <= t
```

Each entry depends only on entries at smaller amounts, so filling the table in ascending order of `t` visits every dependency before it is needed.

```
coins = [1, 2, 5], amount = 11
```

```
t   : 0  1  2  3  4  5  6  7  8  9 10 11
dp  : 0  1  1  2  2  1  2  2  3  3  2  3
```

```
dp[11] was built from dp[11 - 5] + 1 = dp[6] + 1 = 2 + 1 = 3
and 11 = 5 + 5 + 1
```

Implementation

```
///! Coin Change: minimum number of coins to make an amount.
///!
///! Classic dynamic programming. `Option<i32>` expresses "impossible" instead of
///! a magic `-1`.

pub fn coin_change(coins: &[i32], amount: i32) -> Option<i32> {
    if amount < 0 {
        return None;
    }

    let amount = amount as usize;
    let mut dp = vec![usize::MAX; amount + 1];
    dp[0] = 0;

    for total in 1..=amount {
        for &coin in coins {
            let coin = coin as usize;
            if coin > total || dp[total - coin] == usize::MAX {
                continue;
            }
            dp[total] = dp[total].min(dp[total - coin] + 1);
        }
    }

    match dp[amount] {
        usize::MAX => None,
        count => Some(count as i32),
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn computes_minimum_coins() {
        assert_eq!(coin_change(&[1, 2, 5], 11), Some(3)); // 5 + 5 + 1
        assert_eq!(coin_change(&[2], 3), None);
        assert_eq!(coin_change(&[1], 0), Some(0));
    }
}
```

`usize::MAX` is the sentinel for an amount no combination has reached. It is safe as a marker because a real answer can never be that large: any combination uses at most `amount` coins, because every coin is worth at least one.

The guard `if coin > total || dp[total - coin] == usize::MAX` does two things. It skips a coin too large to be the last coin, and it skips a coin whose re-

mainder is itself unreachable. Without the second test the addition below would add one to the sentinel and produce a wrong answer that looks reachable.

`dp[total] = dp[total].min(dp[total - coin] + 1)` keeps the best count found so far for this amount. Because `dp[total]` is read and written in the same statement, the loop order over coins does not matter.

`match dp[amount]` translates `usize::MAX` into `None` and everything else into `Some(count as i32)`, so no caller sees the marker. The answer type is `Option<i32>`, which is the part of the design that matters: "impossible" is not a number.

Intuition

```
coins = [1, 2, 5], amount = 11

total = 1  coin 1: dp[1] = min(MAX, dp[0] + 1) = 1
total = 2  coin 1: dp[2] = min(MAX, dp[1] + 1) = 2
           coin 2: dp[2] = min(2, dp[0] + 1) = 1
total = 3  coin 1: dp[3] = min(MAX, dp[2] + 1) = 2
           coin 2: dp[3] = min(2, dp[1] + 1) = 2
total = 4  coin 1: dp[4] = min(MAX, dp[3] + 1) = 3
           coin 2: dp[4] = min(3, dp[2] + 1) = 2
total = 5  coin 1: dp[5] = min(MAX, dp[4] + 1) = 3
           coin 2: dp[5] = min(3, dp[3] + 1) = 3
           coin 5: dp[5] = min(3, dp[0] + 1) = 1
...
total = 11 coin 1: dp[11] = min(MAX, dp[10] + 1) = 3
           coin 2: dp[11] = min(3, dp[9] + 1) = 3
           coin 5: dp[11] = min(3, dp[6] + 1) = 3
```

```
coins = [2], amount = 3

total = 1  coin 2 is larger than total, skipped;  dp[1] = MAX
total = 2  coin 2: dp[0] = 0, so dp[2] = 1
total = 3  coin 2: dp[1] = MAX, skipped;          dp[3] = MAX

dp[3] is the sentinel, so the result is None.
```

Time and Space Complexity

Resource	Cost	Condition
Time	$O(\text{amount} \times \text{coins.len}())$	every amount considers every denomination
Space	$O(\text{amount})$	one machine word per amount

Limitations

The table is allocated from the amount alone. `coin_change(&[1], i32::MAX)` asks for a vector of `i32::MAX + 1` machine words, which is sixteen gigabytes on a 64-bit target. The allocation failure aborts the process, because the standard library's allocation error handling calls `handle_alloc_error`, which aborts. A version that accepts an amount from a network input would validate it against a limit first, or use `Vec::try_reserve` to turn the failure into a `Result`.

A negative coin is treated as a very large positive one. `coin` as `usize` converts `-1` to `usize::MAX`, and the guard `coin > total` then skips it on every iteration. The function returns an answer that ignores the negative coin, which is the wrong answer if the caller meant the coin to be usable. Nothing reports the conversion.

A zero coin is skipped by the same guard, correctly. `coin = 0` passes the first test only when `total` is zero, and the loop starts at one, so a zero denomination contributes nothing. That is the right behaviour, and it is a consequence of the guard rather than a stated rule.

The function returns the count, not the coins. A caller that wants to know which coins to use needs the predecessor that produced each entry, which means a second table recording decisions. The one-minimum-per-entry design cannot reconstruct the combination.

`i32` bounds both the amount and the answer. An amount above `i32::MAX` cannot be passed, and the conversion `count` as `i32` is safe only because the answer is at most the amount.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Chapter 14, "Dynamic Programming".
- Richard Bellman, *Dynamic Programming*, Princeton University Press, 1957.
- Standard library, `usize::MAX`.
- Standard library, `Vec::try_reserve`.

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Problem Statement

Designing a Solution

```
values = [1, 2, 3]
```

```
graph TD; A["[]"] --- B["[1]"]; A --- C["[2]"]; A --- D["[3]"]; B --- E["[1,2]"]; B --- F["[1,3]"]; C --- G["[2,3]"]; E --- H["[1,2,3]"];
```

record []

record each

record each

record it

8 records: [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]

77

held on entry. Every branch therefore starts from a state that the branches before it did not disturb.

Restricting each recursive call to values at or after `start` is what prevents `[1, 2]` and `[2, 1]` from both appearing. The output is ordered by index, not by value.

Implementation

```

/// Subsets: enumerate every subset of a set of distinct integers.
///
/// Backtracking with clean `path.push(...)` + recursive exploration + `pop`.
/// `Vec<Vec<i32>>` is sorted in the tests to avoid order-dependent assertions.

pub fn subsets(nums: Vec<i32>) -> Vec<Vec<i32>> {
    let mut result = Vec::with_capacity(1 << nums.len());
    let mut path = Vec::new();

    fn backtrack(nums: &[i32], start: usize, path: &mut Vec<i32>, result: &mut
Vec<Vec<i32>>) {
        result.push(path.clone());
        for idx in start..nums.len() {
            path.push(nums[idx]);
            backtrack(nums, idx + 1, path, result);
            path.pop();
        }
    }

    backtrack(&nums, 0, &mut path, &mut result);
    result
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn generates_all_subsets() {
        let mut subsets = subsets(vec![1, 2, 3]);
        subsets.sort_unstable();
        assert_eq!(subsets.len(), 8);
        assert!(subsets.contains(&vec![]));
        assert!(subsets.contains(&vec![1, 2, 3]));
    }

    #[test]
    fn empty_input_has_one_subset() {
        assert_eq!(subsets(vec![]), vec![vec![]]);
    }
}

```

`fn backtrack(...)` is a nested function inside `subsets`. A nested `fn` does not capture its environment, so the values it needs are passed as parameters: `nums`, `start`, `path`, and `result`. That is why the signature is longer than the body.

`result.push(path.clone())` records the current selection. The clone is required because `path` is reused for the rest of the search, and all 2^n recorded subsets are independent vectors.

`backtrack(&nums, idx + 1, path, result)` passes `idx + 1`, not `start + 1`. Each chosen value advances the lower bound past itself, so a value cannot be chosen twice and the indices in a subset always increase.

`path.pop()` after the recursive call restores the invariant. Removing that line makes the selection grow for the remainder of the search, and the output would contain subsets that were never selected.

Intuition

```
subsets(vec![1, 2, 3])

call      path      recorded      loop
backtrack(nums, 0)  []          []          idx 0, 1, 2
  push nums[0] = 1  [1]          [1]          idx 1, 2
    backtrack(nums, 1) [1]          [1]          idx 2
      push nums[1] = 2 [1, 2]        [1, 2]        idx 2
        backtrack(nums, 2) [1, 2]        [1, 2]        loop does not run
          push nums[2] = 3 [1, 2, 3]      [1, 2, 3]
            backtrack(nums, 3) [1, 2, 3] [1, 2, 3]
              pop          [1, 2]
            pop          [1]
          push nums[2] = 3 [1, 3]          [1, 3]          loop does not run
        backtrack(nums, 3) [1, 3]          [1, 3]
          pop          [1]
        pop          []
      push nums[1] = 2    [2]
      backtrack(nums, 2)  [2]          [2]          idx 2
        push nums[2] = 3 [2, 3]        [2, 3]
        backtrack(nums, 3) [2, 3]        [2, 3]
        pop          [2]
      pop          []
    push nums[2] = 3    [3]
    backtrack(nums, 3)  [3]          [3]
    pop          []
```

recorded, in order: [], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]

Time and Space Complexity

Re-source	Cost	Condition
Time	$O(2^n \times n)$	one clone of the selection per recorded subset, of length up to n
Space	$O(2^n \times n)$	the output dominates; the stack is $O(n)$

The clone is the reason for the factor of n . A caller that only needs to visit each subset can pass a closure instead of collecting them, which removes the factor.

Limitations

The capacity hint overflows for large inputs. `Vec::with_capacity(1 << nums.len())` evaluates `1 << nums.len()` as a `usize`. For a list of 64 or more values on a 64-bit target the shift is not representable and panics in a debug build. The panic happens on an input the function is otherwise able to accept: the recursion itself has no such limit, and the allocation would fail long before the work finished.

A list of 40 values cannot be enumerated at all. 2^{40} subsets is about a trillion vectors, and the output alone would exceed any machine's memory. The function has no bound on the input size and no way to report that the work is impossible.

Duplicate values produce duplicate-looking subsets. The function takes a list and treats every position as distinct, so `subsets(vec![1, 1, 1])` returns eight subsets. The documentation says "distinct integers", which is a precondition the type does not enforce.

The output order is an implementation detail, and one test depends on it. `empty_input_has_one_subset` compares the whole output, which is safe for one element. The other test sorts before comparing.

There is no way to stop the search early or to stream the results. A caller that wants the first k subsets, or that is searching for a subset with a property, must collect all of them first.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 5.3, on recurrences, and Chapter 14 on the same recursive-search structure.
- Standard library, `Vec::clone`.
- Standard library, `Vec::with_capacity`.

Part IV: Linked Structures

Reverse a Linked List

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/linked_list.rs`. Test it with `cargo test linked_list`.

Problem Statement

Reverse a singly linked list in place and return its new head. In place means no extra node allocations: the nodes are re-linked, not copied.

Designing a Solution

```
head: Option<Box<ListNode>>

  head
  |
  v
+-----+ +-----+ +-----+
| 1 next | --> | 2 next | --> | 3 next | --> None
+-----+ +-----+ +-----+
  heap      heap      heap
```

Three properties follow from `Option<Box<ListNode>>`.

`None` is the end of the list, so an empty list is the value `None` rather than a node with a flag, and there is no sentinel to keep consistent.

The list owns its nodes. Handing over the head hands over the whole list, so there is no way to hold a list whose nodes have been freed and no separate deallocation call.

A link is one pointer wide. The compiler stores `None` in the null pointer, which is never a valid address for a `Box`, so no discriminant byte is added.

Reversal is one loop over three bindings. In a language with unrestricted pointers it is three assignments per node; in Rust the same algorithm is written by taking ownership of one node at a time, which is what makes it checkable.

```

initial    previous = None                current = Some(1 -> 2 -> 3 -> None)

step 1     current = node.next.take()      current  = Some(2 -> 3 -> None)
           node.next = previous            node     = Some(1 -> None)
           previous = Some(node)            previous = Some(1 -> None)

step 2     current = Some(3 -> None)        previous = Some(2 -> 1 -> None)
step 3     current = None                  previous = Some(3 -> 2 -> 1 -> None)

the loop ends because current is None, and the answer is previous

```

`Option::take` is the operation that makes this work. It moves the successor out of `node.next` and leaves `None` behind, so the node under the loop's cursor stands alone while the rest of the list is held in a separate binding. Without `take`, the assignment `node.next = previous` would have to write through a reference that `current` still holds, which the borrow checker rejects.

Implementation

```

/// Reverse a singly linked list.
///
/// Uses `Option<Box<ListNode>>` and a `while let` loop to move pointers in a
/// borrow-safe way.

#[derive(Debug, PartialEq, Eq)]
pub struct ListNode {
    pub val: i32,
    pub next: Option<Box<ListNode>>,
}

impl ListNode {
    pub fn from_slice(values: &[i32]) -> Option<Box<ListNode>> {
        let mut head = None;
        for &value in values.iter().rev() {
            head = Some(Box::new(ListNode {
                val: value,
                next: head,
            }));
        }
        head
    }

    pub fn to_vec(head: Option<Box<ListNode>>) -> Vec<i32> {
        let mut result = Vec::new();
        let mut current = head;
        while let Some(node) = current {
            result.push(node.val);
            current = node.next;
        }
        result
    }
}

```

```

pub fn reverse_list(head: Option<Box<ListNode>>) -> Option<Box<ListNode>> {
    let mut previous = None;
    let mut current = head;

    while let Some(mut node) = current {
        current = node.next.take();
        node.next = previous;
        previous = Some(node);
    }

    previous
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn reverses_list() {
        let head = ListNode::from_slice(&[1, 2, 3, 4]);
        assert_eq!(ListNode::to_vec(reverse_list(head)), vec![4, 3, 2, 1]);
    }

    #[test]
    fn handles_empty_and_single() {
        assert_eq!(reverse_list(None), None);
        assert_eq!(
            ListNode::to_vec(reverse_list(ListNode::from_slice(&[42]))),
            vec![42]
        );
    }
}

```

`from_slice` builds the list backwards, from the last value to the first, so each new node takes the list built so far as its successor. Building forwards would need a mutable reference to the tail.

`to_vec` takes the list by value, which lets each node be dropped as the traversal proceeds: the value is copied out and the successor moves into `current`, so the node that was just read is freed at the end of the iteration. A version that took `&Option<Box<ListNode>>` would leave the list alive and would need the caller to drop it afterwards.

`while let Some(mut node) = current` binds the node by value, and the `mut` is required because the body assigns to `node.next`. The three statements in the body are the algorithm: detach the successor, point backwards, advance `previous`.

Intuition

input: 1 -> 2 -> 3 -> None

iteration	node	current after take	node.next after assignment	previous
start	-	Some(1 -> 2 -> 3)	-	None
1	1	Some(2 -> 3)	None	Some(1)

2	2	Some(3)	Some(1)	Some(2 -> 1)
3	3	None	Some(2 -> 1)	Some(3 -> 2 -> 1)

current is None, so the loop ends and previous is returned.

Time and Space Complexity

Operation	Time	Space
reverse_list	$O(n)$	$O(1)$; no allocation, only pointer writes
from_slice	$O(n)$	$O(n)$, one allocation per node
to_vec	$O(n)$	$O(n)$ for the result, and it frees the nodes

Limitations

A long list is dropped recursively. The compiler's derived destructor for `Option<Box<ListNode>>` follows the `next` links, one stack frame per node. A list of a few hundred thousand nodes overflows the thread stack when it is dropped, and a stack overflow aborts the process rather than unwinding. This affects every caller, including one that only reverses a list built elsewhere, because the destructor runs when the last owner goes out of scope.

to_vec consumes the list. A caller that wants both the reversed list and its values has to clone one of them, and `ListNode` does not implement `Clone`.

from_slice allocates one box per element. For a long slice that is a long chain of small allocations, each with its own allocator overhead. An array-based representation holds the same data in one allocation, at the cost of not being a linked list.

The value type is fixed at i32. Nothing in the algorithm depends on the value type, and the declaration is not generic.

References

- The Rust Book, [Enabling recursive types with boxes](#).
- Standard library, `Option::take`.
- Standard library, `Box`.

Singly Linked List

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/singly_linked_list.rs`. Run it with `cargo run --bin singly_linked_list`.

Problem Statement

Build a list whose node set is described by an algebraic data type, with constant time insertion and removal at the front.

Designing a Solution

```
List<T> = Empty
      | Node { value: T, next: Box<List<T>> }
```

The layout is the same as `Option<Box<ListNode>>` from Chapter 14, with the empty case named in the type rather than expressed by `None`. The end of the list is `List::Empty`, and an empty list is `List::Empty` as well, so there is no separate sentinel and no nullable field.

```
list = Node { value: 0, next: Box<Node { value: 1, next: Box<Empty> }> }
```

```
list
|
v
+-----+ +-----+ +-----+
| value: 0 | --> | value: 1 | --> | Empty |
| next: Box --- | | next: Box --- | +-----+
+-----+ +-----+
```

`push_front` and `pop_front` both take the whole list out of `self` with `std::mem::replace`, decide what to do with it, and write the result back. That is

what lets the code move a list value without cloning it and without holding a borrow of `self` across the move.

```
push_front(0) on [1, 2]

before: self = Node { 1, next: [2] }

replace(self, Empty) returns the old list [1, 2] and leaves self = Empty
then:    self = Node { 0, next: Box::new([1, 2]) }

pop_front() on [0, 1, 2]

replace(self, Empty) returns [0, 1, 2] and leaves self = Empty
the node is destructured into value = 0 and next = Box([1, 2])
then:    self = *next = [1, 2]
returns: Some(0)
```

Implementation

```
// Singly linked list using an algebraic data type.
//
//   List<T> = Empty
//           | Node { value: T, next: Link<T> }
//
// `Empty` is the end of the list.

type Link<T> = Box<List<T>>;

pub enum List<T> {
    Empty,
    Node { value: T, next: Link<T> },
}

impl<T> List<T> {
    pub fn new() -> Self {
        List::Empty
    }

    pub fn is_empty(&self) -> bool {
        matches!(self, List::Empty)
    }

    pub fn len(&self) -> usize {
        match self {
            List::Empty => 0,
            List::Node { next, .. } => 1 + next.len(),
        }
    }

    /// O(1)
    pub fn push_front(&mut self, value: T) {
        let old = std::mem::replace(self, List::Empty);
        *self = List::Node {
            value,
            next: Link::new(old),
        };
    }

    /// O(1)
    pub fn pop_front(&mut self) -> Option<T> {
        match std::mem::replace(self, List::Empty) {
```

```

        List::Empty => None,
        List::Node { value, next } => {
            *self = *next;
            Some(value)
        }
    }
}

fn main() {
    let mut list = List::new();
    list.push_front(2);
    list.push_front(1);
    list.push_front(0);

    assert_eq!(list.len(), 3);
    assert_eq!(list.pop_front(), Some(0));
    assert_eq!(list.pop_front(), Some(1));
    assert_eq!(list.pop_front(), Some(2));
    println!("ok");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn push_pop_works() {
        let mut list = List::new();
        list.push_front(1);
        list.push_front(2);
        assert_eq!(list.pop_front(), Some(2));
        assert_eq!(list.pop_front(), Some(1));
        assert_eq!(list.pop_front(), None);
    }

    #[test]
    fn len_works() {
        let mut list = List::new();
        assert_eq!(list.len(), 0);
        list.push_front(1);
        list.push_front(2);
        assert_eq!(list.len(), 2);
    }
}

```

`push_front` moves the old list into a new `Box`. The old value comes from `std::mem::replace(self, List::Empty)`, which leaves `self` in a valid state that the following assignment overwrites. There is no moment at which `self` is uninitialised, and the code needs neither `unsafe` nor `Option`.

`pop_front` deconstructs the taken value. `*self = *next` moves the inner list out of its box and into `self`; the box itself is freed, so the popped node is released rather than remembered.

`Link::new(old)` is `Box::new(old)` written through the type alias, which reads as "a new link holding the old list".

The two functions carry `/// 0(1)` doc comments and nothing else. They are the only documentation in the file beyond the module header and the type sketch.

Intuition

```

push_front(2)  self = Node { 2, next: Empty }
push_front(1)  self = Node { 1, next: Node { 2, next: Empty } }
push_front(0)  self = Node { 0, next: Node { 1, next: Node { 2, next: Empty } } }

len()          three recursive calls plus the empty base case, returning 3

pop_front()    replace returns the node holding 0
               value = 0, next = Box(Node { 1, ... })
               self = Node { 1, next: Node { 2, next: Empty } }
               returns Some(0)

pop_front()    returns Some(1), self = Node { 2, next: Empty }
pop_front()    returns Some(2), self = Empty
pop_front()    replace returns Empty, so the match yields None

```

Time and Space Complexity

Operation	Time	Space
push_front	0(1)	one allocation
pop_front	0(1)	freed one allocation
is_empty	0(1)	none
len	0(n)	0(n) stack frames

Limitations

List::new has no Default implementation. Clippy's `new_without_default` lint fires on this pattern, and the reason it exists is that a caller writing a generic function over many types expects `Default` to work. An `impl<T> Default for List<T>` with a two-line body fixes it.

len is recursive. It walks the list with one stack frame per node, so a list long enough to matter cannot be measured. An iterative version keeps a `&List<T>` cursor and counts in a `while let` loop.

Dropping the list is recursive. The derived destructor for `List<T>` follows the `next` links, one frame per node, and a long list aborts the process when it goes out of scope. This is the hazard described in Chapter 14, and

the same fix applies: a manual `Drop` implementation that unlinks nodes into a loop.

`pop_front` is $O(1)$ at any length, but the drop that follows is not. For a list of a million nodes, the operation the API advertises is cheap and the work the program does when the list is released is $O(n)$. The two have to be considered together.

There is no `Clone`, no `Debug`, and no iterator. A caller cannot print the list, cannot copy it, and cannot write `for value in &list`.

References

- The Rust Book, [Defining an enum](#).
- Standard library, `std::mem::replace`.
- Clippy, `new_without_default`.

Doubly Linked List

In this chapter

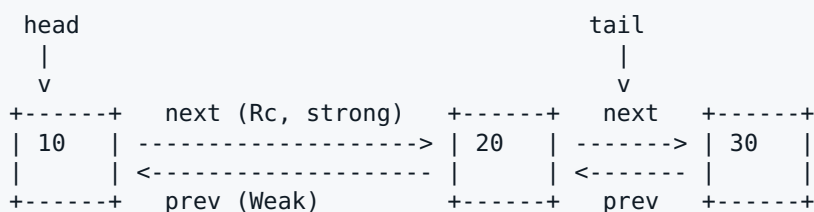
- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/ll.rs`. Run it with `cargo run --bin ll`.

Problem Statement

Build a list with constant time insertion and removal at both ends. A singly linked list cannot remove from the back in constant time, because reaching the penultimate node requires a walk. Something has to point backwards, and that something cannot own what it points at.

Designing a Solution



If both directions owned their target, the reference count of a two-node list would never reach zero and the nodes would leak: each node would be kept alive by the other. The resolution is to let the forward links own and the backward links observe:

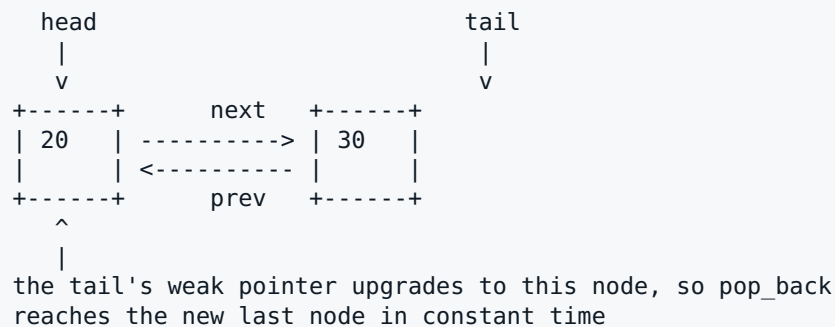
Link	Type	Keeps the node alive	Fails how
<code>next</code>	<code>Rc<RefCell<Node<T>>></code>	yes	the list outlives the value
<code>prev</code>	<code>Weak<RefCell<Node<T>>></code>	no	<code>upgrade()</code> returns <code>None</code>

A `Weak` pointer observes a value without keeping it alive. When the last strong reference goes away, the node is freed and every weak pointer to it starts returning `None` from `upgrade`.

The `RefCell` is the second half of the design. `Rc` alone gives shared ownership but no mutation; `RefCell` adds mutation with the borrow checked at run time rather than at compile time.

after `pop_front` on the list above

(node 10 is freed: nothing strong refers to it)



Implementation

```
// =====
// Type aliases for clarity.
// =====
use std::cell::{Ref, RefCell}; // Ref is needed for peek return types
use std::rc::{Rc, Weak};

type NodeRef<T> = Rc<RefCell<Node<T>>>;
type WeakNodeRef<T> = Weak<RefCell<Node<T>>>;
type OptNodeRef<T> = Option<NodeRef<T>>;
type OptWeakNodeRef<T> = Option<WeakNodeRef<T>>;

// =====
// Node and LinkedList definitions.
// =====
struct Node<T> {
    data: T,
    next: OptNodeRef<T>,
    prev: OptWeakNodeRef<T>,
}

pub struct LinkedList<T> {
    head: OptNodeRef<T>,
    tail: OptNodeRef<T>,
    len: usize,
}

impl<T> LinkedList<T> {
    /// Creates an empty list.
    pub fn new() -> Self {
        LinkedList {
            head: None,
            tail: None,
        }
    }
}
```

```

        len: 0,
    }
}

pub fn len(&self) -> usize {
    self.len
}

pub fn is_empty(&self) -> bool {
    self.len == 0
}

// -----
// push_front
// -----
pub fn push_front(&mut self, data: T) {
    let new_node = Rc::new(RefCell::new(Node {
        data,
        next: None,
        prev: None,
    }));

    // take() moves the old head out of the Option, leaving None.
    match self.head.take() {
        Some(old_head) => {
            // Link old head back to new node (weakly).
            old_head.borrow_mut().prev = Some(Rc::downgrade(&new_node));
            // Link new node forward to old head.
            new_node.borrow_mut().next = Some(old_head);
        }
        None => {
            // List was empty: new node is also the tail.
            self.tail = Some(new_node.clone());
        }
    }
    self.head = Some(new_node);
    self.len += 1;
}

// -----
// push_back
// -----
pub fn push_back(&mut self, data: T) {
    let new_node = Rc::new(RefCell::new(Node {
        data,
        next: None,
        // self.tail.as_ref() gives Option<&NodeRef<T>>,
        // map(Rc::downgrade) turns it into Option<WeakNodeRef<T>>.
        prev: self.tail.as_ref().map(Rc::downgrade),
    }));

    match self.tail.take() {
        Some(old_tail) => {
            old_tail.borrow_mut().next = Some(new_node.clone());
        }
        None => {
            self.head = Some(new_node.clone());
        }
    }
    self.tail = Some(new_node);
    self.len += 1;
}

// -----
// pop_front
// -----
pub fn pop_front(&mut self) -> Option<T> {
    self.head.take().and_then(|old_head| {

```

```

    // Take ownership of the next node (if any).
    let next = old_head.borrow_mut().next.take();

    match next {
        Some(next_node) => {
            // Disconnect the new head's prev pointer.
            next_node.borrow_mut().prev = None;
            self.head = Some(next_node);
        }
        None => {
            self.tail = None; // list becomes empty
        }
    }

    self.len -= 1;

    // Safely extract the data. At this point, old_head is the only
    // strong reference (because we removed it from the list and the
    // new head's prev doesn't point to it).
    let node = Rc::try_unwrap(old_head).ok().unwrap().into_inner();
    Some(node.data)
})
}

// -----
// pop_back
// -----
pub fn pop_back(&mut self) -> Option<T> {
    self.tail.take().and_then(|old_tail| {
        // Get the previous node (upgrade the weak reference).
        let prev = old_tail.borrow_mut().prev.take().and_then(|w| w.upgrade());

        match prev {
            Some(prev_node) => {
                prev_node.borrow_mut().next = None;
                self.tail = Some(prev_node);
            }
            None => {
                self.head = None; // list becomes empty
            }
        }
    })

    self.len -= 1;

    let node = Rc::try_unwrap(old_tail).ok().unwrap().into_inner();
    Some(node.data)
})
}

// -----
// peek_front: returns a guarded reference to the front data.
// The returned Ref<'_, T> holds the borrow of the RefCell until it goes
// out of scope, ensuring the reference stays valid.
// -----
pub fn peek_front(&self) -> Option<Ref<'_, T>> {
    // self.head.as_ref() -> Option<&NodeRef<T>>
    self.head.as_ref().map(|node| {
        // node.borrow() returns Ref<'_, Node<T>>.
        // Ref::map projects that to a Ref<'_, T> pointing to the data field.
        Ref::map(node.borrow(), |n| &n.data)
    })
}

// -----
// peek_back: similar to peek_front.
// -----
pub fn peek_back(&self) -> Option<Ref<'_, T>> {

```

```

        self.tail
            .as_ref()
            .map(|node| Ref::map(node.borrow(), |n| &n.data))
    }
}

// =====
// Demonstration with main().
// =====
fn main() {
    let mut list = LinkedList::new();

    list.push_front(10);
    list.push_front(20);
    list.push_back(30);
    list.push_back(40);

    println!("Length: {}", list.len());

    // Peek returns a Ref; we can dereference it with *.
    if let Some(front) = list.peek_front() {
        println!("Front: {}", *front);
    }
    if let Some(back) = list.peek_back() {
        println!("Back: {}", *back);
    }

    println!("Pop front: {:?}", list.pop_front()); // 20
    println!("Pop back:  {:?}", list.pop_back()); // 40

    // Drain the list.
    while let Some(val) = list.pop_front() {
        println!("Popped: {}", val);
    }
    println!("Empty? {}", list.is_empty());
}

// =====
// Tests (cargo test --bin ll)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn new_list_is_empty() {
        let list: LinkedList<i32> = LinkedList::new();
        assert!(list.is_empty());
        assert_eq!(list.len(), 0);
        assert!(list.peek_front().is_none());
        assert!(list.peek_back().is_none());
    }

    #[test]
    fn push_front_back_and_len() {
        let mut list = LinkedList::new();
        list.push_back(1);
        list.push_back(2);
        list.push_front(0);

        assert_eq!(list.len(), 3);
        assert_eq!(*list.peek_front().unwrap(), 0);
        assert_eq!(*list.peek_back().unwrap(), 2);
    }

    #[test]
    fn pop_front_and_back_work_in_order() {

```

```

    let mut list = LinkedList::new();
    list.push_back(10);
    list.push_back(20);
    list.push_back(30);

    assert_eq!(list.pop_front(), Some(10));
    assert_eq!(list.pop_back(), Some(30));
    assert_eq!(list.pop_front(), Some(20));
    assert_eq!(list.pop_front(), None);
    assert_eq!(list.pop_back(), None);
    assert!(list.is_empty());
}

#[test]
fn popping_last_item_emptyies_the_list() {
    let mut list = LinkedList::new();
    list.push_back(42);
    assert_eq!(list.peek_front().map(|x| *x), Some(42));
    assert_eq!(list.pop_back(), Some(42));
    assert!(list.is_empty());
}

#[test]
fn alternating_push_pop_keeps_invariants() {
    let mut list = LinkedList::new();
    for i in 0..100 {
        list.push_back(i);
        list.push_front(-i);
    }
    assert_eq!(list.len(), 200);
    for i in (0..100).rev() {
        assert_eq!(list.pop_front(), Some(-i));
        assert_eq!(list.pop_back(), Some(i));
    }
    assert!(list.is_empty());
}
}

// =====
// Explanation of key methods used:
//
// - Rc::new / RefCell::new           : create a reference-counted, mutable cell.
// - borrow() / borrow_mut()         : borrow the RefCell's contents (immutably/mutably).
// - Rc::downgrade()                 : turn an Rc into a Weak (no ownership).
// - Weak::upgrade()                 : attempt to get an Rc from a Weak (returns Option).
// - Option::take()                   : take the value out of an Option, leaving None.
// - Option::as_ref()                 : convert Option<T> to Option<&T> without moving.
// - Option::map()                     : apply a function to the inner value if Some.
// - Option::and_then()               : like map but returns an Option (for chaining).
// - Rc::try_unwrap()                 : move the value out of the Rc if it's the only
strong ref.
// - Ref::map()                       : project a Ref to a subfield, keeping the borrow
alive.
// =====

```

Four details carry the implementation.

`Rc::downgrade(&new_node)` is stored in the *old* head's `prev`, not the new node's. The new node already owns the old head through `next`, and a second strong reference would keep the old head alive after the list forgot it.

`self.tail.as_ref().map(Rc::downgrade)` in `push_back` converts `Option<&NodeRef<T>>` into `Option<WeakNodeRef<T>>` without a `match`. When the list is empty the tail is `None` and the new node's `prev` is `None`.

`old_tail.borrow_mut().prev.take().and_then(|w| w.upgrade())` in `pop_back` does three things in one expression: takes the weak pointer out of the node, leaving `None` behind; attempts to upgrade it to a strong reference; and yields `None` if the node has been freed. The `take` is what allows the removed node to be freed afterwards.

`Rc::try_unwrap(old_head).ok().unwrap().into_inner()` moves the node out of the `Rc` and the data out of the `RefCell`. It succeeds because the node has been unlinked and no other strong reference exists. `into_inner` on a `RefCell` consumes the cell, so it cannot fail and needs no borrow check.

Intuition

call	head	tail	len	effect
<code>new()</code>	-	-	0	
<code>push_front(10)</code>	10	10	1	the first node becomes both ends
<code>push_front(20)</code>	20	10	2	10.prev is a weak pointer to 20
<code>push_back(30)</code>	20	30	3	10.next is 30; 30.prev is weak to 10
<code>push_back(40)</code>	20	40	4	30.next is 40; 40.prev is weak to 30
<code>peek_front()</code>	the guard dereferences to 20			
<code>peek_back()</code>	the guard dereferences to 40			
<code>pop_front()</code>	20	40	3	returns <code>Some(20)</code> 10's predecessor link is cleared, so nothing strong refers to 20 and it is freed by the <code>unwrap</code>
<code>pop_back()</code>	10	30	2	returns <code>Some(40)</code> 30.next is cleared

Time and Space Complexity

Operation	Time	Space
<code>push_front</code> , <code>push_back</code>	$O(1)$	one allocation, plus one reference-count update
<code>pop_front</code> , <code>pop_back</code>	$O(1)$	frees one allocation
<code>peek_front</code> , <code>peek_back</code>	$O(1)$	none
<code>len</code> , <code>is_empty</code>	$O(1)$	the length is stored

Limitations

`Rc::try_unwrap(...).ok().unwrap()` is a panic path that the type system does not close. It succeeds while the invariants hold: the removed node has been unlinked and the new end's link to it has been cleared, so the list holds the only strong reference. Nothing in the signature says so, and a future change to `push_back` that keeps an extra strong reference would turn the `unwrap` into a run-time panic rather than a compile error. An `.expect("...")` with the reason would document the assumption; a `match` that recovers the node from the `Err` arm would make the code total.

The list is neither `Send` nor `Sync`. `Rc` and `RefCell` are both single-threaded, so a `LinkedList` cannot be moved to another thread or shared between threads. The compiler rejects both, which puts the limitation at the call site. A concurrent list uses `Arc<Mutex<Node<T>>>` and pays for it with lock contention and the possibility of poison.

`RefCell` borrows are checked at run time. `borrow_mut()` panics if a borrow is already active. Holding the guard that `peek_front` returns while calling `pop_front` on the same list is a run-time panic, where the `Box`-based list of Chapter 14 would have made it a compile error.

Dropping a long list is recursive. Each node holds a strong reference to the next, so releasing the head releases the next node, which releases the next, one stack frame per node. The `RefCell` does not change that; it adds a borrow flag to each node, two more machine words per element.

The demonstration's comment about `pop_front` returning 20 is the only place the expected value is written. `main` prints values and asserts nothing, and the tests cover the same behaviour.

References

- The Rust Book, `Rc<T>`, [the reference-counted smart pointer](#).
- The Rust Book, `RefCell<T>` [and the interior mutability pattern](#).
- Standard library, `Rc::downgrade`.
- Standard library, `Weak::upgrade`.
- Standard library, `Ref::map`.

Part V: Caches

LRU Cache

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/lru_cache.rs`. Test it with `cargo test lru_cache::`.

Problem Statement

A cache with a fixed capacity that evicts the least recently used entry. `get` and `put` must be in expected $O(1)$, and a `get` must count as a use.

Designing a Solution

The recency order is kept as a log of events. Every access appends `(key, generation)` to a queue, where the generation counter increases on every access. A second map holds the generation of each key's most recent access.

access order: a, b, a, c generation counter after each step: 1, 2, 3, 4

event queue	generations map	values map
+-----+	+-----+	+-----+
(a, 1) front	a -> 3	a -> v
(b, 2)	b -> 2	b -> v
(a, 3)	c -> 4	c -> v
(c, 4) back	+-----+	+-----+
+-----+		

Which entry is least recently used? Scan the queue from the front:

- (a, 1) stale: the map says a's generation is 3, not 1
- (b, 2) current: the map says b's generation is 2, so b is the candidate

The first event whose generation matches its key's entry belongs to the least recently used key. The queue is ordered by generation, and every live key's most recent event appears exactly once, so the earliest match is the minimum.

An event whose generation does not match records an earlier access to a key that has been used since. It carries no information about the current recency order and is discarded.

Implementation

```

/// LRU cache that is actually writable in an interview.
///
/// Design: `HashMap<K, (V, generation)>` + `VecDeque<(K, generation)>`.
///
/// Each access pushes a new `(key, generation)` event to the back. On eviction
/// we pop old events until we find the newest generation for a key, then remove
/// it. Stale events are skipped. This keeps amortized O(1) operations while
/// preserving true LRU semantics.

use std::collections::{HashMap, VecDeque};
use std::hash::Hash;

pub struct LruCache<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    capacity: usize,
    map: HashMap<K, (V, u64)>,
    recency: VecDeque<(K, u64)>,
    generation: u64,
}

impl<K, V> LruCache<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "LRU cache capacity must be > 0");
        Self {
            capacity,
            map: HashMap::new(),
            recency: VecDeque::new(),
            generation: 0,
        }
    }

    pub fn len(&self) -> usize {
        self.map.len()
    }

    pub fn is_empty(&self) -> bool {
        self.map.is_empty()
    }

    pub fn get(&mut self, key: &K) -> Option<V> {
        let (value, _) = self.map.get(key)?.clone();
        self.generation += 1;
        self.recency.push_back((key.clone(), self.generation));
        self.map
            .insert(key.clone(), (value.clone(), self.generation));
        Some(value)
    }

    pub fn put(&mut self, key: K, value: V) {

```

```

        self.generation += 1;
        let is_new = self
            .map
            .insert(key.clone(), (value, self.generation))
            .is_none();
        self.recency.push_back((key, self.generation));

        if is_new {
            self.evict_if_needed();
        }
    }

    fn evict_if_needed(&mut self) {
        while self.map.len() > self.capacity {
            let Some((key, generation)) = self.recency.pop_front() else {
                break;
            };

            match self.map.get(&key) {
                Some( (_, current)) if *current == generation => {
                    self.map.remove(&key);
                    break;
                }
                _ => {}
            }
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn put_get_and_len() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);

        assert_eq!(cache.len(), 2);
        assert_eq!(cache.get(&"a".to_string()), Some(1));
        assert_eq!(cache.get(&"missing".to_string()), None);
    }

    #[test]
    fn evicts_least_recently_used() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);
        cache.put("c".to_string(), 3);

        assert_eq!(cache.get(&"a".to_string()), None);
        assert_eq!(cache.get(&"b".to_string()), Some(2));
        assert_eq!(cache.get(&"c".to_string()), Some(3));
    }

    #[test]
    fn get_updates_recency() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);

        // Access "a" so it becomes most recently used.
        assert_eq!(cache.get(&"a".to_string()), Some(1));
    }
}

```

```

        cache.put("c".to_string(), 3); // evicts "b"
        assert_eq!(cache.get(&"b".to_string()), None);
        assert_eq!(cache.get(&"a".to_string()), Some(1));
        assert_eq!(cache.get(&"c".to_string()), Some(3));
    }

    #[test]
    fn updating_existing_key_moves_it_to_mru() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);

        // Update "a"; it becomes MRU, so "b" should be evicted next.
        cache.put("a".to_string(), 10);
        cache.put("c".to_string(), 3);

        assert_eq!(cache.get(&"a".to_string()), Some(10));
        assert_eq!(cache.get(&"b".to_string()), None);
        assert_eq!(cache.get(&"c".to_string()), Some(3));
    }
}

```

`get` clones the value out of the map, increments the generation, appends an event, and writes the entry back with the new generation. The write-back is what keeps the map in agreement with the log; without it, the event just appended would look stale.

`put` inserts first and evicts only when the key was new. An update cannot take the cache over its capacity, so the eviction loop would be wasted work in that case.

`evict_if_needed` pops events until one matches. The `break` after a successful removal ends the loop. `put` adds at most one entry, so the loop removes at most one.

Intuition

capacity 2

call	generation	map after	queue after
<code>new()</code>	0	{}	[]
<code>put("a", 1)</code>	1	{a: (1, 1)}	[(a,1)]
<code>put("b", 2)</code>	2	{a: (1,1), b: (2,2)}	[(a,1), (b,2)]
<code>get(&"a")</code>	3	{a: (1,3), b: (2,2)}	[(a,1), (b,2), (a,3)]
<code>put("c", 3)</code>	4	{a: (1,3), b: (2,2), c: (3,4)}	[(a,1), (b,2), (a,3), (c,4)]

eviction, because the map holds 3 entries and the capacity is 2:

- pop (a, 1) the map says a's generation is 3, so this event is stale
- pop (b, 2) the map says b's generation is 2, so b is the least recently used
remove b, and the loop ends

result: {a: (1,3), c: (3,4)}

Time and Space Complexity

Re-source	Cost	Condition
get	expected $O(1)$, plus one clone of the value	the hash function distributes keys evenly
put	expected $O(1)$ amortised	the eviction scan pops events, each of which is removed once
Space	$O(\text{entries})$ for the map, and $O(\text{accesses since the last eviction})$ for the queue	see below

Limitations

The event queue is not bounded by the capacity. Every `get` and every `put` appends one entry to `recency`, and entries are removed only during eviction. A cache that is read far more often than it is written grows its queue without bound: with a capacity of two and a million reads of one key, the queue holds a million events, and nothing in the API releases them. The queue is proportional to the number of operations rather than to the number of entries, so a long-lived read-heavy cache will exhaust memory. Chapter 18 shows a design whose storage is bounded by the capacity.

get clones the value on every hit. The signature returns `Option<V>` rather than `Option<&V>`, so a cached value is copied out on every access. For a small integer that is free; for a cached response body it is a `memcpy` per hit, and it also forces the `V: Clone` bound on the type.

capacity == 0 panics. `new` asserts `capacity > 0`, so a cache cannot be disabled by configuration. A caller that receives a capacity from a file must validate it or accept the panic.

The break in the eviction loop is correct only because put adds at most one entry. If the function were reused where several entries could be over capacity, it would leave some of them in place.

A generation counter of u64 wraps after 2⁶⁴ accesses. The wrap is unreachable in practice, about 584 years at a billion accesses per second,

and its consequence would be a misidentified stale event rather than memory unsafety.

References

- Standard library, `VecDeque`.
- Standard library, `HashMap`.
- Standard library, `u64`.

LRU Cache, Array-Backed

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/lru_cache_easy.rs`. Test it with `cargo test lru_cache_easy`.

Problem Statement

The same problem as the previous chapter, solved a second way: a fixed-capacity cache that evicts the least recently used entry, with `get` and `put` in constant time.

Designing a Solution

Keep the entries in a `Vec` and use the index as the pointer. The map goes from key to index, and the arena is stitched into a doubly linked list by two fields on each node, `prev` and `next`, with `head` at the most recently used end and `tail` at the least recently used one.

```
lookup_table: "A" -> 1, "C" -> 0
nodes:  index 0          index 1
        +-----+      +-----+
        | key "C"  |      | key "A"  |
        | value 30  |      | value 10  |
        | prev: None|      | prev: Some(0)|
        | next: Some(1)|    | next: None  |
        +-----+      +-----+
          ^              ^
        head            tail
    most recently used  least recently used
```

An index needs no reference count and no borrow flag, so a node costs two integers more than the entry itself and nothing else. The arena is dense and in-

dex-stable: when the cache is full, the slot at the tail is overwritten rather than freed, so no node is deallocated while the cache lives.

One method moves a node to the front, and it handles both cases in a single pass. A node that is already linked is spliced out of its current position first; a brand new node has no neighbours and is spliced in directly. That is what `touch` does.

Implementation

```
/// A fixed-capacity LRU (Least Recently Used) cache.
///
/// ## Design
///
/// Two structures cooperate to give  $O(1)$  `get` and `put`:
///
/// * `map: HashMap<K, usize>` – key → index into the arena.
/// * `nodes: Vec<Node<K, V>>` – an arena of nodes; indices play the role
///   of pointers, sidestepping `Rc<RefCell<...>>` and its borrow-checker pain.
///
/// The arena is stitched into a doubly linked list through `head`/`tail`:
///
/// ```text
/// MRU → head → ... → tail → LRU
/// ```
///
/// On eviction we reuse the tail slot instead of removing it, so no node
/// is ever deallocated mid-life. That keeps the arena dense and index-stable.

use std::collections::HashMap;
use std::hash::Hash;

/// A single cache entry, linked into the recency list by index.
struct Node<K, V> {
    /// Retained so we can delete the corresponding map entry when evicting.
    key: K,
    value: V,
    /// Neighbour closer to the MRU end (or `None` if this is `head`).
    prev: Option<usize>,
    /// Neighbour closer to the LRU end (or `None` if this is `tail`).
    next: Option<usize>,
}

/// Fixed-capacity LRU cache.
pub struct LruCache<K, V> {
    /// Maximum number of entries; `>= 1` (enforced by `new`).
    cap: usize,
    /// Key → arena index. The single source of truth for "is this key cached?".
    lookup_table: HashMap<K, usize>,
    /// Arena of nodes; indices are the "pointers" of the linked list.
    nodes: Vec<Node<K, V>>,
    /// Arena index of the most recently used entry.
    head: Option<usize>,
    /// Arena index of the least recently used entry (next to be evicted).
    tail: Option<usize>,
}

impl<K, V> LruCache<K, V>
where
    K: Hash + Eq + Clone,
```

```

{
    /// Create a cache holding at most `cap` entries.
    ///
    /// # Panics
    /// Panics if `cap == 0`, since a zero-capacity cache cannot store anything.
    pub fn new(cap: usize) -> Self {
        assert!(cap > 0, "capacity must be > 0");
        Self {
            cap,
            lookup_table: HashMap::new(),
            nodes: Vec::new(),
            head: None,
            tail: None,
        }
    }

    /// Promote node `i` to `head` (MRU).
    ///
    /// Handles both cases in one pass:
    /// * **Already linked** – detach from its current neighbours first.
    /// * **Brand new** – the node has `prev = next = None` and is not yet
    ///   referenced by `head`/`tail`, so the detach step is skipped.
    fn touch(&mut self, i: usize) {
        let (prev, next) = (self.nodes[i].prev, self.nodes[i].next);

        // A node is "linked" if any of its four references exist.
        let linked = prev.is_some()
            || next.is_some()
            || self.head == Some(i)
            || self.tail == Some(i);

        if linked {
            // Splice `i` out of the list.
            match prev {
                Some(p) => self.nodes[p].next = next,
                None => self.head = next,
            }
            match next {
                Some(n) => self.nodes[n].prev = prev,
                None => self.tail = prev,
            }
        }

        // Splice `i` in at the front.
        self.nodes[i].prev = None;
        self.nodes[i].next = self.head;
        if let Some(h) = self.head {
            self.nodes[h].prev = Some(i);
        }
        self.head = Some(i);
        if self.tail.is_none() {
            self.tail = Some(i);
        }
    }

    /// Look up `key`, promoting it to MRU on a hit.
    ///
    /// Returns `None` if the key is not cached.
    pub fn get(&mut self, key: &K) -> Option<&V> {
        let i = *self.lookup_table.get(key)?;
        self.touch(i);
        Some(&self.nodes[i].value)
    }

    /// Insert or update `key`. Promotes it to MRU. Evicts the LRU entry if
    /// the cache is full.
    pub fn put(&mut self, key: K, value: V) {

```

```

    // Case 1: key already present → update value, promote.
    if let Some(&i) = self.lookup_table.get(&key) {
        self.nodes[i].value = value;
        self.touch(i);
        return;
    }

    // Case 2: room to grow → append a fresh node.
    if self.nodes.len() < self.cap {
        let i = self.nodes.len();
        self.nodes.push(Node {
            key: key.clone(),
            value,
            prev: None,
            next: None,
        });
        self.lookup_table.insert(key, i);
        self.touch(i);
        return;
    }

    // Case 3: at capacity → reuse the LRU slot.
    // `tail` is guaranteed to be `Some` here because `cap >= 1` and the
    // arena is full, but we still handle `None` gracefully.
    if let Some(i) = self.tail {
        let old_key = std::mem::replace(&mut self.nodes[i].key, key);
        self.nodes[i].value = value;
        self.lookup_table.remove(&old_key);
        self.lookup_table.insert(self.nodes[i].key.clone(), i);
        self.touch(i);
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn basic_eviction() {
        let mut c = LruCache::new(2);
        c.put("A", 1);
        c.put("B", 2);

        assert_eq!(c.get(&"A"), Some(&1)); // A is now MRU
        c.put("C", 3); // evicts B

        assert_eq!(c.get(&"B"), None);
        assert_eq!(c.get(&"A"), Some(&1));
        assert_eq!(c.get(&"C"), Some(&3));
    }

    #[test]
    fn updating_existing_key_keeps_it() {
        let mut c = LruCache::new(2);
        c.put("A", 1);
        c.put("B", 2);

        c.put("A", 100); // update A, A becomes MRU
        c.put("C", 3); // evicts B

        assert_eq!(c.get(&"A"), Some(&100));
        assert_eq!(c.get(&"B"), None);
        assert_eq!(c.get(&"C"), Some(&3));
    }

    #[test]

```

```

fn get_refreshes_recency() {
    let mut c = LruCache::new(2);
    c.put("A", 1);
    c.put("B", 2);

    assert_eq!(c.get(&"A"), Some(&1)); // A refreshed
    c.put("C", 3);                       // evicts B

    assert_eq!(c.get(&"B"), None);
    assert_eq!(c.get(&"A"), Some(&1));
}

#[test]
fn capacity_one() {
    let mut c = LruCache::new(1);
    c.put("A", 1);
    c.put("B", 2); // evicts A

    assert_eq!(c.get(&"A"), None);
    assert_eq!(c.get(&"B"), Some(&2));
}

#[test]
fn missing_key_returns_none() {
    let mut c: LruCache<&str, i32> = LruCache::new(2);
    assert_eq!(c.get(&"nope"), None);
}

#[test]
#[should_panic(expected = "capacity must be > 0")]
fn zero_capacity_panics() {
    let _: LruCache<&str, i32> = LruCache::new(0);
}
}

```

`touch` reads the node's two links, decides whether the node is linked at all, and acts. The `linked` test is what lets one method serve both callers. Without it, a brand new node would be spliced out of a list it is not in, and `head` and `tail` would be overwritten with `None`.

`get` takes `&mut self` because a hit reorders the list. The index comes from the lookup table, `touch` promotes it, and the reference returned is into the arena.

`put` has three cases in order. An existing key takes the update path, which writes the new value and promotes the slot. A key that is not present and a cache below capacity appends a node. A full cache overwrites the tail slot, which removes the old key from the lookup table in the same step. The `assert!` in `new` is what makes the third case safe to write with `tail` assumed to be present.

The key is cloned once per insertion because it lives in two places: the node keeps it for eviction, and the lookup table keeps it for search. The map cannot be keyed on a reference into the arena, so one of the two copies has to be made.

Intuition

capacity 2, keys A B C

call	nodes	lookup_table	head	tail
new(2)	[]	{}	None	None
put("A", 10)	[(A,10,-,-)]	{A:0}	0	0
put("B", 20)	[(A,10,-,1), (B,20,0,-)]	{A:0,B:1}	1	0
get(&"A")	touch 0: detach 0, attach 0 [(A,10,-,1), (B,20,0,-)]	{A:0,B:1}	0	1
put("C", 30)	at capacity, so the tail slot 1 is reused detach 1, remove B from the table nodes[1] = (C,30,-,-) attach 1: nodes[1].next = 0, nodes[0].prev = Some(1) [(A,10,1,-), (C,30,-,0)]			
		{A:0,C:1}	1	0
get(&"B")	the table has no entry for B		None	
get(&"A")	touch 0		Some(10)	
get(&"C")	touch 1		Some(30)	
put("A", 99)	case 1: nodes[0].value = 99, touch 0			
get(&"A")			Some(99)	

Time and Space Complexity

Operation	Time	Space
get	$O(1)$ expected	none
put of an existing key	$O(1)$ expected	none
put of a new key	$O(1)$ expected	one slot, reused when the cache is full
memory	$O(\text{capacity})$	one node per slot, never more

touch performs a constant number of assignments in every case. There is no allocation, no copying of values, and no recursion.

Limitations

The arena never shrinks. Slots are reused, so the cache holds one `K` and one `V` per slot for its whole life, including after those values have been replaced. That is what bounds the memory, and it also means a cache created with a large capacity keeps that memory after it becomes empty.

A capacity of zero panics at construction. `new` asserts `cap > 0`, so the panic happens before any use and names the capacity, which is better than failing on the first `put`. A caller that receives a capacity from a file still has to validate it or accept the panic; a `Result` would report it as a value.

get borrows the cache mutably, so the returned reference cannot be held across another call. `let v = cache.get(&k)` keeps `cache` borrowed while `v` is live, and `put` cannot be called until `v` goes out of scope. A caller that needs to read a value and then store something has to copy the value out first.

There is no `len`, no `is_empty`, and no iteration. The lookup table's length is the entry count, and a caller outside the module cannot read it. A two-line accessor would close the gap.

The indexes are not checked. Every method indexes `self.nodes[i]` directly. The indexes come from the lookup table or from `head` and `tail`, which this module maintains, so they are valid while the invariants hold. A mistake in `touch` would be an index panic rather than a wrong answer.

The tests do not exercise growth past the capacity with distinct keys. They cover eviction, update, refresh, capacity one, a missing key and a zero capacity. A test that fills a cache of capacity three, evicts one entry, and then reads the other two would check that the reused slot and the surviving links agree.

References

- Standard library, `Vec`.
- Standard library, `HashMap`.
- Standard library, `std::mem::replace`.
- Standard library, `Option::get_or_insert`.

Part VI: Concurrency and the Operating Sys- tem

Workload State Machine

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/state_machine.rs`. Test it with `cargo test state_machine`.

Problem Statement

Model the lifecycle of a workload that an SDK exposes, so that the legal transitions are stated once and an illegal transition is refused with a reason. Then model the errors the same API returns, so that a caller can decide whether to retry without parsing a message.

Designing a Solution

An enumeration rather than a status code. A status code makes the set of legal values a convention that lives in a document; an enumeration makes it a type, and a `match` that omits a case does not compile.



The same argument applies to the error type. A retry decision depends on facts, such as the delay a server asked for and the status code it returned, so those facts are attached to the variants that need them. A caller that matches on `RateLimited` has the delay in hand.

Implementation

```

/// Algebraic data types / enum state machines.
///
/// NVIDIA cloud SDKs model workloads as finite state machines. Rust `enum` is
/// perfect for this: invalid states are unrepresentable and transitions are
/// explicit `match` arms, not nested `if` chains.

use std::time::Duration;

/// Lifecycle of a GPU cloud workload exposed through an SDK/API.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum WorkloadState {
    Pending,
    Provisioning,
    Running,
    Terminating,
    Terminated,
    Failed,
}

impl WorkloadState {
    /// Returns an error string for impossible transitions, or `Ok(())`.
    pub fn can_transition_to(self, next: Self) -> Result<(), String> {
        match (self, next) {
            // Forward lifecycle.
            (Self::Pending, Self::Provisioning)
            | (Self::Pending, Self::Failed)
            | (Self::Provisioning, Self::Running)
            | (Self::Provisioning, Self::Failed)
            | (Self::Running, Self::Terminating)
            | (Self::Running, Self::Failed)
            | (Self::Terminating, Self::Terminated)
            | (Self::Terminating, Self::Failed) => Ok(()),

            // Retry loops allowed from failed states.
            (Self::Failed, Self::Pending) | (Self::Failed, Self::Provisioning) => Ok(()),

            // Everything else is invalid.
            _ => Err(format!("cannot transition from {self:?} to {next:?}")),
        }
    }

    pub fn is_terminal(self) -> bool {
        matches!(self, Self::Terminated | Self::Failed)
    }

    pub fn summary(self) -> &'static str {
        match self {
            Self::Pending => "workload accepted, waiting for scheduler",
            Self::Provisioning => "GPU resources being allocated",
            Self::Running => "workload is running",
            Self::Terminating => "cleanup in progress",
            Self::Terminated => "workload stopped and billing ended",
            Self::Failed => "workload failed; inspect logs",
        }
    }
}

/// API error modeled as an enum with associated data.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ApiError {
    InvalidRequest(String),
    Unauthorized,
}

```

```

RateLimited { retry_after: Duration },
NotFound { resource_id: String },
Server { status: u16, message: String },
}

impl ApiError {
    pub fn is_retryable(&self) -> bool {
        matches!(
            self,
            Self::RateLimited { .. }
            | Self::Server {
                status: 500..=599,
                ..
            }
        )
    }

    pub fn user_message(&self) -> String {
        match self {
            Self::InvalidRequest(msg) => format!("invalid request: {msg}"),
            Self::Unauthorized => "authentication required or token expired".to_string(
),
            Self::RateLimited { retry_after } => {
                format!("rate limited, retry in {}ms", retry_after.as_millis())
            }
            Self::NotFound { resource_id } => format!("resource not found:
{resource_id}"),
            Self::Server { status, message } => format!("server error {status}:
{message}"),
        }
    }
}

/// Typical NVIDIA SDK helper: never expose nested `if let` soup to callers.
pub fn parse_gpu_count(raw: &str) -> Result<u32, ApiError> {
    match raw.trim() {
        "" => Err(ApiError::InvalidRequest("gpuCount is empty".into())),
        parsed => parsed.parse::<u32>().map_err(|_| {
            ApiError::InvalidRequest(format!("gpuCount must be a number, got {raw:?}"))
        }),
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn state_machine_rejects_invalid_transitions() {
        assert!(
            WorkloadState::Pending
                .can_transition_to(WorkloadState::Provisioning)
                .is_ok()
        );
        assert!(
            WorkloadState::Running
                .can_transition_to(WorkloadState::Running)
                .is_err()
        );
        assert!(
            WorkloadState::Terminated
                .can_transition_to(WorkloadState::Running)
                .is_err()
        );
        assert!(WorkloadState::Terminating.is_terminal() == false);
        assert!(WorkloadState::Terminated.is_terminal());
    }
}

```

```

#[test]
fn api_error_retry_policy() {
    let rate_limited = ApiError::RateLimited {
        retry_after: Duration::from_millis(100),
    };
    let not_found = ApiError::NotFound {
        resource_id: "gpu-123".into(),
    };
    let server = ApiError::Server {
        status: 503,
        message: "maintenance".into(),
    };

    assert!(rate_limited.is_retryable());
    assert!(not_found.is_retryable());
    assert!(server.is_retryable());
}

#[test]
fn parsing_returns_typed_errors() {
    assert_eq!(parse_gpu_count("4"), Ok(4));
    assert!(parse_gpu_count("").is_err());
    assert!(parse_gpu_count("abc").is_err());
}
}

```

`can_transition_to` matches on the pair `(self, next)`, listing every permitted transition as an alternative pattern. The whole rule set is one expression, and the fall-through arm catches everything else, including a self-transition such as `Running` to `Running`.

A self-transition is refused rather than treated as a no-op. A controller that reports a transition it did not perform writes an audit log that cannot be trusted, and refusing the case costs one line.

`is_retryable` uses `matches!` with a range pattern: a status in `500..=599` is retryable, everything else is not. `NotFound` and `Unauthorized` are not retryable because waiting does not change them: the resource still does not exist and the credential is still expired.

`parse_gpu_count` trims the field, distinguishes an empty field from an unparseable one in the message, and quotes the received text with `{raw:?}` so that trailing whitespace is visible. The error variant carries a message; the variant itself is what a caller matches on.

The repository's version of this function is named for the employer's product line, and its error messages name the field after that product. This edition names the field `gpuCount` and leaves the logic unchanged.

Intuition

state machine, over the whole table

from	to	result
Pending	Provisioning	Ok
Pending	Failed	Ok
Pending	Running	Err "cannot transition from Pending to Running"
Provisioning	Running	Ok
Provisioning	Failed	Ok
Provisioning	Terminated	Err
Running	Terminating	Ok
Running	Failed	Ok
Running	Running	Err
Terminating	Terminated	Ok
Terminating	Failed	Ok
Terminating	Running	Err
Terminated	Pending	Err
Terminated	Running	Err
Failed	Pending	Ok
Failed	Provisioning	Ok
Failed	Terminated	Err

```

parse_gpu_count(" 4 ")  trim gives "4", parse succeeds      -> Ok(4)
parse_gpu_count("")     trim gives "", so the empty arm runs  -> Err(InvalidRequest)
parse_gpu_count("four") parse fails, map_err replaces the error
                        with InvalidRequest quoting "four"    ->
Err(InvalidRequest)
parse_gpu_count("-1")    parse::

```

Time and Space Complexity

Operation	Time	Space
can_transition_to	O(1)	none when permitted; one String when refused
is_terminal	O(1)	none
summary	O(1)	none; the return type is &'static str
is_retryable	O(1)	none
user_message	O(1)	one String allocation per call
parse_gpu_count	O(digits)	one String on the error path

Limitations

can_transition_to returns `Result<(), String>`. A refused transition is a value a caller may want to match on, to log it, to count it, or to distinguish "illegal transition" from "the workload no longer exists", and a `String` can-

not be matched. It also allocates on every refusal. The fix is a small error type with two fields, `from` and `next`, both of which the state machine already has.

`user_message` allocates for every call, including the two variants that need no formatting. `Unauthorized` returns a `to_string()` of a constant. A `Display` implementation writes into the caller's buffer and allocates nothing.

`is_retryable` treats any status in `500..=599` as retryable, and a `Server` variant carrying a `4xx` status as not retryable. The range test is doing double duty: it also asserts that the `Server` variant holds a server-side status, and nothing enforces that at construction time.

The transition table is tested by example, not exhaustively. The test checks four transitions. There are thirty-six ordered pairs of six states, and the table permits twelve of them. A test that loops over all pairs and compares each answer with a written list would catch a missing or an extra pattern.

`parse_gpu_count` reports a value that overflows `u32` as a parse failure. `"4294967296"` and `"four"` produce the same variant, so a caller cannot tell a malformed field from an out-of-range one.

References

- The Rust Book, [Defining an enum](#).
- Standard library, `std::time::Duration`.
- Standard library, `str::parse`.

Token Bucket Rate Limiter

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/rate_limiter.rs`. Test it with `cargo test rate_limiter`.

Problem Statement

Limit how often a caller may do something, allow a burst, and do it safely from several threads. The interface is one question: may I do it now?

Designing a Solution

A token bucket holds at most `capacity` tokens and refills at `rate` tokens per second. Each accepted request removes one token; a request that arrives with fewer than one token available is refused. The bucket therefore admits a burst of up to `capacity` requests and then holds the sustained rate at `rate`.

```
tokens = 3.0, capacity = 5.0, rate = 1.0/s, last_refill = t
t + 0.0s  try_acquire  tokens 3.0 >= 1.0  true   tokens = 2.0
t + 0.0s  try_acquire  tokens 2.0 >= 1.0  true   tokens = 1.0
t + 0.0s  try_acquire  tokens 1.0 >= 1.0  true   tokens = 0.0
t + 0.0s  try_acquire  tokens 0.0 < 1.0   false  tokens = 0.0
t + 0.5s  try_acquire  refill adds 0.5, so 0.5 < 1.0  false
t + 1.5s  try_acquire  refill adds 1.5, so 1.5 >= 1.0 true, tokens = 0.5
t + 10.0s try_acquire  refill is capped at capacity, so the result is 5.0
```

The refill is computed from the elapsed time rather than by a timer thread. A timer thread would cost one thread per bucket and would make the bucket's state depend on how the scheduler ran that thread.

Implementation

```

/// Thread-safe token bucket rate limiter.

use std::sync::Mutex;
use std::time::Instant;

pub struct TokenBucket {
    /// Max tokens.
    capacity: f64,
    /// Tokens per second.
    rate: f64,
    state: Mutex<BucketState>,
}

struct BucketState {
    tokens: f64,
    last_refill: Instant,
}

impl TokenBucket {
    pub fn new(capacity: usize, rate_per_sec: f64) -> Self {
        assert!(capacity > 0);
        assert!(rate_per_sec > 0.0);
        Self {
            capacity: capacity as f64,
            rate: rate_per_sec,
            state: Mutex::new(BucketState {
                tokens: capacity as f64, // Start full
                last_refill: Instant::now(),
            }),
        }
    }

    pub fn try_acquire(&self) -> bool {
        let mut s = self.state.lock().unwrap();
        let now = Instant::now();

        // Refill tokens based on elapsed time
        let elapsed = now - s.last_refill;
        let new_tokens = elapsed.as_secs_f64() * self.rate;
        s.tokens = (s.tokens + new_tokens).min(self.capacity);
        s.last_refill = now;

        // Try to take one
        if s.tokens >= 1.0 {
            s.tokens -= 1.0;
            true
        } else {
            false
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::thread;
    use std::time::Duration;

    #[test]
    fn it_works() {
        let bucket = TokenBucket::new(5, 1.0); // 5 tokens, refill 1 per sec

        // Use all 5
    }
}

```

```

    for _ in 0..5 {
        assert!(bucket.try_acquire());
    }
    // 6th fails
    assert!(!bucket.try_acquire());

    // Wait 1 second
    thread::sleep(Duration::from_secs(1));
    // Should have 1 token back
    assert!(bucket.try_acquire());
    // And none left
    assert!(!bucket.try_acquire());
}
}

```

`try_acquire` takes `&self` rather than `&mut self`. That is what allows one bucket to be shared by reference across threads; the mutation happens behind the `Mutex`.

The four statements inside the lock are the whole algorithm: read the clock, add the tokens that time has earned, cap at the capacity, and take one if there is one. `last_refill` is updated on every call, so no interval is counted twice.

The lock is held across a clock read and a floating-point multiply, which is the shortest critical section this design permits.

Intuition

```

new(5, 1.0)
state = { tokens: 5.0, last_refill: t0 }

five calls to try_acquire, all at t0:
call 1 elapsed 0, tokens 5.0, 5.0 >= 1.0 -> true,  tokens 4.0
call 2 elapsed 0, tokens 4.0, 4.0 >= 1.0 -> true,  tokens 3.0
call 3 elapsed 0, tokens 3.0, 3.0 >= 1.0 -> true,  tokens 2.0
call 4 elapsed 0, tokens 2.0, 2.0 >= 1.0 -> true,  tokens 1.0
call 5 elapsed 0, tokens 1.0, 1.0 >= 1.0 -> true,  tokens 0.0

call 6 at t0:
elapsed 0, tokens 0.0, 0.0 < 1.0          -> false, tokens 0.0

thread::sleep(1s), then call 7 at t0 + 1s:
elapsed 1.0s, new_tokens = 1.0 * 1.0 = 1.0
tokens = min(0.0 + 1.0, 5.0) = 1.0
1.0 >= 1.0                               -> true,  tokens 0.0

call 8 at t0 + 1s:
elapsed ~0, tokens 0.0                     -> false

```

Time and Space Complexity

Operation	Time	Space
<code>try_acquire</code>	$O(1)$ plus one uncontended mutex acquisition	three machine words of state
<code>new</code>	$O(1)$	one allocation-free struct

Limitations

`new` panics on an invalid configuration. `capacity == 0` and `rate_per_sec <= 0.0` both assert, and a rate read from a configuration file is external input. The rate test also rejects `f64::NAN`, because every comparison with `NAN` is false, so the assertion fires. `f64::INFINITY` passes the same test and makes every request succeed, because the refill is capped at the capacity on the first call and the bucket then stays full. The validation is therefore incomplete and the failure is a panic rather than a value.

`self.state.lock().unwrap()` panics if a thread panicked while holding the lock. One failed request handler would then take down every later caller that touches this bucket. Chapter 22 describes the recovery pattern that avoids this.

There is no blocking acquisition. A caller that is refused must sleep and try again, and the bucket offers no way to learn how long to sleep, so the retry policy lives in the caller and every caller invents one.

`try_acquire` reads the clock on every call. `Instant::now()` is cheap, and on some platforms it goes through the vDSO rather than a system call. A bucket that is called millions of times per second spends a measurable fraction of its time in the clock.

The bucket is per-process. Two processes sharing a limit need a shared store, and nothing in this design provides one.

The test sleeps for a second. That makes the test suite slower and makes the result depend on how the operating system scheduled the test thread. On a loaded machine the sleep may return later than requested, which only makes the assertion more likely to pass, so a bucket whose rate were too high would not be caught.

References

- Standard library, `Mutex`.
- Standard library, `Instant`.
- Standard library, `f64::min`.

Worker Pool

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/worker_pool.rs`. Test it with `cargo test worker_pool`.

Problem Statement

Apply a function to every element of a vector using a bounded number of threads, and return the results in the order of the input. Order matters because a caller that reports per-item outcomes must be able to match an outcome with the item it came from.

Designing a Solution

Jobs carry their input index on a channel. Workers take jobs as they become free, apply the function, and send `(index, result)` back on a second channel. Results arrive in completion order, and the caller places each one at its index.

```

                                job channel          result channel
inputs  ---> [(0,a), (1,b), (2,c), ...]  [(2,x), (0,y), (1,z), ...]

worker 0 ---+
worker 1 ---+----> take one job at a time ---> send (index, output)
worker 2 ---+

the caller collects the pairs and writes them into a slot vector:
ordered[2] = x
ordered[0] = y
ordered[1] = z

when every slot is filled, the ordered vector is the answer

```

The bound on concurrency is the number of workers. The bound on memory is the number of jobs in flight, which is the input length plus the results not yet collected.

Implementation

```

/// Bounded worker pool with ordered results.
///
/// Cloud CLIs/SDKs often fan out API calls across a bounded number of workers.
/// This example maps `Vec<T>` to `Vec<R>` with N threads and preserves input
/// order in the output. It demonstrates `Arc<Mutex<_>>`, channels, scoped
/// ownership, and panic-free dispatch.

use std::sync::{mpsc, Arc, Mutex};
use std::thread;

pub fn map_order<T, R, F>(worker_count: usize, inputs: Vec<T>, mapper: F) -> Vec<R>
where
    T: Send + 'static,
    R: Send + 'static,
    F: Fn(T) -> R + Send + Sync + 'static,
{
    assert!(worker_count > 0, "worker_count must be > 0");
    assert!(inputs.is_empty(), "inputs must not be empty");

    let expected = inputs.len();
    let mapper = Arc::new(mapper);
    let (job_tx, job_rx) = mpsc::channel::<(usize, T)>();
    let job_rx = Arc::new(Mutex::new(job_rx));
    let (result_tx, result_rx) = mpsc::channel::<(usize, R)>();

    let mut handles = Vec::with_capacity(worker_count);
    for _worker_id in 0..worker_count {
        let mapper = Arc::clone(&mapper);
        let job_rx = Arc::clone(&job_rx);
        let result_tx = result_tx.clone();
        handles.push(thread::spawn(move || {
            loop {
                let job = {
                    let guard = job_rx.lock().expect("job_rx poisoned");
                    guard.recv()
                };
                match job {
                    Ok((idx, input)) => {
                        let output = mapper(input);
                        if result_tx.send((idx, output)).is_err() {
                            break;
                        }
                    }
                    Err(_) => break, // all senders dropped
                }
            }
        }));
    }

    // Send all jobs, then close the job channel so workers finish after draining it.
    for (idx, input) in inputs.into_iter().enumerate() {
        job_tx
            .send((idx, input))
            .expect("bounded by memory; channel only blocks until a worker receives");
    }
    drop(job_tx);

    // Drop our result sender. Only the worker clones remain, so `iter()` ends
    // after every worker exits.
    drop(result_tx);

    // Reassemble results in input order.
    let mut ordered: Vec<Option<R>> = Vec::new();

```

```

    for (idx, value) in result_rx.iter() {
        if ordered.len() <= idx {
            ordered.resize_with(idx + 1, || None);
        }
        ordered[idx] = Some(value);
    }

    for handle in handles {
        let _ = handle.join();
    }

    assert_eq!(
        ordered.len(),
        expected,
        "every job must produce exactly one result"
    );
    ordered
        .into_iter()
        .map(|value| value.expect("all results present"))
        .collect()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn maps_all_inputs_and_preserves_order() {
        let inputs: Vec<i32> = (0..100).collect();
        let results = map_order(4, inputs, |x| x * x);
        assert_eq!(results.len(), 100);
        for (i, value) in results.iter().enumerate() {
            assert_eq!(*value, (i as i32) * (i as i32));
        }
    }

    #[test]
    fn one_worker_works() {
        let results = map_order(1, vec![1, 2, 3], |x| x + 10);
        assert_eq!(results, vec![11, 12, 13]);
    }
}

```

`mpsc::Receiver` is single-consumer, so sharing it between workers requires `Arc<Mutex<Receiver<_>>>`. The lock is taken inside a block and released before the mapper runs:

```

let job = {
    let guard = job_rx.lock().expect("job_rx poisoned");
    guard.recv()
};

```

The guard is alive while `recv` blocks, so a worker waiting for work holds the lock. The block ends before `mapper(input)` is called, so the mapper never runs with the lock held, which is what keeps the pool from serialising all of its work.

`drop(job_tx)` is what tells the workers to stop. A receive on an empty channel returns an error once every sender is dropped, so the loop breaks. `drop(result_tx)` does the same job for the caller's own copy of the result

sender: without it, `result_rx.iter()` would wait for a sender that only the caller holds.

`ordered.resize_with(idx + 1, || None)` grows the vector on demand, so results that arrive out of order for low indices still land in the right slots.

Intuition

```
map_order(2, vec![10, 20, 30], |x| x + 1)

jobs on the channel: (0, 10), (1, 20), (2, 30)

possible interleaving:
  worker 0 takes (0, 10), computes 11, sends (0, 11)
  worker 1 takes (1, 20), computes 21, sends (1, 21)
  worker 0 takes (2, 30), computes 31, sends (2, 31)

result channel delivers, in some order: (0, 11), (1, 21), (2, 31)

caller:
  ordered = []                receives (0, 11)
  ordered.len() is 0 <= 0, so resize to 1; ordered[0] = Some(11)
  receives (1, 21)
  ordered.len() is 1 <= 1, so resize to 2; ordered[1] = Some(21)
  receives (2, 31)
  ordered.len() is 2 <= 2, so resize to 3; ordered[2] = Some(31)

every worker has dropped its result sender, so iter() ends
ordered.len() == 3 == expected
flattened: [11, 21, 31]
```

Time and Space Complexity

Re-source	Cost
Time	$O(n)$ mapper calls, $O(\text{worker_count})$ thread creations, $O(n)$ lock acquisitions and $O(n)$ sends
Space	$O(n)$ for the jobs, the results, and the slot vector

Limitations

The receivers are contended, and the lock is held while a worker waits. A worker that finds the queue empty holds the mutex while blocking in `recv`, so any other worker that finishes a job and comes back for more waits for the mutex to be released. `recv` releases the lock internally once it starts waiting, the standard library's channel implementation releases the

"blocking" case through a condition variable, but the window between the lock being taken and the wait beginning is real, and it is the part that a multiple-consumer channel exists to remove. The `crossbeam-channel` crate provides such a channel, at the cost of one dependency.

A panic in the mapper is invisible. `let _ = handle.join()` discards the join result, so a worker that panics mid-item leaves that item's slot empty. The `assert_eq!(ordered.len(), expected)` in the common case does not fire, because `ordered` was resized by the results that did arrive, so the failure that reaches the caller is the `expect("all results present")` on the missing slot, which names no item and no cause.

The function asserts on empty input. `assert!(!inputs.is_empty())` refuses a call that has an obvious correct answer, an empty vector, and turns it into a panic. That is a design choice rather than a necessity: the loop would send nothing, the workers would exit, and the result would be empty.

The order of results on the wire is unspecified, and only the reassembly makes the output ordered. A caller that wants results as they arrive, in completion order, cannot use this function.

The closure must be 'static. `F: Fn(T) -> R + Send + Sync + 'static` means the mapper cannot borrow anything from the caller's stack. Scoped threads would lift that restriction, and they would also let `T` and `R` borrow.

Summary

- Two channels carry the work. The index recorded with each job and returned with each result is the mechanism that restores the input order.
- `Arc<Mutex<Receiver>>` is present because `mpsc` is single-consumer. The mutex makes the receiver shareable rather than protecting data.
- The lock is released before the mapper runs, so the workers execute in parallel rather than one at a time.
- A panic in the mapper is discarded by `let _ = handle.join()`. The slot for that item stays empty, and the failure surfaces as an `expect` that names neither the item nor the cause.
- `assert!(!inputs.is_empty())` turns a call with an obvious correct answer into a panic; the loop would send nothing and the result would be empty.

- The closure bound is `'static`, so the mapper cannot borrow from the caller's stack. Scoped threads would lift that restriction.

References

- Standard library, `std::sync::mpsc`.
- Standard library, `Mutex`.
- Standard library, `std::thread::scope`, the alternative that removes the `'static` bounds.
- Standard library, `JoinHandle::join`.

Mutex Poisoning

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/mutex_poisoning.rs`. Run it with `cargo run --bin mutex_poisoning`.

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

Leslie Lamport, *Distribution*, 1987

Problem Statement

Handle the case where a thread panics while holding a lock. The data behind the lock may be half-updated, and the next thread to acquire it has to decide whether to trust what it finds.

Designing a Solution

A panic can happen in the middle of a sequence of writes that were meant to succeed or fail together. If the lock were released and the data left as it was, the next reader would see the partial update. Rust makes that failure visible rather than silent: a `Mutex` whose guard was held during a panic is *poisoned*, and later calls to `lock` return `Err(PoisonError)` instead of a guard.

<pre>thread A ----- lock() -> guard stock_value -= 1000 cash_on_hand += 1200 panic! while guard is alive</pre>	<pre>thread B ----- lock() -> Err(PoisonError)</pre>
---	---

the Mutex records that a panic happened while the lock was held	the guard is still reachable through <code>PoisonError::into_inner()</code> , so B can read the state, correct it, and release the lock
--	---

The recovery path in the file is a reconciliation: an audit prints the discrepancy, corrects the cash figure, marks the transaction closed, and takes a copy of the corrected state.

The important detail is what poisoning does *not* do. Poisoning is not access control, and recovering the data does not clear the mark. Every later `lock()` on that mutex still returns `Err`, so the decision to trust the repaired state is made by each caller rather than by the act of repair.

Implementation

```
// =====
// Mutex poisoning: failed transaction and audit recovery
//
// Ported from the Rust playground repo (`scam_transaction.rs`).
//
// Story:
//   A branch executes a financial transaction that mutates shared state under
//   a `Mutex`. Mid-transaction the thread panics. In Rust, a `Mutex` whose
//   guard was held during a panic becomes **poisoned**. Future lock() calls
//   return Err(PoisonError) instead of silently letting another thread
//   observe an inconsistent partial update.
//
// Why this appears in cloud interviews:
//   - Controllers/reconcilers mutate shared state; a panic while holding a lock
//     is a real failure mode in worker pools, caches, and rate limiters.
//   - Rust chooses fail-fast over data corruption: a poisoned mutex tells
//     callers that the protected invariant may be broken. The caller must
//     decide whether to abort, recover, or reset state.
//   - PoisonError::into_inner() returns the MutexGuard, so a recovery path
//     can inspect/correct the last known state without waiting for the lock to
//     be released by the panicked thread.
//
// Run:
//   cargo run --bin mutex_poisoning
// =====

use std::error::Error;
use std::sync::{Arc, Mutex, MutexGuard, PoisonError};
use std::thread;

/// Shared ledger state. In a real system this would be behind a transactional
/// store; here it is plain in-memory state to illustrate lock poisoning.
#[derive(Debug, Clone)]
struct Ledger {
    cash_on_hand: i64,
    stock_value: i64,
    transaction_id: u32,
    is_settled: bool,
}

impl Ledger {
    fn new() -> Self {
        Ledger {

```

```

        cash_on_hand: 10_000,
        stock_value: 5_000,
        transaction_id: 0,
        is_settled: false,
    }
}

/// Recovers from a poisoned mutex by taking ownership of the guard that the
/// panicked thread left behind.
///
/// `PoisonError::into_inner()` gives us the `MutexGuard`, which means we still
/// hold the lock while repairing state. The guard is dropped when this function
/// returns.
fn recover_ledger(poisoned: PoisonError<MutexGuard<'_, Ledger>>) -> Ledger {
    // into_inner() returns the MutexGuard. We own the lock exclusively even
    // though the previous holder panicked.
    let mut guard = poisoned.into_inner();

    println!("[Reconciliation] Physical audit reveals a cash discrepancy.");
    println!(
        "[Reconciliation] Digital cash: {} | Corrected cash: {}",
        guard.cash_on_hand,
        guard.cash_on_hand - 200
    );

    // Correct the data while we still hold the guard.
    guard.cash_on_hand -= 200;
    guard.is_settled = true;
    guard.transaction_id += 1;

    println!("[Reconciliation] Ledger corrected. Transaction marked failed and
closed.");

    // Clone the inner data to return an owned value. The guard drops here and
    // releases the lock.
    guard.clone()
}

fn main() -> Result<(), Box<dyn Error>> {
    let ledger = Arc::new(Mutex::new(Ledger::new()));

    // Thread A: the faulty transaction. It mutates one side of the ledger and
    // then panics before marking the transaction settled.
    let ledger_a = Arc::clone(&ledger);
    let handle_a = thread::spawn(move || {
        // SAFETY-free normal code: if the mutex is already poisoned, unwrap()
        // panics; in a controlled demo this is the first lock so it is fine.
        let mut guard = ledger_a.lock().unwrap();

        println!(
            "[Branch A] Transaction {} commenced.",
            guard.transaction_id + 1
        );
        guard.stock_value -= 1_000;
        println!(
            "[Branch A] Stock deducted. New value: {}",
            guard.stock_value
        );
        guard.cash_on_hand += 1_200;
        println!(
            "[Branch A] Cash updated. New balance: {}",
            guard.cash_on_hand
        );

        // Panic while `guard` is still alive -> the Mutex becomes poisoned.
        panic!("[Branch A] Settlement failure: arithmetic overflow in reconciliation.")
    });
}

```

```

;
});

// A panic in a spawned thread does not abort the process unless the panic
// strategy is set to abort. `join()` returns Err, which is expected here.
let _ = handle_a.join();

// Thread B: the forensic audit. It sees Err(PoisonError), inspects the
// recovered state, and repairs the ledger.
let ledger_b = Arc::clone(&ledger);
let handle_b = thread::spawn(move || {
    println!("\n[Audit] Attempting to access the ledger...");

    match ledger_b.lock() {
        Ok(guard) => {
            println!("[Audit] Ledger is not poisoned: {:?}", *guard);
        }
        Err(poisoned) => {
            println!("[Audit] Mutex is poisoned. Recovered state before
correction:");
            println!("[Audit] {:?}", *poisoned.get_ref());

            let corrected = recover_ledger(poisoned);
            println!("[Audit] Corrected ledger: {:?}", corrected);
        }
    }
});

handle_b.join().unwrap();

// Important: recovering the data does NOT clear the poison bit on the
// Mutex itself. Subsequent lock() calls still return Err(PoisonError) so
// callers can decide whether the repaired state is trustworthy.
// Uncommenting the next lines would panic:
//
// let _guard = ledger.lock().unwrap();

Ok(())
}

// =====
// Tests (cargo test --bin mutex_poisoning)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn ledger_starts_in_expected_state() {
        let ledger = Ledger::new();
        assert_eq!(ledger.cash_on_hand, 10_000);
        assert_eq!(ledger.stock_value, 5_000);
        assert!(!ledger.is_settled);
    }

    #[test]
    fn recovery_repairs_a_poisoned_ledger() {
        let ledger = Arc::new(Mutex::new(Ledger::new()));

        // Poison the mutex from a helper thread, exactly like the demo.
        let ledger_a = Arc::clone(&ledger);
        let handle = thread::spawn(move || {
            let _guard = ledger_a.lock().unwrap();
            panic!("boom");
        });
        let _ = handle.join();
    }
}

```

```

    let lock_result = ledger.lock();
    assert!(lock_result.is_err(), "mutex should be poisoned");

    let recovered = recover_ledger(lock_result.unwrap_err());
    assert!(recovered.is_settled);
    assert_eq!(recovered.transaction_id, 1);
    assert_eq!(recovered.cash_on_hand, 10_000 - 200);
}

```

`recover_ledger` takes the `PoisonError` by value. That is the signature that makes the type system enforce what the function needs: a `PoisonError<MutexGuard<...>>` can only be produced by a failed `lock`, so the function cannot be called with a healthy lock by mistake.

`poisoned.into_inner()` consumes the error and returns the guard. The lock is now held by this thread, so the repair happens under the lock rather than beside it.

`guard.clone()` returns an owned copy of the state. `MutexGuard` is not `Clone`, so the clone is of the inner value, taken through the guard. The guard is dropped when the function returns, which releases the lock.

`handle_b.join().unwrap()` propagates a panic from the audit thread. `main` returns `Result`, so an `unwrap` here would panic rather than return an error; the join result is discarded for the first thread with `let _ = handle_a.join()`, and the comment explains that the panic is the point of the demonstration.

The test calls `recover_ledger` with the error from a real poisoned lock, so the recovery path is exercised rather than described.

Intuition

```

ledger after new:  { cash 10000, stock 5000, transaction_id 0, is_settled false }

thread A          lock() -> guard, the mutex is now marked as locked
                  stock_value -= 1000    -> 4000
                  cash_on_hand += 1200   -> 11200
                  panic! while the guard is alive

                  the guard is dropped during unwinding, the mutex is unlocked,
                  and the poison bit is set

join A            returns Err, which the code discards

thread B          lock() -> Err(PoisonError)
                  poisoned.get_ref() shows the partial update:
                  { cash 11200, stock 4000, transaction_id 0, is_settled false }
                  recover_ledger:
                  cash 11200 - 200 = 11000
                  is_settled = true
                  transaction_id = 1

```

```

the guard is dropped, the lock is released

after the run    the mutex is still poisoned, and the ledger inside it reads
                  { cash 11000, stock 4000, transaction_id 1, is_settled true }

```

Time and Space Complexity

Re-source	Cost
Time	one lock acquisition per access; the recovery path prints four lines and clones the state once
Space	the ledger is four machine words; a <code>PoisonError</code> is the size of a guard

Limitations

The repair rule is a constant in the code. `guard.cash_on_hand -= 200` is the audit's correction, and it is hardcoded. In a real reconciliation the correct figure comes from an external source, a bank statement, a database, and the function would take that source as a parameter. As written, the demonstration shows the mechanism and not a policy.

Recovering does not clear the poison mark. The comment at the end of `main` says so, and the line that would panic is left commented out. Every subsequent `lock()` therefore returns `Err`, and a program whose recovery is complete has no way to say so to the other threads without dropping the mutex entirely. `Mutex::clear_poison` exists for that, and it is not used here.

The failed thread's own recovery is absent. Thread A panics and its work is lost; nothing rolls back the partial update except thread B's audit. A system where either thread might be the one to recover needs the repair to be idempotent and runnable by any caller.

`ledger_a.lock().unwrap()` panics if the mutex is already poisoned. The comment acknowledges it. In a program where several threads may touch the same lock, `unwrap` on a `lock` result converts one thread's failure into a cascade of panics, which is the opposite of what poisoning is for.

The audit thread's panic would be propagated with `unwrap`. `main` returns `Result<(), Box<dyn Error>>`, so the error type on the happy path is unused here: the only failure that can occur is a panic, and a panic cannot

be converted into `Box<dyn Error>` by `?. join()` returns `Result<T, Box<dyn Any + Send>>`, and `Box<dyn Any + Send>` does not implement `Error`.

Summary

- Poisoning records that a panic occurred while the guard was held. It repairs nothing and it does not deny access to the data.
- `PoisonError::into_inner()` returns the guard, so the thread that recovers also holds the lock while it repairs the state, and the repair does not race with other readers.
- Recovering is appropriate where the invariant can be re-established from data the program trusts, and propagating is appropriate where it cannot. In both cases the defect is a recovery with no stated repair rule.
- The repair rule here is a constant, `guard.cash_on_hand -= 200`. A reconciliation would take the correct figure from an external source and pass it in.
- Recovering does not clear the poison mark, so every later `lock()` returns `Err` and the other threads cannot be told that the repair is complete. `Mutex::clear_poison` exists for that purpose and is not used here.
- `ledger_a.lock().unwrap()` converts one thread's failure into a cascade of panics, which is the opposite of what poisoning is for.

References

- Standard library, `Mutex`, including the section on poisoning.
- Standard library, `PoisonError::into_inner`.
- Standard library, `Mutex::clear_poison`, stabilised in Rust 1.77.0.
- Standard library, `JoinHandle::join`.

Counting Islands

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/count_islands.rs`. Run it with `cargo run --bin count_islands`.

Problem Statement

Given a rectangular grid of cells, count the connected regions of a chosen value. Two cells are connected when they share an edge, so a cell touches at most four neighbours. The function is generic over the cell type, so the caller decides whether land is `'1'`, `true`, `1u8`, or something else.

Designing a Solution

Scan the grid and flood-fill each unvisited land cell. Counting the number of flood-fills is the answer, because a flood-fill visits exactly one connected region.

```

grid
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1

count
scan finds (0,0), fills the four cells of the top-left block
scan finds (2,2), fills that single cell
scan finds (3,3), fills (3,3) and (3,4)
answer: 3

the fill order for the first region, breadth first
queue: [(0,0)]
pop (0,0), push (0,1) and (1,0)
pop (0,1), push (1,1)
pop (1,0), neighbours already visited
pop (1,1), neighbours already visited

```

A cell is marked when it enters the queue rather than when it leaves. Marking on entry is what keeps a cell from being enqueued twice: two neighbours of the

same cell can both see it as unvisited if the mark happens later, and the queue then holds duplicates and the work grows.

Implementation

```
// =====
// Count Islands
//
// Ported from the Rust playground repo.
//
// Problem: given a grid of cells, count the number of connected "land"
// regions. Two land cells are connected when they are adjacent horizontally
// or vertically (4-directional). Water cells are not traversable.
//
// Why this appears in cloud interviews:
// - "Islands" are connected-component counting, the same shape used to find
//   connected GPU nodes, failure domains, network partitions, or storage
//   groups in distributed systems.
// - The generic implementation accepts any land type (char, u8, bool) and
//   uses BFS with an explicit visited set, which is easy to reason about.
//
// Time complexity: O(rows * cols)
// Space complexity: O(rows * cols) for the visited set and BFS queue.
// =====

use std::collections::{HashSet, VecDeque};

/// Counts 4-directionally connected components of `land` in a rectangular grid.
///
/// The grid is generic over cell values so callers can use `'1'/'0'`,
/// `true/false`, `1u8/0u8`, or any other PartialEq value.
fn count_islands<T: PartialEq>(grid: &[&[T]], land: T) -> usize {
    if grid.is_empty() || grid[0].is_empty() {
        return 0;
    }

    let rows = grid.len();
    let cols = grid[0].len();
    let mut visited = HashSet::new();
    let mut count = 0;
    let dirs = [(0, 1), (1, 0), (0, -1), (-1, 0)];

    // BFS closure: flood-fill one island starting from a known land cell.
    // Passing `visited` explicitly avoids borrow-checker friction when calling
    // the closure from the outer loop.
    let bfs = |start: (usize, usize), visited: &mut HashSet<(usize, usize)>| {
        let mut queue = VecDeque::new();
        queue.push_back(start);
        visited.insert(start);

        while let Some((cr, cc)) = queue.pop_front() {
            for &(dr, dc) in &dirs {
                let nr = cr as i32 + dr;
                let nc = cc as i32 + dc;

                if (0..rows as i32).contains(&nr) && (0..cols as i32).contains(&nc) {
                    let (ur, uc) = (nr as usize, nc as usize);
                    if grid[ur][uc] == land && !visited.contains(&(ur, uc)) {
                        visited.insert((ur, uc));
                        queue.push_back((ur, uc));
                    }
                }
            }
        }
    };

    for (r, row) in grid.iter().enumerate() {
        for (c, cell) in row.iter().enumerate() {
            if *cell == land && !visited.contains(&(r, c)) {
                bfs((r, c), &visited);
                count += 1;
            }
        }
    }

    count
}
```

```

    }
};

for r in 0..rows {
    for c in 0..cols {
        if grid[r][c] == land && !visited.contains(&(r, c)) {
            count += 1;
            bfs((r, c), &mut visited);
        }
    }
}

count
}

// =====
// Runnable demonstration
// =====
fn main() {
    let grid = [
        &['1', '1', '0', '0', '0'][..],
        &['1', '1', '0', '0', '0'][..],
        &['0', '0', '1', '0', '0'][..],
        &['0', '0', '0', '1', '1'][..],
    ];

    println!("Islands: {}", count_islands(&grid, '1'));

    let boolean_grid = [
        &[true, true, false][..],
        &[false, false, false][..],
        &[false, true, true][..],
    ];
    println!("Boolean islands: {}", count_islands(&boolean_grid, true));
}

// =====
// Tests (cargo test --bin count_islands)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn empty_grid_has_zero_islands() {
        let empty: [&[char]; 0] = [];
        assert_eq!(count_islands(&empty, '1'), 0);

        let no_cols: [&[char]; 2] = [&[], &[]];
        assert_eq!(count_islands(&no_cols, '1'), 0);
    }

    #[test]
    fn counts_classic_example() {
        let grid = [
            &['1', '1', '0', '0', '0'][..],
            &['1', '1', '0', '0', '0'][..],
            &['0', '0', '1', '0', '0'][..],
            &['0', '0', '0', '1', '1'][..],
        ];
        assert_eq!(count_islands(&grid, '1'), 3);
    }

    #[test]
    fn all_water_is_zero() {
        let grid = [&['0', '0'][..], &['0', '0'][..]];
        assert_eq!(count_islands(&grid, '1'), 0);
    }
}

```

```

    }

    #[test]
    fn generic_bool_grid_works() {
        let grid = [
            &[true, false, false][..],
            &[true, false, true][..],
            &[false, false, true][..],
        ];
        assert_eq!(count_islands(&grid, true), 2);
    }

    #[test]
    fn single_row_and_column() {
        let row = [&['1', '0', '1', '1'][..]];
        assert_eq!(count_islands(&row, '1'), 2);

        let col = [&['1'][..], &['1'][..], &['0'][..], &['1'][..]];
        assert_eq!(count_islands(&col, '1'), 2);
    }
}

```

The grid type is `&[&[T]]`: a slice of row slices. This accepts a fixed-size array of slices (`&['1', '1', '0'][..]`) and a vector of vectors, and it requires every row to have the same length. The code reads `grid[0].len()` as the column count and assumes the rest of the rows match.

`count_islands<T: PartialEq>(grid: &[&[T]], land: T)` takes `land` by value. The comparison `grid[r][c] == land` therefore needs `PartialEq<T>` for `T`, which the single bound provides. The bound is minimal: nothing else about the cell type is required.

The bounds test casts to `i32` before adding the delta:

```

let nr = cr as i32 + dr;
let nc = cc as i32 + dc;

if (0..rows as i32).contains(&nr) && (0..cols as i32).contains(&nc) {

```

A `usize` cannot represent `-1`, so a neighbour above or to the left of the origin would wrap to a very large index rather than failing a range check. Signed arithmetic makes the range test correct for all four directions in one expression.

The `bfs` closure takes `visited` as a parameter rather than capturing it mutably. A closure that captured `visited` would hold a mutable borrow across the whole outer loop, and the loop also reads `visited`. Passing the reference in makes the borrow begin and end at the call.

The demonstration prints two lines, and the tests exercise the same two grids plus four boundary cases.

Intuition

```

grid
row 0: 1 1 0 0 0
row 1: 1 1 0 0 0
row 2: 0 0 1 0 0
row 3: 0 0 0 1 1

scan reach  cell  is land  visited?  action
(0,0)      yes   no       count = 1, fill from (0,0)
           visit (0,0), (1,0), (0,1), (1,1)
(0,1)      yes   yes      skip
(0,2)      no    -        skip
(0,3)      no    -        skip
(0,4)      no    -        skip
(1,0)      yes   yes      skip
(1,1)      yes   yes      skip
(2,2)      yes   no       count = 2, fill from (2,2), visiting one cell
(3,3)      yes   no       count = 3, fill from (3,3), visiting (3,3) and (3,4)

the run prints:  Islands: 3
                  Boolean islands: 2

```

Time and Space Complexity

Re-source	Cost
Time	$O(\text{rows} \times \text{cols})$, each cell is visited once and each visit examines four neighbours
Space	$O(\text{rows} \times \text{cols})$ for the visited set and the queue

Limitations

The visited set is a hash set of coordinate pairs. Each entry holds two `usize` values plus the hash table's overhead: control bytes, a bucket array that may be twice the entry count, and a random seed per set. A bitmap of one bit per cell answers the same question in $\text{rows} \times \text{cols} / 8$ bytes and one shift per lookup. For a 4096×4096 grid the hash set holds up to sixteen million entries, while the bitmap holds two megabytes. That factor, roughly fifty to a hundred, depending on load factor, is the difference between a large allocation and a small one, and the file's own comment says only " $O(\text{rows} * \text{cols})$ for the visited set".

The rows are assumed to be the same length. `cols` is read from the first row and used to bound every row. A ragged input does not panic on

the bound check for the shorter rows; it panics on `grid[ur][uc]` when a shorter row is indexed at a column the first row had. A ragged grid is a malformed input, and the failure mode is an index panic deep inside the fill.

A very large connected region is held in the queue. The BFS queue can hold a frontier proportional to the region's size, so memory is bounded by the input area in the worst case. A depth-first fill with an explicit stack has the same bound.

The function is private to the binary. `count_islands` is in `src/bin/`, so it is not part of the library's API, and a second binary that wants it must copy it. Moving it into `src/problems/` would export it with the rest of the problems.

The cell type must be `PartialEq` and the land value is taken by value. So `count_islands` cannot accept a predicate, which would let a caller count something other than strict equality, for example, a region of cells above a threshold. The predicate version is the same code with `land` replaced by a closure.

Summary

- A flood fill visits exactly one connected component, so the number of fills is the number of components.
- A cell is marked when it is enqueued rather than when it is dequeued. Without that, several neighbours can enqueue the same cell and the queue holds it more than once.
- The neighbour offsets use signed arithmetic, which is what prevents the fill from wrapping around the edge of the grid.
- The visited set is a hash set of coordinate pairs, so each entry holds two `usize` values plus the table's overhead. A bitmap of one bit per cell answers the same question in `rows × cols / 8` bytes: for a 4096×4096 grid the hash set holds up to sixteen million entries against two megabytes for the bitmap.
- The rows are assumed to be of equal length. A ragged grid does not fail the bound check on a short row; it panics on `grid[ur][uc]` when a short row is indexed at a column the first row had.

- The BFS queue holds a frontier proportional to the region being filled, so memory is bounded by the area of the input in the worst case. A depth-first fill with an explicit stack has the same bound.
- The function is private to its binary, and the cell type must be `PartialEq`, because the land value is taken by value and no predicate is accepted.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 20.2, "Breadth-first search".
- Standard library, `VecDeque`.
- Standard library, `std::ops::Range::contains`.

Syscall Overhead

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/syscall_overhead.rs`. Run it with `cargo run --release --bin syscall_overhead`.

Latency lags bandwidth.

David A. Patterson, *Latency lags bandwidth*, Communications of the ACM 47(10), 2004

Problem Statement

The cost of crossing from user space into the kernel is measured on one machine, and the figure is reported in a form that can be reproduced.

Designing a Solution

Every system call is a controlled transfer of control: the processor switches to kernel mode, the kernel validates the arguments, does the work, checks whether the calling thread should be descheduled, and returns. For a call that does almost nothing, that machinery is the entire cost.

`getpid` is such a call. It returns the caller's process identifier from a field the kernel already holds, takes no arguments, and touches no user memory. Its cost is therefore close to the irreducible price of the boundary itself, which is why it is the standard probe.

The measurement has three parts, and each exists because of a way a benchmark can lie.

warm-up	run the call 100 000 times without timing it the first calls fault in pages and ramp the clock
timing	read the clock, run the call 10 000 000 times, read the clock the loop body is one call, so the elapsed time per iteration is the cost of the call plus the cost of the loop
division	elapsed nanoseconds / iterations the result is an average, and the spread is not reported

Implementation

```
// =====
// Syscall overhead measurement
//
// Ported from the Rust playground repo (`mkdir_syscall.rs`).
//
// What this measures:
// - Every syscall crosses the user/kernel boundary: argument validation,
//   kernel work, scheduling checks, and return to user space.
// - This program calls `getpid()` in a tight loop and estimates the average
//   per-call cost. `getpid()` is a tiny syscall, so the measured cost is
//   close to the irreducible syscall overhead on this kernel/hardware.
//
// Why this appears in cloud interviews:
// - High-performance data planes care about per-packet or per-I/O syscall
//   cost. The difference between one syscall per event and batching via
//   io_uring/recvmmsg can be enormous.
// - It also demonstrates unsafe FFI, safety comments, warm-up loops, and
//   benchmark methodology: warm the code path, run enough iterations, and
//   report average cost rather than one lucky sample.
//
// Run:
// cargo run --bin syscall_overhead
// =====

use std::time::Instant;

/// Calls `getpid()` `iterations` times and returns the average cost in
/// nanoseconds per call.
fn measure_getpid_ns_per_call(iterations: u64) -> f64 {
    let start = Instant::now();

    for _ in 0..iterations {
        // SAFETY: `libc::getpid` is a pure read-only system call with no
        // pointer arguments and no memory side effects. The FFI declaration in
        // the `libc` crate matches the Linux kernel ABI.
        //
        // `black_box` prevents the optimizer from assuming the loop has no
        // observable effect, which matters because the syscall result is
        // otherwise unused.
        unsafe {
            libc::getpid();
        }
    }

    let elapsed = start.elapsed();
    elapsed.as_nanos() as f64 / iterations as f64
}

fn main() {
    const ITERATIONS: u64 = 10_000_000;
}
```

```

// Warm up the code path before timing. The first calls may fault in the
// libc/PLT, cache lines, or CPU frequency may still be ramping.
for _ in 0..100_000 {
    // SAFETY: same reasoning as above; read-only syscall.
    unsafe { libc::getpid() };
}

let ns_per_call = measure_getpid_ns_per_call(ITERATIONS);

println!("Iterations:      {}", ITERATIONS);
println!(
    "Total time:          {:.3}s",
    ns_per_call * ITERATIONS as f64 / 1e9
);
println!("Per syscall:        {:.1} ns", ns_per_call);

// Example framing: if a service processed 10M packets/s and used one
// getpid-style syscall per packet, this fraction of each second would be
// spent crossing into the kernel just for the syscall overhead.
let cpu_seconds_per_second = ns_per_call * 10_000_000.0 / 1_000_000_000.0;
println!(
    "At 10M calls/s, syscall overhead would consume: {:.2} CPU-seconds per second",
    cpu_seconds_per_second
);
}

// =====
// Tests (cargo test --bin syscall_overhead)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn measurement_returns_a_positive_finite_number() {
        let ns = measure_getpid_ns_per_call(10_000);
        assert!(ns.is_finite(), "nanoseconds per call must be finite");
        assert!(ns > 0.0, "a syscall cannot cost zero time");
        // On any real kernel this is microseconds or tens/hundreds of ns, not
        // seconds per call.
        assert!(
            ns < 1_000_000.0,
            "suspiciously slow syscall measurement: {ns} ns"
        );
    }

    #[test]
    fn getpid_returns_nonzero_pid() {
        // SAFETY: read-only syscall, no pointers, no side effects.
        let pid = unsafe { libc::getpid() };
        assert!(pid > 0);
    }
}

```

Three parts of this file need comment.

The SAFETY comments state the obligation: no pointer arguments, no memory side effects, and an FFI declaration that matches the kernel ABI. That is what an `unsafe` block's comment is for, the reasoning that justifies the claim that the block cannot cause undefined behaviour.

The comment in `measure_getpid_ns_per_call` also mentions `black_box`, and the code does not call `black_box`. The call would be the right thing to do: without it, the loop's result is unused, and the optimiser is entitled to remove work whose result nobody observes. What prevents removal today is that a call through `libc` is opaque to the optimiser, an external function call is a side effect it cannot delete, so the loop survives, and the comment describes an intention rather than the code. `std::hint::black_box(libc::getpid())` would make the code match its comment.

The trailing space after the `unsafe` block in the warm-up loop is invisible in a rendered page and present in the source. It is noted here because the listing is otherwise a faithful copy of the file.

The final projection multiplies the measured cost by ten million calls per second and divides by a billion, which yields a figure in CPU-seconds per second. A result above 1.0 means that one core cannot sustain that rate: the arithmetic is saying that the kernel crossing alone would need more than one core's worth of time.

Intuition

Run on an AArch64 Linux machine, release build, with `rustc` 1.96.0-nightly, while no other work was running on the machine:

```
Iterations:      10000000
Total time:      2.076s
Per syscall:     207.6 ns
At 10M calls/s, syscall overhead would consume: 2.08 CPU-seconds per second
```

The last line is the answer to the question this program exists to raise. A service that makes one such call per request at ten million requests per second needs more than two cores doing nothing but crossing the boundary.

Two qualifications belong with the number. The machine is a development board with a specific kernel, and a different machine will produce a different figure; the program prints its platform for that reason. And `getpid` is cheaper than most system calls: a read or a write copies data and an `open` walks the file system, so the number here is a floor for the boundary and not a typical cost.

Time and Space Complexity

Resource	Cost
Time	one clock read per measurement, one system call per iteration
Space	$O(1)$; nothing is allocated

Limitations

The loop is not empty, so the measurement includes it. A call, a bounds check that the optimiser removes, and an increment are part of every iteration. For a 200-nanosecond system call the loop's own cost is a rounding error, and for the control measurement of an empty loop it is the whole measurement. The program has no control row, so the loop's cost is not reported separately.

The average hides the distribution. One number cannot show that a call takes 150 nanoseconds in the common case and a microsecond occasionally, which is what an interference-heavy machine produces. The minimum of several batches is the value least disturbed by other activity on the machine, and the program does not compute it.

getpid may be served without a trap on some architectures. Some kernels implement particular calls through a vDSO shared page that the process reads directly, which turns a system call into a memory read. The program measures what the call costs on this kernel, and the figure is a boundary cost only if the kernel traps.

The program builds the library it links against, and the library emits a warning. Running `cargo run --bin syscall_overhead` prints a `dead_code` warning for an unused function in the library. The warning comes from `merge_intervals2`, which Chapter 5 identifies, and it appears in this chapter's output because Cargo compiles the whole crate.

Summary

- `getpid` gives a floor for the cost of crossing the process boundary. It is not the cost a service would see per request.

- The first iterations pay for page faults, for lazy symbol resolution, and for a clock that has not reached its steady frequency, so the run includes a warm-up pass.
- The figure converts into a consequence. At 207 nanoseconds per call and ten million calls per second, the boundary alone consumes two cores.
- Two measurements would strengthen the result: a control measurement of the empty loop, and the minimum over several batches. Neither is present.

References

- `libc`, `getpid`.
- Standard library, `std::time::Instant`.
- Standard library, `std::hint::black_box`.
- David A. Patterson, "Latency lags bandwidth", *Communications of the ACM* 47(10), 2004, pages 71–75.

Shadowing and Mutation

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/shadowing.rs`. Run it with `cargo run --bin shadowing`.

Problem Statement

Two constructs look similar in source and behave differently: giving a name a new value with `let`, and changing the value a binding already holds.

Designing a Solution

A `let` introduces a binding. Writing a second `let` with the same name introduces a *second* binding that shadows the first; the first is still there, still owns whatever it owns, and is no longer reachable by that name. Because a new binding can have a new type, shadowing is the mechanism for converting a value.

Mutation is a different operation. `let mut count = 0` creates one binding whose value may be replaced through it, and `count += 1` writes to that place. The type is fixed at the declaration, and no new binding appears.

shadowing

```
let value: u32 = 7;
  binding A: u32

let value = value.to_string();
  binding B: String
  A is unreachable by name
  A is dropped at the end of
  its scope
```

mutation

```
let mut count = 0;
  binding: i32, mutable

count += 1;
  the same binding, new value

count += 1;
  again
```

```
let value = value.len();      count
binding C: usize             one binding in the whole block
B is unreachable by name
```

The two mechanisms solve different problems. Shadowing changes the type, and it makes every later reader of the name see the new one. Mutation keeps the type and the binding, which is what a counter or an accumulator needs.

Implementation

```
// =====
// Variable shadowing vs mutation
//
// Ported from the Rust playground repo (`shadowing.rs`).
//
// What this demonstrates:
// - Rust allows a new `let` binding with the same name as an existing
//   binding. This is called shadowing.
// - Shadowing can change the value and the type of a variable without
//   declaring it `mut`.
// - The old binding is no longer reachable after the shadowing `let`.
//
// Why this appears in interviews:
// - Rust SDK code often shadows parsed/deserialized values to convert them
//   from raw input to typed domain values:
//
//     let raw = request.header("X-GPU-Count")?;
//     let raw: u64 = raw.parse()?; // shadow with a typed value
//
// - Shadowing is also useful after unwrapping an `Option`/`Result` when the
//   guard value is no longer needed.
//
// Run:
// cargo run --bin shadowing
// =====

fn demonstrate_shadowing() -> String {
    // First binding is an integer.
    let value: u32 = 7;
    let mut log = format!("integer: {value}");

    // Shadow the integer with a String. The old integer is gone.
    let value = value.to_string();
    log.push_str(&format!("\nstring: {value}"));

    // Shadow the String with a usize computed from it.
    let value = value.len();
    log.push_str(&format!("\nusize: {value}"));

    log
}

fn demonstrate_mutation() -> String {
    // If you need to mutate the same binding's value (same type) without
    // creating a new binding, mark it `mut`.
    let mut count = 0;
    let mut log = String::from("mut count:");
    for _ in 0..3 {
        count += 1;
        log.push_str(&format!(" {count}"));
    }
}
```



```

    log
}

fn main() {
    println!("=== Shadowing ===");
    println!("{}", demonstrate_shadowing());

    println!("\n=== Mutation ===");
    println!("{}", demonstrate_mutation());
}

// =====
// Tests (cargo test --bin shadowing)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn shadowing_allows_type_changes() {
        let log = demonstrate_shadowing();
        assert!(log.contains("integer: 7"));
        assert!(log.contains("string: 7"));
        assert!(log.contains("usize: 1"));
    }

    #[test]
    fn mutation_requires_mut_binding() {
        let log = demonstrate_mutation();
        assert!(log.contains("mut count: 1 2 3"));
    }
}

```

`let value = value.to_string()` uses the old `value` on the right and introduces the new binding on the left. The order is what makes the idiom legal: the right side is evaluated with the old binding in scope, and the new binding takes effect after the statement.

`let value = value.len()` then shadows the `String`, and the value becomes `1` because `"7"` is one character. The chain of three bindings ends with a `usize`.

The conversion idiom in the header comment is the reason the feature matters in application code:

```

let raw = request.header("X-Device-Count")?;
let raw: u64 = raw.parse()?;

```

The second `let` reuses the name for the converted value. There is no chance of using the unparsed text afterwards, because the name now refers to the parsed number. A `mut` version would need two names or an early `drop`, and neither is as clear about which value is in play.

`demonstrate_mutation` shows the other side: one binding, marked `mut`, whose value changes three times. The loop's counter has one name and one type,

which is exactly what shadowing cannot express, because each `let` in a loop body would create a fresh binding that dies at the end of the iteration.

Intuition

demonstrate_shadowing

statement	bindings in scope	log so far
<code>let value: u32 = 7</code>	<code>value: u32 = 7</code>	
<code>let mut log = format!(...)</code>	<code>value: u32 = 7</code> <code>log: String</code>	<code>"integer: 7"</code>
<code>let value = value.to_string()</code>	<code>value: String = "7"</code> (the <code>u32</code> binding is unreachable)	<code>"integer: 7\nstring: 7"</code>
<code>let value = value.len()</code>	<code>value: usize = 1</code> (the <code>String</code> binding is unreachable)	<code>"integer: 7\nstring: 7\nusize: 1"</code>
<code>return log</code>		

the function returns `"integer: 7\nstring: 7\nusize: 1"`, and the test asserts on the three lines separately

demonstrate_mutation

statement	bindings	log
<code>let mut count=0</code>	<code>count: i32 = 0</code>	
<code>let mut log</code>	<code>count: i32 = 0</code> <code>log: String</code>	<code>"mut count:"</code>
iteration 1	<code>count = 1</code>	<code>"mut count: 1"</code>
iteration 2	<code>count = 2</code>	<code>"mut count: 1 2"</code>
iteration 3	<code>count = 3</code>	<code>"mut count: 1 2 3"</code>

one binding for `count`, changed three times

Time and Space Complexity

Resource	Cost
Shadowing	no run-time cost; the old binding is dropped at the end of its scope, if it owns anything
Mutation	no run-time cost beyond the write itself
<code>format!</code>	one <code>String</code> allocation per call

The only allocation in either function is the `log`, which is built with `format!` and `push_str`. The conversion `value.to_string()` also allocates a `String`, and it is freed at the end of `demonstrate_shadowing` when the last binding to it goes out of scope.

Limitations

The comment says the old binding is "gone"; the value is not necessarily freed. Shadowing hides a name. The original binding's storage lasts until the end of its scope, and for a type with a destructor the resource is released at the end of the scope, not at the shadowing `let`. For `u32` and `String` in this file the distinction has no observable effect; for a file handle or a lock guard it does, because the guard stays alive until the end of the block. The comment's wording is more absolute than the rule.

Shadowing hides mistakes as well as values. A `let` that accidentally reuses a name produces no warning, and the compiler's unused-variable lint may not fire because the first binding was used before it was shadowed. The mechanism is powerful and it is not checked.

The file has no test for the shadowed value being unreachable. The tests assert on the log's contents, which proves that each binding was used. Nothing demonstrates the *lifetime* point, that an owned resource in a shadowed binding is released at the end of the scope, not at the shadowing statement.

Two `demonstrate_*` functions return a `String` built by `format!`. A version that took `&mut String` and appended to the caller's buffer would allocate less, and the tests would be the same. For a demonstration the difference is irrelevant; it is the kind of allocation that shows up in a profile.

Summary

- `let` introduces a binding and `mut` allows a place to be written. The two are independent.
- The conversion idiom is the reason shadowing appears in ordinary code. Parsing input and reusing the name avoids inventing a second name for the same value.
- Shadowing does not destroy the shadowed binding. Its storage lasts until the end of the scope, so a value with a destructor is released at the end of the block rather than at the shadowing statement.
- A `mut` binding fixes the type, which is why a counter uses `mut` and a conversion uses `let`.

- Shadowing hides mistakes as well as values: a `let` that reuses a name by accident produces no warning.

References

- The Rust Book, [Shadowing](#).
- The Rust Book, [Variables and mutability](#).
- The Rust Reference, [Names, scopes, and shadowing](#).

Part VII: Files, Directories, and Arguments

Reading and Writing Files

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/read_write_file.rs`, `src/bin/append_to_file_open_options.rs`, and `src/bin/readfile_line_by_line.rs`. Run them with `cargo run --bin read_write_file`, `cargo run --bin append_to_file_open_options`, and `cargo run --bin readfile_line_by_line`.

Problem Statement

Three programs cover the file operations a command-line tool needs: write a file from a string, add lines to an existing file, and read a file back, either whole or one line at a time. Each reports a failure through `io::Result` rather than terminating on the first error.

Designing a Solution

`std::fs` provides the whole-value operations. `fs::write` creates a file or replaces its contents in one call, and `fs::read_to_string` reads an entire file into a `String`. Both return `io::Result`, so `?` propagates a failure to the caller, and returning that result from `main` prints the error and sets a nonzero exit status.

Appending needs `OpenOptions`, because `fs::write` truncates. `OpenOptions::new()` assembles the flags for a single `open` call: `create(true)` makes the file when it is missing, and `append(true)` places every write at the end. Two `writeln!` calls then add two lines without reading the existing contents.

Reading line by line uses a `BufReader`. The reader owns a buffer and fills it in blocks, so `lines()` does not issue one system call per line. `BufRead` must be in

scope for the method to resolve, even though the value is a `BufReader`; that is the only trait import the program needs. Each item is an `io::Result<String>` without its terminator, so an unreadable line is reported rather than skipped.

Implementation

`read_write_file.rs` writes a string and reads it back through two paths built in different ways:

```
use std::fs;
use std::io;
use std::path::{Path, PathBuf};

// `&Path` accepts both a `&Path` and a `&PathBuf` argument; `&PathBuf` would not.
fn read_content_of_file(path: &Path) -> io::Result<String> {
    fs::read_to_string(path)
}

fn main() -> io::Result<()> {
    fs::write("hello.txt", "Hello, NVIDIA!\n")?;

    // `join` builds a new PathBuf and leaves `base_dir` unchanged.
    let base_dir = PathBuf::from(".");
    let joined_path = base_dir.join("hello.txt");

    // `push` mutates the PathBuf in place instead of returning a new one.
    let mut mut_base_dir = PathBuf::from(".");
    mut_base_dir.push("hello.txt");

    let text = read_content_of_file(&joined_path)?;
    println!("File contents (via .join()):\n{}", text);

    let text_inplace = read_content_of_file(&mut_base_dir)?;
    println!("File contents (via .push()):\n{}", text_inplace);

    Ok(())
}
```

`append_to_file_open_options.rs` adds two lines to a file that survives the run:

```
use std::fs::OpenOptions;
use std::io::Write;

fn main() -> std::io::Result<()> {
    let mut file = OpenOptions::new()
        .create(true) // create if missing
        .append(true) // always write at the end
        .open("log.txt")?;

    writeln!(file, "New log line")?;
    writeln!(file, "Another line")?;

    Ok(())
}
```

`readfile_line_by_line.rs` numbers the lines as it prints them:

```

use std::fs::File;
use std::io::{BufRead, BufReader}; // BufRead brings `lines()` into scope.

fn main() -> std::io::Result<()> {
    let file = File::open("log.txt");
    let reader = BufReader::new(file);

    for (i, line) in reader.lines().enumerate() {
        let line = line?;
        println!("{}", i + 1, line);
    }

    Ok(())
}

```

`read_content_of_file` takes `&Path` rather than `&PathBuf`. A `&PathBuf` coerces to `&Path` at the call site, but a `&Path` does not coerce to `&PathBuf`, so the `&Path` signature accepts strictly more callers: both `&joined_path` and `&mut_base_dir` below borrow as `&PathBuf`, and the coercion is what lets a `&Path`-typed parameter accept both. `fs::read_to_string` already returns `io::Result<String>`, the function's own return type, so the body returns that value directly rather than unwrapping it with `?` and rewrapping it in `Ok`.

`OpenOptions` is consumed by `open`, which is why the builder chain ends in a single statement. The two flags are independent: `create` decides what happens when the file is absent, and `append` decides where each write lands. Without `create`, opening a missing `log.txt` returns `ErrorKind::NotFound`.

Intuition

`read_write_file.rs`, from an empty working directory:

statement	effect
<code>fs::write(...)</code>	<code>hello.txt</code> is created with <code>"Hello, NVIDIA!\n"</code>
<code>PathBuf::from(".").join(...)</code>	<code>joined_path = "./hello.txt"</code>
<code>push("hello.txt")</code>	<code>pushed_path = "./hello.txt"</code>
<code>read_content_of_file(joined)</code>	returns <code>"Hello, NVIDIA!\n"</code>
<code>read_content_of_file(pushed)</code>	returns the same string

`append_to_file_open_options.rs`, run twice with `log.txt` absent at the start:

```

run 1  log.txt -> "New log line\nAnother line\n"
run 2  log.txt -> "New log line\nAnother line\nNew log line\nAnother line\n"

```

the file is opened for append, so the second run adds to the end rather than replacing the contents

`readfile_line_by_line.rs` over that file:


```
1: New log line
2: Another line
```

The index is one-based because `enumerate` starts at zero and the program prints `index + 1`.

Time and Space Complexity

Operation	Cost	Condition
<code>fs::read_to_string</code>	$O(n)$ time, $O(n)$ space	n bytes in the file; the whole file is held in memory
<code>fs::write</code>	$O(n)$ time	one truncation followed by the writes
<code>writeln!</code> on an append handle	one write per call	the handle is not line-buffered; each call reaches the kernel
<code>BufReader::lines</code>	$O(n)$ time, $O(1)$ space in the buffer	one <code>String</code> allocation per line

`read_to_string` fails when the bytes are not valid UTF-8, and it allocates the whole file before returning. A program that reads a large file should keep the `File` and read through the `BufReader` rather than calling `read_to_string`.

Limitations

Neither writer is atomic. A process that stops between the truncation and the writes leaves a partial file. Replacing a file without that window is done by writing a temporary file in the same directory and renaming it over the target.

No call requests a flush. `fs::write` and `writeln!` return once the data has reached the operating system, not the storage device, so a power loss can lose recent writes. `File::sync_all` is the call that waits for the device.

Relative paths depend on the working directory. All three programs use names such as `hello.txt` and `log.txt`, which resolve against the directory the program was started in. Running `readfile_line_by_line` before `append_to_file_open_options` reports `NotFound`.

lines() splits on one separator. It divides on `\n` and removes a trailing `\r`, so a file separated by `;` or by a bare `\r` is read as one line. The final line is yielded even when it has no terminator.

The two writers ignore the returned length. `writeln!` can write fewer bytes than requested; the callers rely on `io::Write` to report that as an error, and neither program checks how many bytes reached the file.

Summary

- `fs::write` replaces contents, `OpenOptions` with `append(true)` adds to them, and the difference is the flag rather than a separate function.
- `?` converts an `io::Error` into the caller's error type, which is what lets these programs read as a sequence of statements.
- `BufReader` turns many small reads into block reads, and `BufRead` has to be imported for `lines` to resolve.
- Relative file names resolve against the working directory, so these programs are not independent of where they are started.

References

- Standard library, `std::fs`.
- Standard library, `OpenOptions`.
- Standard library, `BufReader`.
- Standard library, `BufRead::lines`.
- Standard library, `PathBuf`.

Paths and Directory Traversal

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/path_buff.rs`, `src/bin/list_directory.rs`, and `src/bin/recursive_directory_walk.rs`. Run them with `cargo run --bin path_buff`, `cargo run --bin list_directory`, and `cargo run --bin recursive_directory_walk`.

Problem Statement

Three programs answer the questions a tool needs about the filesystem before it opens anything: how a path is assembled and taken apart, what a directory contains, and how to visit every file below a starting directory.

Designing a Solution

A path is a sequence of components, not a string. `Path` is the borrowed form and `PathBuf` is the owned form, with the same relationship as `str` and `String`. `join` builds a new `PathBuf` and leaves the receiver alone; `push` extends the receiver in place. Because a directory separator differs between platforms, the two methods insert the platform's separator, which string concatenation does not.

The inspection methods return `Option` rather than a sentinel, because a path need not have the component asked for. `/tmp/data.txt` has a parent, a file name and an extension; `/` has none of the three. Returning `None` for those cases keeps the caller from inventing a value.

`fs::read_dir` yields one `io::Result<DirEntry>` per directory entry. The `Result` matters when an entry cannot be read, which `for entry in fs::read_dir(dir)?` would otherwise hide. `DirEntry::path` produces an owned `PathBuf`, and

`DirEntry::file_type` returns the entry's type from the directory listing, which avoids the metadata lookup that `Path::is_dir` performs per entry.

A recursive walk is the same loop with a call to itself for each subdirectory, and `?` on that call stops the walk at the first unreadable directory.

Implementation

`path_buff.rs` builds one path with `join` and another with `push`, then reads parts of the second:

```
use std::path::{Path, PathBuf};

fn main() {
    let dir = Path::new("/tmp");
    let file = dir.join("data.txt");

    let mut buf = PathBuf::from(dir);
    buf.push("nested");
    buf.push("file.txt");

    println!("file: {}", file.display());
    println!("buf:  {}", buf.display());
    println!("parent: {:?}", buf.parent());
    println!("file_name: {:?}", buf.file_name());
    println!("ext: {:?}", buf.extension());
}
```

`list_directory.rs` prints one line per entry of the working directory:

```
use std::fs;
use std::io;

fn main() -> io::Result<()> {
    for entry in fs::read_dir(".")? {
        let entry = entry?;
        let path = entry.path();

        if path.is_dir() {
            println!("[DIR]  {}", path.display());
        } else {
            println!("[FILE] {}", path.display());
        }
    }
    Ok(())
}
```

`recursive_directory_walk.rs` repeats that loop for every subdirectory:

```
use std::fs;
use std::io;
use std::path::Path;

fn walk(dir: &Path) -> io::Result<()> {
    for entry in fs::read_dir(dir)? {
```

```

    let entry = entry?;
    let path = entry.path();
    let file_type = entry.file_type()?;

    if file_type.is_dir() {
        println!("[DIR] {}", path.display());
        walk(&path)?;
    } else {
        println!("[FILE] {}", path.display());
    }
}
Ok(())
}

fn main() -> io::Result<()> {
    walk(Path::new("."))?;
    Ok(())
}

```

`Path::display` is the conversion for printing. `Path` is not guaranteed to be valid UTF-8, so it does not implement `Display`; `display` returns a wrapper that replaces unrepresentable bytes, which is what a terminal needs.

`DirEntry::file_type` is used in the walk while the single-directory program uses `Path::is_dir`. The two differ for a symbolic link: `file_type` reports the link itself, and `is_dir` follows the link. The walk therefore prints a link to a directory as `[FILE]` and does not descend into it, which is also what keeps a link cycle from producing an unbounded walk.

Intuition

`path_buff.rs` on a Unix host:

```

dir                "/tmp"
dir.join("data.txt")  "/tmp/data.txt"
PathBuf::from(dir)   "/tmp"
push("nested")       "/tmp/nested"
push("file.txt")     "/tmp/nested/file.txt"

file: /tmp/data.txt
buf:  /tmp/nested/file.txt
parent: Some("/tmp/nested")
file_name: Some("file.txt")
ext: Some("txt")

```

`list_directory.rs` in a directory holding `Cargo.toml` and `src`:

```

[FILE] ./Cargo.toml
[DIR]  ./src

```

`recursive_directory_walk.rs` from the same directory:

```
[FILE] ./Cargo.toml
[DIR]  ./src
[FILE] ./src/lib.rs
[FILE] ./src/main.rs
```

Neither listing sorts its output, so the order above is one possible order rather than the order the filesystem guarantees.

Time and Space Complexity

Operation	Cost	Condition
<code>Path::join</code> , <code>PathBuf::push</code>	$O(\text{length})$ time, one allocation for join	the path is copied into the result
<code>read_dir</code> iteration	one directory read plus one <code>DirEntry</code> per entry	the order is unspecified
<code>DirEntry::file_type</code>	no extra metadata call on most platforms	the type comes with the entry
<code>Path::is_dir</code>	one metadata call per call	it resolves the path again
recursive walk	$O(\text{entries})$ time, $O(\text{depth})$ stack	one frame per open directory

Limitations

The walk has no cycle protection beyond not following links. `entry.file_type` does not follow a symbolic link, so a link to a directory is not entered. A bind mount or a hard-linked directory can still present a cycle, and the walk would not terminate.

Depth is bounded by the stack. `walk` calls itself once per directory level, so a tree several thousand levels deep exhausts the stack. An explicit stack of `PathBuf` values removes that limit.

One unreadable directory stops the walk. `walk(&path)?` propagates the error to `main`, so a permission failure at one level abandons the remaining directories. A tool that should continue past such an entry has to collect the error and resume.

The order is unspecified and unsorted. `read_dir` returns entries in the order the filesystem supplies. A reproducible listing has to collect the entries and sort them, which the programs do not do.

Relative names depend on the working directory. Both directory programs walk `"."`, so their output changes with the directory they are started in, and `list_directory` does not recurse.

Summary

- `Path` and `PathBuf` are the borrowed and owned forms of a path, and `join` is the non-mutating builder against `push`.
- The inspection methods return `Option`, because the component asked for may not exist.
- `DirEntry::file_type` avoids a per-entry metadata call, and its refusal to follow symbolic links is what keeps the walk finite.
- The recursive walk trades an explicit stack for call frames, so its depth is limited by the stack.

References

- Standard library, `std::path`.
- Standard library, `PathBuf::push`.
- Standard library, `std::fs::read_dir`.
- Standard library, `DirEntry::file_type`.

File Operations and Errors

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/copy_rename_delete_file.rs` and `src/bin/file_error_handling.rs`. Run them with `cargo run --bin copy_rename_delete_file` and `cargo run --bin file_error_handling`.

Problem Statement

Two programs cover the mutating file operations and the other half of the interface: copy, rename and delete a file, and decide what to report when a read fails.

Designing a Solution

`fs::copy` reads one file and writes another, returning the number of bytes copied. `fs::rename` changes a name, and on one filesystem it is an atomic replacement: the target either has the old contents or the new ones, never a mixture. `fs::remove_file` removes a name for a file, not a directory.

Errors are values. `io::Error` carries an `ErrorKind`, which is the part a caller can branch on without examining the message text. `ErrorKind::NotFound` is the common case for a path that does not exist, and `Err(e) if e.kind() == io::ErrorKind::NotFound` matches it before the general error arm. The guard is the mechanism: the arm is only selected when its condition holds, so the ordering of the arms is explicit rather than incidental.

Implementation

`copy_rename_delete_file.rs` performs the three operations in sequence:


```

use std::fs;
use std::io;

fn main() -> io::Result<()> {
    fs::write("a.txt", "hello")?;

    fs::copy("a.txt", "b.txt")?;
    fs::rename("b.txt", "c.txt")?;
    fs::remove_file("a.txt")?;

    println!("Done");
    Ok(())
}

```

`file_error_handling.rs` turns a failed read into a message that depends on the kind of failure:

```

use std::fs;
use std::io;

fn read_file(path: &str) -> Result<String, io::Error> {
    fs::read_to_string(path)
}

fn main() {
    match read_file("missing.txt") {
        Ok(text) => println!("{}", text),
        Err(e) if e.kind() == io::ErrorKind::NotFound => {
            eprintln!("File not found");
        }
        Err(e) => eprintln!("Error: {}", e),
    }
}

```

Each call in the first program is a fallible operation, and `?` returns early on the first failure. That is the correct default for a sequence in which each step depends on the previous one: copying before the file exists has no meaning.

In the second program the error is matched rather than propagated, because `main` returns `()`. A `main` that returns `Result` instead would let `?` do the work, at the cost of printing the error through the runtime's `Debug` implementation rather than the message chosen here. The `eprintln!` writes to standard error, which keeps diagnostics out of a pipeline that reads standard output.

Intuition

`copy_rename_delete_file.rs` from an empty directory:

statement	files afterwards
<code>fs::write("a.txt", ...)</code>	a.txt
<code>fs::copy("a.txt", "b.txt")</code>	a.txt b.txt

```
fs::rename("b.txt","c.txt") a.txt  c.txt
fs::remove_file("a.txt")    c.txt
println!("Done")           prints Done
```

`file_error_handling.rs` with no `missing.txt`:

```
read_file("missing.txt")  Err(kind = NotFound)
first arm                 does not match Ok
guarded arm               matches, prints "File not found" to stderr
main                      returns ()
```

Time and Space Complexity

Operation	Cost	Condition
<code>fs::copy</code>	$O(n)$ time and one extra file's worth of writes	n bytes copied; the kernel may choose the buffer size
<code>fs::rename</code>	$O(1)$ time	source and target are on one filesystem
<code>fs::remove_file</code>	$O(1)$ time	the name is unlinked; the data is freed when the last handle closes
<code>ErrorKind::match</code>	$O(1)$ time	the kind is a field of the error

Limitations

rename is not portable across filesystems. When the source and target are on different mounts, the operation fails rather than copying, and the error kind is platform specific. Moving between filesystems needs a copy followed by a remove, which is not atomic.

copy does not preserve everything. It writes the file contents and copies the permission bits, but it does not preserve ownership, timestamps, or extended attributes. A tool that needs those has to set them explicitly.

remove_file refuses a directory. A directory needs `remove_dir`, which itself requires the directory to be empty. A recursive delete has to walk the tree and remove children before parents.

A failure in the middle leaves the earlier steps applied. The first program returns as soon as a call fails, so a failure at `rename` leaves `a.txt` and

`b.txt` in place. Nothing here rolls back, and a tool that must leave the directory unchanged needs a temporary directory and a final rename.

The error message in the last arm is the library's. `eprintln!("{}", e)` prints `Display` for `io::Error`, which includes the operating system message but not the path. Adding the path is the caller's job, because the error does not carry it.

Summary

- `copy`, `rename` and `remove_file` are the mutating operations, and `rename` is the atomic replacement primitive within one filesystem.
- Errors are values; `ErrorKind` is the part a program can branch on, and a match guard is how the branch is written.
- `?` is the right default for a sequence of dependent steps, and an explicit match is the right form when the program chooses the message.
- Diagnostics go to standard error, which keeps standard output usable as data.

References

- Standard library, `fs::copy`.
- Standard library, `fs::rename`.
- Standard library, `fs::remove_file`.
- Standard library, `io::ErrorKind`.

Command-Line Arguments

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/command_line_args.rs`. Run it with `cargo run --bin command_line_args -- --name ada --count 3 --verbose`.

Problem Statement

A command-line tool receives its input as a list of strings from the operating system. The program has to know where that list starts and ends, which entry is the name of the program rather than an argument to it, and how to turn the remaining entries into the options the tool accepts. It also has to report a bad command line in a way the shell can see, which means a non-zero exit status.

Designing a Solution

`env::args()` returns an iterator over the arguments as `String`. The first item is the name the program was invoked with, so the arguments proper begin at index one and `args().skip(1)` is the boundary between the two.

The program parses three options. `--name` takes a value, `--count` takes a number, and `--verbose` is a flag. The parser walks the arguments once with an iterator, so a value option consumes the next item and a flag consumes only itself. It returns `Result<Options, String>`: every failure is a message that names the argument at fault, and `main` prints that message and a usage line to standard error before returning `ExitCode::FAILURE`. A successful run returns `ExitCode::SUCCESS`, so the exit status is a fact the shell can test.

`main` returns `ExitCode` rather than `()`. The return value is the program's report to its caller, and the shell reads the status byte and branches on it. Returning

`ExitCode` keeps that decision in `main` instead of in a call to `std::process::exit`, which would skip the destructors of everything still alive.

The iterator has one failure mode worth knowing. `args()` panics when an argument is not valid UTF-8. `args_os` yields `OsString` instead and never panics, which is what a tool that forwards an arbitrary path should use.

Implementation

```
use std::env::args;
use std::process::ExitCode;

/// The options this program understands.
#[derive(Debug, PartialEq)]
struct Options {
    name: String,
    count: u32,
    verbose: bool,
}

impl Default for Options {
    fn default() -> Self {
        Self { name: String::from("world"), count: 1, verbose: false }
    }
}

/// Turn the arguments after the program name into `Options`.
fn parse(arguments: impl IntoIterator<Item = String>) -> Result<Options, String> {
    let mut options = Options::default();
    let mut arguments = arguments.into_iter();

    while let Some(argument) = arguments.next() {
        match argument.as_str() {
            "--name" => {
                options.name = arguments.next().ok_or("--name needs a value")?;
            }
            "--count" => {
                let value = arguments.next().ok_or("--count needs a value")?;
                options.count = value
                    .parse()
                    .map_err(|_| format!("--count needs a number, got {value:?}"))?;
            }
            "--verbose" => options.verbose = true,
            other => return Err(format!("unknown argument {other:?}")),
        }
    }

    Ok(options)
}

fn main() -> ExitCode {
    let options = match parse(args().skip(1)) {
        Ok(options) => options,
        Err(message) => {
            eprintln!("error: {message}");
            eprintln!("usage: command_line_args [--name NAME] [--count N] [--verbose]");
        }
    };

    return ExitCode::FAILURE;
};
```

```

    for index in 0..options.count {
        if options.verbose {
            println!("[{index}] hello, {}", options.name);
        } else {
            println!("hello, {}", options.name);
        }
    }

    ExitCode::SUCCESS
}

#[cfg(test)]
mod tests {
    use super::*;

    fn parse_args(arguments: &[&str]) -> Result<Options, String> {
        parse(arguments.iter().map(|value| value.to_string()))
    }

    #[test]
    fn defaults_apply_when_there_are_no_arguments() {
        assert_eq!(parse_args(&[]).unwrap(), Options::default());
    }

    #[test]
    fn a_value_option_consumes_the_next_argument() {
        let options = parse_args(&["--name", "ada", "--count", "3"]).unwrap();
        assert_eq!(options.name, "ada");
        assert_eq!(options.count, 3);
    }

    #[test]
    fn verbose_is_a_flag() {
        assert!(parse_args(&["--verbose"]).unwrap().verbose);
        assert_eq!(!parse_args(&[]).unwrap().verbose);
    }

    #[test]
    fn a_value_option_without_a_value_is_an_error() {
        assert!(parse_args(&["--name"]).is_err());
        assert!(parse_args(&["--count"]).is_err());
    }

    #[test]
    fn a_non_numeric_count_is_an_error() {
        assert!(parse_args(&["--count", "many"]).is_err());
    }

    #[test]
    fn an_unknown_argument_is_an_error() {
        assert!(parse_args(&["--colour", "red"]).is_err());
        assert!(parse_args(&["extra"]).is_err());
    }
}

```

The listing is the whole file. Taken in order:

- `parse` accepts any iterator of `String`, so the tests can pass a slice and `main` can pass `args().skip(1)`. The argument is not collected into a `Vec` first, so a value is consumed exactly when the parser reaches it.
- `--name` and `--count` both call `arguments.next()`. When the iterator is exhausted they receive `None`, and `ok_or` turns that into the message `--name`

needs a value. A flag that silently accepted a missing value would hide a typo.

- `--count` parses with `value.parse()`. `map_err` replaces the parser's error with one that quotes the offending text, because the caller needs to know which word was not a number.
- The final arm catches every other argument, including a stray positional one, and returns an error. An unknown option is a failure rather than something ignored, so a misspelled flag does not silently do nothing.
- `main` returns `ExitCode`. On success the loop prints `count` greetings, and `--verbose` prefixes each line with its index.

The `Default` implementation gives the program the values it uses when an option is absent. Defaults live in one place, so the parser only writes a field when the command line supplies one.

Intuition

With no arguments, the defaults apply:

```
$ cargo run --bin command_line_args
hello, world
```

With the options:

```
$ cargo run --bin command_line_args -- --name ada --count 3 --verbose
[0] hello, ada
[1] hello, ada
[2] hello, ada
```

A bad command line goes to standard error and sets the exit status:

```
$ cargo run --bin command_line_args -- --count many
error: --count needs a number, got "many"
usage: command_line_args [--name NAME] [--count N] [--verbose]
$ echo $?
1
```

`cargo run` passes everything after a bare `--` to the program, which is why the examples use it. Without the separator, `cargo` would treat `--name` as one of its own flags.

The process receives its arguments after the shell has removed quoting and performed expansion. `--name "two words"` arrives as one element, and `*` ar-

rives already expanded by the shell, so the program never sees the command line as a single string.

Time and Space Complexity

Operation	Cost	Condition
<code>env::args()</code>	no allocation until an item is taken	each item is a <code>String</code> when produced
<code>parse</code>	$O(n)$ time	one pass, one <code>next()</code> per option and value
<code>skip(1)</code>	$O(1)$	it drops the first item of the iterator
memory	$O(1)$ beyond the strings already produced	the parser keeps no list of its own

Collecting into a `Vec<String>` would cost one allocation per argument plus the vector, and it would hold the whole command line at once. The parser avoids both, because it only ever needs the current argument and, for a value option, the one after it.

Limitations

Only the separated form is accepted. `--name ada` is an option and its value; `--name=ada` is an unknown argument, because the match is on the whole word. Accepting the second form means splitting each argument at its first `=` before the match.

There is no `--` separator of its own. The program does not treat a bare `--` as the end of the options, so a value that begins with `--` cannot be passed as a name except by changing the parser.

There is no help flag. `--help` is reported as an unknown argument. A tool that ships to other people usually answers `--help` with the usage line and the exit status for success, rather than with the failure path.

`args()` panics on a non-UTF-8 argument. On Unix an argument is a byte string, and a file name that is not valid UTF-8 reaches the program as

bytes. A program that must accept such a name has to use `args_os` and handle `OsString`.

The program name is not the same as the executable. The first argument is whatever the caller passed as `argv[0]`, which is the path used to invoke the program and can be changed by the caller. It is not a reliable source of the program's location.

Precedence between sources is not implemented. A flag over an environment variable over a file over a default is the usual order, and a tool that reads more than one source has to state that order somewhere. This parser has one source.

Summary

- `env::args()` yields `String` values whose first item is the invocation name; `skip(1)` is the boundary between the program and its arguments.
- The parser walks the arguments once with an iterator, so a value option consumes the next item and a flag consumes only itself.
- Every failure returns a message that names the argument at fault, and `main` turns it into standard error plus `ExitCode::FAILURE`.
- `args()` panics on non-UTF-8 input, and `args_os` is the version that does not.

References

- Standard library, `std::env::args`.
- Standard library, `std::env::args_os`.
- Standard library, `std::process::ExitCode`.
- The Rust Book, [Accepting command line arguments](#).

Paginating a File with a Byte Cursor

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/file_pagination.rs`. Run it from the `rust-interview-lab` directory with `cargo run --bin file_pagination`, so that the relative path in `main` resolves.

Problem Statement

A file holds more records, one per line, than a caller can receive at once. Serve it a page at a time: each request asks for at most `page_size` lines and gets back those lines together with a cursor, and passing that cursor to the next request resumes exactly where the previous page stopped. The last page must say that it is the last, and no request may read the whole file.

Designing a Solution

The obvious cursor is a line number, and it is the wrong one. Resuming at line `N` means reading and discarding `N` lines, so serving the last page of a million-line file costs a million lines of work, and serving the whole file page by page costs $O(n^2)$. A byte offset costs one `lseek` regardless of how far into the file it points, which makes every page cost the same.

That choice is what keeps the reader stateless. Nothing is remembered between calls: the file is opened, the offset is sought to, the page is read, and the handle is dropped. The cursor is the entire session, which is what an HTTP API needs when consecutive requests may land on different processes.

Two details decide whether the cursor is correct. `BufRead::read_line` returns the number of bytes it consumed, and that count includes the line terminator, so adding it to the offset gives the position of the next line rather than the position of its newline. `trim_end` then removes the terminator from the string that is handed back, and does not touch the count. Conflating the two, by advancing the offset with the trimmed length, moves the cursor one byte short of each line and repeats a newline on every page boundary.

Reporting the last page needs a look at what comes next without consuming it. `BufRead::fill_buf` returns the reader's buffer, filling it from the file if it is empty, and an empty slice means the file is exhausted. Checking it after the loop distinguishes a final page from a page that merely happens to be full, so the caller stops at the right moment instead of making one more request that returns nothing.

Implementation

```
use std::fs::File;
use std::io::{self, BufRead, BufReader, Seek, SeekFrom};

struct Page {
    jobs: Vec<String>,
    next_cursor: Option<u64> // Byte offset where the next page starts
}

fn read_page(path: &str, cursor: u64, page_size: usize) -> io::Result<Page> {
    let mut file = File::open(path)?;
    file.seek(SeekFrom::Start(cursor))?;
    let mut reader = BufReader::new(file);

    // Optimization which preserves as large as the max page size
    let mut jobs: Vec<String> = Vec::with_capacity(page_size);
    let mut offset = cursor;
    let mut line = String::new();

    while jobs.len() < page_size {
        // Clear the previously read line
        line.clear();

        match reader.read_line(&mut line)? {
            0 => break,
            bytes_read => {
                offset += bytes_read as u64;
                jobs.push(line.trim_end().to_string());
            },
        }
    }

    let at_end = reader.fill_buf()?.is_empty();
    let next_cursor = if at_end { None } else { Some(offset) };

    Ok(Page { jobs, next_cursor })
}

fn main() -> io::Result<()> {
```

```

let mut cursor = 0;

loop {
    let page = read_page("src/bin/large_file.txt", cursor, 3)?;

    println!("--- page ---");
    for job in &page.jobs {
        println!("{job}");
    }

    match page.next_cursor {
        Some(next) => cursor = next,
        None => break,
    }
}

Ok(())
}

```

`Page` is the return type rather than a tuple, so neither field can be read as the other. `next_cursor` is an `Option<u64>`, and `None` is the end of the file. An `Option` rather than a sentinel offset means the caller cannot accidentally continue past the end, because there is no value to pass.

`read_page` seeks before it wraps the file. The order matters: `File` implements `Seek`, and `BufReader` starts with an empty buffer, so seeking first and buffering second means the reader's first fill begins at the cursor. Seeking through the `BufReader` afterwards would also work, and it discards the buffer to do so.

`Vec::with_capacity(page_size)` sizes the page once, since the exact upper bound is known before the loop starts. `line.clear()` at the top of each iteration is required rather than tidy: `read_line` appends to the string it is given, so a string reused without clearing accumulates the whole page into its first entry.

The `match` on the byte count separates the two outcomes. Zero means end of file, which ends the loop early and leaves the page short. Any other count is both the evidence that a line arrived and the amount by which the offset moves, so the same value does both jobs.

`reader.fill_buf()?.is_empty()` is the end-of-file test, and it is a peek rather than a read: the bytes stay in the reader's buffer, which is then dropped along with the reader. The cost is at most one extra read of the underlying file per page.

Intuition

The sample file holds thirteen lines in fifty-eight bytes, and `main` asks for three lines at a time. The cursor after each page is the offset of the line that has not been read yet:

page	lines	bytes consumed	next_cursor
1	"First Line", "Second Line", "Thrid Line"	12 + 12 + 11	35
2	"4", "5", "6"	2 + 2 + 2	41
3	"7", "8", "9"	2 + 2 + 2	47
4	"10", "11", "12"	3 + 3 + 3	56
5	"13"	2	None

The first line is eleven characters and twelve bytes, because the trailing newline is counted in the offset and trimmed from the string. The fifth page stops after one line: `read_line` returns zero, the loop breaks early, and `fill_buf` then confirms the file is exhausted, so the cursor is `None` and the caller's loop ends. Running the program prints the five pages:

```

--- page ---
First Line
Second Line
Thrid Line
--- page ---
4
5
6
--- page ---
7
8
9
--- page ---
10
11
12
--- page ---
13

```

Time and Space Complexity

Resource	Cost	Condition
One page	$O(b)$	b bytes in the page; the seek is $O(1)$ regardless of the offset
The whole file, page by page	$O(n)$	each byte is read once, which a line-number cursor would not achieve
Space per call	8 KiB buffer, plus the page	<code>BufReader</code> 's default capacity, allocated and dropped per call

Resource	Cost	Condition
Syscalls per page	one open, one lseek, and $\text{ceil}(b / 8192) + 1$ reads	the final read is the fill_buf peek

Limitations

Every page opens the file and allocates a new buffer. `File::open` and an 8 KiB `BufReader` are paid on each call, and the buffer is discarded after serving a few hundred bytes. A server handling many small pages would keep a pool of open handles, or accept a `&mut BufReader` from its caller.

The file may change between pages. Nothing pins the file's contents, so an offset recorded for one version of the file can land in the middle of a line of another, and the first record of the following page is then a fragment. Cursors in a real API carry something that detects this, such as a generation number or the file's modification time, and reject a cursor that no longer applies.

A line has no length limit. `read_line` grows the string until it meets a newline, so a file with no newlines allocates its whole length into one `String`, whatever `page_size` says. `BufRead::take` with a cap, or `read_until` on a bounded buffer, puts a ceiling on it.

trim_end removes more than the terminator. It strips every trailing whitespace character, so records ending in a space or a tab come back altered. `trim_end_matches('\n')` followed by the same for `'\r'` trims exactly the terminator and nothing else.

Invalid UTF-8 ends the page with an error. `read_line` requires valid UTF-8 and returns `InvalidData` otherwise, which `?` propagates, discarding the lines already collected. A reader over arbitrary bytes uses `read_until(b'\n', &mut Vec<u8>)` and leaves the decoding to the caller.

The path is hard-coded and relative. `main` names `src/bin/large_file.txt`, so the program only works when run from the `rust-interview-lab` directory. Chapter 29 shows the argument handling that would replace it.

Summary

- A byte offset is the right cursor for a file, because seeking to it costs the same wherever it points, while a line number makes the last page the most expensive one.
- The offset advances by the count `read_line` returns, which includes the terminator that `trim_end` removes from the string.
- `fill_buf` peeks at what follows the page, which is what lets the last page be reported as the last one instead of being discovered by an empty request.
- The reader keeps no state between calls, so the cursor alone is enough to resume, and nothing prevents the file from changing underneath it.

References

- Standard library, `BufRead::read_line`.
- Standard library, `BufRead::fill_buf`.
- Standard library, `Seek::seek` and `SeekFrom`.
- Chapter 26, on reading and writing files, and chapter 24, on what a system call costs.

Part VIII: Coordination and Parallelism

A Bounded Buffer

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/bounded_buffer.rs`. Run it with `cargo run --bin bounded_buffer`.

Problem Statement

Build a queue with a fixed capacity that several threads share: producers must wait while the queue is full, and consumers must wait while it is empty. No thread may spin on a flag, and no item may be lost or handed out twice.

Designing a Solution

A `Mutex<VecDeque<T>>` makes the deque safe to share, but a lock alone cannot express waiting. A consumer that finds the queue empty has to release the lock and sleep, and it has to be woken when an item arrives. That is what a condition variable is for.

The queue uses two condition variables. `not_empty` is the one consumers wait on and producers signal after a push; `not_full` is the one producers wait on and consumers signal after a pop. With a single variable, every waiter would be woken by every change, and the threads of the wrong kind would find their own condition still false and go back to sleep.

`Condvar::wait` takes the guard by value. It releases the lock, parks the thread, and returns a new guard once the lock has been reacquired. That reacquisition is why the condition has to be checked in a `while` loop rather than an `if`: the thread can wake without a signal, and another thread can take the slot between the wake-up and the reacquisition. The `while` re-checks the condition

with the lock held, so the thread only proceeds when the state is what it observed.

`notify_one` is used rather than `notify_all`. Each signal follows exactly one state change, one item added or one slot freed, so exactly one waiter can make progress. Waking every waiter would produce a crowd that immediately re-checks the condition and sleeps again.

Implementation

```
use std::collections::VecDeque;
use std::sync::{Arc, Condvar, Mutex};
use std::thread;
use std::time::Duration;

// A thread-safe bounded buffer (a "shelf" that holds at most `capacity` items).
//
// Three fields do all the work:
// - `inner` : the actual VecDeque, wrapped in a Mutex so only one thread
//             touches it at a time.
// - `not_empty` : waiting room for CONSUMERS. A consumer sleeps here when the
//                 shelf is empty. Producers ring this bell after pushing.
// - `not_full` : waiting room for PRODUCERS. A producer sleeps here when the
//                shelf is full. Consumers ring this bell after popping.
//
// Two separate condvars so we always wake the RIGHT kind of thread.
// One shared condvar would cause livelock: consumers waking consumers,
// producers waking producers, everyone re-checking and going back to sleep.
struct BoundedQueue<T> {
    inner: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    fn new(capacity: usize) -> Self {
        // Capacity 0 makes no sense: no producer could ever push,
        // and every pop would block forever.
        assert!(capacity > 0, "capacity must be > 0");

        Self {
            inner: Mutex::new(VecDeque::with_capacity(capacity)),
            capacity,
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    fn push(&self, item: T) {
        // Acquire the shelf lock. `guard` is our exclusive handle to the deque.
        let mut guard = self.inner.lock().unwrap();

        // Wait while the shelf is full.
        //
        // Why `while` and not `if`:
        // 1. Condvars are allowed to wake up for no reason (spurious wakeup).
        // 2. Even on a real wakeup, another producer may have raced us and
        //    taken the free slot before we reacquired the lock.
        // So we must re-check the condition every time we wake.
```

```

//
// Why `guard = ...wait(guard)`?
//   `wait` consumes the guard (it releases the lock on our behalf),
//   puts us to sleep, then re-locks and hands us a NEW guard.
//   The reassignment just means "here's my fresh key to the shelf."
//   The VecDeque inside is the same one – we're not replacing the queue.
while guard.len() == self.capacity {
    guard = self.not_full.wait(guard).unwrap();
}

// We have room. Add the item to the back of the deque.
guard.push_back(item);

// Release the lock BEFORE ringing the bell.
// This is an optimization, not a correctness requirement.
// If we notify while still holding the lock, the thread we wake will
// immediately block on `lock()` until we release it – a wasted wakeup.
drop(guard);

// Ring the "not empty" bell: one consumer can now make progress.
// `notify_one` (not `notify_all`) because exactly ONE item was added,
// so only ONE consumer can succeed. Waking all of them = stampede.
self.not_empty.notify_one();
}

fn pop(&self) -> T {
    // Acquire the shelf lock.
    let mut guard = self.inner.lock().unwrap();

    // Wait while the shelf is empty.
    // Same `while` reasoning as push: spurious wakeups + races with other
    // consumers who may have grabbed the item before we reacquired the lock.
    while guard.is_empty() {
        guard = self.not_empty.wait(guard).unwrap();
    }

    // Safe to unwrap: the `while` loop guarantees the deque is non-empty,
    // and we hold the lock so nobody can change that between the check
    // and this line.
    let item = guard.pop_front().unwrap();

    // Release before notifying (same optimization as in push).
    drop(guard);

    // Ring the "not full" bell: one producer can now make progress.
    // Exactly one slot opened, so wake exactly one producer.
    self.not_full.notify_one();

    item
}

fn main() {
    // Small capacity (3) so that with two slow consumers you can actually
    // SEE producers block: the queue fills up and push() sleeps in `not_full`.
    let queue = Arc::new(BoundedQueue::new(3));

    let mut handles = vec![];

    // ---- Producers ----
    // Two producers, each pushes 5 items. `Arc::clone` bumps the refcount so
    // each thread shares the same queue; ownership of the clone moves in.
    for p in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for i in 0..5 {
                let item = format!("p{}:item{}", p, i);

```

```

println!(" [producer {}] pushing {}", p, item);
q.push(item);
// Slow down so we don't blast through the whole buffer instantly.
thread::sleep(Duration::from_millis(30));
    }
    }));
}

// ---- Consumers ----
// Two consumers, each pops 5 items. Consumers are SLOWER than producers
// here (60ms vs 30ms), so the queue fills up and you'll see producers
// stall on `not_full.wait(...)`.
for c in 0..2 {
    let q = Arc::clone(&queue);
    handles.push(thread::spawn(move || {
        for _ in 0..5 {
            let item = q.pop();
            println!("[consumer {}] got {}", c, item);
            thread::sleep(Duration::from_millis(60));
        }
    }));
}

// Join everything. If our synchronization is wrong, this is where you'd
// hang (a thread sleeping forever) or panic (unwrap on empty / on a
// poisoned mutex after another thread panicked while holding it).
for h in handles {
    h.join().unwrap();
}
println!("done");
}

```

The lock is released with `drop(guard)` before `notify_one`. Holding the lock while signalling does not lose the wake-up, but the thread that is woken immediately blocks on `lock` until the signaller releases it, so the signal costs a context switch that accomplishes nothing.

`Arc` is what lets two threads own the same queue. The clone is passed to the thread and the original stays in `main`; the queue is freed when the last handle is dropped.

The variant that returns an error instead of panicking

Two of the limitations above are the same limitation seen from different sides. A thread that panics while holding the lock poisons the mutex, every later `lock()` returns `Err`, and the `unwrap` turns that into a second panic that takes down a thread which did nothing wrong. A queue meant to be used by other people has to hand the failure back to its caller instead. The version below does that, and it carries the tests that prove it.

Source file: `src/bin/bounded_buffer_with_error_handling.rs`. Run it with `cargo run --bin bounded_buffer_with_error_handling`, and test it with `cargo test --bin bounded_buffer_with_error_handling`.

```

use std::collections::VecDeque;
use std::fmt;
use std::sync::{Arc, Condvar, Mutex, PoisonError};
use std::thread;
use std::time::Duration;

#[derive(Debug)]
pub struct QueuePoisonedError;

impl fmt::Display for QueuePoisonedError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "The queue's internal mutex was poisoned by a panicking thread")
    }
}

impl std::error::Error for QueuePoisonedError {}

trait PoisonMap<T> {
    fn map_poison(self) -> Result<T, QueuePoisonedError>;
}

impl<T> PoisonMap<T> for Result<T, PoisonError<T>> {
    fn map_poison(self) -> Result<T, QueuePoisonedError> {
        self.map_err(|_| QueuePoisonedError)
    }
}

struct BoundedQueue<T> {
    inner: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            inner: Mutex::new(VecDeque::with_capacity(capacity)),
            capacity,
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    fn push(&self, item: T) -> Result<(), QueuePoisonedError> {
        let mut guard = self.inner.lock().map_poison()?;
        while guard.len() == self.capacity {
            guard = self.not_full.wait(guard).map_poison()?;
        }
        guard.push_back(item);
        drop(guard);
        self.not_empty.notify_one();
        Ok(())
    }

    fn pop(&self) -> Result<T, QueuePoisonedError> {
        let mut guard = self.inner.lock().map_poison()?;
        while guard.is_empty() {
            guard = self.not_empty.wait(guard).map_poison()?;
        }
        let item = guard.pop_front().unwrap();
        drop(guard);
        self.not_full.notify_one();
        Ok(item)
    }
}

```

```

fn main() {
    let queue = Arc::new(BoundedQueue::new(3));
    let mut handles = vec![];

    // ---- Producers ----
    for p in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for i in 0..5 {
                let item = format!("p{}:item{}", p, i);
                println!("[producer {}] pushing {}", p, item);

                // Uses match, avoids naming an error variable, uses dbg!
                match q.push(item) {
                    Ok(_) => {}
                    Err(err) => {
                        dbg!(err);
                        break;
                    }
                }
                thread::sleep(Duration::from_millis(30));
            }
        }));
    }

    // ---- Consumers ----
    for c in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for _ in 0..5 {
                // Uses match, avoids naming an error variable, uses dbg!
                match q.pop() {
                    Ok(item) => {
                        println!("[consumer {}] got {}", c, item);
                    }
                    Err(err) => {
                        dbg!(err);
                        break;
                    }
                }
                thread::sleep(Duration::from_millis(60));
            }
        }));
    }

    // Join everything.
    for h in handles {
        match h.join() {
            Ok(_) => {}
            Err(err) => {
                dbg!(err);
            }
        }
    }
    println!("done");
}

#[cfg(test)]
mod tests {
    use super::*;

    // 1. Mark the test as one that MUST panic to pass
    // 2. Optional: use 'expected' to verify it panics with the correct error message
    #[test]
    #[should_panic(expected = "capacity must be > 0")]
    fn test_zero_capacity_panics() {

```

```

// This line invokes the assert!(capacity > 0) inside new()
let _queue: BoundedQueue<i32> = BoundedQueue::new(0);
}

#[test]
fn test_queue_poisoning_on_thread_panic() {
    // 1. Create a queue shared via Arc so multiple threads can touch it
    let queue = Arc::new(BoundedQueue::new(5));
    let queue_clone = Arc::clone(&queue);

    // 2. Spawn a thread designed to crash while holding the internal lock
    let handle = thread::spawn(move || {
        // Manually lock the inner mutex so this thread owns the lock guard
        let _guard = queue_clone.inner.lock().unwrap();

        // Trigger a serious runtime panic while holding the lock guard!
        panic!("Intentional worker thread crash!");
    });

    // 3. Wait for the thread to die. It will return an Err because it panicked.
    let join_result = handle.join();
    assert!(join_result.is_err(), "The thread was supposed to panic");

    // 4. Now try to interact with the queue.
    // Because the thread panicked while holding the guard, the Mutex is poisoned.
    // Our push and pop methods should cleanly catch this and return our custom
    error.
    match queue.push("test_item") {
        Err(QueuePoisonedError) => {
            // Success! The queue correctly caught the poisoned state
            // instead of panicking the main thread.
        }
        Ok(_) => {
            panic!("Expected QueuePoisonedError, but push succeeded!");
        }
    }

    // Verify pop behaves the same way
    match queue.pop() {
        Err(QueuePoisonedError) => {}
        Ok(_) => panic!("Expected QueuePoisonedError, but pop succeeded!"),
    }
}
}

```

`QueuePoisonedError` is a unit struct with a `Display` implementation and an empty `std::error::Error` implementation, which is the minimum a type needs to travel inside a `Box<dyn Error>` or to be printed by a caller that knows nothing about the queue. It carries no data, because there is nothing useful to say beyond the fact that the queue can no longer be trusted.

The conversion from the standard library's error is done by an extension trait rather than by a free function. Both `Mutex::lock` and `Condvar::wait` fail with a `PoisonError<T>`, but `T` differs between them: `lock` gives back a `MutexGuard`, and `wait` gives back the guard it was handed. Writing `map_poison` as a method on `Result<T, PoisonError<T>>` covers both call sites with one implementation, and leaves `?` usable at the end of each line.

Discarding the `PoisonError` is a decision worth naming. It holds the guard, so a caller that wanted to could reach through it with `into_inner` and use the data the panicking thread left behind. Dropping it says that this queue treats poisoning as permanent: the contents may be half-updated, and no caller should act on them.

The panicking thread no longer costs the whole program. `push` and `pop` return `Result`, the producers and consumers match on it, and a poisoned queue ends each of their loops with a `break` rather than an abort. What the variant does not fix is the third limitation: there is still no way to close the queue, so a consumer that outlives the producers still waits forever. Chapter 46 adds that.

The tests cover the two failures the original had no way to express. `#[should_panic(expected = "capacity must be > 0")]` pins the assertion in `new`, and the `expected` string means the test fails if the code panics for some other reason. The second test poisons the queue on purpose: a thread locks the inner mutex, panics while holding the guard, and after `join` reports the panic, both `push` and `pop` are checked for the error. Matching `Err(QueuePoisonedError)` as a pattern works because the error is a unit struct, so its name is both the type and its only value.

Intuition

With `BoundedQueue::new(1)` and one producer and one consumer, the state after each operation is:

step	queue	producer	consumer
<code>push(1)</code>	<code>[1]</code>	running	
<code>pop()</code>	<code>[]</code>	running	takes 1
<code>push(2)</code>	<code>[2]</code>	running	
<code>pop()</code>	<code>[]</code>	running	takes 2
if the consumer starts first:			
<code>pop()</code>	<code>[]</code>	-	waits on <code>not_empty</code>
<code>push(1)</code>	<code>[1]</code>	running	wakes, takes 1

The consumer that finds the queue empty parks on `not_empty` and holds no lock while parked, so the producer can take the lock and push.

Time and Space Complexity

Operation	Cost	Condition
<code>push</code> , <code>pop</code>	$O(1)$	one lock, at most one signal

Operation	Cost	Condition
block and wake	one context switch	the lock is released while the thread is parked
memory	one deque at capacity, plus one word per blocked thread	the blocked threads are in the kernel, not in the queue
memory	$O(n)$ per VecDeque	the buffer is sized at construction and does not grow

The mutex is held for the length of the critical section, which is a deque operation and a length check. No work that can block is performed under the lock, which is what keeps the queue from serialising its callers.

Limitations

There is no way to close the queue. A consumer that calls `pop` after the producers have finished waits forever, and with no `close` method there is no state that would release it. A production queue needs a shutdown flag, checked inside the same loop, and a `notify_all` when it is set.

Poisoning is handled with `unwrap`. A thread that panics while holding the lock marks the mutex poisoned, and every later `lock()` returns `Err`. The `unwrap` turns that into a second panic rather than a recoverable error, which is acceptable for a demonstration and not for a library.

There is no fairness guarantee. `notify_one` may wake any waiter, so a pushed item can be taken by a later arrival while an earlier consumer is still parked. A fair queue needs an ordered wait list, which this one does not provide.

The program's output is interleaved. The print statements run outside the lock, so the two producers can print in either order and the lines from different threads can be adjacent. The set of lines is fixed; the order is not.

The file has no tests. Correctness here is a property of the interleaving, and the tests that would catch a missing `while` are the ones that run a producer and a consumer concurrently rather than the ones that assert on a single-threaded push and pop.

Summary

- A mutex makes the deque shared; condition variables make waiting possible.
- The condition is re-checked in a `while` loop, because a wake-up does not prove the condition is true.
- Two condition variables keep each signal directed at a thread that can act on it; `notify_one` follows each single state change.
- The queue has no shutdown path, and a consumer that outlives the producers waits indefinitely.
- Returning a `Result` from `push` and `pop` keeps a panicking producer from taking down the threads that were using the queue correctly.
- An extension trait on `Result<T, PoisonError<T>>` converts both the `lock` and the `wait` failure with one implementation.

References

- Standard library, `std::sync::Condvar`.
- Standard library, `Condvar::wait`.
- Standard library, `std::sync::Mutex`.
- The Rust Book, [Shared-state concurrency](#).

Parallel Sum

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/parallel_sum.rs`. Run it with `cargo run --release --bin parallel_sum`, optionally followed by the thread count and the element count.

Problem Statement

Sum a large array twice, once on one thread and once on several, and check that the two results agree. The program is a measurement as much as a demonstration: the same work is done with three element types so that the effect of the element size on the parallel speedup is visible.

Designing a Solution

`thread::scope` creates a scope that waits for every thread spawned inside it, and it lets those threads borrow values from the surrounding stack. That is why the chunks can be `&[T]` slices of one array rather than `Arc<Vec<T>>` clones: the scope's lifetime covers the borrow, so no `'static` bound is needed.

The work is divided by position, not by value. Each thread receives an index, computes the half-open range `[start, end)`, and sums that slice. The last thread takes everything after its start, which covers the array when its length is not a multiple of the thread count. The partial sums are then reduced in order with `sum()`.

The result is order-independent because integer addition is associative here. The same structure with floating-point addition would not be bit-for-bit reproducible, because the grouping would change the rounding.

Implementation

```

use std::env;
use std::thread;
use std::time::{Duration, Instant};

const SIZE: usize = 1_000_000_000;

fn seq_i32(data: &[i32]) -> (i32, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i32>();
    (sum, t.elapsed())
}

fn par_i32(data: &[i32], n: usize) -> (i32, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i32>()
                })
            })
        .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i32>()
    });
    (sum, t.elapsed())
}

fn seq_i64(data: &[i64]) -> (i64, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i64>();
    (sum, t.elapsed())
}

fn par_i64(data: &[i64], n: usize) -> (i64, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i64>()
                })
            })
        .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i64>()
    });
    (sum, t.elapsed())
}

fn seq_i128(data: &[i128]) -> (i128, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i128>();
    (sum, t.elapsed())
}

fn par_i128(data: &[i128], n: usize) -> (i128, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {

```

```

    let handles: Vec<_> = (0..n)
      .map(|i| {
        s.spawn(move || {
          let chunk = data.len() / n;
          let start = i * chunk;
          let end = if i == n - 1 { data.len() } else { start + chunk };
          data[start..end].iter().sum::<i128>()
        })
      })
      .collect();
    handles.into_iter().map(|h| h.join().unwrap()).sum::<i128>()
  });
  (sum, t.elapsed())
}

fn main() {
  let n: usize = env::args()
    .nth(1)
    .and_then(|s| s.parse().ok())
    .unwrap_or(8);

  {
    let data: Vec<i32> = vec![1; SIZE];
    let (s1, t1) = seq_i32(&data);
    let (s2, t2) = par_i32(&data, n);
    assert_eq!(s1, s2);
    println!("i32    seq sum = {s1}, elapsed = {t1:?}");
    println!("i32    par sum = {s2}, elapsed = {t2:?}  (n = {n})");
  }

  {
    let data: Vec<i64> = vec![1; SIZE];
    let (s1, t1) = seq_i64(&data);
    let (s2, t2) = par_i64(&data, n);
    assert_eq!(s1, s2);
    println!("i64    seq sum = {s1}, elapsed = {t1:?}");
    println!("i64    par sum = {s2}, elapsed = {t2:?}  (n = {n})");
  }

  {
    let data: Vec<i128> = vec![1; SIZE];
    let (s1, t1) = seq_i128(&data);
    let (s2, t2) = par_i128(&data, n);
    assert_eq!(s1, s2);
    println!("i128  seq sum = {s1}, elapsed = {t1:?}");
    println!("i128  par sum = {s2}, elapsed = {t2:?}  (n = {n})");
  }
}

```

`assert_eq!(s1, s2)` is the correctness check. It is what makes the program a test as well as a demonstration: if the chunk boundaries overlapped or left a gap, the two sums would differ.

The three blocks are separate so that each array is dropped before the next is allocated. `SIZE` is one billion, so the `i128` array alone is sixteen gigabytes; the blocks keep the peak at one array rather than three.

Intuition

With `n = 4` and `data.len() = 10`, the ranges each thread receives are:

```

thread  chunk  start  end    elements
0       2      0     2     2
1       2      2     4     2
2       2      4     6     2
3       2      6    10     4      (the last thread takes the remainder)

partial sums  a  b  c  d
reduction    ((a + b) + c) + d

```

The four sub-slices partition `0..10`, and the last range is longer because it absorbs the remainder.

Time and Space Complexity

Re-source	Cost	Condition
Time	$O(n)$ additions	independent of the thread count
Threads	$O(\text{threads})$ stacks	each thread has its own stack
Memory	$O(n)$ elements in the array, $O(\text{threads})$ for the partial sums	the input is not partitioned
Speedup	at most $\min(\text{threads}, \text{cores})$	the reduction adds a small serial tail

The sequential sum is memory-bound: each element is read once and nothing else is done with it. Adding threads divides the arithmetic, but not the memory traffic, so the speedup saturates once the bus or the cache hierarchy is saturated. The three element types are in the file to make that visible: the `i128` pass does more arithmetic per byte than the `i32` pass, so it scales differently even on the same machine.

Limitations

The default size needs a large machine. One billion elements is four gigabytes for `i32`, eight for `i64`, and sixteen for `i128`. The program was

written for a workstation and it will exhaust memory on a smaller host; the size is not a parameter, only the thread count is.

The timing has no warm-up and one pass. `Instant::now()` is called once per function, so the first touch of each page and any frequency scaling are inside the measurement. A measurement intended to be quoted needs a warm-up and several repetitions, as the benchmark harness in this repository does.

More threads than cores do not help. The thread count defaults to eight and is taken from the first argument. Nothing reads the available parallelism, so on a smaller machine the extra threads contend for cores rather than adding work.

The chunking is unbalanced. `data.len() / n` is the same for every thread except the last, which takes the remainder. When `n` is close to the length, the last thread does most of the work. A balanced split would distribute the remainder one element at a time.

The array is not partitioned or aligned for the access pattern. Each thread reads a contiguous range, which is the friendly case. A false-sharing problem would appear if threads wrote partial sums into adjacent fields of one object, which this program avoids by returning each sum through `join`.

Summary

- `thread::scope` allows the workers to borrow the input, which removes the `Arc<Vec<T>>` the naive version would need.
- Dividing by index gives each thread a disjoint half-open range, and the last range absorbs the remainder so that the whole array is covered.
- The correctness check is the equality of the two sums; a boundary error makes the program fail rather than print a wrong number.
- The measured speedup is bounded by memory bandwidth, which is why the element type is part of the experiment.

References

- Standard library, `thread::scope`.

- Standard library, `Instant`.
- Standard library, `Iterator::sum`.

Part IX: Sequences, Re-visited

Running Median

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/median_finder.rs`. Run it with `cargo run --bin median_finder`.

Problem Statement

Accept a stream of numbers one at a time and report the median of everything seen so far after each insertion. The median is the middle value when the count is odd and the average of the two middle values when it is even. Resorting the values on every query is the obvious answer and the wrong one; the type has to maintain the median as the stream arrives.

Designing a Solution

Two heaps split the values into a lower half and an upper half. `low` is a `BinaryHeap<i32>`, which is a max-heap, so its top is the largest value in the lower half. `high` is a `BinaryHeap<Reverse<i32>>`, and `Reverse` flips the ordering, so its top is the smallest value in the upper half. `BinaryHeap` is already a priority queue; `Reverse` is the wrapper that gives the second one the opposite ordering.

Two invariants keep the halves together:

- every value in `low` is less than or equal to every value in `high`;
- `low.len()` equals `high.len()`, or exceeds it by one.

If both hold, the median is `low.peek()` when `low` is longer, and the average of the two tops otherwise. The insertion maintains them in three steps: push onto

low, move low's maximum to high so the halves stay ordered, and move high's minimum back if high has grown too large.

Implementation

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

#[derive(Default)]
pub struct MedianFinder {
    low: BinaryHeap<i32>,
    high: BinaryHeap<Reverse<i32>>,
}

impl MedianFinder {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn add_num(&mut self, num: i32) {
        self.low.push(num);

        if let Some(val) = self.low.pop() {
            self.high.push(Reverse(val));
        }

        if self.high.len() > self.low.len() {
            if let Some(Reverse(val)) = self.high.pop() {
                self.low.push(val);
            }
        }
    }

    pub fn find_median(&self) -> Option<f64> {
        let max_low = *self.low.peek()?;

        if self.low.len() > self.high.len() {
            Some(max_low as f64)
        } else {
            let Reverse(min_high) = *self.high.peek()?;
            Some((max_low as f64 + min_high as f64) / 2.0)
        }
    }
}

const SAMPLE: [i32; 11] = [6, 10, 2, 6, 5, 0, 6, 3, 1, 0, 0];

fn main() {
    let mut finder = MedianFinder::new();

    for value in SAMPLE {
        finder.add_num(value);
        match finder.find_median() {
            Some(median) => println!("after {value:>2}: median {median}"),
            None => println!("after {value:>2}: no median"),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
```

```

/// The median of a sorted slice, computed directly.
fn median_of_sorted(values: &[i32]) -> f64 {
    let middle = values.len() / 2;
    if values.len() % 2 == 1 {
        values[middle] as f64
    } else {
        (values[middle - 1] as f64 + values[middle] as f64) / 2.0
    }
}

#[test]
fn an_empty_finder_has_no_median() {
    assert_eq!(MedianFinder::new().find_median(), None);
}

#[test]
fn one_value_is_the_median() {
    let mut finder = MedianFinder::new();
    finder.add_num(7);
    assert_eq!(finder.find_median(), Some(7.0));
}

#[test]
fn an_even_count_averages_the_middle_two() {
    let mut finder = MedianFinder::new();
    finder.add_num(1);
    finder.add_num(2);
    assert_eq!(finder.find_median(), Some(1.5));
}

#[test]
fn three_values_are_ordered() {
    let mut finder = MedianFinder::new();
    for (value, median) in [1, 2, 3].into_iter().zip([1.0, 1.5, 2.0]) {
        finder.add_num(value);
        assert_eq!(finder.find_median(), Some(median));
    }
}

#[test]
fn duplicates_are_kept() {
    let mut finder = MedianFinder::new();
    for value in [4, 4, 4, 4] {
        finder.add_num(value);
    }
    assert_eq!(finder.find_median(), Some(4.0));
}

#[test]
fn the_stream_matches_a_sort_on_every_step() {
    let mut finder = MedianFinder::new();
    let mut seen = Vec::new();

    for value in SAMPLE {
        finder.add_num(value);
        seen.push(value);
        seen.sort_unstable();

        assert_eq!(finder.find_median(), Some(median_of_sorted(&seen)));
    }
}

#[test]
fn the_two_heaps_never_differ_by_more_than_one() {
    let mut finder = MedianFinder::new();

    for value in SAMPLE {

```

```

        finder.add_num(value);
        assert!(finder.low.len().abs_diff(finder.high.len()) <= 1);
    }
}
}

```

The listing is the whole file. Taken in order:

- `add_num` pushes onto `low` before rebalancing. That order means the new value is considered for the lower half first, and the move of `low`'s maximum to `high` then restores the ordering if the value belongs above the median. The second `if` runs only when `high` became longer, which moves one value back so `low` is never the shorter heap.
- `find_median` returns `Option<f64>`. `self.low.peek()? returns None for an empty finder, so find_median reports absence rather than dividing by zero. When low is longer, low's maximum is the median; otherwise the two tops are averaged.`
- The median is returned as `f64` even when it is a whole number, because the two middle values can be a half-integer. Casting `i32` to `f64` is exact for every `i32`, so no precision is lost on the way in.
- `main` drives the type with the `SAMPLE` stream and prints the median after each insertion, which is the behaviour the chapter claims in one run.
- The tests check the empty case, one value, an even count, the ordered sequence, a repeated value, and the whole `SAMPLE` stream against a sort of the values seen so far. That last test is a differential check: it computes the median a second, slower way and requires the heaps to agree with it on every step. A further test asserts the balancing invariant itself, that the two lengths never differ by more than one.

Intuition

Insert the values 1, then 2, then 3:

step	low (max-heap)	high (min-heap)	median
start	[]	[]	None
add_num(1)	[1]	[]	1.0
add_num(2)	[1]	[2]	1.5
add_num(3)	[2, 1]	[3]	2.0

The third step in detail. 3 goes onto `low`, so `low` is `[3, 1]`. Its maximum, 3, moves to `high`, which becomes `[2, 3]` read as a min-heap: the top is 2. `high` is now longer than `low`, so its minimum, 2, moves back and `low` becomes `[2, 1]`. The median is `low`'s top, 2.0.

The `SAMPLE` stream in the program produces:

```
after 6: median 6
after 10: median 8
after 2: median 6
after 6: median 6
after 5: median 6
after 0: median 5.5
after 6: median 6
after 3: median 5.5
after 1: median 5
after 0: median 4
after 0: median 3
```

Each insertion moves at most two values between the heaps, so the two lengths never differ by more than one and the median is always one or two heap tops away.

Time and Space Complexity

Operation	Cost	Condition
<code>add_num</code>	$O(\log n)$	two heap pushes and at most two pops
<code>find_median</code>	$O(1)$	the median is one or two heap tops
memory	$O(n)$	every value is stored once

The alternative, keeping a sorted `Vec` and re-sorting on each query, costs $O(n \log n)$ per insertion. Sorting once per query and caching the result trades that for $O(1)$ queries but still pays $O(n \log n)$ when the stream continues.

Limitations

The type is fixed to `i32`. The heaps are `BinaryHeap<i32>`, so the caller cannot use `i64` or `f64`. Making the type generic needs an ordering bound plus a way to average two values, which is more than `Ord` provides.

A median is returned as `f64`, which changes the type of the answer. For `i32` inputs the conversion and the average are exact, but a caller that needs an exact fraction of two integers has to compute it from the two tops rather than from the returned value.

There is no removal. The structure answers the median of everything seen. A sliding-window median needs a structure that can remove the oldest value, which the two heaps cannot do without locating that value in one of them.

The tests cover one stream in depth and a few cases by hand. `SAMPLE` is eleven values. The differential test compares against a sort, so it catches a rebalancing error on those eleven steps, but a random or exhaustive stream would cover more of the state space than a fixed array does.

Summary

- Two heaps hold the lower and upper halves, and `Reverse` turns the second into a min-heap.
- The three-step insertion keeps the halves ordered and balanced, which is what makes the median a constant-time query.
- `Option<f64>` states that an empty finder has no median, instead of returning a sentinel.
- The program prints the median after every value of a sample stream, and the tests check the result against a sort of the values seen so far.

References

- Standard library, `BinaryHeap`.
- Standard library, `std::cmp::Reverse`.
- Standard library, `Option`.

Reversing a String In Place

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/reverse_string.rs`. Run it with `cargo run --bin reverse_string`.

Problem Statement

Reverse the characters of a `String` without allocating a second one. The constraint is what makes the problem interesting in Rust: a `String` is UTF-8, so its bytes are not the same as its characters, and the safe API does not expose a mutable byte buffer.

Designing a Solution

The function works on bytes and swaps from the two ends inward. Two indices start at the first and last byte, exchange their values, and move toward each other until they meet. Each swap fixes two positions, so the loop runs half as many times as there are bytes and requires no extra storage.

Reversing bytes only produces a valid `String` when every byte is a complete character. For ASCII that is true, because one byte is one code point. For general UTF-8 it is not: a multi-byte character read backwards is not a character. The function therefore asserts that the input is ASCII before it touches the buffer, and the assertion is the precondition that makes the unsafe access sound.

`String::as_mut_vec` is the only way to reach the bytes. It is `unsafe` because a caller that leaves invalid UTF-8 in the buffer creates a `String` that violates its own invariant, and every later operation on it is undefined. The `SAFETY` comment states the obligation and the assertion that discharges it.

Implementation

```
fn reverse_str(s: &mut String) {
    // SAFETY: s.as_mut_vec() is unsafe because it exposes the inner mutability to
    // it's character buffer, and we must ensure
    // to leave valid UTF-8. We enforce ASCII only characters.
    assert!(s.is_ascii());
    let bytes = unsafe { s.as_mut_vec() };

    let (mut start, mut end) = (0, bytes.len() - 1);

    while start < end {
        bytes.swap(start, end);
        start+=1; end-=1;
    }
}

fn main() {
    for value in ["abcd", "abc", "a", "rust"] {
        let mut s = String::from(value);
        reverse_str(&mut s);
        println!("{value:>6} -> {s}");
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn an_even_length_string_is_reversed() {
        let mut s = String::from("abcd");
        reverse_str(&mut s);
        assert_eq!(s, "dcba");
    }

    #[test]
    fn an_odd_length_string_keeps_its_middle_byte() {
        let mut s = String::from("abc");
        reverse_str(&mut s);
        assert_eq!(s, "cba");
    }

    #[test]
    fn a_single_byte_is_unchanged() {
        let mut s = String::from("a");
        reverse_str(&mut s);
        assert_eq!(s, "a");
    }

    #[test]
    fn the_buffer_is_reused() {
        let mut s = String::from("abcd");
        let capacity = s.capacity();
        let address = s.as_ptr();
        reverse_str(&mut s);
        assert_eq!(s.capacity(), capacity);
        assert_eq!(s.as_ptr(), address);
    }

    #[test]
    fn reversing_twice_restores_the_string() {
        let mut s = String::from("interview");
        reverse_str(&mut s);
        reverse_str(&mut s);
    }
}
```

```

    assert_eq!(s, "interview");
}

#[test]
#[should_panic]
fn non_ascii_input_panics() {
    let mut s = String::from("café");
    reverse_str(&mut s);
}
}

```

The listing is the whole file. Taken in order:

- `assert!(s.is_ascii())` walks the string before the reversal. It is $O(n)$, which does not change the overall bound, and it runs before the unsafe block rather than inside it, so the panic leaves the string untouched.
- `bytes.swap(start, end)` is `slice::swap`, which exchanges two elements without moving either out. A manual exchange through a temporary would be three assignments on `u8`; `swap` states the intent and compiles to the same place.
- `main` reverses four strings and prints each result, so one run shows the even case, the odd case, the single byte, and a second four-letter word.
- The tests cover both parities, the single byte, the in-place property, and the round trip. `the_buffer_is_reused` reads the capacity and the address before and after and requires both to be unchanged, which is what separates this function from one that builds a new `String`. `non_ascii_input_panics` is marked `#[should_panic]`, so the assertion is checked rather than assumed.

Intuition

Reverse "abcd":

```

before  a  b  c  d
        ^      ^
        start  end

swap 0,3 b  a  c  d
swap 1,2 b  c  a  d
        start >= end, stop
after   b  c  a  d

```

Reverse "abc":

```

before  a  b  c
swap 0,2 c  b  a
        start = 1, end = 1, stop
after   c  b  a

```

The middle byte of an odd-length string is never moved, which is why the loop condition is `start < end` rather than `start <= end`.

The program prints:

```
abcd -> dcba
abc  -> cba
a    -> a
rust -> tsur
```

The buffer is the same allocation in every line. Only the bytes inside it move.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$	<code>is_ascii</code> reads every byte; the swaps touch half of them
Space	$O(1)$	the swap is in place; no second buffer
Allocations	zero	the <code>String</code> keeps its buffer

The reversal itself is $n / 2$ swaps, and it is performed in place, so the buffer's address and capacity do not change. The assertion adds a second pass over the bytes, which is the price of the safe wrapper around the unsafe block.

Limitations

Non-ASCII input panics. `assert!(s.is_ascii())` refuses any string containing a byte above `0x7F`, which includes every accented letter and every emoji. A general reversal needs to decode the string into characters, reverse the characters, and rebuild the `String`, which allocates.

Empty input panics. With `bytes.len() == 0`, the expression `bytes.len() - 1` underflows. In a debug build that panics with an arithmetic error; in a release build it wraps and the following `swap` is out of bounds. The function needs a check for an empty string, and it does not have one, so the tests do not cover the empty case either.

The `unsafe` block is sound only because of the assertion above it. Removing the assertion makes the function undefined behaviour on any non-

ASCII input. The `SAFETY` comment is the record of that dependency, and a future edit that drops the check has to drop the unsafe approach as well.

The function reverses bytes, not grapheme clusters. For ASCII the two are the same. For text that passes the assertion they cannot differ, but a reader should not carry the assumption to a general string.

The largest test input is nine bytes. The tests establish the two parities and the round trip. A larger input, or one built to exercise the exact midpoint of an even-length string, would test the loop bound more of the way.

Summary

- Byte reversal is $O(n)$ time and $O(1)$ space, and only the ASCII precondition makes it a valid `String` transformation.
- `as_mut_vec` is `unsafe` because it can leave the buffer invalid; the assertion ahead of it is what discharges the obligation recorded in the `SAFETY` comment.
- The loop runs $n / 2$ times and leaves the middle byte of an odd-length string in place.
- The program reverses four strings and prints them, and the tests check both parities, the single byte, the unchanged allocation, and the round trip.
- Empty and non-ASCII inputs are not handled: the first panics on the subtraction, and the second is refused by the assertion.

References

- Standard library, `String::as_mut_vec`.
- Standard library, `str::is_ascii`.
- Standard library, `slice::swap`.
- The Rustonomicon, [Working with unsafe](#).

Part X: Reliability

Idempotent Operations

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/idempotent_operation.rs` and `src/bin/idempotent_operation_with_error_propagation.rs`. Run them with `cargo run --bin idempotent_operation` and `cargo run --bin idempotent_operation_with_error_propagation`.

Problem Statement

A client sends a request, the server performs the work, and the response is lost before the client receives it. The client retries. If the server performs the work again, a card is charged twice or an order is placed twice. An operation is *idempotent* when performing it more than once has the same effect as performing it once. The task here is to give an arbitrary fallible operation that property: run it at most once for a given key, and return the same result to every later call that presents the same key.

Designing a Solution

Keep a map from key to the result already produced for that key. When a call arrives, look the key up. On a hit, return a copy of the stored result and do not run the operation. On a miss, run the operation, store the result, and return it. The map sits behind a `Mutex`, so two threads cannot both observe a miss for the same key.

Holding the lock across the operation is the decision that carries the design. It is what makes the second of two concurrent callers wait on the lock, then find the stored result instead of running the operation. The cost is that the lock is held for the whole of the operation, so callers with *different* keys also wait for

one another. Only a successful result is stored: when the operation returns an error, the key is left absent and a later retry runs it again.

The operation is passed as `FnOnce`, because the function may run zero times (on a hit) or one time (on a miss), and never more than once. The error type is `Box<dyn std::error::Error>`, which lets the caller supply any error without the store naming it.

Implementation

```
use std::collections::HashMap;
use std::hash::Hash;
use std::sync::Mutex;

pub struct Idempotent<K, V> {
    store: Mutex<HashMap<K, V>>,
}

impl<K, V> Idempotent<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    pub fn execute<F>(&self, key: K, f: F) -> Result<V, Box<dyn std::error::Error>>
    where
        F: FnOnce() -> Result<V, Box<dyn std::error::Error>>,
    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned"?);
        if let Some(v) = store.get(&key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key, v.clone());
        Ok(v)
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let idem = Idempotent::new();

    let charge = || -> Result<String, Box<dyn std::error::Error>> {
        println!("charging card...");
        Ok("txn_42".into())
    };

    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    Ok(())
}
```

The lock is taken first and held until `execute` returns. `store.get(&key)` borrows the key, so the borrowed value is cloned out with `v.clone()` and the borrow ends before `insert` moves the key into the map.

`let v = f()?` is where the operation runs. The `?` returns early on an error, so the map is never written for a failed call. That is the choice that makes the store *at-most-once for successes*: an error is treated as "the operation did not happen", and a retry is allowed to run it again.

`store.insert(key, v.clone())` stores one copy and returns the other. The value is cloned twice in all, once for the map and once for the caller. A version that returned `v` and let the map own a clone, or that used the previous value `insert` returns, would avoid one of the two.

The `K: Clone` bound is present but unused: `get` borrows the key and `insert` moves it, and neither needs a copy. `V: Clone` is required, because both the hit path and the store path copy the value.

The variant with a named error type

The repository holds a second version of the same type, `idempotent_operation_with_error_propagation.rs`. It differs in one way: the error type is given a name, `RuntimeError`, and the alias is used in every signature.

```
use std::collections::HashMap;
use std::hash::Hash;
use std::sync::Mutex;

type RuntimeError = Box<dyn std::error::Error>;

pub struct Idempotent<K, V> {
    store: Mutex<HashMap<K, V>>,
}

impl<K, V> Idempotent<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    pub fn execute<F>(&self, key: K, f: F) -> Result<V, RuntimeError>
    where
        F: FnOnce() -> Result<V, RuntimeError>,
    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned")?;
        if let Some(v) = store.get(&key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key, v.clone());
        Ok(v)
    }
}

fn main() -> Result<(), RuntimeError> {
    let idem = Idempotent::new();
```



```

let charge = || -> Result<String, RuntimeError> {
  println!("charging card...");
  Ok("txn_42".into())
};

assert_eq!(idem.execute("order-1", charge)?, "txn_42");
assert_eq!(idem.execute("order-1", charge)?, "txn_42");
Ok(())
}

```

The alias names the error in one place, so a change to the error type is a change to one line and the signatures stay short. The behaviour is the same in both files: the early return on an error, the lock held across the operation, and the rule that only successes are stored.

Intuition

Two sequential calls with the same key, tracing the map:

execute("order-1", charge)	store = {}
lock	
get("order-1")	miss
charge()	prints "charging card..." -> "txn_42"
insert("order-1", "txn_42")	store = {order-1: "txn_42"}
unlock	
return "txn_42"	
execute("order-1", charge)	store = {order-1: "txn_42"}
lock	
get("order-1")	hit
return "txn_42"	charge() is not called
unlock	
output:	
charging card...	

The program prints the line once. The two assertions both hold, but they would hold even without caching, because the same closure produces the same value; the evidence that the closure ran once is the single line of output.

Two concurrent calls with the same key show what the lock is for:

thread A	thread B
lock	
get: miss	
charge() (runs)	
	lock (blocks)
insert	
unlock	
	get: hit
	unlock
	return stored value
charge() ran once; thread B returned the value thread A stored.	

Without the lock held across `charge`, both threads would miss, both would run the operation, and the effect would happen twice.

Time and Space Complexity

Operation	Time	Space
execute, hit	$O(1)$ expected	one clone of the stored value
execute, miss	$O(1)$ expected, plus the cost of <code>f</code>	one map entry and one clone
store	$O(k)$ entries	one entry per distinct key, held for the life of the value

$O(1)$ expected for the map rests on the usual assumption that the hash function distributes keys evenly. The store is linear in the number of distinct keys and is never pruned.

Limitations

The lock is held across the operation. Every call, for every key, is serialised for the duration of `f`. A slow `f` blocks unrelated keys as well as duplicates of its own. An operation that calls back into the same store deadlocks, because the lock is not reentrant.

The store has no bound and no expiry. Entries live until the process exits, so the map grows with the number of distinct keys. A production store needs a time-to-live and an eviction policy, because the set of keys is not bounded by anything the client controls.

The store is in memory. A restart forgets every key, so a retry after a crash runs the operation again. Durability requires persisting the key and the result, or relying on a unique constraint in the datastore that performs the effect.

Only successes are stored. A failed call leaves the key absent, so a retry runs the operation again. For an operation that can partially succeed, that is the wrong default; the record of the attempt has to be written before the effect, not after it.

The error type erases the concrete error. Callers receive `Box<dyn std::error::Error>` and cannot match on the failure without downcasting. The lock-poisoning case is a string, which is the least informative variant in the file.

A panic while the lock is held poisons the mutex. Every later call returns the `"lock poisoned"` error, and the store never recovers. The error is reported rather than panicking the caller, which is the better of the two available choices, but the store is unusable from that point on.

The key is not scoped. A production idempotency key is scoped to a client and an endpoint, so that two callers cannot collide by choosing the same string. This store treats the key as global.

K: Clone is an unused bound. It constrains callers without buying anything. The bound can be removed with no other change to the implementation.

Summary

- A `Mutex<HashMap<K, V>>` gives at-most-once execution per key and a cached result for every later call with that key.
- Holding the lock across the operation is what prevents two concurrent callers from running it twice; the price is that different keys are serialised too.
- Only success is recorded, so failure leaves the key absent and a retry is permitted.
- The store is in memory, unbounded, and without expiry or scope, so it is a faithful model of the pattern rather than a production idempotency layer.

References

- Rust standard library, `Mutex` and `HashMap`.
- Rust standard library, `FnOnce`, for a closure that runs at most once.
- RFC 9110, [HTTP Semantics](#), Section 9.2.2, on idempotent methods.

Part XI: Types and Pointers

Types That Reject Invalid Values

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/adt_idioms.rs`. Test it with `cargo test adt_idioms`.

Problem Statement

A command-line client for a GPU service accepts four commands, one per line:

```
list
get <id>
create <name> <gpu-count>
delete <id>
```

A GPU count must be an integer from 1 to 64. The parser must turn a line into a value the rest of the program can use without checking it again, and it must report each kind of malformed input in a way a caller can distinguish: an empty line, an unknown verb, a missing argument, and a count that is not a number or is out of range.

Designing a Solution

The obvious representation is a `String` for the verb, an `Option<String>` for each argument, and a `u32` for the count. Every function that receives those values would have to re-check them, because nothing in the types says that a `create` has a name or that the count is in range. Three types remove the re-checking.

A newtype for the count. `struct GpuCount(u32)` is a distinct type whose only constructor validates the range. Code that receives a `GpuCount` knows the value

is between 1 and 64 without looking at it, and a raw `u32` cannot be passed where a `GpuCount` is expected.

An enumeration for the command. Each variant carries exactly the fields its command needs. `Command::List` has none, `Command::Get` has an `id`, and `Command::Create` has a `name` and a `GpuCount`. A `match` on a `Command` must handle all four variants, and a variant cannot be built without its fields.

An enumeration for the failure. `CommandError` names each way a line can be rejected, and carries the text that caused the rejection. A caller can match on the variant to decide what to do, and print it through `Display` to tell a user what happened.

The diagram below shows the flow from text to typed value.

```

"create triton 4"
  |
  v
Command::parse ----> Err(CommandError::UnknownVerb | MissingArgument |
InvalidGpuCount)
  |
  v
Ok(Command::Create { name: "triton", gpus: GpuCount(4) })
  |
  v
the rest of the program: no further checks on name or gpus

```

Implementation

The module has four parts: the newtype, its error, the command and its error, and a short iterator example. The first listing shows the newtype.

```

use std::fmt;

/// A GPU count that is validated at construction.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct GpuCount(u32);

impl GpuCount {
    pub const MIN: u32 = 1;
    pub const MAX: u32 = 64;

    /// Validate a raw count.
    pub fn new(value: u32) -> Result<Self, GpuCountError> {
        if (Self::MIN..=Self::MAX).contains(&value) {
            Ok(Self(value))
        } else {
            Err(GpuCountError(value))
        }
    }

    pub fn get(self) -> u32 {
        self.0
    }
}

```

```

}

/// Why a raw GPU count was rejected.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct GpuCountError(pub u32);

impl fmt::Display for GpuCountError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            formatter,
            "gpu count {} is outside {}..={}",
            self.0,
            GpuCount::MIN,
            GpuCount::MAX
        )
    }
}

impl std::error::Error for GpuCountError {}

impl TryFrom<u32> for GpuCount {
    type Error = GpuCountError;

    fn try_from(value: u32) -> Result<Self, Self::Error> {
        Self::new(value)
    }
}

impl From<GpuCount> for u32 {
    fn from(count: GpuCount) -> u32 {
        count.0
    }
}

```

The tuple field of `GpuCount` is private, so code outside the module cannot write `GpuCount(0)`. The only ways to obtain a value are `GpuCount::new` and `GpuCount::try_from`, and both run the range check.

`(Self::MIN..=Self::MAX).contains(&value)` states the valid range once, using the associated constants that the error message also prints. Changing the limit is a one-line edit that updates the check and the message together.

`GpuCountError` is itself a newtype. It carries the rejected value, which lets the `Display` implementation produce a message such as `gpu count 99 is outside 1..=64`. The empty `impl std::error::Error for GpuCountError {}` is enough to make the type usable with `?` in a function that returns `Box<dyn Error>`; the trait's methods all have default implementations.

`TryFrom<u32>` connects the newtype to the standard conversion traits. A caller can write `GpuCount::try_from(raw)?` or `let gpus: GpuCount = raw.try_into()?`, and generic code that accepts `T: TryFrom<u32>` works with it. The implementation delegates to `new`, so there is one validation path. `From<GpuCount> for u32` goes the other way, and it cannot fail, which is why it is `From` rather than `TryFrom`.

The next listing shows the command type and its parser.

```

/// A closed set of commands, each carrying exactly the data it needs.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    List,
    Get { id: String },
    Create { name: String, gpus: GpuCount },
    Delete { id: String },
}

impl Command {
    /// Parse one command line.
    pub fn parse(line: &str) -> Result<Self, CommandError> {
        let mut parts = line.split_whitespace();
        let verb = parts.next().ok_or(CommandError::Empty)?;

        match verb {
            "list" => Ok(Self::List),
            "get" => Ok(Self::Get {
                id: argument(&mut parts, "id")?.to_string(),
            }),
            "delete" => Ok(Self::Delete {
                id: argument(&mut parts, "id")?.to_string(),
            }),
            "create" => {
                let name = argument(&mut parts, "name")?.to_string();
                let raw = argument(&mut parts, "gpu count")?;
                let value: u32 = raw
                    .parse()
                    .map_err(|_| CommandError::InvalidGpuCount(raw.to_string()))?;
                let gpus = GpuCount::new(value)
                    .map_err(|_| CommandError::InvalidGpuCount(raw.to_string()))?;
                Ok(Self::Create { name, gpus })
            }
            other => Err(CommandError::UnknownVerb(other.to_string())),
        }
    }

    pub fn verb(&self) -> &'static str {
        match self {
            Self::List => "list",
            Self::Get { .. } => "get",
            Self::Create { .. } => "create",
            Self::Delete { .. } => "delete",
        }
    }
}

/// A typed parse failure, so callers can distinguish the cases.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum CommandError {
    Empty,
    UnknownVerb(String),
    MissingArgument(&'static str),
    InvalidGpuCount(String),
}

impl fmt::Display for CommandError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::Empty => write!(formatter, "empty command"),
            Self::UnknownVerb(verb) => write!(formatter, "unknown verb {verb:?}"),
            Self::MissingArgument(name) => write!(formatter, "missing argument:
{name}"),
            Self::InvalidGpuCount(raw) => write!(formatter, "invalid gpu count:

```



```

    {raw:?}"},
    }
}

impl std::error::Error for CommandError {}

fn argument<'a>(
    parts: &mut impl Iterator<Item = &'a str>,
    name: &'static str,
) -> Result<&'a str, CommandError> {
    parts.next().ok_or(CommandError::MissingArgument(name))
}

```

`Command::parse` splits the line on whitespace and takes the verb with `parts.next().ok_or(CommandError::Empty)?`. `ok_or` converts `Option` to `Result`, and `?` returns the error immediately, so the rest of the function runs only when a verb exists.

The helper `argument` has the same shape and adds the name of the missing argument to the error. Its signature is worth reading closely. `parts: &mut impl Iterator<Item = &'a str>` borrows the iterator mutably, so each call consumes one item from the same sequence. The lifetime `'a` ties the returned `&str` to the original line, not to the iterator, which lets `parse` convert it with `to_string` after the call.

The `create` arm distinguishes two failures that produce the same variant. A count that does not parse as `u32` and a count that parses but is out of range both return `InvalidGpuCount`, carrying the raw text. The design choice is that the caller cares about which argument was wrong, not about which check rejected it.

`verb` returns `&'static str` for each variant. The `..` in patterns such as `Self::Get { .. }` ignores the fields, and the `match` has no wildcard arm, so adding a fifth command makes this function fail to compile until the new variant is handled.

The next listing shows the last function in the module, which is unrelated to parsing and illustrates a different idiom.

```

/// Count and sum in one pass, without managing an index.
pub fn summarize(values: &[i32]) -> (usize, i64) {
    values.iter().fold((0usize, 0i64), |(count, sum), value| {
        (count + 1, sum + i64::from(*value))
    })
}

```

`fold` carries a tuple of accumulators through the iteration. The count is `usize` and the sum is `i64`, and each element is widened with `i64::from(*value)`.

Widening before adding means that summing a slice of `i32` values cannot overflow until the total exceeds the range of `i64`. `From` is used instead of `as` because it compiles only for conversions that cannot lose information.

The last listing shows the tests. They exercise every variant of `Command`, every variant of `CommandError`, the boundaries of the newtype, and the rendered messages.

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn parses_each_command_variant() {
        assert_eq!(Command::parse("list"), Ok(Command::List));
        assert_eq!(
            Command::parse("get wl-7"),
            Ok(Command::Get { id: "wl-7".into() })
        );
        assert_eq!(
            Command::parse("create triton 4"),
            Ok(Command::Create {
                name: "triton".into(),
                gpus: GpuCount::new(4).expect("valid")
            })
        );
        assert_eq!(
            Command::parse("delete wl-7"),
            Ok(Command::Delete { id: "wl-7".into() })
        );
    }

    #[test]
    fn rejects_bad_input_with_typed_errors() {
        assert_eq!(Command::parse(""), Err(CommandError::Empty));
        assert_eq!(
            Command::parse("froblicate"),
            Err(CommandError::UnknownVerb("froblicate".into()))
        );
        assert_eq!(
            Command::parse("get"),
            Err(CommandError::MissingArgument("id"))
        );
        assert_eq!(
            Command::parse("create triton 0"),
            Err(CommandError::InvalidGpuCount("0".into()))
        );
        assert_eq!(
            Command::parse("create triton many"),
            Err(CommandError::InvalidGpuCount("many".into()))
        );
    }

    #[test]
    fn newtype_rejects_out_of_range_values() {
        assert!(GpuCount::new(1).is_ok());
        assert!(GpuCount::new(64).is_ok());
        assert!(GpuCount::new(0).is_err());
        assert!(GpuCount::new(65).is_err());
        assert_eq!(u32::from(GpuCount::try_from(8).expect("valid")), 8);
        assert!(GpuCount::try_from(0).is_err());
    }
}
```

```

#[test]
fn summarize_reports_count_and_sum() {
    assert_eq!(summarize(&[]), (0, 0));
    assert_eq!(summarize(&[1, 2, 3]), (3, 6));
    assert_eq!(summarize(&[-5, 5]), (2, 0));
}

#[test]
fn errors_render_a_reason() {
    assert_eq!(
        Command::parse("create triton 0").unwrap_err().to_string(),
        "invalid gpu count: \"0\""
    );
    assert!(
        GpuCount::new(99)
            .unwrap_err()
            .to_string()
            .contains("outside")
    );
}
}

```

Intuition

The table below follows six inputs through `Command::parse`.

`Command::parse` on valid and invalid lines

input	verb	arguments consumed	result
"list"	list	none	<code>Ok(Command::List)</code>
"get wl-7"	get	wl-7	<code>Ok(Command::Get { id: "wl-7" })</code>
"create triton 4"	create	triton, 4	<code>Ok(Command::Create { name: "triton", gpus: GpuCount(4) })</code>
"create triton 0"	create	triton, 0	"0" parses as <code>0u32</code> , <code>GpuCount::new(0)</code> fails, <code>Err(InvalidGpuCount("0"))</code>
"get"	get	none available	<code>Err(MissingArgument("id"))</code>
" "	none	none	<code>Err(Empty)</code>

`Command::parse("create triton 0").unwrap_err().to_string()` produces `invalid gpu count: "0"`. The `{raw:?}` format prints the text with quotes, which makes trailing whitespace or an empty string visible in the message.

Time and Space Complexity

Operation	Time	Space
<code>GpuCount::new</code>	$O(1)$	none; <code>GpuCount</code> is the size of a <code>u32</code>
<code>Command::parse</code>	$O(\text{len})$ for the line	one <code>String</code> per stored argument
<code>verb</code>	$O(1)$	none
<code>summarize</code>	$O(n)$	none

A newtype has no run-time cost. `GpuCount` has the same size and alignment as `u32`, and its methods compile to the same instructions as the unwrapped code. The type exists only for the compiler.

Limitations

The parser allocates for every argument. `Command::Get { id: String }` owns its identifier, so each successful parse copies the argument out of the line. A version that returns `Command<'a>` with `&'a str` fields would allocate nothing, at the cost of tying the command's lifetime to the input line.

Extra arguments are ignored. `"get wl-7 extra"` parses as `Get { id: "wl-7" }`. The parser never checks that `parts` is exhausted. Adding `if parts.next().is_some() { return Err(...) }` after each arm would reject the input, and it would require a new error variant.

The two count failures are merged. `InvalidGpuCount` does not say whether the text was not a number or the number was out of range. A caller that wants to print "must be between 1 and 64" for the second case has to parse the text again. Keeping the `GpuCountError` inside the variant, for example `OutOfRange(GpuCountError)`, would preserve the information.

`CommandError::MissingArgument(&'static str)` limits the argument names to string literals. This is fine for a fixed grammar and prevents building the name at run time. It also keeps the error type `Clone` and cheap.

`summarize` is unrelated to the rest of the module. It sits here as an example of an iterator fold. In a real codebase it would live with the code that uses it.

Summary

- A newtype with a private field and a validating constructor makes an invalid value unrepresentable, at no run-time cost.
- `TryFrom` and `From` connect a newtype to the standard conversion traits, so `?` and `try_into` work with it.
- An enumeration whose variants carry only the fields they need replaces a bag of optional fields, and `match` without a wildcard makes a new variant a compile error at every use.
- A typed error enumeration lets a caller branch on the failure, and implementing `Display` and `std::error::Error` lets it travel through `?` and `Box<dyn Error>`.
- Widening with `i64::from` before adding avoids overflow, and `From` rejects lossy conversions at compile time.

References

- The Rust Book, [Using the newtype pattern](#).
- Rust API Guidelines, [Newtypes provide static distinctions](#).
- Standard library, `TryFrom`.
- Standard library, `std::error::Error`.
- Standard library, `Iterator::fold`.

Smart Pointers and Interior Mutability

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/smart_pointers.rs`. Test it with `cargo test smart_pointers`.

Problem Statement

Five situations recur in systems code:

1. A type contains itself, such as an expression tree.
2. Two parts of one thread need to change the same value.
3. A child needs to reach its parent without keeping the parent alive.
4. Several threads need to change the same value.
5. A function usually returns its input unchanged and occasionally needs a modified copy.

Each has a type designed for it, and choosing the wrong one produces either a compile error or a run-time failure. The module documentation summarises the choices in a table, reproduced here.

What each pointer type provides

Type	Ownership	Mutation	Across threads
<code>Box<T></code>	single	through <code>&mut</code>	if <code>T</code> : <code>Send</code>
<code>Rc<T></code>	shared, non-atomic count	no	no
<code>Arc<T></code>	shared, atomic count	no	if <code>T</code> : <code>Send + Sync</code>

Type	Ownership	Mutation	Across threads
<code>RefCell<T></code>	single	checked at run time	no
<code>Mutex<T></code>	single	blocking lock	yes
<code>Weak<T></code>	non-owning	no	as <code>Rc</code> or <code>Arc</code>
<code>Cow<'a, T></code>	borrowed or owned	no	as the borrow

Designing a Solution

The types combine along two axes. The first axis is ownership: `Box` has one owner, `Rc` and `Arc` have many, and `Weak` has none. The second is mutation through a shared reference, which Rust forbids by default and permits only through a cell: `RefCell` checks borrows at run time on one thread, and `Mutex` enforces exclusion with a lock across threads.

The common pairings follow from those axes:

- `Rc<RefCell<T>>` for shared, mutable state inside one thread, such as the doubly linked list in chapter 16.
- `Arc<Mutex<T>>` for shared, mutable state between threads, such as the counters and queues in chapters 21, 22, and 30.
- `Rc` forward and `Weak` backward whenever two objects refer to each other, so that the reference counts can reach zero.

`Cow` is on a different axis. It is not about sharing; it lets a function defer the decision to allocate until it knows whether an allocation is needed.

Implementation

The module contains five short examples and a test for each. The file begins with the table from the Problem Statement in its documentation comment and the imports, shown below.

```

/// Smart pointers and interior mutability.
///
/// Each pointer type answers a different ownership question, and picking the
/// wrong one shows up as either a compile error or a runtime bug.
///
/// | Type | Ownership | Mutation | Across threads |
/// |---|---|---|---|
/// | `Box<T>` | single | through `&mut` | if `T: Send` |
/// | `Rc<T>` | shared, non-atomic count | no | no |
/// | `Arc<T>` | shared, atomic count | no | if `T: Send + Sync` |
/// | `RefCell<T>` | single | runtime checked | no |

```

```

/// | `Mutex<T>` | single | blocking lock | yes |
/// | `Weak<T>` | non-owning | no | as `Arc` |
/// | `Cow<'a, T>` | borrow or own | no | as the borrow |
///
/// Two rules cover most decisions. `Rc`/`RefCell` is for graphs and caches inside
/// one thread; `Arc`/`Mutex` is for state shared between threads. `Weak` exists to
/// break reference cycles, because two strong references in a cycle never reach a
/// count of zero and the memory is never freed.

use std::borrow::Cow;
use std::cell::RefCell;
use std::rc::{Rc, Weak};
use std::sync::{Arc, Mutex};
use std::thread;

```

Box for a recursive type

A type that contains itself by value would have infinite size. `Box<Expr>` is one pointer wide whatever the expression contains, so `Expr` has a finite size.

```

/// A recursive type needs indirection, because `Expr` cannot contain itself by
/// value. `Box` provides the fixed-size indirection.
#[derive(Debug, PartialEq, Eq)]
pub enum Expr {
    Lit(i64),
    Add(Box<Expr>, Box<Expr>),
}

/// Evaluate an expression tree.
pub fn eval(expr: &Expr) -> i64 {
    match expr {
        Expr::Lit(value) => *value,
        Expr::Add(left, right) => eval(left) + eval(right),
    }
}

```

`eval(left)` receives a `&Box<Expr>`, and deref coercion turns it into the `&Expr` the function expects, so the recursive calls need no explicit dereference.

Rc<RefCell<T>> inside one thread

```

/// Shared mutation inside one thread, with `Rc` for aliasing and `RefCell` for
/// the runtime borrow check.
pub fn shared_counter_with_rc_refcell() -> i32 {
    let counter = Rc::new(RefCell::new(0));
    let alias = Rc::clone(&counter);

    *alias.borrow_mut() += 1;
    *counter.borrow_mut() += 10;

    *counter.borrow()
}

```

`Rc::clone(&counter)` increments a reference count and returns a second handle to the same allocation; it does not copy the integer. `borrow_mut` returns a `RefMut` guard that dereferences to `&mut i32`. Each guard is a temporary that

ends with its statement, so the two mutable borrows never overlap and neither panics. The function returns 11.

If the two guards were alive at the same time, for example `let a = alias.borrow_mut(); let b = counter.borrow_mut();`, the second call would panic with `already borrowed`. The compiler cannot see the conflict because both handles are shared references; `RefCell` checks it at run time instead.

Weak for a back-reference

```
/// A node whose parent link is weak, so a child does not keep its parent alive.
pub struct TreeNode {
    pub value: i32,
    parent: RefCell<Weak<TreeNode>>,
}

impl TreeNode {
    /// A node with no parent.
    pub fn root(value: i32) -> Rc<Self> {
        Rc::new(Self {
            value,
            parent: RefCell::new(Weak::new()),
        })
    }

    /// A node that points back weakly at `parent`.
    pub fn child_of(parent: &Rc<Self>, value: i32) -> Rc<Self> {
        Rc::new(Self {
            value,
            parent: RefCell::new(Rc::downgrade(parent)),
        })
    }

    /// The parent's value, or `None` once the parent has been dropped.
    pub fn parent_value(&self) -> Option<i32> {
        self.parent.borrow().upgrade().map(|parent| parent.value)
    }
}
```

`Rc::downgrade(parent)` creates a `Weak` that does not contribute to the strong count. When the last `Rc` to the parent is dropped, the parent is freed even though the child still holds the `Weak`. `upgrade` then returns `None`, and `parent_value` reports absence rather than reading freed memory.

The `parent` field is a `RefCell<Weak<TreeNode>>`. This module only reads the cell, but the cell is what would let a parent be attached or replaced after the child is created, through a shared `Rc<TreeNode>`.

Arc<Mutex<T>> across threads

```
/// Shared mutation across threads, with `Arc` for ownership and `Mutex` for
/// exclusive access.
pub fn total_with_arc_mutex(threads: usize, per_thread: usize) -> usize {
    let total = Arc::new(Mutex::new(0usize));
```

```

let mut handles = Vec::with_capacity(threads);

for _ in 0..threads {
    let total = Arc::clone(&total);
    handles.push(thread::spawn(move || {
        for _ in 0..per_thread {
            let mut guard = total.lock().expect("mutex poisoned");
            *guard += 1;
        }
    }));
}

for handle in handles {
    handle.join().expect("worker panicked");
}

*total.lock().expect("mutex poisoned")
}

```

`Arc` is the thread-safe version of `Rc`: its reference count is updated with atomic operations, so handles can be cloned and dropped on different threads. `Mutex` provides the exclusive access that `RefCell` provides on one thread, and it blocks rather than panics when the value is already borrowed.

The lock is taken once per increment. That is correct and slow: each increment pays for a lock acquisition. Chapter 43 replaces the mutex with an `AtomicUsize` for this exact workload.

Cow to allocate only when necessary

```

/// Borrow when no change is needed, allocate only when there is one.
///
/// `Cow` is the idiomatic return type for a function that usually passes its
/// input through unchanged.
pub fn normalize(input: &str, uppercase: bool) -> Cow<'_, str> {
    if uppercase && input.bytes().any(|byte| byte.is_ascii_lowercase()) {
        Cow::Owned(input.to_uppercase())
    } else {
        Cow::Borrowed(input)
    }
}

```

The function scans the bytes once. If uppercasing is not requested, or the input has no lowercase ASCII letter, it returns `Cow::Borrowed(input)` and allocates nothing. Otherwise it allocates the uppercased `String`. A caller treats both cases the same way, because `Cow<str>` dereferences to `&str`, and a caller that needs ownership calls `into_owned`.

The lifetime in `Cow<'_, str>` is elided to the lifetime of `input`, which is what ties a borrowed result to the argument it came from.

The last listing shows the tests, one per example.

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn boxed_expression_evaluates() {
        let expr = Expr::Add(Box::new(Expr::Lit(1)), Box::new(Expr::Lit(2)));
        assert_eq!(eval(&expr), 3);
    }

    #[test]
    fn rc_refcell_shares_mutation_within_a_thread() {
        assert_eq!(shared_counter_with_rc_refcell(), 11);
    }

    #[test]
    fn weak_parent_link_does_not_keep_the_parent_alive() {
        let root = TreeNode::root(1);
        let child = TreeNode::child_of(&root, 2);

        assert_eq!(child.parent_value(), Some(1));
        drop(root);
        assert_eq!(child.parent_value(), None);
    }

    #[test]
    fn arc_mutex_sums_concurrent_updates() {
        assert_eq!(total_with_arc_mutex(8, 250), 2_000);
    }

    #[test]
    fn cow_borrows_and_owns_as_needed() {
        assert!(matches!(normalize("A100", true), Cow::Borrowed(_)));
        assert!(matches!(normalize("a100", true), Cow::Owned(_)));
        assert_eq!(normalize("a100", true), "A100");
        assert_eq!(normalize("a100", false), "a100");
    }
}
```

Intuition

The weak-parent test is the one that shows a pointer type changing state over time. The table below records the reference counts at each step.

Counts during `weak_parent_link_does_not_keep_the_parent_alive`

step	strong count of root	weak count of root	child.parent_value()
<code>let root = TreeNode::root(1)</code>	1	0	not called
<code>let child = TreeNode::child_of(&root, 2)</code>	1	1	<code>Some(1)</code>
<code>drop(root)</code>	0, the node is dropped	1, the allocation remains until the Weak is gone	<code>None</code>

The allocation that held the parent's control block is released when the child, and with it the last `Weak`, is dropped at the end of the test. The `TreeNode` value itself was dropped at `drop(root)`.

The `Cow` test produces the following results:

```
normalize("A100", true)  no lowercase byte    Cow::Borrowed("A100")
normalize("a100", true)  'a' is lowercase  Cow::Owned("A100")
normalize("a100", false) uppercase not wanted Cow::Borrowed("a100")
```

Time and Space Complexity

Operation	Time	Space
<code>Box::new</code>	one allocation	the value plus nothing
<code>Rc::clone</code> , <code>Rc drop</code>	one non-atomic increment or decrement	two counts per allocation
<code>Arc::clone</code> , <code>Arc drop</code>	one atomic increment or decrement	two counts per allocation
<code>RefCell::borrow_mut</code>	one flag check and write	one isize flag per cell
<code>Mutex::lock</code>	uncontended: one atomic operation; contended: a system call	a few bytes of lock state
<code>Weak::upgrade</code>	one count check and increment	none
<code>normalize</code>	$O(n)$ scan, plus $O(n)$ copy when owned	zero or one allocation

Limitations

`Rc<RefCell<T>>` moves errors from compile time to run time. A borrow conflict in the `Rc<RefCell<T>>` example is a panic, not a compile error. Code that can be written with plain ownership and `&mut` should be, and the arena in chapter 18 shows a design that avoids `RefCell` altogether.

`Arc<Mutex<T>>` serializes the work it protects. In the `Arc<Mutex<T>>` example, eight threads spend most of their time waiting for one lock. The type makes sharing safe; it does not make it fast.

Weak does not prevent cycles on its own. It is a tool for breaking them, and the program has to choose which link is weak. Two `Rc` links in a cycle leak both values, and no warning is issued.

normalize looks only for ASCII lowercase letters. An input such as `"straße"` has lowercase letters, and so is converted, but an input consisting only of non-ASCII lowercase letters such as `"é"` is returned borrowed and unchanged even though `to_uppercase` would change it. The check and the conversion disagree on what "lowercase" means.

Summary

- `Box` gives a recursive type a finite size and has a single owner.
- `Rc` shares ownership within a thread, and `Arc` shares it across threads with atomic counts.
- `RefCell` and `Mutex` allow mutation through a shared handle; `RefCell` panics on a conflicting borrow, and `Mutex` blocks.
- `Weak` observes without owning, which lets back-references coexist with reference counting; `upgrade` returns `None` after the target is dropped.
- `Cow` returns a borrow when no change is needed and allocates only when one is.

References

- The Rust Book, [Smart pointers](#).
- The Rust Book, [Reference cycles can leak memory](#).
- Standard library, `std::rc`, `std::cell`, and `std::sync::Arc`.
- Standard library, `std::borrow::Cow`.

Part XII: Recursion, Memory, and Caches

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Problem Statement

Designing a Solution

```

fib(5)
  /      \
fib(4)    fib(3)
 /  \    /  \
fib(3) fib(2) fib(2) fib(1)
 /  \  /  \  /  \
fib(2) fib(1) fib(1) fib(0) fib(1) fib(0)
 /  \
fib(1) fib(0)

```

Two changes remove the duplication.

Memoization keeps the recursion and stores each answer the first time it is computed. Every later request for the same subproblem is a table read, so each of the $n + 1$ subproblems is computed once.

Iteration reverses the direction. It starts from `fib(0)` and `fib(1)` and advances a pair of accumulators `n` times. It needs only the previous two values, so it uses constant space.

Implementation

The first listing shows the three functions.

```
///! Three ways to compute Fibonacci: naive recursion, iteration, and memoization.
///! The first shows why exponential blowup happens; the other two fix it.
///!
///! Run with: cargo run --bin cs_fib

use std::time::{Duration, Instant};

///! Exponential: fib(n) recomputes fib(n - 1) and fib(n - 2) from scratch.
fn recursive(n: u64) -> u64 {
    if n < 2 {
        n
    } else {
        recursive(n - 1) + recursive(n - 2)
    }
}

///! Linear time, constant space.
fn iterative(n: u64) -> u64 {
    let (mut previous, mut current) = (0u64, 1u64);
    for _ in 0..n {
        (previous, current) = (current, previous + current);
    }
    previous
}

///! Linear time, linear space: recursion plus a cache of subproblem answers.
fn memoized(n: u64) -> u64 {
    fn go(n: u64, cache: &mut [u64]) -> u64 {
        if n < 2 {
            return n;
        }
        let index = n as usize;
        if cache[index] == 0 {
            cache[index] = go(n - 1, cache) + go(n - 2, cache);
        }
        cache[index]
    }

    let mut cache = vec![0u64; n as usize + 1];
    go(n, &mut cache)
}
```

`recursive` is the recurrence written as an `if` expression. Nothing in it is wrong except its cost.

`iterative` keeps `(previous, current)` equal to `(fib(i), fib(i + 1))` at the start of iteration `i`. The statement `(previous, current) = (current, previous + current)` is a destructuring assignment, stable since Rust 1.59. The right side is evaluated in full before either variable is written, so no temporary is needed. After `n` iterations, `previous` holds `fib(n)`.

`memoized` declares a nested function `go` that receives the cache as `&mut [u64]`. A nested `fn` cannot capture variables from the enclosing function, so the cache is passed explicitly, as in chapter 13. A cache slot holding `0` means "not yet computed". That sentinel is safe because `fib(n)` is zero only for `n = 0`, and `n < 2` is handled before the cache is consulted.

The second listing shows the timing helper and `main`.

```
fn timed(label: &str, n: u64, operation: impl Fn(u64) -> u64) {
    let start = Instant::now();
    let value = operation(n);
    report(label, n, value, start.elapsed());
}

fn report(label: &str, n: u64, value: u64, elapsed: Duration) {
    println!(" {label:<10} fib({n:>2}) = {value:<20} in {elapsed:?}");
}

fn main() {
    println!("small input, all three should agree");
    timed("recursive", 20, recursive);
    timed("iterative", 20, iterative);
    timed("memoized", 20, memoized);

    println!("\nrecursive cost grows quickly");
    timed("recursive", 30, recursive);
    timed("iterative", 30, iterative);

    println!("\nlarge input: recursion alone would not finish");
    timed("iterative", 90, iterative);
    timed("memoized", 90, memoized);

    for n in 0..20 {
        assert_eq!(recursive(n), iterative(n));
        assert_eq!(memoized(n), iterative(n));
    }
    assert_eq!(iterative(30), 832_040);
    assert_eq!(iterative(90), 2_880_067_194_370_816_120);

    println!("\nall checks passed");
}
```

`timed` accepts the function as `impl Fn(u64) -> u64`. Each of the three function items has its own zero-sized type, so each call to `timed` is monomorphised for that function, and the call inside `timed` is a direct call.

The assertions at the end of `main` compare the three versions for `n` below 20 and pin two known values. The program prints `all checks passed` only when they hold.

Intuition

`iterative(5)` runs five iterations:

```
iteration  previous  current  after (previous, current) = (current, previous +
current)
start           0         1
1              1         1
2              1         2
3              2         3
4              3         5
5              5         8
return previous = 5
```

A release build on an NVIDIA Jetson board with an 8-core Cortex-A78AE printed the following:

```
small input, all three should agree
recursive fib(20) = 6765           in 39.489µs
iterative fib(20) = 6765           in 64ns
memoized  fib(20) = 6765           in 928ns

recursive cost grows quickly
recursive fib(30) = 832040         in 4.508408ms
iterative fib(30) = 832040         in 64ns

large input: recursion alone would not finish
iterative fib(90) = 2880067194370816120 in 96ns
memoized  fib(90) = 2880067194370816120 in 7.681µs

all checks passed
```

Going from `n = 20` to `n = 30` multiplies the recursive time by about 114, close to $1.618^{10} \approx 123$. At that rate `recursive(90)` would take about 1.5×10^{10} seconds, several centuries. The memoized version is slower than the iterative one because it allocates a vector of `n + 1` entries and makes `2n - 1` function calls. A single timed run of a function this short is dominated by measurement noise, so the nanosecond figures indicate order of magnitude only.

Time and Space Complexity

Version	Time	Space
recursive	$O(\phi^n)$, where $\phi \approx 1.618$	$O(n)$ stack frames

Version	Time	Space
memoized	$O(n)$	$O(n)$ for the cache and $O(n)$ stack frames
iterative	$O(n)$	$O(1)$

Limitations

u64 overflows early. `fib(93)` is the largest Fibonacci number that fits in a `u64`. `iterative` computes one term ahead, so `iterative(93)` evaluates `fib(94)` in its last iteration and panics with an arithmetic overflow in a debug build, even though the value it returns would fit. In a release build the addition wraps silently. `checked_add` would turn both into an explicit error, and `u128` extends the range to `fib(186)`.

memoized recurses to depth `n`. Each level uses one stack frame. With a `u64` result the value overflows at `n = 94`, long before the depth matters, but a version widened to an arbitrary-precision integer would reach the stack limit for large `n`. The iterative version has no such limit.

memoized uses 0 as a sentinel. The choice is correct for this sequence and would be wrong for a recurrence whose answers can be zero. `Option<u64>` in the cache, or a separate `computed` bitmap, removes the dependency on the values.

`n` as `usize` assumes a 64-bit target. On a 32-bit target a `u64` larger than `u32::MAX` truncates when cast. The values of `n` that can produce a `u64` result are far below that limit, so the cast is safe in practice and not in general.

The timings are single runs. Chapter 24 and chapter 40 describe the warm-up and repetition a measurement needs before its figures can be quoted.

Summary

- Direct recursion on the Fibonacci recurrence makes $2 \cdot \text{fib}(n + 1) - 1$ calls, which is exponential in `n`.
- Memoization stores each subproblem's answer on first computation, reducing the work to $O(n)$ while keeping the recursive structure.

- Iteration computes the same values bottom-up with two accumulators in `O(1)` space; tuple assignment updates both without a temporary.
- `fib(93)` is the largest value that fits in `u64`, and the iterative loop overflows one step earlier because it computes one term ahead.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Chapter 7, "Dynamic Programming".
- The Rust Reference, [Destructuring assignments](#).
- Standard library, `u64::checked_add`.

Four List Layouts and the Recursive Drop

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `benchmarking_examples/lists/`, the three demonstration programs `benchmarking_examples/list_box.rs`, `benchmarking_examples/list_enum.rs`, and `benchmarking_examples/list_drop.rs`, and the benchmark in `benchmarking_examples/benchmark.rs`. Run the benchmark with `cargo run --release --bin benchmark list`.

Problem Statement

Implement a stack-shaped singly linked list with `push_front`, `pop_front`, and `is_empty`, in each of four variants:

variant	node layout	deallocation
box	<code>Option<Box<Node<T>>></code>	compiler-generated
enum	<code>enum ListNode<T> { Empty, Next(T, Box<ListNode<T>>) }</code>	compiler-generated
box+drop	<code>Option<Box<Node<T>>></code>	a hand-written iterative <code>Drop</code>
enum+drop	<code>enum ListNode<T></code>	a hand-written iterative <code>Drop</code>

Measure the time to push and to drop a list in each variant, find the largest list the recursive variants can free, and check that the iterative variants can free a much larger one.

Designing a Solution

One trait, four implementations. A trait with the operations every variant supports lets the benchmark be written once as a generic function. Each call is monomorphised for its variant, so the trait adds no dispatch cost to the measurement.

Why the default drop recurses. When a `Box<Node<T>>` is dropped, the compiler drops the node's fields, and one of those fields is the next `Box`. Dropping it drops its node, whose `next` is dropped in turn. Each step is a nested call, so freeing a list of `n` nodes needs `n` stack frames at once. The diagram below shows the stack at the deepest point.

<code>drop(head: Option<Box<Node>>)</code>	frame 1, node 1 not yet freed
<code>drop(node1.next)</code>	frame 2, node 2 not yet freed
<code>drop(node2.next)</code>	frame 3, node 3 not yet freed
<code>drop(node3.next = None)</code>	frame 4, returns
<code>free node 3</code>	
<code>free node 2</code>	
<code>free node 1</code>	

The iterative fix. A hand-written `Drop` moves each node's `next` out before the node is freed. When the node is then dropped, its `next` is already `None`, so the drop does not descend, and the loop moves to the detached tail. The stack depth is constant.

Implementation

The trait

```
/// Singly linked lists in four variants, so one benchmark loop can cover all of
/// them.
///
/// The variants differ only in how a node is stored and in how the list frees
/// itself. `boxed` and `enum_node` free a chain of boxes recursively, which
/// overflows the stack past roughly 265,000 nodes. `boxed_drop` and `enum_drop`
/// unlink iteratively and scale.

pub mod boxed;
pub mod boxed_drop;
pub mod enum_drop;
pub mod enum_node;

/// The operations every variant supports, so the benchmark can be written once.
pub trait SinglyList<T>: Sized {
    /// A new, empty list.
    fn new() -> Self;

    /// Adds `value` to the front of the list.
    fn push_front(&mut self, value: T);
```

```

    /// Removes and returns the front value, if there is one.
    fn pop_front(&mut self) -> Option<T>;

    /// Whether the list holds nothing.
    fn is_empty(&self) -> bool;

    /// The variant's name, for benchmark output and error messages.
    fn variant() -> &'static str;
}

```

`fn new() -> Self` in a trait requires `Self: Sized`, which the bound `SinglyList<T>: Sized` states for every method at once. `variant` is an associated function rather than a method, so the benchmark can print a name without constructing a list.

The boxed layout

```

    /// `Option<Box<Node<T>>>`: one heap allocation per node.
    ///
    /// Pushing takes the old head and boxes it inside the new node. Freeing the head
    /// frees its `next`, which frees its `next`, so the drop recurses once per
    /// element and overflows the stack on a long list.

    use super::SinglyList;

    struct Node<T> {
        value: T,
        next: Option<Box<Node<T>>>,
    }

    pub struct LinkedList<T> {
        head: Option<Box<Node<T>>>,
    }

    impl<T> SinglyList<T> for LinkedList<T> {
        fn new() -> Self {
            Self { head: None }
        }

        fn push_front(&mut self, value: T) {
            self.head = Some(Box::new(Node {
                value,
                // .take() leaves None in its place and extracts the old head
                next: self.head.take(),
            }));
        }

        fn pop_front(&mut self) -> Option<T> {
            self.head.take().map(|node| {
                // Point the list head to the next node in line
                self.head = node.next;
                // Return the unboxed value
                node.value
            })
        }

        fn is_empty(&self) -> bool {
            self.head.is_none()
        }

        fn variant() -> &'static str {

```

```

    "box"
  }
}

```

`self.head.take()` moves the old head out and leaves `None` in the field, so the new node can own it without an intermediate state in which the list has two owners. `pop_front` uses `map` on the taken head: the closure moves `node.next` back into the list and returns `node.value`, and the `Box` holding the node is freed when the closure returns.

The enumeration layout

```

///! `enum ListNode<T> { Empty, Next(T, Box<ListNode<T>>) }`.
///!
///! Same shape as the boxed variant, written with an enum instead of an
///! `Option`. The enum cannot contain itself by value, so the recursive arm holds
///! a `Box`. The drop is recursive for the same reason.

use super::SinglyList;

enum ListNode<T> {
    Empty,
    Next(T, Box<ListNode<T>>),
}

pub struct LinkedList<T> {
    head: ListNode<T>,
}

impl<T> SinglyList<T> for LinkedList<T> {
    fn new() -> Self {
        Self {
            head: ListNode::Empty,
        }
    }

    fn push_front(&mut self, value: T) {
        // Extract the old head and leave Empty in its place
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);

        // Wrap the old head inside a Box behind the new value
        self.head = ListNode::Next(value, Box::new(old_head));
    }

    fn pop_front(&mut self) -> Option<T> {
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);

        match old_head {
            ListNode::Empty => None,
            ListNode::Next(value, next_node) => {
                // Point the list head to the next node in the sequence
                self.head = *next_node;
                Some(value)
            }
        }
    }

    fn is_empty(&self) -> bool {
        matches!(self.head, ListNode::Empty)
    }
}

```



```
fn variant() -> &'static str {
    "enum"
}
}
```

The enumeration cannot use `take`, which exists only on `Option`, so it uses `std::mem::replace(&mut self.head, ListNode::Empty)` to the same effect. `self.head = *next_node` moves the boxed tail out of its box into the list head and frees the box.

The two layouts differ in what the list head holds. `Option<Box<Node<u64>>>` is 8 bytes, one pointer, because the null pointer encodes `None`. `ListNode<u64>` is 16 bytes: a `u64` and a `Box`. It needs no separate discriminant, because `Box` is never null and the compiler uses the null pattern to encode `Empty`. The heap nodes are 16 bytes in both layouts. The difference is in the moves: the boxed variant's push moves an 8-byte pointer out of the head, while the enumeration variant's push moves a 16-byte `ListNode` out of the head and into a new box.

The iterative destructors

The two `+drop` variants repeat the push and pop code of their counterparts unchanged and add a `Drop` implementation.

```
/// `Option<Box<Node<T>>>` with an iterative `Drop`.
///
/// Push and pop are identical to [`boxed`](super::boxed); only freeing changes.
/// Dropping a node would otherwise drop its `next` and recurse once per element,
/// so the list is unlinked in a loop before any node is freed.

use super::SinglyList;

struct Node<T> {
    value: T,
    next: Option<Box<Node<T>>>,
}

pub struct LinkedList<T> {
    head: Option<Box<Node<T>>>,
}

impl<T> SinglyList<T> for LinkedList<T> {
    fn new() -> Self {
        Self { head: None }
    }

    fn push_front(&mut self, value: T) {
        self.head = Some(Box::new(Node {
            value,
            next: self.head.take(),
        }));
    }

    fn pop_front(&mut self) -> Option<T> {
        self.head.take().map(|node| {
            self.head = node.next;
            node.value
        })
    }
}
```

```

    })
}

fn is_empty(&self) -> bool {
    self.head.is_none()
}

fn variant() -> &'static str {
    "box+drop"
}
}

impl<T> Drop for LinkedList<T> {
    fn drop(&mut self) {
        // Walk the chain and unlink each node before it is freed, so no node's
        // drop has to carry the rest of the list on the stack.
        let mut current = self.head.take();
        while let Some(mut node) = current {
            current = node.next.take();
        }
    }
}

```

while let Some(mut node) = current takes ownership of one node per iteration. current = node.next.take() detaches the rest of the list before node goes out of scope at the end of the loop body. The node is then freed with next == None, so its destructor has nothing to recurse into.

```

///! `enum ListNode<T>` with an iterative `Drop`.
///!
///! Push and pop are identical to [enum_node](super::enum_node); only freeing
///! changes. The recursive arm holds a `Box`, so the default drop walks the chain
///! on the stack. This one replaces the head with `Empty`, then takes the box out
///! of each node in turn.

use super::SinglyList;

enum ListNode<T> {
    Empty,
    Next(T, Box<ListNode<T>>),
}

pub struct LinkedList<T> {
    head: ListNode<T>,
}

impl<T> SinglyList<T> for LinkedList<T> {
    fn new() -> Self {
        Self {
            head: ListNode::Empty,
        }
    }

    fn push_front(&mut self, value: T) {
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);
        self.head = ListNode::Next(value, Box::new(old_head));
    }

    fn pop_front(&mut self) -> Option<T> {
        let old_head = std::mem::replace(&mut self.head, ListNode::Empty);

        match old_head {
            ListNode::Empty => None,

```

```

        ListNode::Next(value, next_node) => {
            self.head = *next_node;
            Some(value)
        }
    }
}

fn is_empty(&self) -> bool {
    matches!(self.head, ListNode::Empty)
}

fn variant() -> &'static str {
    "enum+drop"
}
}

impl<T> Drop for LinkedList<T> {
    fn drop(&mut self) {
        // Take the chain out of the list first, then step it forward one node at
        // a time. Each node has already lost its `next` by the time it is freed,
        // so the drop stays on the heap.
        let mut current = std::mem::replace(&mut self.head, ListNode::Empty);
        while let ListNode::Next(_, next) = current {
            current = *next;
        }
    }
}
}

```

while `let ListNode::Next(_, next) = current` deconstructs the current node, binding the boxed tail and dropping the value. `current = *next` moves the tail out of its box. The assignment drops the previous value of `current`, which at that point has already been moved out of by the pattern, so nothing recursive happens.

The demonstration programs

```

/// Demo of the `Option<Box<Node<T>>>` list in `lists::boxed`.
///
/// Run with: cargo run --bin list_box

use nvidia_rust_interview_lab::lists::SinglyList;
use nvidia_rust_interview_lab::lists::boxed::LinkedList;

fn main() {
    let mut list = LinkedList::new();

    list.push_front(10);
    list.push_front(20);
    list.push_front(30);

    assert_eq!(list.pop_front(), Some(30));
    assert_eq!(list.pop_front(), Some(20));
    assert_eq!(list.pop_front(), Some(10));
    assert_eq!(list.pop_front(), None);
    assert!(list.is_empty());

    println!("All tests passed successfully!");
}

```

```

///! Demo of the `enum ListNode<T>` list in `lists::enum_node`.
///!
///! Run with: cargo run --bin list_enum

use nvidia_rust_interview_lab::lists::SinglyList;
use nvidia_rust_interview_lab::lists::enum_node::LinkedList;

fn main() {
    let mut list = LinkedList::new();
    list.push_front(42);
    list.push_front(100);

    assert_eq!(list.pop_front(), Some(100));
    assert_eq!(list.pop_front(), Some(42));
    assert_eq!(list.pop_front(), None);
    assert!(list.is_empty());
}

```

```

///! Demo of the iterative `Drop` in `lists::boxed_drop`.
///!
///! The same program against `lists::boxed` aborts with a stack overflow at this
///! size, because that variant frees its chain recursively.
///!
///! Run with: cargo run --release --bin list_drop

use nvidia_rust_interview_lab::lists::SinglyList;
use nvidia_rust_interview_lab::lists::boxed_drop::LinkedList;

fn main() {
    const N: u64 = 5_000_000;

    let mut list = LinkedList::new();
    for value in 0..N {
        list.push_front(value);
    }
    println!("built a list of {N} nodes");

    list.pop_front();
    println!("dropping the remaining {} nodes", N - 1);

    drop(list);
    println!("dropped without recursing");
}

```

The programs import both the trait and the concrete type. Without `use nvidia_rust_interview_lab::lists::SinglyList`, the calls to `LinkedList::new` and `push_front` would not resolve, because trait methods are in scope only when the trait is.

The benchmark loop

The first listing below shows `main`, which dispatches between the cache benchmark of chapter 40 and the list benchmark, and the second shows the list benchmark itself.

```

///! One main for every benchmark in the lab.
///!
///! Usage:

```

```

/// cargo run --release --bin benchmark # cache, then lists
/// cargo run --release --bin benchmark cache # cache only
/// cargo run --release --bin benchmark list # all four variants
/// cargo run --release --bin benchmark list enum 500000 # one variant, one size
///
/// The recursive list variants abort with a stack overflow past roughly 265,000
/// nodes, so run them below that size or on their own.

use std::hint::black_box;
use std::time::{Duration, Instant};

use nvidia_rust_interview_lab::cache::Cache;
use nvidia_rust_interview_lab::cache::arena::LruCache as ArenaCache;
use nvidia_rust_interview_lab::cache::rc_list::LruCache as RcCache;
use nvidia_rust_interview_lab::lists::SinglyList;
use nvidia_rust_interview_lab::lists::{boxed, boxed_drop, enum_drop, enum_node};

const CACHE_CAPACITY: usize = 65_536;
const KEY_SPACE: u64 = 131_072;
const OPERATIONS: u64 = 10_000_000;
const DEFAULT_LIST_ELEMENTS: usize = 200_000;
/// Timed passes per cache; the fastest is reported, since a slow pass is noise.
const CACHE_RUNS: usize = 3;
/// Timed passes per list variant.
const LIST_RUNS: usize = 3;

fn main() {
    let arguments: Vec<String> = std::env::args().skip(1).collect();

    match arguments.first().map(String::as_str) {
        Some("cache") => cache_benchmark(),
        Some("list") => {
            let variant = arguments.get(1).map(String::as_str);
            let elements = arguments
                .get(2)
                .and_then(|value| value.parse().ok())
                .unwrap_or(DEFAULT_LIST_ELEMENTS);
            list_benchmark(variant, elements);
        }
        _ => {
            cache_benchmark();
            println!();
            list_benchmark(None, DEFAULT_LIST_ELEMENTS);
        }
    }
}

```

```

// -----
// Lists: four variants, one loop
// -----

fn list_benchmark(variant: Option<&str>, elements: usize) {
    println!(
        "Singly linked list: push {} values, pop them back in order, then drop a full",
        list",
        with_thousands(elements as u64)
    );
    println!("Best of {LIST_RUNS} runs per variant, allocator warmed up.\n");
    println!(
        " {:<10} {:>12} {:>13} {:>13}",
        "variant", "elements", "push", "drop"
    );
    println!(" {} ", "-".repeat(52));

    match variant {
        Some("box") => run_list::boxed::LinkedList<u64>>(elements),

```

```

Some("enum") => run_list::<enum_node::LinkedList<u64>>(elements),
Some("box+drop") => run_list::<boxed_drop::LinkedList<u64>>(elements),
Some("enum+drop") => run_list::<enum_drop::LinkedList<u64>>(elements),
_ => {
    run_list::<boxed::LinkedList<u64>>(elements);
    run_list::<enum_node::LinkedList<u64>>(elements);
    run_list::<boxed_drop::LinkedList<u64>>(elements);
    run_list::<enum_drop::LinkedList<u64>>(elements);
}
}
}

fn run_list<L: SinglyList<u64>>(elements: usize) {
    // Warm the allocator first: without this the first variant in a process pays
    // for fresh heap pages and looks several times slower than an identical one.
    let mut warm_up = L::new();
    for value in 0..elements as u64 {
        warm_up.push_front(value);
    }
    drop(warm_up);

    let mut best_push = Duration::MAX;
    let mut best_drop = Duration::MAX;

    for _ in 0..LIST_RUNS {
        let mut list = L::new();

        let start = Instant::now();
        for value in 0..elements as u64 {
            list.push_front(value);
        }
        best_push = best_push.min(start.elapsed());

        // Pop every value back and check the order, so a broken list cannot post
        // a fast time.
        let mut expected = elements as u64;
        while let Some(value) = list.pop_front() {
            expected -= 1;
            assert_eq!(value, expected, "{ } lost LIFO order", L::variant());
        }
        assert_eq!(expected, 0);
        black_box(expected);

        // Rebuild, then time the drop of a full list. This is where the
        // recursive variants overflow the stack.
        for value in 0..elements as u64 {
            list.push_front(value);
        }
        let start = Instant::now();
        drop(list);
        best_drop = best_drop.min(start.elapsed());
    }

    println!(
        " {:<10} {:>12} {best_push:>13.2?} {best_drop:>13.2?}",
        L::variant(),
        with_thousands(elements as u64)
    );
}

fn with_thousands(value: u64) -> String {
    let digits = value.to_string();
    let mut out = String::with_capacity(digits.len() + digits.len() / 3);
    for (index, digit) in digits.chars().enumerate() {
        if index > 0 && (digits.len() - index) % 3 == 0 {
            out.push(',');
        }
    }
}

```

```

        out.push(digit);
    }
    out
}

```

`run_list::<boxed::LinkedList<u64>>(elements)` names the variant with a turbofish. The generic function calls `L::new()`, `push_front`, and `pop_front` through the trait, and each instantiation compiles to direct calls on its variant.

Four measurement decisions are visible in `run_list`:

- **A warm-up pass.** The first list built in a process pays for fresh heap pages. Building and dropping one list before timing removes that cost from the first variant measured.
- **Best of three.** Each timed pass records its duration, and the minimum is reported, because interference from the rest of the system only ever adds time.
- **A correctness check inside the timed program.** Every value is popped back and compared with the expected LIFO order, so a broken variant fails instead of posting a fast time.
- **black_box.** `std::hint::black_box(expected)` tells the optimiser to treat the value as used, so the check cannot be removed as dead code.

`with_thousands` formats counts with separators for the report.

Intuition

The benchmark was run in release mode on an NVIDIA Jetson board, an 8-core Cortex-A78AE with an 8 MiB main-thread stack. The recorded results are in `BENCHMARKS.md` in the repository.

The largest list each variant can free

variant	260,000 nodes	270,000 nodes	5,000,000 nodes
<code>box</code>	completed	aborted	not attempted
<code>enum</code>	completed	aborted	not attempted
<code>box+drop</code>	completed	completed	completed
<code>enum+drop</code>	completed	completed	completed

The recursive variants abort between 260,000 and 270,000 nodes with:

```
thread 'main' has overflowed its stack
fatal runtime error: stack overflow, aborting
```

An 8 MiB stack divided by roughly 265,000 frames is about 32 bytes per frame.

The timed results, as best of three passes:

variant	elements	push	drop
box	200,000	2.40ms	3.23ms
enum	200,000	3.42ms	3.20ms
box+drop	200,000	2.32ms	2.91ms
enum+drop	200,000	3.62ms	2.52ms
box+drop	5,000,000	58.11ms	72.32ms
enum+drop	5,000,000	90.98ms	67.51ms

Two conclusions follow. First, the enumeration layout pushes more slowly, about 17 ns per element against 12 ns, which is consistent with each push moving a 16-byte node value rather than an 8-byte pointer. Second, the iterative destructor is not meaningfully slower than the recursive one; at 200,000 nodes it is slightly faster. Its value is that it removes the size limit.

`box` and `box+drop` share the same push code, and their push times of 2.40 ms and 2.32 ms agree within run-to-run variation. That agreement is a check on the measurement itself.

Time and Space Complexity

Operation	Time	Stack space	Heap
push_front	$O(1)$	$O(1)$	one allocation
pop_front	$O(1)$	$O(1)$	one deallocation
default drop of n nodes	$O(n)$	$O(n)$ frames	n deallocations
iterative drop of n nodes	$O(n)$	$O(1)$	n deallocations

Limitations

The limit depends on the stack size. 265,000 nodes is the figure for an 8 MiB main-thread stack. A spawned thread has a 2 MiB stack by default,

so the same program on a worker thread fails at roughly a quarter of that length. The iterative variants have no such dependency.

The `+drop` variants duplicate code. Push and pop are copied from their counterparts so that each file is complete. A shared inner type with the `Drop` added in a wrapper would remove the duplication, at the cost of one more type.

The enumeration layout stores a boxed `Empty`. The first push boxes the initial `Empty` head, so the last heap node of every non-empty list holds no value. The allocation count is still one per push, because the most recent value lives inline in the list head, but one of the `n` heap blocks carries no data.

The measurement covers one allocator and one machine. The ratios are the figures intended to carry over. The absolute times depend on the system allocator, the compiler version, and the processor.

Summary

- A trait with a `Sized` bound lets one generic benchmark exercise several implementations with no dispatch overhead.
- A compiler-generated destructor for a chain of boxes recurses once per node, and an 8 MiB stack is exhausted at roughly 265,000 nodes.
- An iterative `Drop` that detaches each node's successor before freeing it runs in constant stack space and frees five million nodes without difficulty.
- The iterative destructor is not slower. Its benefit is the removal of the size limit.
- A benchmark needs a warm-up pass, several repetitions reported as the minimum, and a correctness check that the optimiser cannot remove.

References

- The Rust Book, [Running code on cleanup with the `Drop` trait](#).
- Aria Beingessner and contributors, *Learning Rust With Entirely Too Many Linked Lists*, [Drop](#).
- Standard library, `std::hint::black_box`.
- Standard library, `std::thread`, on the default stack size of spawned threads.

A Bump Allocator

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/bump_allocator.rs`. Test it with `cargo test bump_allocator`.

Problem Statement

Given a buffer of `capacity` bytes, implement `alloc(size, align)` that returns a mutable slice of `size` bytes whose start is a multiple of `align`, or `None` when the request does not fit. Individual allocations are never freed; `reset` makes the whole buffer available again. A failed allocation must leave the arena unchanged.

Designing a Solution

The arena keeps one integer, `offset`, the index of the first unused byte. An allocation rounds `offset` up to the next multiple of `align`, checks that `size` bytes fit after that point, and moves `offset` past them. The diagram below shows two allocations.

```
alloc(1, 8)    offset 0 is already a multiple of 8; bytes [0, 1) are returned
               offset = 1

alloc(4, 16)   align_up(1, 16) = 16; bytes [16, 20) are returned
               offset = 20; bytes [1, 16) are padding and are never handed out

0   1               16   20                               64
[a][ padding ..... ][bbbb][ free ..... ]
```

Rounding up to a power of two does not need division. For `align = 2k`, the mask `align - 1` has the low `k` bits set. Adding the mask and then clearing

those bits gives the smallest multiple of `align` that is not less than the original offset:

```
align_up(offset, align) = (offset + (align - 1)) & !(align - 1)

align_up(1, 16)  = (1 + 15) & !15 = 16 & ...11110000 = 16
align_up(16, 16) = (16 + 15) & !15 = 31 & ...11110000 = 16
align_up(17, 16) = (17 + 15) & !15 = 32 & ...11110000 = 32
```

The only arithmetic that can overflow is `offset + mask` and `start + size`. Both use `checked_add`, and an overflow is reported as `None` like any other request that does not fit.

Implementation

```
/// A bump (arena) allocator over a fixed byte buffer.
///
/// Bump allocation is the simplest allocator: hand out bytes from a cursor and
/// never free an individual block. It is O(1) per allocation, has no fragmentation
/// bookkeeping, and is freed all at once with [BumpArena::reset]. Parsers,
/// request handlers, and per-frame GPU upload buffers use exactly this shape.
///
/// The point of the exercise is not to beat the system allocator; it is to show
/// that you understand alignment, the memory layout you hand to hardware, and why
/// "free everything at once" is both the strength and the limitation.

/// A fixed-capacity bump allocator.
pub struct BumpArena {
    buffer: Vec<u8>,
    offset: usize,
}

impl BumpArena {
    /// Create an arena backed by `capacity` zeroed bytes.
    pub fn with_capacity(capacity: usize) -> Self {
        Self {
            buffer: vec![0; capacity],
            offset: 0,
        }
    }

    /// Allocate `size` bytes aligned to `align`.
    ///
    /// Returns `None` when the request does not fit. `align` must be a power of
    /// two.
    pub fn alloc(&mut self, size: usize, align: usize) -> Option<&mut [u8]> {
        assert!(align.is_power_of_two(), "align must be a power of two");

        let start = align_up(self.offset, align)?;
        let end = start.checked_add(size)?;
        if end > self.buffer.len() {
            return None;
        }

        self.offset = end;
        Some(&mut self.buffer[start..end])
    }

    /// Bytes handed out so far, including alignment padding.
}
```

```

pub fn used(&self) -> usize {
    self.offset
}

/// Bytes still available before the next allocation.
pub fn remaining(&self) -> usize {
    self.buffer.len() - self.offset
}

/// Total capacity.
pub fn capacity(&self) -> usize {
    self.buffer.len()
}

/// Free everything at once, as an arena does.
pub fn reset(&mut self) {
    self.offset = 0;
}
}

fn align_up(offset: usize, align: usize) -> Option<usize> {
    let mask = align - 1;
    offset.checked_add(mask).map(|value| value & !mask)
}

```

`alloc` returns `Option<&mut [u8]>`. The slice borrows the arena mutably, so the caller must finish using one allocation before requesting another. That restriction is what makes the function safe without `unsafe`: two live allocations would be two mutable borrows of the same `Vec`, which the borrow checker rejects. A production arena lifts the restriction by handing out raw pointers or by using interior mutability, and it then needs `unsafe` code to justify that the regions do not overlap.

The order of the statements in `alloc` implements the "failed allocation changes nothing" requirement. `start` and `end` are computed and checked first, and `self.offset = end` is the only write, performed after every check has passed.

`align_up` returns `Option<usize>` so that an offset near `usize::MAX` cannot wrap to a small value and appear to fit. `map(|value| value & !mask)` applies the mask only when the addition succeeded.

`reset` sets the cursor to zero and does not clear the bytes, so an allocation made after a reset can see data written before it. Only a fresh arena is guaranteed to be zeroed, by `vec![0; capacity]`, and the test `writes_are_isolated_between_allocations` depends on that.

The next listing shows the tests.

```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]

```

```

fn allocations_are_aligned() {
    let mut arena = BumpArena::with_capacity(64);

    let first = arena.alloc(1, 8).expect("fits");
    assert_eq!(first.as_ptr() as usize % 8, 0);
    assert_eq!(first.len(), 1);

    let second = arena.alloc(4, 16).expect("fits");
    assert_eq!(second.as_ptr() as usize % 16, 0);

    // The first block occupies 1 byte. Aligning the next block to 16 adds 15
    // bytes of padding, so the arena has consumed 16 + 4 = 20 bytes.
    assert_eq!(arena.used(), 20);
    assert_eq!(arena.remaining(), 44);
}

#[test]
fn exhaustion_returns_none_and_keeps_state() {
    let mut arena = BumpArena::with_capacity(16);
    assert!(arena.alloc(16, 1).is_some());

    let used_before = arena.used();
    assert!(arena.alloc(1, 1).is_none());
    assert_eq!(arena.used(), used_before, "a failed alloc must not advance");
}

#[test]
fn reset_reclaims_everything() {
    let mut arena = BumpArena::with_capacity(32);
    assert!(arena.alloc(20, 4).is_some());
    assert_eq!(arena.remaining(), 12);

    arena.reset();
    assert_eq!(arena.used(), 0);
    assert_eq!(arena.remaining(), arena.capacity());
    assert!(arena.alloc(32, 1).is_some());
}

#[test]
fn zero_sized_allocation_is_allowed() {
    let mut arena = BumpArena::with_capacity(8);
    let empty = arena.alloc(0, 8).expect("fits");
    assert!(empty.is_empty());
    assert_eq!(arena.used(), 0);
}

#[test]
fn writes_are_isolated_between_allocations() {
    let mut arena = BumpArena::with_capacity(16);
    arena
        .alloc(4, 4)
        .expect("fits")
        .copy_from_slice(&[1, 2, 3, 4]);
    let second = arena.alloc(4, 4).expect("fits");
    assert!(second.iter().all(|&byte| byte == 0));
    second.copy_from_slice(&[9, 9, 9, 9]);

    let view = arena.buffer.as_slice();
    assert_eq!(&view[0..4], &[1, 2, 3, 4]);
    assert_eq!(&view[4..8], &[9, 9, 9, 9]);
}

```

`writes_are_isolated_between_allocations` reads `arena.buffer` directly. The test module is a child of the module that defines `BumpArena`, so it can see the private

field. The test ends the borrow of `second` before it borrows `arena.buffer`, which is why `second` is not used after `copy_from_slice`.

Intuition

`allocations_are_aligned` on a 64-byte arena

call	offset before	start = align_up(offset, align)	end	offset after	remaining
<code>alloc(1, 8)</code>	0	0	1	1	63
<code>alloc(4, 16)</code>	1	16	20	20	44

`exhaustion_returns_none_and_keeps_state` on a 16-byte arena

call	start	end	end > 16 ?	result	offset after
<code>alloc(16, 1)</code>	0	16	no	<code>Some(16 bytes)</code>	16
<code>alloc(1, 1)</code>	16	17	yes	<code>None</code>	16, unchanged

A zero-sized allocation at offset 0 with alignment 8 returns an empty slice and leaves `offset` at 0. A zero-sized allocation at offset 1 with alignment 8 would move the cursor to 8 and consume seven bytes of padding for an allocation of no bytes.

Time and Space Complexity

Operation	Time	Space
<code>with_capacity</code>	$O(\text{capacity})$, to zero the buffer	capacity bytes
<code>alloc</code>	$O(1)$	none beyond the buffer; padding is lost until reset
<code>reset</code>	$O(1)$	none

Internal fragmentation is bounded by `align - 1` bytes per allocation. There is no external fragmentation, because nothing is freed individually.

Limitations

The allocator aligns offsets, not addresses. `alloc(size, 8)` returns a slice that starts at an offset divisible by 8 from the start of the buffer. The returned pointer is divisible by 8 only if the buffer's own address is. `Vec<u8>` has an alignment of 1, and nothing in the type guarantees that its heap block is aligned to 8 or 16. The test `allocations_are_aligned` passes because the system allocator on common platforms returns blocks aligned to at least 16 bytes. An arena intended to store typed values needs to allocate its buffer with `std::alloc::alloc` and a `Layout` of the maximum alignment it will serve, or align relative to the address rather than the offset.

One allocation at a time. The `&mut [u8]` return value prevents holding two allocations simultaneously, which is the normal use of an arena. The type is a correct model of the arithmetic and not yet a usable allocator.

Bytes, not values. `alloc` returns bytes. Placing a `T` in them requires `unsafe` code, and dropping that `T` requires the arena to remember to run its destructor, which a bump allocator does not do. Crates such as `bumpalo` address both problems.

reset does not zero. Memory handed out after a reset contains whatever the previous cycle wrote. For byte buffers that are always written before they are read this is acceptable; for anything else it is a source of stale data.

align is checked with assert!. A non-power-of-two alignment panics rather than returning `None`. The caller controls the alignment, so a panic reports a programming error rather than a runtime condition, which is a reasonable choice, and it should be documented as such.

Summary

- A bump allocator is a buffer and a cursor. Allocation rounds the cursor up to the requested alignment and advances it; `reset` frees everything at once.
- For a power-of-two alignment, `(offset + align - 1) & !(align - 1)` rounds up without division.

- `checked_add` on both additions turns arithmetic overflow into an ordinary failed allocation, and writing `offset` only after every check keeps a failure side-effect free.
- Returning `&mut [u8]` keeps the code safe and limits the caller to one live allocation.
- Aligning an offset inside a `Vec<u8>` does not align the address; a real arena allocates its buffer with the alignment it intends to serve.

References

- Standard library, `std::alloc::Layout`.
- Standard library, `usize::is_power_of_two` and `usize::checked_add`.
- The `bumpalo` crate, [documentation](#).
- The Rust Reference, [Type layout](#), on size and alignment.

Measuring Two LRU Layouts

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `benchmarking_examples/cache/` and `benchmarking_examples/benchmark.rs`. Run it with `cargo run --release --bin benchmark cache`.

Problem Statement

Implement a fixed-capacity LRU cache twice with identical behaviour:

- **arena**: nodes in a `Vec`, linked by `Option<usize>`;
- **rc_list**: one `Rc<RefCell<Node>>` per entry, linked forward by `Rc` and backward by `Weak`.

Drive both with the same request stream, check that they produce the same hits, and report the time per request.

Designing a Solution

A shared trait. Both caches implement `Cache<K, V>`, with `new`, `get`, `put`, `len`, and `variant`. The benchmark is a generic function over `C: Cache<u64, u64>`, so the only difference between the two measured loops is the cache type.

Values that are Copy. The trait requires `V: Copy`, and `get` returns `Option<V>` rather than a reference. This lets the `Rc<RefCell>` version return a value without holding a `Ref` guard past the end of the method, and keeps the two signatures identical.

A realistic request stream. A uniform stream over the key space would make every cache miss most of the time. The benchmark instead draws four requests in five from a hot set of 1 percent of the keys and the rest from the whole

space. The stream comes from a linear congruential generator seeded with a constant, so both caches see the same sequence and the hit counts can be compared.

Where the two designs differ. On a hit, both look up the key in a `HashMap` and move the entry to the front. The arena writes a few integers in one vector. The list clones an `Rc`, upgrades a `Weak`, takes `RefCell` borrows on up to three nodes, and follows pointers to separate heap blocks. On a miss at capacity, the arena overwrites the evicted slot, while the list frees the evicted node and allocates a new one.

Implementation

The contract

```

/// Two LRU caches of the same design, stored differently, so one benchmark loop
/// can cover both.
///
/// [`arena`] keeps nodes in a `Vec` and links them by `usize` index.
/// [`rc_list`] allocates an `Rc<RefCell<_>>` per node and links them by pointer.

pub mod arena;
pub mod rc_list;

use std::hash::Hash;

/// The operations every cache supports.
pub trait Cache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    /// A cache that holds at most `capacity` entries.
    ///
    /// # Panics
    /// Panics if `capacity == 0`, since a zero-capacity cache cannot store
    /// anything.
    fn new(capacity: usize) -> Self;

    /// Looks up `key`, promoting it to most recently used on a hit.
    fn get(&mut self, key: &K) -> Option<V>;

    /// Inserts or updates `key`, evicting the least recently used entry when
    /// the cache is full.
    fn put(&mut self, key: K, value: V);

    /// How many entries are cached.
    fn len(&self) -> usize;

    /// The variant's name, for benchmark output.
    fn variant() -> &'static str;
}

```

The arena

The arena version is the cache from chapter 18 with its public methods moved into the trait implementation. The first listing shows the types and `touch`, and the second the trait implementation.

```

/// Fixed-capacity LRU cache backed by an arena.
///
/// ## Design
///
/// Two structures cooperate to give O(1) `get` and `put`:
///
/// * `lookup_table: HashMap<K, usize>` – key to index into the arena.
/// * `nodes: Vec<Node<K, V>>` – an arena of nodes; indices play the role of
///   pointers, sidestepping `Rc<RefCell<...>` and its borrow-checker pain.
///
/// The arena is stitched into a doubly linked list through `head`/`tail`:
///
/// ```text
///   MRU -> head -> ... -> tail -> LRU
/// ```
///
/// On eviction the tail slot is reused instead of removed, so no node is
/// deallocated mid-life and the arena stays dense and index-stable.

use std::collections::HashMap;
use std::hash::Hash;

use super::Cache;

/// A single cache entry, linked into the recency list by index.
struct Node<K, V> {
    /// Retained so the corresponding map entry can be removed when evicting.
    key: K,
    value: V,
    /// Neighbour closer to the MRU end (`None` if this is `head`).
    prev: Option<usize>,
    /// Neighbour closer to the LRU end (`None` if this is `tail`).
    next: Option<usize>,
}

pub struct LruCache<K, V> {
    /// Maximum number of entries; `>= 1`.
    cap: usize,
    /// Key to arena index. The single source of truth for "is this cached?".
    lookup_table: HashMap<K, usize>,
    /// Arena of nodes; indices are the "pointers" of the linked list.
    nodes: Vec<Node<K, V>>,
    /// Arena index of the most recently used entry.
    head: Option<usize>,
    /// Arena index of the least recently used entry (next to evict).
    tail: Option<usize>,
}

impl<K, V> LruCache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    /// Promotes node `i` to `head` (MRU).
    ///
    /// Handles both cases in one pass:
    /// * Already linked – detach from its current neighbours first.
    /// * Brand new – the node has `prev = next = None` and is not yet

```

```

/// referenced by `head`/`tail`, so the detach step is skipped.
fn touch(&mut self, i: usize) {
    let (prev, next) = (self.nodes[i].prev, self.nodes[i].next);

    // A node is "linked" if any of its four references exist.
    let linked =
        prev.is_some() || next.is_some() || self.head == Some(i) || self.tail == So
me(i);

    if linked {
        // Splice `i` out of the list.
        match prev {
            Some(p) => self.nodes[p].next = next,
            None => self.head = next,
        }
        match next {
            Some(n) => self.nodes[n].prev = prev,
            None => self.tail = prev,
        }
    }

    // Splice `i` in at the front.
    self.nodes[i].prev = None;
    self.nodes[i].next = self.head;
    if let Some(h) = self.head {
        self.nodes[h].prev = Some(i);
    }
    self.head = Some(i);
    if self.tail.is_none() {
        self.tail = Some(i);
    }
}
}

```

```

impl<K, V> Cache<K, V> for LruCache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            cap: capacity,
            lookup_table: HashMap::new(),
            nodes: Vec::new(),
            head: None,
            tail: None,
        }
    }

    fn get(&mut self, key: &K) -> Option<V> {
        let i = *self.lookup_table.get(key)?;
        self.touch(i);
        Some(self.nodes[i].value)
    }

    fn put(&mut self, key: K, value: V) {
        // Case 1: key already present, update the value and promote.
        if let Some(&i) = self.lookup_table.get(&key) {
            self.nodes[i].value = value;
            self.touch(i);
            return;
        }

        // Case 2: room to grow, append a fresh node.
        if self.nodes.len() < self.cap {

```

```

        let i = self.nodes.len();
        self.nodes.push(Node {
            key: key.clone(),
            value,
            prev: None,
            next: None,
        });
        self.lookup_table.insert(key, i);
        self.touch(i);
        return;
    }

    // Case 3: at capacity, reuse the LRU slot. `tail` is `Some` here because
    // `cap >= 1` and the arena is full, but handle `None` anyway.
    if let Some(i) = self.tail {
        let old_key = std::mem::replace(&mut self.nodes[i].key, key);
        self.nodes[i].value = value;
        self.lookup_table.remove(&old_key);
        self.lookup_table.insert(self.nodes[i].key.clone(), i);
        self.touch(i);
    }

    }

    fn len(&self) -> usize {
        self.lookup_table.len()
    }

    fn variant() -> &'static str {
        "arena (Vec + indices)"
    }
}

```

get returns `Some(self.nodes[i].value)`, a copy, because the trait bound is `V: Copy`. Chapter 18's version returned `Option<&V>`.

The reference-counted list

```

/// LRU cache built from an `Rc<RefCell<_>>` doubly linked list.
///
/// The textbook version: `Rc` forward, `Weak` back, and a `HashMap` from key to
/// node. Every node is its own heap allocation, and every move to the front
/// clones an `Rc`, upgrades a `Weak`, and borrows the `RefCell`.
///
/// The default drop would free a node, which frees its `next`, one stack frame
/// per element, so `Drop` is written to unlink iteratively.

use std::cell::RefCell;
use std::collections::HashMap;
use std::hash::Hash;
use std::rc::{Rc, Weak};

use super::Cache;

type Link<K, V> = Option<Rc<RefCell<Node<K, V>>>>;

struct Node<K, V> {
    key: K,
    value: V,
    prev: Weak<RefCell<Node<K, V>>>,
    next: Link<K, V>,
}

pub struct LruCache<K, V> {

```

```

    cap: usize,
    map: HashMap<K, Rc<RefCell<Node<K, V>>>>,
    head: Link<K, V>,
    tail: Link<K, V>,
}

```

`Link<K, V>` is the type alias for an owning, optional forward link. The backward link `prev` is a `Weak`, so two neighbours do not keep each other alive. The cache holds three kinds of strong reference to a node: the map entry, the previous node's `next` (or `head`), and, for the last node, `tail`.

```

impl<K, V> LruCache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    fn detach(&mut self, node: &Rc<RefCell<Node<K, V>>>) {
        let (prev, next) = {
            let entry = node.borrow();
            (entry.prev.clone(), entry.next.clone())
        };

        match prev.upgrade() {
            Some(previous) => previous.borrow_mut().next = next.clone(),
            None => self.head = next.clone(),
        }
        match &next {
            Some(following) => following.borrow_mut().prev = prev,
            None => self.tail = prev.upgrade(),
        }
    }

    fn push_front(&mut self, node: Rc<RefCell<Node<K, V>>>) {
        let head = self.head.clone();
        {
            let mut entry = node.borrow_mut();
            entry.prev = Weak::new();
            entry.next = head.clone();
        }
        match &head {
            Some(previous) => previous.borrow_mut().prev = Rc::downgrade(&node),
            None => self.tail = Some(node.clone()),
        }
        self.head = Some(node);
    }
}

```

`detach` first clones both links out of the node inside a block, so the `Ref` guard from `node.borrow()` ends before any `borrow_mut` runs. `RefCell` checks borrows at run time, and a guard that lives longer than it needs to is the usual cause of an `already borrowed` panic. The block rules that out by construction, at the cost of one `Weak` clone and one `Rc` clone.

`prev.upgrade()` turns the weak back-link into a strong one for the duration of the statement. When it returns `None`, the node was the head, and `head` is

moved to the successor. When `next` is `None`, the node was the tail, and `tail` becomes the upgraded predecessor.

`push_front` clears the node's `prev`, points its `next` at the old head, and makes the old head's `prev` a `Weak` to the node. If the list was empty, the node becomes the tail as well.

```
impl<K, V> Cache<K, V> for LruCache<K, V>
where
    K: Hash + Eq + Clone,
    V: Copy,
{
    fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            cap: capacity,
            map: HashMap::new(),
            head: None,
            tail: None,
        }
    }

    fn get(&mut self, key: &K) -> Option<V> {
        let node = self.map.get(key)?.clone();
        self.detach(&node);
        self.push_front(node.clone());
        Some(node.borrow().value)
    }

    fn put(&mut self, key: K, value: V) {
        if let Some(node) = self.map.get(&key).cloned() {
            node.borrow_mut().value = value;
            self.detach(&node);
            self.push_front(node);
            return;
        }

        if self.map.len() >= self.cap {
            if let Some(victim) = self.tail.clone() {
                let old_key = victim.borrow().key.clone();
                self.map.remove(&old_key);
                self.detach(&victim);
            }
        }

        let node = Rc::new(RefCell::new(Node {
            key: key.clone(),
            value,
            prev: Weak::new(),
            next: None,
        }));
        self.map.insert(key, node.clone());
        self.push_front(node);
    }

    fn len(&self) -> usize {
        self.map.len()
    }

    fn variant() -> &'static str {
        "Rc<RefCell> list"
    }
}
```

```

impl<K, V> Drop for LruCache<K, V> {
    fn drop(&mut self) {
        // Unlink iteratively: dropping a million-node chain recursively overflows
        // the stack.
        let mut current = self.head.take();
        while let Some(node) = current {
            let next = node.borrow_mut().next.take();
            node.borrow_mut().prev = Weak::new();
            current = next;
        }
    }
}

```

`get` clones the `Rc` out of the map before calling `detach`, because `detach` takes `&mut self` and the map entry borrows `self`. The clone is a reference-count increment. Each hit therefore performs several count updates, a `Weak` upgrade, and four or more `RefCell` borrow checks, none of which the arena needs.

The `Drop` implementation unlinks the chain iteratively for the reason chapter 38 measured: the default destructor would free a 65,536-node chain recursively.

The benchmark

```

// -----
// Cache: arena vs Rc<RefCell> linked list
// -----

/// Request stream with realistic locality: four requests in five come from a hot
/// set of 1% of the keys, the rest from the whole space.
fn key_at(state: u64) -> u64 {
    let mixed = state >> 17;
    if (state >> 33) % 100 < 80 {
        mixed % (KEY_SPACE / 100)
    } else {
        mixed % KEY_SPACE
    }
}

/// The fastest of `CACHE_RUNS` timed passes, with the hit count and final size
/// from the last one. Each pass starts from an empty cache, so the stream is the
/// same every time.
fn best_cache_run<C: Cache<u64, u64>>() -> (Duration, u64, usize) {
    let mut best = Duration::MAX;
    let mut hits = 0;
    let mut len = 0;

    for _ in 0..CACHE_RUNS {
        let mut cache = C::new(CACHE_CAPACITY);
        let (elapsed, run_hits) = cache_workload(&mut cache);
        best = best.min(elapsed);
        hits = run_hits;
        len = cache.len();
    }
    (best, hits, len)
}

/// The same request stream against any cache.
fn cache_workload<C: Cache<u64, u64>>(cache: &mut C) -> (Duration, u64) {

```



```

let mut state = 0x9e37_79b9_7f4a_7c15u64;
let mut hits = 0u64;

let start = Instant::now();
for _ in 0..OPERATIONS {
    state = state
        .wrapping_mul(6_364_136_223_846_793_005)
        .wrapping_add(1_442_695_040_888_963_407);
    let key = key_at(state);

    match cache.get(&key) {
        Some(value) => {
            hits += 1;
            black_box(value);
        }
        None => cache.put(key, key.rotate_left(17)),
    }
}
(start.elapsed(), hits)
}

fn cache_benchmark() {
    println!(
        "LRU cache: {} entries, {} requests over {} keys",
        with_thousands(CACHE_CAPACITY as u64),
        with_thousands(OPERATIONS),
        with_thousands(KEY_SPACE)
    );
    println!("A miss inserts and evicts the least recently used entry.");
    println!("Best of {CACHE_RUNS} runs per cache.\n");

    let (arena_time, arena_hits, arena_len) = best_cache_run::<ArenaCache<u64, u64>>();
    let (list_time, list_hits, list_len) = best_cache_run::<RcCache<u64, u64>>();

    // Same population and same hit count, or this compares nothing.
    assert_eq!(arena_len, list_len);
    assert_eq!(
        arena_hits, list_hits,
        "the two runs saw different request streams"
    );

    println!(
        " hit rate: {:.1}% ({} hits, {} inserts)\n",
        arena_hits as f64 * 100.0 / OPERATIONS as f64,
        with_thousands(arena_hits),
        with_thousands(OPERATIONS - arena_hits)
    );

    let per_request = |elapsed: Duration| elapsed.as_nanos() as f64 / OPERATIONS as
f64;

    // `{:0.2?}` uses Duration's own Debug format, rounded to two decimals.
    println!("  {:<22} {:>12} {:>13}", "storage", "total", "per request");
    println!("  {} ", "-".repeat(48));
    for (name, elapsed) in [
        (
            <ArenaCache<u64, u64> as Cache<u64, u64>>::variant(),
            arena_time,
        ),
        (<RcCache<u64, u64> as Cache<u64, u64>>::variant(), list_time),
    ] {
        println!(
            " {name:<22} {elapsed:>12.2?} {:>8.0} ns",
            per_request(elapsed)
        );
    }
}

```

```

let speedup = list_time.as_secs_f64() / arena_time.as_secs_f64();
println!(
    "\n the arena is {speedup:.2}x faster on the same requests: {arena_time:.2?}
    against {list_time:.2?}"
);
println!(" the arena reuses the evicted slot; the list frees a node and allocates
another");
}

```

`key_at` uses two different ranges of bits from the generator state: bits above 33 choose between the hot set and the whole space, and bits above 17 choose the key. The low bits of a linear congruential generator have short periods, so both shifts discard them.

The two assertions after the runs are the benchmark's correctness check. If the two caches had different eviction behaviour, the same stream would produce different hit counts and the program would stop before printing a time.

`<ArenaCache<u64, u64> as Cache<u64, u64>::variant()` is fully qualified syntax. It is needed because `variant` is an associated function with no `self`, and the type has no inherent function of that name to fall back on.

The arena cache as a standalone program

The repository also holds the arena cache as a runnable program, `src/bin/lru_cache_arena.rs`. Its `Node`, `LruCache`, `touch`, `get`, `put`, and tests are the same as `src/problems/lru_cache_easy.rs`, which chapter 18 prints in full; its test module is named `tests_for_lru`. It adds a `main` that runs the eviction sequence traced in chapter 18. Run it with `cargo run --bin lru_cache_arena`.

```

fn main() {
    let mut cache = LruCache::new(2);

    cache.put("A", 10);
    cache.put("B", 20);
    println!("get A = {:?}", cache.get(&"A")); // Some(10), A now MRU

    cache.put("C", 30); // evicts B (LRU)

    println!("get B = {:?}", cache.get(&"B")); // None
    println!("get A = {:?}", cache.get(&"A")); // Some(10)
    println!("get C = {:?}", cache.get(&"C")); // Some(30)

    cache.put("A", 99);
    println!("get A = {:?}", cache.get(&"A")); // Some(99)
}

```

The program prints:

```

get A = Some(10)
get B = None
get A = Some(10)
get C = Some(30)
get A = Some(99)

```

Intuition

The recorded result from `BENCHMARKS.md`, release build, on an NVIDIA Jetson board with an 8-core Cortex-A78AE:

```

LRU cache: 65,536 entries, 10,000,000 requests over 131,072 keys
A miss inserts and evicts the least recently used entry.
Best of 3 runs per cache.

```

```
hit rate: 89.8% (8,979,314 hits, 1,020,686 inserts)
```

storage	total	per request
arena (Vec + indices)	1.03s	103 ns
Rc<RefCell> list	1.86s	186 ns

```

the arena is 1.81x faster on the same requests: 1.03s against 1.86s
the arena reuses the evicted slot; the list frees a node and allocates another

```

Both caches pay for the same `HashMap` lookup on every request, and that lookup is a large part of the 103 ns the arena spends. The ratio between the two designs is therefore bounded by the share of each request spent on list maintenance. The repository's report measured the ratio across several access patterns: it ranged from 1.1 to 1.9, reaching 1.83 for a stream of hits only and falling to 1.30 for a stream that never repeats a key.

Time and Space Complexity

Operation	arena	rc_list
get hit	$O(1)$ expected; integer writes in one Vec	$O(1)$ expected; count updates, a Weak upgrade, several borrow checks
put miss at capacity	$O(1)$ expected; no allocation	$O(1)$ expected; one deallocation and one allocation
memory per entry	key, value, two <code>Option<usize></code> , map entry	key, value, Rc counts, RefCell flag, Weak, <code>Option<Rc></code> , map entry, allocator header
drop	one Vec deallocation	one deallocation per entry

Limitations

One workload, one machine. The 1.81 ratio holds for this stream, this capacity, and this processor. The report's range of 1.1 to 1.9 shows how much the answer depends on the access pattern.

Single-threaded only. Neither cache is shared between threads. A concurrent version would add a lock or sharding, as in chapter 41, and the lock would likely dominate both figures.

V: Copy narrows the contract. A cache of `String` values cannot use this trait. The restriction keeps the two implementations comparable and hides the cost that a `Clone` on every hit would add.

Clippy findings. The trait has `len` without `is_empty` (`len_without_is_empty`), and the eviction block in `rc_list::put` nests an `if let` inside an `if` that can be collapsed into one condition (`collapsible_if`). Neither affects the measurement.

Summary

- A shared trait and generic benchmark functions make two implementations comparable with identical measurement code.
- The `Rc<RefCell>` list pays for reference-count updates, `Weak` upgrades, runtime borrow checks, and an allocation per eviction; the arena pays for integer writes in one vector.
- A deterministic generator with locality gives both caches the same request stream, and comparing hit counts verifies that they behave identically.
- On ten million requests the arena was 1.81 times faster. The shared hash-map lookup bounds the ratio, which ranged from 1.1 to 1.9 across access patterns.

References

- The repository's `BENCHMARKS.md` and `benchmarking_examples/benchmarking-report.pdf`, for the full procedure and the threats to validity.
- Standard library, `std::rc::Weak` and `std::cell::RefCell`.

- Donald E. Knuth, *The Art of Computer Programming*, Volume 2, 3rd edition, Addison-Wesley, 1997, Section 3.2.1, on linear congruential generators.

A Sharded Concurrent Cache

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

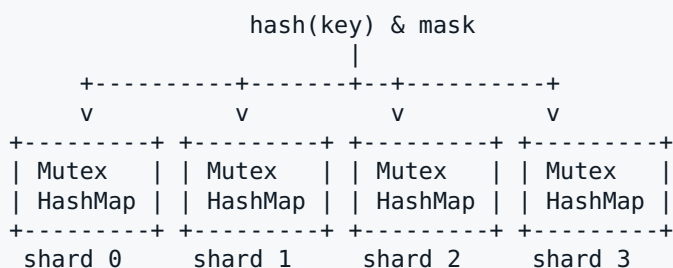
Source file: `src/problems/sharded_cache.rs`. Test it with `cargo test sharded_cache`.

Problem Statement

Build a map from `K` to `V` that many threads can read and write concurrently. Two operations on different keys should usually not block each other. The map needs `insert`, `get`, `remove`, `contains_key`, `len`, `is_empty`, `clear`, and a way to inspect a value without cloning it.

Designing a Solution

Allocate `N` independent `Mutex<HashMap<K, V>>` values. To find a key's shard, hash the key and take the hash modulo `N`. Operations on keys in different shards take different locks and proceed in parallel. The diagram below shows the layout for four shards.



If `N` is a power of two, `hash % N` equals `hash & (N - 1)`, and the mask is cheaper than a division. The constructor therefore rounds the requested shard count up to a power of two and stores the mask.

The hash that selects a shard must be the same for a key every time. The cache keeps one `RandomState`, created once, and builds a hasher from it for each lookup. A `RandomState` holds random keys chosen at construction, so shard selection is stable for the life of the cache and unpredictable to an outside party, which prevents a client from choosing keys that all land in one shard.

Operations that touch one key lock one shard. Operations that touch every key, `len` and `clear`, lock each shard in turn. They do not hold all the locks at once, so they do not observe a single consistent state of the whole map.

Implementation

```
/// Sharded concurrent map to reduce lock contention.
///
/// A single `Mutex<HashMap>` serializes every access. Splitting the map into N
/// shards, each with its own lock, lets independent keys proceed in parallel. The
/// cost is that `len` and `clear` must visit every shard, and there is no global
/// atomic snapshot.
///
/// This is a common SDK/CLI cache shape: per-account token caches, node metadata
/// caches, and request de-duplication tables.

use std::collections::HashMap;
use std::collections::hash_map::RandomState;
use std::hash::{BuildHasher, Hash, Hasher};
use std::sync::Mutex;

/// A fixed-size map split across independently locked shards.
///
/// The shard count is rounded up to a power of two so the index is a mask instead
/// of a modulo.
pub struct ShardedCache<K, V> {
    shards: Vec<Mutex<HashMap<K, V>>>,
    hasher: RandomState,
    mask: usize,
}

impl<K, V> ShardedCache<K, V>
where
    K: Hash + Eq,
{
    /// Create a cache with at least `shard_count` shards (rounded to a power of two).
    pub fn new(shard_count: usize) -> Self {
        let shard_count = shard_count.max(1).next_power_of_two();

        let shards = (0..shard_count)
            .map(|_| Mutex::new(HashMap::new()))
            .collect();

        Self {
            shards,
            hasher: RandomState::new(),
            mask: shard_count - 1,
        }
    }

    fn shard(&self, key: &K) -> &Mutex<HashMap<K, V>> {
        let mut hasher = self.hasher.build_hasher();
        key.hash(&mut hasher);
        let index = (hasher.finish() as usize) & self.mask;
    }
}
```

```

        &self.shards[index]
    }

    /// Insert a value, returning the previous one if the key was present.
    pub fn insert(&self, key: K, value: V) -> Option<V> {
        self.shard(&key)
            .lock()
            .expect("shard mutex poisoned")
            .insert(key, value)
    }

    /// Read a value by cloning it out of the shard.
    pub fn get(&self, key: &K) -> Option<V>
    where
        V: Clone,
    {
        self.shard(key)
            .lock()
            .expect("shard mutex poisoned")
            .get(key)
            .cloned()
    }

    /// Run `inspect` against the value under the shard lock, without cloning.
    ///
    /// Use this for read-modify-write on a single key; the closure holds the lock
    /// for its duration, so keep it short and do not call back into the cache.
    pub fn with<R>(&self, key: &K, inspect: impl FnOnce(Option<&V>) -> R) -> R {
        let guard = self.shard(key).lock().expect("shard mutex poisoned");
        inspect(guard.get(key))
    }

    pub fn remove(&self, key: &K) -> Option<V> {
        self.shard(key)
            .lock()
            .expect("shard mutex poisoned")
            .remove(key)
    }

    pub fn contains_key(&self, key: &K) -> bool {
        self.shard(key)
            .lock()
            .expect("shard mutex poisoned")
            .contains_key(key)
    }

    pub fn len(&self) -> usize {
        self.shards
            .iter()
            .map(|shard| shard.lock().expect("shard mutex poisoned").len())
            .sum()
    }

    pub fn is_empty(&self) -> bool {
        self.len() == 0
    }

    pub fn clear(&self) {
        for shard in &self.shards {
            shard.lock().expect("shard mutex poisoned").clear();
        }
    }

    pub fn shard_count(&self) -> usize {
        self.shards.len()
    }
}

```


`new` builds the shard vector with an iterator, `(0..shard_count).map(...).collect()`. `Mutex<HashMap<K, V>>` is not `Clone`, so `vec![Mutex::new(HashMap::new()); n]` would not compile; the iterator constructs each shard separately.

`shard` returns `&Mutex<HashMap<K, V>>` borrowed from `self`. The caller locks it, and the guard lives only for the expression that uses it, so each public method holds its lock for exactly one map operation.

`get` requires `V: Clone` through a `where` clause on the method rather than on the `impl` block. A cache of values that are not `Clone` can still use every other method, including `with`.

`with` takes a closure `FnOnce(Option<&V>) -> R` and calls it with a reference into the map while the shard lock is held. The caller can read a large value, or compute something small from it, without copying the value out. The documentation comment states the two rules that make this safe to use: keep the closure short, because it holds the lock, and do not call back into the cache from inside it, because a second lock of the same shard on the same thread deadlocks.

`len` sums the shard lengths with an iterator. Each shard is locked, measured, and unlocked before the next is locked.

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::thread;

    #[test]
    fn insert_get_remove() {
        let cache: ShardedCache<String, u32> = ShardedCache::new(4);
        assert_eq!(cache.insert("gpu-a100".into(), 8), None);
        assert_eq!(cache.get(&"gpu-a100".into()), Some(8));
        assert_eq!(cache.insert("gpu-a100".into(), 16), Some(8));
        assert_eq!(cache.get(&"gpu-a100".into()), Some(16));
        assert_eq!(cache.remove(&"gpu-a100".into()), Some(16));
        assert!(!cache.contains_key(&"gpu-a100".into()));
        assert!(cache.is_empty());
    }

    #[test]
    fn with_exposes_the_value_without_cloning() {
        let cache: ShardedCache<&str, String> = ShardedCache::new(2);
        cache.insert("node", "A100".to_string());
        let observed = cache.with(&"node", |value| value.map(|v| v.len()));
        assert_eq!(observed, Some(4));
    }

    #[test]
    fn concurrent_writers_all_land() {
        let cache = Arc::new(ShardedCache::new(16));
        let mut handles = Vec::new();
```

```

    for writer in 0..8 {
        let cache = Arc::clone(&cache);
        handles.push(thread::spawn(move || {
            for key in 0..500 {
                cache.insert(format!("{writer}-{key}"), key);
            }
        }));
    }
    for handle in handles {
        handle.join().expect("writer panicked");
    }

    assert_eq!(cache.len(), 8 * 500);
}

#[test]
fn keys_spread_across_shards() {
    let cache: ShardedCache<u32, u32> = ShardedCache::new(8);
    for key in 0..4_096 {
        cache.insert(key, key);
    }
    // Every shard should hold roughly 1/8 of the keys; assert none is starved.
    let smallest = cache
        .shards
        .iter()
        .map(|shard| shard.lock().expect("shard mutex poisoned").len())
        .min()
        .expect("at least one shard");
    assert!(smallest > 0, "at least one shard was empty");
}
}

```

`concurrent_writers_all_land` runs eight threads that insert 500 distinct keys each and checks that the total is 4,000. A lost update would show as a smaller count.

`keys_spread_across_shards` inserts 4,096 keys into eight shards and asserts that no shard is empty. The test module can read the private `shards` field because it is a child module.

Intuition

Shard selection for a cache created with `new(6)`

step	value
<code>shard_count.max(1)</code>	6
<code>.next_power_of_two()</code>	8
<code>mask</code>	$8 - 1 = 7$, binary <code>0b111</code>
<code>hasher.finish()</code> for some key	for example <code>0x9f3a_52c1_0b7e_44d5</code>
<code>as usize & mask</code>	the low three bits, <code>0b101</code> , so shard 5

Two threads inserting `"gpu-a100"` and `"node-17"` lock shard 5 and, say, shard 2. The two inserts do not wait for each other. Two threads inserting keys that both hash to shard 5 serialize on that shard's lock, exactly as they would on a single global map.

Time and Space Complexity

Operation	Time	Locks taken
<code>insert</code> , <code>get</code> , <code>remove</code> , <code>contains_key</code> , <code>with</code>	$O(1)$ expected, plus hashing the key	one shard
<code>len</code> , <code>is_empty</code>	$O(\text{shards})$	every shard, one at a time
<code>clear</code>	$O(\text{entries})$	every shard, one at a time
memory	$O(\text{entries} + \text{shards})$	one <code>Mutex</code> and one <code>HashMap</code> header per shard

With S shards and keys spread evenly, the chance that two concurrent single-key operations need the same lock is about $1/S$.

Limitations

No cross-shard atomicity. `len` can return a value the map never held, if one thread inserts into shard 0 after `len` has counted it and another removes from shard 7 before `len` reaches it. A read-modify-write over two keys, such as moving a value from one key to another, cannot be done atomically with this API.

get then insert is not atomic either. Two threads that both call `get`, see `None`, and then `insert` will both insert. An `entry`-style method that performs the check and the insert under one shard lock is the idiomatic addition.

No eviction. Despite its name, this is a concurrent map with no capacity. A long-running process that caches without bound will grow without bound.

with can deadlock its caller. Calling any method of the same cache inside the closure, for a key in the same shard, locks a `std::sync::Mutex` twice on one thread. The standard mutex does not detect this, and the thread waits forever.

Hot keys defeat sharding. If most traffic targets one key, that key's shard is a single global lock. Sharding spreads keys, not load.

Hashing is written out by hand. `shard` creates a hasher, feeds the key, and calls `finish`. Since Rust 1.71, `BuildHasher::hash_one` does the same in one call, `self.hasher.hash_one(key)`, and Clippy suggests it through `manual_hash_one`. The constructor also contains a line with trailing whitespace after `let shard_count`, which `rustfmt` removes.

A poisoned shard panics every caller. Each method uses `expect("shard mutex poisoned")`. One panic while a shard is locked makes every later access to any key in that shard panic as well. Chapter 22 describes the alternatives.

Summary

- Splitting a map into `N` independently locked shards lets operations on different keys run in parallel while keeping the simplicity of a `Mutex`.
- Rounding `N` to a power of two turns `hash % N` into `hash & (N - 1)`.
- A single `RandomState` gives stable, unpredictable shard selection for the life of the cache.
- `with` exposes a reference under the lock without cloning, and must not call back into the cache.
- Whole-map operations visit shards one at a time and do not see a consistent snapshot, and check-then-insert sequences need an atomic `entry`-style method.

References

- Standard library, `std::collections::hash_map::RandomState`.
- Standard library, `BuildHasher::hash_one`.
- The `dashmap` crate, [documentation](#), a production sharded map.
- Maurice Herlihy and Nir Shavit, *The Art of Multiprocessor Programming*, 2nd edition, Morgan Kaufmann, 2020, Chapter 32, "Concurrent Hashing and Natural Parallelism".

Consistent Hashing

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/consistent_hash.rs`. Test it with `cargo test consistent_hash`.

Problem Statement

Map string keys to node names so that:

- the same key maps to the same node while the set of nodes is unchanged;
- adding a node moves only keys that the new node takes over;
- removing a node moves only the keys that node owned;
- keys spread roughly evenly across nodes.

Designing a Solution

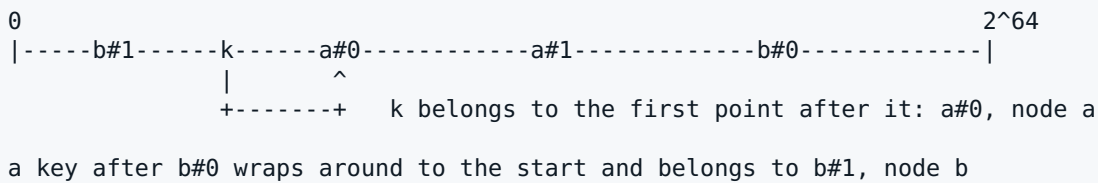
Consider $N = 3$ backends and placement by `hash(key) % 3`. Adding a fourth backend changes the divisor, and a key keeps its backend only when `hash % 3 == hash % 4`, which holds for about one key in four. Three quarters of the cache misses at once.

A hash ring avoids the divisor. Every node is hashed to a point on the circle of 64-bit values, and every key is hashed to a point on the same circle. A key belongs to the first node point clockwise from it. Adding a node inserts new points, and only the keys between each new point and its predecessor change owner, all of them moving to the new node.

One point per node gives uneven arcs. The implementation places each node at `replicas` points, called virtual nodes, by hashing the node name together with

the replica number. With 128 replicas the arcs average out, and each node owns close to its share of the circle.

The diagram below shows a ring with two nodes and two replicas each.



The ring is stored as a vector of (hash, node) pairs sorted by hash. Lookup is a binary search for the first point greater than the key's hash, wrapping to index 0 when the key's hash is beyond the last point.

Implementation

```
///! Consistent hashing ring.
///!
///! When a distributed gateway shards work across N backends, a plain
///! `hash(key) % N` remaps nearly every key the moment N changes. Consistent
///! hashing places each node at several points on a hash ring, so adding or
///! removing one node only moves the keys that node owns. That property is the
///! reason it shows up in caches, sharded queues, and load balancers.
///!
///! Each physical node is placed at `replicas` points (virtual nodes) to smooth the
///! key distribution; more replicas means a less lumpy ring.

use std::collections::HashSet;
use std::collections::hash_map::DefaultHasher;
use std::hash::{Hash, Hasher};
use std::sync::Arc;

/// A hash ring mapping keys to node names.
pub struct ConsistentHash {
    replicas: usize,
    /// Sorted `(hash, node)` pairs. The node name is an `Arc<str>` so that the
    /// many virtual nodes for one physical node share a single allocation.
    ring: Vec<(u64, Arc<str>)>,
}

impl ConsistentHash {
    /// Create an empty ring with `replicas` virtual nodes per physical node.
    /// Panics if `replicas` is zero.
    pub fn new(replicas: usize) -> Self {
        assert!(replicas > 0, "replicas must be > 0");
        Self {
            replicas,
            ring: Vec::new(),
        }
    }

    /// Add a node and its virtual nodes. Adding a node that already exists is a
    /// no-op.
    pub fn add_node(&mut self, node: &str) {
        if self.ring.iter().any(|(&, name)| name.as_ref() == node) {
            return;
        }
    }
}
```

```

    }

    // One allocation for the name, then a refcount bump per virtual node.
    let name: Arc<str> = Arc::from(node);
    for replica in 0..self.replicas {
        let hash = virtual_node_hash(node, replica);
        self.ring.push((hash, Arc::clone(&name)));
    }
    self.ring.sort_by_key(|(hash, _)| *hash);
}

/// Remove a node and all of its virtual nodes.
pub fn remove_node(&mut self, node: &str) {
    self.ring.retain(|(_, name)| name.as_ref() != node);
}

/// Return the node responsible for `key`, or `None` when the ring is empty.
pub fn get(&self, key: &str) -> Option<&str> {
    if self.ring.is_empty() {
        return None;
    }

    let hash = hash_of(key);
    // First virtual node strictly clockwise from the key.
    let index = self.ring.partition_point(|(point, _)| *point <= hash);
    let index = if index == self.ring.len() { 0 } else { index };
    Some(self.ring[index].1.as_ref())
}

/// Number of distinct physical nodes.
pub fn node_count(&self) -> usize {
    self.ring
        .iter()
        .map(|(_, name)| name.as_ref())
        .collect::<HashSet<_>>()
        .len()
}

/// Number of virtual nodes on the ring.
pub fn ring_len(&self) -> usize {
    self.ring.len()
}
}

fn hash_of(input: &str) -> u64 {
    let mut hasher = DefaultHasher::new();
    input.hash(&mut hasher);
    hasher.finish()
}

/// Hash of the `replica`-th virtual node for `node`.
///
/// Hashing the two fields separately avoids allocating a "{node}#{replica}"
/// string for every virtual node on the ring.
fn virtual_node_hash(node: &str, replica: usize) -> u64 {
    let mut hasher = DefaultHasher::new();
    node.hash(&mut hasher);
    replica.hash(&mut hasher);
    hasher.finish()
}
}

```

The node name is stored as `Arc<str>`. With 128 replicas, a `String` per virtual node would allocate 128 copies of the name. `Arc::from(node)` allocates once,

and each `Arc::clone` increments a count. `Arc<str>` is two words wide, a pointer and a length, the same as `&str`.

`add_node` checks for an existing node with a linear scan over the ring, pushes `replicas` new points, and re-sorts the vector. `sort_by_key` is a stable sort that performs well on input that is already mostly sorted, which is the state after pushing new points onto the end of a sorted vector.

`get` uses `partition_point(|(point, _)| *point <= hash)`, which returns the index of the first element for which the predicate is false: the first point strictly greater than the key's hash. When every point is less than or equal to the hash, the result is `ring.len()`, and the next line wraps it to 0.

`virtual_node_hash` feeds the node name and the replica number into one hasher rather than hashing a formatted string such as `"node-a#17"`. This avoids an allocation per virtual node and produces a well-mixed hash of both values.

`DefaultHasher::new()` creates a hasher with fixed keys, unlike `RandomState`. The ring therefore places nodes at the same points in every process, which is required when several clients must agree on placement. The standard library does not promise that `DefaultHasher` produces the same values across Rust releases, so clients built with different compiler versions could disagree.

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn empty_ring_has_no_owner() {
        let ring = ConsistentHash::new(64);
        assert_eq!(ring.get("workload-1"), None);
        assert_eq!(ring.node_count(), 0);
    }

    #[test]
    fn lookup_is_stable_for_a_fixed_ring() {
        let mut ring = ConsistentHash::new(64);
        ring.add_node("node-a");
        ring.add_node("node-b");

        let first = ring.get("gpu-job-42").map(str::to_string);
        for _ in 0..100 {
            assert_eq!(ring.get("gpu-job-42").map(str::to_string), first);
        }
        assert!(first.is_some());
    }

    #[test]
    fn keys_spread_across_nodes() {
        let mut ring = ConsistentHash::new(128);
        for node in ["node-a", "node-b", "node-c"] {
            ring.add_node(node);
        }
    }
}
```



```

    let mut counts = std::collections::HashMap::new();
    for key in 0..3_000 {
        let owner = ring.get(&format!("key-{}", key)).expect("ring is populated");
        *counts.entry(owner.to_string()).or_insert(0) += 1;
    }

    assert_eq!(counts.len(), 3, "every node should own some keys");
    let smallest = *counts.values().min().expect("non-empty");
    assert!(smallest > 300, "distribution is too lumpy: {counts:?}");
}

#[test]
fn adding_a_node_only_moves_keys_to_the_new_node() {
    let keys: Vec<String> = (0..2_000).map(|i| format!("workload-{}", i)).collect();

    let mut ring = ConsistentHash::new(128);
    ring.add_node("node-a");
    ring.add_node("node-b");
    let before: Vec<String> = keys
        .iter()
        .map(|key| ring.get(key).expect("ring is populated").to_string())
        .collect();

    ring.add_node("node-c");
    let after: Vec<String> = keys
        .iter()
        .map(|key| ring.get(key).expect("ring is populated").to_string())
        .collect();

    for (old, new) in before.iter().zip(after.iter()) {
        assert!(
            old == new || new == "node-c",
            "adding a node moved a key from {old} to {new}"
        );
    }
}

#[test]
fn removing_a_node_drops_it_from_the_ring() {
    let mut ring = ConsistentHash::new(32);
    ring.add_node("node-a");
    ring.add_node("node-b");
    assert_eq!(ring.node_count(), 2);
    assert_eq!(ring.ring_len(), 64);

    ring.remove_node("node-a");
    assert_eq!(ring.node_count(), 1);
    for _ in 0..500 {
        assert!(matches!(ring.get("any-key"), Some("node-b")));
    }
}
}

```

`adding_a_node_only_moves_keys_to_the_new_node` records the owner of 2,000 keys on a two-node ring, adds a third node, and checks every key: either its owner is unchanged or its new owner is `node-c`. That is the defining property of consistent hashing, tested directly.

`keys_spread_across_nodes` requires each of three nodes to own more than 300 of 3,000 keys, a loose bound that catches a ring whose virtual nodes are not being used.

Intuition

Lookup on a small ring, with hashes shortened to two digits for readability

ring after sorting	key	key hash	partition_point result	owner
[(12, b), (40, a), (71, b), (93, a)]	job-1	55	2, the first point > 55 is 71	b
same	job-2	12	1, the first point > 12 is 40	a
same	job-3	97	4 = len, wraps to 0	b

Adding node **c** with points (60, c) and (20, c) changes the ring to [(12, b), (20, c), (40, a), (60, c), (71, b), (93, a)]. **job-1** at 55 now finds 60 and moves to **c**. **job-2** at 12 now finds 20 and moves to **c**. **job-3** at 97 still wraps to **b**. No key moved from **a** to **b** or from **b** to **a**.

Time and Space Complexity

N is the number of physical nodes, R the number of replicas, and $V = N \cdot R$ the length of the ring.

Operation	Time	Space
add_node	$O(V)$ scan plus $O(V \log V)$ sort	R new entries
remove_node	$O(V)$	none
get	$O(\log V)$ for the search, plus hashing the key	none
node_count	$O(V)$, and it allocates a HashSet	$O(N)$
memory	$O(V)$ entries of 24 bytes each, plus one name per node	

When a node joins a ring of N nodes, about $1/(N + 1)$ of the keys move.

Limitations

DefaultHasher is not a stable hash. Its algorithm is unspecified and may change between Rust releases. Two services that must agree on placement should use a hash function with a published, fixed definition.

No weights. Every node receives the same number of virtual nodes. A backend with twice the capacity needs twice the replicas, which the API cannot express.

add_node is quadratic over many additions. Adding N nodes one at a time sorts the ring N times. Building the ring from a list of nodes and sorting once would be $O(V \log V)$ in total.

Hash collisions between virtual nodes are not handled. If two virtual nodes hash to the same point, `partition_point` always lands after both of them, so the one that sorts first can never own a key. The effect is a slightly smaller share for its node. With 64-bit hashes and a few thousand points the probability is negligible.

node_count builds a HashSet on every call. A field that counts physical nodes, updated in `add_node` and `remove_node`, would make it $O(1)$.

Summary

- Modulo placement moves most keys when the node count changes. A hash ring moves only the keys between a new node's points and their predecessors.
- Each physical node appears at `replicas` virtual points, which evens out the share of keys each node owns.
- A sorted `Vec<(u64, Arc<str>)>` with `partition_point` gives $O(\log V)$ lookup and wraps to index 0 past the last point.
- `Arc<str>` stores one copy of each node name, shared by all its virtual nodes.
- Placement must be computed with a hash whose definition is stable across processes and releases, which `DefaultHasher` does not guarantee.

References

- David Karger, Eric Lehman, Tom Leighton, Matthew Levine, Daniel Lewin, and Rahul Panigrahy, "Consistent hashing and random trees", *Proceedings of the 29th ACM Symposium on Theory of Computing*, 1997.
- Giuseppe DeCandia et al., "Dynamo: Amazon's highly available key-value store", *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, 2007.
- Standard library, `slice::partition_point`.
- Standard library, `DefaultHasher`, on its unspecified algorithm.

Part XIII: Threads and Synchronization

Threads, Send, and Sync

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/threads.rs`. Test it with `cargo test threads`.

Problem Statement

Write the basic concurrent building blocks a systems interview expects:

- sum a slice on several threads without copying it or wrapping it in `Arc`;
- count events from several threads without a lock;
- initialize a process-wide value exactly once, even if several threads ask for it at the same time;
- make a thread-safety assumption about a type fail the build when it is false.

Designing a Solution

Send and Sync. A type is `Send` when a value of it may be moved to another thread, and `Sync` when a shared reference `&T` may be used from several threads at once. Equivalently, `T: Sync` exactly when `&T: Send`. Both are auto traits: the compiler implements them for a type whose fields all implement them. `Rc<T>` is neither, because its reference count is updated without synchronization, and `RefCell<T>` is `Send` but not `Sync`, because its borrow flag is not synchronized. Passing either where the other thread could touch it is a compile error.

Three ways to run work on threads, in increasing order of ceremony:

1. `thread::scope` for work that borrows local data and finishes before the scope ends. The closures may capture references, because the scope guarantees they are joined before the borrowed data can be dropped.
2. `thread::spawn` for work that owns its data and may outlive the caller. The closure must be `'static` and `Send`, so shared data travels in an `Arc`.
3. A thread pool when there are many small jobs and threads should be reused, which chapter 21 builds.

Atomics instead of locks. When the shared state is a single integer, an `AtomicUsize` updates it with one processor instruction such as `lock xadd` on x86 or an exclusive-load and exclusive-store pair on ARM. No thread blocks, and no guard exists that could be held across a panic.

Implementation

```
/// Threads and fearless concurrency.
///
/// Rust's claim about concurrency is narrow and checkable: safe code cannot
/// produce a data race, because `Send` and `Sync` describe what may cross a
/// thread boundary and the compiler refuses programs that violate the rules.
///
/// - `Send` means a value may be moved to another thread.
/// - `Sync` means `&T` may be shared across threads.
///
/// Both are auto traits: a type is `Send`/`Sync` when its fields are. `Rc` and
/// `RefCell` are the standard counter-examples, which is precisely why sharing
/// them across threads does not compile. The compile error is the feature, not an
/// obstacle.
///
/// Three ways to get work onto other threads, in increasing order of ceremony:
///
/// 1. `thread::scope` for work that borrows local data and finishes with the
///    scope. No `static`, no `Arc`.
/// 2. `thread::spawn` for work that must own its data and outlive the caller.
/// 3. A thread pool when the number of units of work is large and threads should
///    be reused.

use std::sync::OnceLock;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

/// Number of hardware threads the runtime can use, defaulting to 1.
pub fn available_workers() -> usize {
    thread::available_parallelism()
        .map(|count| count.get())
        .unwrap_or(1)
}

/// Sum a slice by splitting it into chunks that run on scoped threads.
///
/// `thread::scope` is what makes this ergonomic: the workers borrow `values`
/// directly, so there is no `Arc` and no `static` bound. The scope waits for
/// every thread, which removes the forgotten-join bug.
```

```

pub fn parallel_sum(values: &[i32]) -> i64 {
    if values.is_empty() {
        return 0;
    }

    let workers = available_workers().min(values.len());
    let chunk_size = values.len().div_ceil(workers);

    thread::scope(|scope| {
        let handles: Vec<_> = values
            .chunks(chunk_size)
            .map(|chunk| {
                scope.spawn(move || chunk.iter().map(|value| i64::from(*value)).sum::<i
64>())
            })
            .collect();

        handles
            .into_iter()
            .map(|handle| handle.join().expect("worker panicked"))
            .sum()
    })
}

```

`thread::available_parallelism` returns `io::Result<NonZeroUsize>`. It can fail, for example inside a restricted container, and the function falls back to 1 rather than propagating the error, which is a reasonable default for sizing a pool.

`parallel_sum` returns early for an empty slice. Without that check, `workers` would be 0 and `values.len().div_ceil(0)` would panic with a division by zero.

`values.chunks(chunk_size)` yields at most `workers` sub-slices. The closure passed to `scope.spawn` is `move`, so it captures `chunk`, a `&[i32]`, by value. The reference borrows from `values`, and `thread::scope` is what makes that borrow legal: the scope does not return until every spawned thread has finished.

The handles are collected into a `Vec` before any is joined. Joining inside the same iterator chain that spawns would make the chain lazy and sequential: each thread would be spawned and joined before the next was spawned.

```

/// A counter that several threads may increment without a lock.
///
/// `Relaxed` is correct here because the counter is the only value being
/// communicated; there is no other data whose visibility depends on it.
#[derive(Default)]
pub struct AtomicCounter {
    value: AtomicUsize,
}

impl AtomicCounter {
    pub const fn new() -> Self {
        Self {
            value: AtomicUsize::new(0),
        }
    }

    /// Increment and return the new value.
    pub fn increment(&self) -> usize {

```



```

        self.value.fetch_add(1, Ordering::Relaxed) + 1
    }

    pub fn get(&self) -> usize {
        self.value.load(Ordering::Relaxed)
    }
}

/// Increment a counter from several scoped threads that share it by reference.
pub fn scoped_increment(counter: &AtomicCounter, threads: usize, per_thread: usize) {
    thread::scope(|scope| {
        for _ in 0..threads {
            scope.spawn(|| {
                for _ in 0..per_thread {
                    counter.increment();
                }
            });
        }
    });
}

```

`increment` takes `&self`, not `&mut self`. The atomic provides interior mutability that is safe across threads, so `AtomicCounter` is `Sync` and can be shared by reference.

`fetch_add(1, Ordering::Relaxed)` returns the previous value, and the method adds one to return the new value. The `Ordering` argument does not control the addition, which is always atomic. It describes the guarantees about when other atomic operations observe the result. For a single counter with no relationship to other variables, `Relaxed` is sufficient. `SeqCst` would be required only if the program's correctness depended on a particular interleaving of this counter with other atomic variables.

`scoped_increment` spawns closures that capture `counter: &AtomicCounter` without `move`. Each closure borrows the reference, and all the borrows are shared, which is legal because `AtomicCounter: Sync`.

`AtomicCounter::new` is a `const fn`, so a counter can be a `static` initialized at compile time: `static REQUESTS: AtomicCounter = AtomicCounter::new();`

```

/// A value initialized once and shared by every caller.
///
/// `OnceLock` is the primitive behind lazy process-wide configuration, and it is
/// safe because initialization happens at most once even under concurrent calls.
pub fn build_channel() -> &'static str {
    static CHANNEL: OnceLock<&'static str> = OnceLock::new();
    CHANNEL.get_or_init(|| "stable")
}

/// A compile-time assertion that `T` may cross thread boundaries.
///
/// Calling `assert_send_sync::<Client>()` fails to compile if `Client` is not
/// thread-safe, which turns a concurrency assumption into a build error.
pub fn assert_send_sync<T: Send + Sync>() {}

```

`OnceLock::get_or_init` runs the closure at most once, even when several threads call it concurrently. The other callers block until the first finishes and then receive a reference to the same value. The `static` is local to the function, so the value is private to it and still lives for the whole process.

`assert_send_sync::<T>()` has an empty body and a bound. Calling it with a type that is not `Send + Sync` is a compile error, which turns an assumption such as "our client can be shared between threads" into a check that runs on every build.

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::thread;

    #[test]
    fn parallel_sum_matches_the_sequential_sum() {
        let values: Vec<i32> = (1..=10_000).collect();
        let expected: i64 = values.iter().map(|value| i64::from(*value)).sum();
        assert_eq!(parallel_sum(&values), expected);
    }

    #[test]
    fn parallel_sum_handles_small_and_empty_inputs() {
        assert_eq!(parallel_sum(&[]), 0);
        assert_eq!(parallel_sum(&[7]), 7);
        assert_eq!(parallel_sum(&[-1, 1, -2, 2]), 0);
    }

    #[test]
    fn scoped_threads_share_a_counter_without_arc() {
        let counter = AtomicCounter::new();
        scoped_increment(&counter, 8, 500);
        assert_eq!(counter.get(), 4_000);
    }

    #[test]
    fn spawn_returns_a_join_handle_that_carries_the_result() {
        let handle = thread::spawn(|| 6 * 7);
        assert_eq!(handle.join().expect("worker panicked"), 42);
    }

    #[test]
    fn once_lock_initializes_a_single_value() {
        assert_eq!(build_channel(), "stable");
        assert_eq!(build_channel(), build_channel());
    }

    #[test]
    fn counter_is_thread_safe_by_construction() {
        assert_send_sync::<AtomicCounter>();
        assert!(available_workers() >= 1);
    }
}
```

`spawn_returns_a_join_handle_that_carries_the_result` shows the owned alternative to `scoped threads`: `thread::spawn(|| 6 * 7)` returns a

`JoinHandle<i32>`, and `join` returns `Result<i32, Box<dyn Any + Send>>`, whose error case carries the panic payload if the thread panicked.

Intuition

`parallel_sum(&[1, 2, 3, 4, 5, 6, 7])` on a machine with eight hardware threads

step	value
<code>available_workers()</code>	8
<code>workers = 8.min(7)</code>	7
<code>chunk_size = 7.div_ceil(7)</code>	1
<code>values.chunks(1)</code>	seven slices of one element
threads spawned	7, each summing one element
joined results	1, 2, 3, 4, 5, 6, 7
return	28

For a slice of 10,000 values on the same machine, `workers` is 8, `chunk_size` is 1,250, and eight threads each sum 1,250 values. The test compares the result with a sequential sum.

Time and Space Complexity

Operation	Time	Space
<code>parallel_sum</code>	$O(n)$ total work, spread over up to <code>available_parallelism</code> threads, plus thread creation	one stack per thread and one handle per chunk
<code>AtomicCounter::increment</code>	$O(1)$, one atomic instruction	none
<code>build_channel</code>	$O(1)$ after the first call	one <code>&'static str</code>
<code>assert_send_sync</code>	compile time only	none

Creating a thread costs tens of microseconds and a stack reservation. For a slice of a few thousand integers the sequential loop is faster than any paral-

lel version, and the parallel version pays off only when the per-chunk work dominates the cost of starting threads.

Limitations

parallel_sum spawns threads on every call. A program that calls it in a loop pays thread creation each time. A pool, or a library such as `rayon`, re-uses threads.

Contention on a shared atomic. Every `fetch_add` from every thread writes to the same cache line, so eight threads incrementing one counter scale poorly. Chapter 48 measures the related effect of false sharing between separate counters.

Relaxed is correct here, and easy to misuse elsewhere. The justification in the documentation comment applies to an isolated counter. Code that reads one atomic to decide how to update another needs an argument about ordering, and usually needs a lock or a compare-and-swap loop.

build_channel is a demonstration. Its initializer returns a constant, which a plain `const` would express without `OnceLock`. The pattern pays off when initialization reads the environment or a file.

Summary

- `Send` permits moving a value to another thread and `Sync` permits sharing `&T` between threads; the compiler derives both from a type's fields.
- `thread::scope` lets threads borrow local data because it joins them before returning, which removes the need for `Arc` and `'static`.
- Collect spawned handles before joining them, or the threads run one after another.
- An `AtomicUsize` counts from many threads without a lock; `Relaxed` ordering suffices for a value with no relationship to other atomics.
- `OnceLock` initializes a shared value exactly once, and a generic function with a `Send + Sync` bound turns a thread-safety assumption into a compile-time check.

References

- The Rust Book, [Fearless concurrency](#).
- The Rustonomicon, [Send and Sync](#).
- Mara Bos, *Rust Atomics and Locks*, O'Reilly Media, 2023, Chapters 1 to 35.
- Standard library, `std::thread::scope`, `std::sync::atomic::Ordering`, and `std::sync::OnceLock`.

A Spin Lock

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/spin_lock.rs`. Test it with `cargo test spin_lock`.

Problem Statement

Implement `SpinLock<T>` with `lock`, which waits until the lock is free and returns a guard, and `try_lock`, which returns `None` immediately if the lock is held. The guard must give `&T` and `&mut T` access to the value and release the lock when it is dropped. The lock must be shareable between threads through `Arc` or a scoped borrow.

Designing a Solution

State. The lock is an `AtomicBool`, `false` when free, and the value is an `UnsafeCell<T>`. `UnsafeCell` is the only legal way in Rust to obtain a mutable pointer to data reachable through a shared reference; every interior-mutability type, including `Mutex` and `RefCell`, is built on it.

Acquire. `compare_exchange(false, true)` stores `true` only if the current value is `false`, and reports whether it did. Exactly one of several competing threads succeeds. The losers wait for the flag to become `false` and try again.

Release. The guard's `Drop` stores `false`. Because release happens in a destructor, it happens on every path out of the critical section, including a panic that unwinds through it.

Soundness. The guard dereferences to `&mut T` using the `UnsafeCell`. That is sound only while the guard is the only way to reach the value, which the flag guarantees: while one guard exists, no other thread can create a second one.

<pre>thread A compare_exchange(false, true) -> Ok guard A: exclusive access to the value *guard += 1 drop(guard A): locked.store(false)</pre>	<pre>thread B compare_exchange(false, true) -> Err while locked.load() { spin_loop() } ... load() sees false, loop exits compare_exchange(false, true) -> Ok guard B: exclusive access</pre>
--	--

Implementation

```
/// A spin lock built from a single `AtomicBool`.
///
/// This is the primitive to write when an interviewer asks for a lock from
/// scratch. The interesting part is not the loop; it is the memory ordering and
/// the ownership argument that make the unsafe block sound.
///
/// - `lock`/`try_lock` use a compare-exchange with `Acquire` ordering, so the
///   critical section observes every write the previous holder made before it
///   released.
/// - `unlock` uses a `Release` store, so writes inside the critical section are
///   visible to whoever acquires next.
/// - The guard derefs to `&T`/`&mut T`. Mutual exclusion is what makes handing out
///   `&mut T` sound, and that is exactly what the lock guarantees.
///
/// A spin lock is the wrong choice when the critical section can block or is
/// long: every waiting thread burns a core. Use it for very short sections on
/// dedicated cores, and prefer `std::sync::Mutex` (which parks the thread) in
/// general code.

use std::cell::UnsafeCell;
use std::hint::spin_loop;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::{AtomicBool, Ordering};

/// A mutual-exclusion lock that busy-waits instead of parking.
pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}

// SAFETY: `value` is only accessed while `locked` is held, and the acquire/release
// ordering on `locked` makes a hand-off between threads well defined. Sharing
// `&SpinLock` across threads therefore only requires the protected value to be
// `Send`, which lets one thread move data in and another move it out.
unsafe impl<T: Send> Sync for SpinLock<T> {}

/// Grants access to the protected value and releases the lock when dropped.
pub struct SpinGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

impl<T> SpinLock<T> {
    /// Create an unlocked spin lock.
    pub const fn new(value: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            value: UnsafeCell::new(value),
        }
    }

    /// Acquire the lock, spinning until it is free.
    pub fn lock(&self) -> SpinGuard<'_, T> {
```

```

    loop {
        if self
            .locked
            .compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxe
d)
            .is_ok()
        {
            return SpinGuard { lock: self };
        }

        // Spin on a relaxed load so the contention loop does not keep
        // bouncing the cache line with exclusive accesses.
        while self.locked.load(Ordering::Relaxed) {
            spin_loop();
        }
    }
}

/// Acquire the lock only if it is free right now.
pub fn try_lock(&self) -> Option<SpinGuard<'_, T>> {
    self.locked
        .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
        .ok()
        .map(|_| SpinGuard { lock: self })
}

/// Whether the lock is currently held.
pub fn is_locked(&self) -> bool {
    self.locked.load(Ordering::Relaxed)
}

fn unlock(&self) {
    self.locked.store(false, Ordering::Release);
}

impl<T: Default> Default for SpinLock<T> {
    fn default() -> Self {
        Self::new(T::default())
    }
}

impl<T> Drop for SpinGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.unlock();
    }
}

impl<T> Deref for SpinGuard<'_, T> {
    type Target = T;

    fn deref(&self) -> &T {
        // SAFETY: holding the guard means this thread has exclusive access.
        unsafe { &*self.lock.value.get() }
    }
}

impl<T> DerefMut for SpinGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        // SAFETY: holding the guard means this thread has exclusive access.
        unsafe { &mut *self.lock.value.get() }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

```



```

use std::sync::Arc;
use std::thread;

#[test]
fn serializes_concurrent_increments() {
    let lock = Arc::new(SpinLock::new(0usize));
    let mut handles = Vec::new();

    for _ in 0..8 {
        let lock = Arc::clone(&lock);
        handles.push(thread::spawn(move || {
            for _ in 0..1_000 {
                *lock.lock() += 1;
            }
        }));
    }
    for handle in handles {
        handle.join().expect("worker panicked");
    }

    assert_eq!(*lock.lock(), 8_000);
}

#[test]
fn try_lock_reports_contention() {
    let lock = SpinLock::new(1u32);
    let guard = lock.lock();
    assert!(lock.try_lock().is_none());
    assert!(lock.is_locked());
    drop(guard);
    assert!(lock.try_lock().is_some());
    assert!(!lock.is_locked());
}

#[test]
fn guard_gives_mutable_access() {
    let lock = SpinLock::new(vec![1, 2, 3]);
    {
        let mut guard = lock.lock();
        guard.push(4);
    }
    assert_eq!(*lock.lock(), vec![1, 2, 3, 4]);
}
}

```

The acquire loop

`lock` has two nested loops. The outer loop attempts the exchange with `compare_exchange_weak`, which is allowed to fail spuriously even when the value is `false`. On some architectures, including ARM, the weak version maps directly to a load-link/store-conditional instruction pair and avoids an internal retry loop, and the outer loop already handles failure.

When the exchange fails, the inner loop waits with plain `load` calls until the flag reads `false`. A failed `compare_exchange` still requests exclusive ownership of the cache line holding the flag, while a `load` can proceed from a shared copy. Spinning on `load` keeps contending threads from bouncing the cache line between cores. `std::hint::spin_loop` emits the processor's pause instruction,

which reduces power use and lets the processor schedule the waiting loop efficiently.

The orderings

The success ordering of the exchange is `Acquire` and the release uses `Release`. For a lock built on one atomic, these names document intent: `Acquire` for the operation that takes the lock and `Release` for the one that gives it up. The protected data does not depend on the ordering argument. It is synchronized by mutual exclusion: a write made under the lock happens before the release, and the next holder cannot start before it observes that release. The failure ordering `Relaxed` applies to the load performed when the exchange fails, where no guarantee beyond the atomic's own is needed.

The trait implementations

`unsafe impl<T: Send> Sync for SpinLock<T> {}` tells the compiler that `&SpinLock<T>` may be shared between threads. The bound is `T: Send`, not `T: Sync`, and it matches `std::sync::Mutex`. Through the lock, only one thread at a time can reach the value, so the value never needs to be accessed concurrently; it only needs to be movable between threads, because one thread may write it and another read it later.

`SpinLock<T>` is automatically `Send` when `T: Send`, because `AtomicBool` and `UnsafeCell<T>` are.

The guard

`SpinGuard` holds `&'a SpinLock<T>`, and the lifetime ties it to the lock, so a guard cannot outlive its lock. `Deref` and `DerefMut` each contain one `unsafe` block that turns the `UnsafeCell`'s raw pointer into a reference. The `SAFETY` comments state the invariant that justifies both: holding a guard means this thread has exclusive access.

Intuition

`try_lock_reports_contention`

step	locked	result
<code>SpinLock::new(1u32)</code>	<code>false</code>	
<code>let guard = lock.lock()</code>	<code>true</code>	exchange succeeds

step	locked	result
<code>lock.try_lock()</code>	true	exchange fails, returns <code>None</code>
<code>lock.is_locked()</code>	true	returns <code>true</code>
<code>drop(guard)</code>	false	<code>Drop</code> stores <code>false</code>
<code>lock.try_lock()</code>	true for the temporary guard, then false when it is dropped at the end of the statement	returns <code>Some</code>
<code>lock.is_locked()</code>	false	returns <code>false</code>

In `serializes_concurrent_increments`, eight threads each perform 1,000 increments of `*lock.lock() += 1`. Every increment reads and writes the counter under the lock, so the final value is exactly 8,000. Without the lock, lost updates would make it smaller.

Time and Space Complexity

Operation	Time	Space
lock, uncontended	one compare-and-swap	none
lock, contended	unbounded busy-waiting; the thread uses a full core while it waits	none
try_lock	one compare-and-swap	none
guard drop	one store	none
memory		<code>size_of::<t>()</t></code> plus one byte, rounded up to T's alignment

Limitations

The guard is `Sync` for any `T: Send`, which is unsound. `SpinGuard` has no explicit trait implementations, so the compiler derives them from its one field, `&'a SpinLock<T>`. That reference is `Sync` whenever `SpinLock<T>` is, which is whenever `T: Send`. The guard therefore becomes `Sync` for a type

such as `Cell<i32>`, which is `Send` but not `Sync`. Sharing `&SpinGuard<Cell<i32>>` between two scoped threads lets both call `Cell::set` through `Deref` at the same time, a data race in safe code. The following program compiles against this lock:

```
let lock = SpinLock::new(Cell::new(0));
let guard = lock.lock();
std::thread::scope(|s| {
    s.spawn(|| guard.set(1));
    guard.set(2);
});
```

`std::sync::MutexGuard` avoids this with an explicit `unsafe impl<T: ?Sized + Sync> Sync for MutexGuard<'_, T>`. The smallest fix for `SpinGuard` is a marker field that makes the guard's automatic `Sync` implementation require `T: Sync`:

```
use std::marker::PhantomData;

pub struct SpinGuard<'a, T> {
    lock: &'a SpinLock<T>,
    // `&mut T` is `Sync` only if `T: Sync`, so the guard is too.
    _not_sync_unless_t_is: PhantomData<&'a mut T>,
}
```

Each place that constructs a guard then writes `SpinGuard { lock: self, _not_sync_unless_t_is: PhantomData }`. The same issue appears in the short spin lock in `drills.rs`, described in chapter 59.

Spinning wastes a core. A thread that cannot acquire the lock keeps running. If the holder is descheduled, or the critical section is long, waiting threads burn CPU time without progress. On a machine with more runnable threads than cores this can be much slower than a mutex that parks the thread. Spin locks suit very short critical sections on dedicated cores; `std::sync::Mutex` spins briefly and then parks, and is the right default.

No fairness. A thread that has waited a long time has no advantage over one that arrived a moment ago. Under steady contention, a thread can wait indefinitely.

No poisoning. A panic while the guard is held releases the lock through `Drop`, and the next thread sees whatever partial update the panicking thread left. Chapter 22 explains why `Mutex` records that event.

Not reentrant. Calling `lock` twice on one thread without dropping the first guard spins forever.

Summary

- A spin lock is an `AtomicBool` guarding an `UnsafeCell<T>`; `compare_exchange` lets exactly one thread set the flag.
- `compare_exchange_weak` in a loop, with a `load-and-spin_loop` wait on failure, is the conventional acquire path.
- The guard releases the lock in `Drop`, so every exit from the critical section, including unwinding, unlocks it.
- `unsafe impl<T: Send> Sync for SpinLock<T>` matches `Mutex` and is justified by mutual exclusion.
- The guard's automatically derived `Sync` is too permissive; a `PhantomData<&mut T>` field, or an explicit implementation bounded by `T: Sync`, closes the hole.

References

- Mara Bos, *Rust Atomics and Locks*, O'Reilly Media, 2023, Chapter 4, "Building Our Own Spin Lock".
- Standard library, `AtomicBool::compare_exchange_weak`.
- Standard library, `std::cell::UnsafeCell` and `std::hint::spin_loop`.
- Standard library, `MutexGuard`, for its `Send` and `Sync` implementations.
- The Rustonomicon, [Send and Sync](#).

A Counting Semaphore

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/semaphore.rs`. Test it with `cargo test semaphore`.

Problem Statement

Implement `Semaphore::new(permits)` with:

- `acquire_guard`, which blocks until a permit is free, takes it, and returns a guard that gives it back when dropped;
- `try_acquire`, which takes a permit only if one is free now;
- `try_acquire_timeout`, which waits at most a given duration;
- `available_permits`, for metrics.

At no time may more guards be alive than the semaphore has permits.

Designing a Solution

The state is one integer, `available`, in a `Mutex`. Taking a permit decrements it and releasing one increments it. A thread that finds `available == 0` waits on a `Condvar`, `released`, and every release signals that condition variable once.

```
acquire_guard                release (in SemaphoreGuard::drop)
lock state                  lock state
while available == 0:        available += 1
    state = released.wait(state)  released.notify_one()
available -= 1
return SemaphoreGuard
```

`notify_one` is enough, because a release frees exactly one permit, so exactly one waiter can proceed. The `while` loop re-checks the count after every wake-

up, because a wake-up can be spurious and another thread can take the permit between the signal and the moment the woken thread reacquires the lock.

The guard is what makes the permit count trustworthy. Code written as "acquire, work, release" leaks a permit whenever the work returns early or panics. With a guard, the release runs in `Drop` on every path.

Implementation

```

/// Counting semaphore built from `Mutex` + `Condvar`.
///
/// A semaphore bounds how many callers may proceed at once without protecting any
/// data. It is the primitive behind connection pools, per-account GPU slot limits,
/// and bounded fan-out in an SDK or CLI.
///
/// The RAII guard returns the permit on drop, so a panicking worker cannot leak a
/// permit and slowly deadlock the pool.

use std::sync::{Condvar, Mutex};
use std::time::Duration;

struct State {
    available: usize,
}

/// A counting semaphore. Cloning is not provided; share one behind `Arc`.
pub struct Semaphore {
    state: Mutex<State>,
    released: Condvar,
}

impl Semaphore {
    /// Create a semaphore with `permits` permits available.
    pub fn new(permits: usize) -> Self {
        Self {
            state: Mutex::new(State { available: permits }),
            released: Condvar::new(),
        }
    }

    /// Block until a permit is available.
    pub fn acquire(&self) {
        let _guard = self.acquire_guard();
    }

    /// Block until a permit is available, returning a guard that releases it.
    pub fn acquire_guard(&self) -> SemaphoreGuard<'_> {
        let mut state = self.state.lock().expect("semaphore mutex poisoned");
        while state.available == 0 {
            state = self.released.wait(state).expect("semaphore mutex poisoned");
        }
        state.available -= 1;
        SemaphoreGuard { semaphore: self }
    }

    /// Take a permit only if one is free right now.
    pub fn try_acquire(&self) -> Option<SemaphoreGuard<'_>> {
        let mut state = self.state.lock().expect("semaphore mutex poisoned");
        if state.available == 0 {
            None
        } else {

```

```

        state.available -= 1;
        Some(SemaphoreGuard { semaphore: self })
    }
}

/// Take a permit, waiting at most `timeout`.
pub fn try_acquire_timeout(&self, timeout: Duration) -> Option<SemaphoreGuard<'_>>
{
    let mut state = self.state.lock().expect("semaphore mutex poisoned");
    if state.available > 0 {
        state.available -= 1;
        return Some(SemaphoreGuard { semaphore: self });
    }

    let (mut state, _timed_out) = self
        .released
        .wait_timeout(state, timeout)
        .expect("semaphore mutex poisoned");

    if state.available == 0 {
        None
    } else {
        state.available -= 1;
        Some(SemaphoreGuard { semaphore: self })
    }
}

/// Permits currently free. Useful for metrics, not for control flow.
pub fn available_permits(&self) -> usize {
    self.state
        .lock()
        .expect("semaphore mutex poisoned")
        .available
}

fn release(&self) {
    let mut state = self.state.lock().expect("semaphore mutex poisoned");
    state.available += 1;
    self.released.notify_one();
}

}

/// Releases one permit when dropped.
pub struct SemaphoreGuard<'a> {
    semaphore: &'a Semaphore,
}

impl Drop for SemaphoreGuard<'_> {
    fn drop(&mut self) {
        self.semaphore.release();
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::thread;

    #[test]
    fn never_exceeds_the_permit_count() {
        let permits = 4;
        let semaphore = Arc::new(Semaphore::new(permits));
        let active = Arc::new(AtomicUsize::new(0));
        let peak = Arc::new(AtomicUsize::new(0));

```



```

let mut handles = Vec::new();
for _ in 0..32 {
    let semaphore = Arc::clone(&semaphore);
    let active = Arc::clone(&active);
    let peak = Arc::clone(&peak);
    handles.push(thread::spawn(move || {
        let _guard = semaphore.acquire_guard();
        let now = active.fetch_add(1, Ordering::SeqCst) + 1;
        peak.fetch_max(now, Ordering::SeqCst);
        thread::sleep(Duration::from_millis(1));
        active.fetch_sub(1, Ordering::SeqCst);
    }));
}
for handle in handles {
    handle.join().expect("worker panicked");
}

assert!(peak.load(Ordering::SeqCst) <= permits);
assert_eq!(semaphore.available_permits(), permits);
}

#[test]
fn guard_returns_the_permit_on_drop() {
    let semaphore = Semaphore::new(1);
    {
        let _guard = semaphore.acquire_guard();
        assert_eq!(semaphore.available_permits(), 0);
    }
    assert_eq!(semaphore.available_permits(), 1);
}

#[test]
fn try_acquire_reports_contention() {
    let semaphore = Semaphore::new(1);
    let first = semaphore.try_acquire().expect("first permit is free");
    assert!(semaphore.try_acquire().is_none());
    drop(first);
    assert!(semaphore.try_acquire().is_some());
}

#[test]
fn timeout_gives_up_when_no_permit_arrives() {
    let semaphore = Semaphore::new(0);
    assert!(
        semaphore
            .try_acquire_timeout(Duration::from_millis(5))
            .is_none()
    );
}
}

```

`State` is a struct with a single field rather than a bare `usize`. The indirection costs nothing and leaves room for more state, such as a closed flag, without changing every lock site.

`acquire_guard` returns `SemaphoreGuard<'_>`, whose lifetime borrows the semaphore. The guard cannot outlive the semaphore, and code that shares the semaphore between threads must therefore share it by reference, through `Arc` or a scope.

`try_acquire` returns `Option<SemaphoreGuard<'_>>`, so the caller either holds a permit or knows it does not, and cannot forget to release one it took.

`try_acquire_timeout` checks the count, and if no permit is free, waits once with `wait_timeout`. After the wait it checks the count again and returns `None` if it is still zero. The boolean that reports whether the wait timed out is ignored, because the count is the authoritative answer.

`release` is private. The only way to return a permit is to drop a guard, which rules out releasing a permit that was never taken.

`never_exceeds_the_permit_count` runs 32 threads against four permits. Each thread increments an `active` counter after acquiring, records the maximum with `fetch_max`, sleeps for a millisecond, and decrements. The test requires the recorded peak to be at most four and all permits to be back at the end.

Intuition

Three threads contending for a semaphore with two permits

time	thread	action	available after
1	A	<code>acquire_guard</code> : count is 2, take one	1
2	B	<code>acquire_guard</code> : count is 1, take one	0
3	C	<code>acquire_guard</code> : count is 0, <code>wait</code> releases the lock and parks	0
4	A	guard dropped: <code>release</code> increments and calls <code>notify_one</code>	1
5	C	wakes, reacquires the lock, loop sees 1, takes it	0
6	B	guard dropped	1
7	C	guard dropped	2

Time and Space Complexity

Operation	Time	Space
<code>acquire_guard</code> , permit free	one lock acquisition	none
		the parked thread's stack

Operation	Time	Space
acquire_guard, no permit	blocks until a release; one context switch to wake	
try_acquire	one lock acquisition	none
guard drop	one lock acquisition and one notification	none
memory		one Mutex<usize> and one Condvar

Limitations

acquire does not hold a permit. Its body is `let _guard = self.acquire_guard();`. The guard is bound to `_guard` and dropped at the end of the function, so `acquire` waits until a permit is free, takes it, and returns it before the caller runs any code. A caller that writes `semaphore.acquire(); do_work();` has no concurrency limit at all. The method should either be removed or return the guard. The tests do not call it.

The timeout is not a deadline. `try_acquire_timeout` waits once. A spurious wake-up, or a notification whose permit is taken by another thread first, makes it return `None` before the timeout has elapsed. `Condvar::wait_timeout_while` re-checks the condition and continues waiting for the remaining time, and it is the idiomatic replacement:

```
pub fn try_acquire_timeout(&self, timeout: Duration) ->
Option<SemaphoreGuard<'_>> {
    let state = self.state.lock().expect("semaphore mutex poisoned");
    let (mut state, result) = self
        .released
        .wait_timeout_while(state, timeout, |state| state.available == 0)
        .expect("semaphore mutex poisoned");
    if result.timed_out() {
        return None;
    }
    state.available -= 1;
    Some(SemaphoreGuard { semaphore: self })
}
```

`wait_timeout_while` returns immediately without waiting when a permit is already free, so the separate fast path is no longer needed.

No fairness. A thread that arrives just as a permit is released can take it ahead of one that has waited longer.

No way to change the permit count. A pool that grows or shrinks at run time needs an `add_permits` method, and a shutdown path needs a way to wake all waiters with an error.

`available_permits` is stale on return. The documentation comment says so: the count can change before the caller reads it, so it is suitable for metrics and not for deciding whether to acquire.

Summary

- A counting semaphore is a count under a `Mutex` plus a `Condvar` that waiters park on; each release notifies one waiter.
- The condition is re-checked in a `while` loop after every wake-up.
- A guard that releases in `Drop` makes it impossible to leak a permit on an early return or a panic, and a private `release` makes it impossible to release one twice.
- `acquire` drops its guard before returning, so it does not limit concurrency.
- `wait_timeout_while` implements a time-limited wait that tolerates spurious wake-ups.

References

- Edsger W. Dijkstra, "Cooperating sequential processes", technical report EWD-123, 1965, which introduced semaphores.
- Standard library, `Condvar::wait_timeout_while`.
- Tokio, `tokio::sync::Semaphore`, the asynchronous equivalent with owned permits.

A Closable Bounded Queue

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/bounded_queue.rs`. Test it with `cargo test bounded_queue`.

Problem Statement

Chapter 30's bounded buffer runs a fixed number of pushes and pops, so every consumer knows when to stop. A service does not have that knowledge. Its consumers loop until the producers are finished, and something has to tell a consumer that is parked waiting for an item that no item will ever arrive. Build a bounded queue shared by many producers and many consumers that:

- makes producers wait while the queue is full, and consumers wait while it is empty;
- can be closed, after which `push` fails and returns the item to the caller;
- lets consumers drain items pushed before the close, and then report that no more work exists.

Designing a Solution

The queue uses the same structure as chapter 30: a `Mutex` around a `VecDeque`, a `not_empty` condition variable for consumers, and a `not_full` condition variable for producers. It adds three behaviours.

A `closed` flag inside the protected state. It lives in the same `Mutex` as the items, so a thread reads it and the item count under one lock acquisition and cannot observe one without the other.

push returns Err(item) after close. The caller gets its value back instead of losing it, which matters when the item is a request that must be answered.

pop returns Option<T>. It returns `Some` while items remain, including items pushed before `close`, and `None` only when the queue is both closed and drained. A consumer can therefore be written as `while let Some(item) = queue.pop()`.

<pre> push lock state loop: closed? -> Err(item) Some(item) room? -> push, notify not_empty, Ok wait on not_full close: lock, closed = true, unlock, notify_all on both condition variables </pre>	<pre> pop lock state loop: item available? -> notify not_full, closed? -> None wait on not_empty </pre>
---	--

Implementation

```

/// Bounded blocking queue with backpressure.
///
/// Unbounded queues are a classic production failure: a fast producer and a slow
/// consumer turn into an OOM kill instead of a slow response. A bounded queue makes
/// the producer wait, which is backpressure, and it is the first thing a senior
/// systems interviewer looks for.
///
/// This is an MPMC queue (many producers, many consumers) built from one `Mutex`
/// and two condition variables: `not_empty` for waiting consumers and `not_full`
/// for waiting producers. `close` unblocks everyone so shutdown cannot hang.

use std::collections::VecDeque;
use std::sync::{Condvar, Mutex};

struct State<T> {
    items: VecDeque<T>,
    capacity: usize,
    closed: bool,
}

/// A bounded MPMC queue. Share it behind `Arc`; all methods take `&self`.
pub struct BoundedQueue<T> {
    state: Mutex<State<T>>,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    /// Create a queue that holds at most `capacity` items.
    ///
    /// Panics if `capacity` is zero, because a zero-capacity queue can never make
    /// progress.
    pub fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "capacity must be > 0");
        Self {
            state: Mutex::new(State {
                items: VecDeque::with_capacity(capacity),

```

```

        capacity,
        closed: false,
    }},
    not_empty: Condvar::new(),
    not_full: Condvar::new(),
}

/// Push an item, blocking while the queue is full.
///
/// Returns `Err(item)` if the queue is closed, so the caller keeps ownership
/// instead of losing the value.
pub fn push(&self, item: T) -> Result<(), T> {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    loop {
        if state.closed {
            return Err(item);
        }
        if state.items.len() < state.capacity {
            state.items.push_back(item);
            self.not_empty.notify_one();
            return Ok(());
        }
        state = self.not_full.wait(state).expect("queue mutex poisoned");
    }
}

/// Pop an item, blocking while the queue is empty.
///
/// Returns `None` once the queue is closed and drained, which is how a
/// consumer loop terminates cleanly.
pub fn pop(&self) -> Option<T> {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    loop {
        if let Some(item) = state.items.pop_front() {
            self.not_full.notify_one();
            return Some(item);
        }
        if state.closed {
            return None;
        }
        state = self.not_empty.wait(state).expect("queue mutex poisoned");
    }
}

/// Pop without blocking.
pub fn try_pop(&self) -> Option<T> {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    let item = state.items.pop_front();
    if item.is_some() {
        self.not_full.notify_one();
    }
    item
}

/// Close the queue.
///
/// Pending and future `pop` calls return `None` after the queue drains, and
/// future `push` calls fail. Idempotent.
pub fn close(&self) {
    let mut state = self.state.lock().expect("queue mutex poisoned");
    state.closed = true;
    drop(state);
    self.not_empty.notify_all();
    self.not_full.notify_all();
}

```

```

pub fn is_closed(&self) -> bool {
    self.state.lock().expect("queue mutex poisoned").closed
}

pub fn capacity(&self) -> usize {
    self.state.lock().expect("queue mutex poisoned").capacity
}

pub fn len(&self) -> usize {
    self.state.lock().expect("queue mutex poisoned").items.len()
}

pub fn is_empty(&self) -> bool {
    self.state
        .lock()
        .expect("queue mutex poisoned")
        .items
        .is_empty()
}
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::thread;
    use std::time::Duration;

    #[test]
    fn preserves_order_across_threads() {
        let queue = Arc::new(BoundedQueue::new(8));
        let producer = {
            let queue = Arc::clone(&queue);
            thread::spawn(move || {
                for value in 0..1_000 {
                    queue.push(value).expect("queue stays open");
                }
                queue.close();
            })
        };

        let mut received = Vec::new();
        while let Some(value) = queue.pop() {
            received.push(value);
        }
        producer.join().expect("producer panicked");

        assert_eq!(received, (0..1_000).collect::<Vec<_>>());
    }

    #[test]
    fn never_exceeds_capacity() {
        let queue = Arc::new(BoundedQueue::new(4));
        let observed_peak = Arc::new(AtomicUsize::new(0));
        let producer = {
            let queue = Arc::clone(&queue);
            let observed_peak = Arc::clone(&observed_peak);
            thread::spawn(move || {
                for value in 0..200u32 {
                    queue.push(value).expect("queue stays open");
                    observed_peak.fetch_max(queue.len(), Ordering::SeqCst);
                }
                queue.close();
            })
        };
    }
}

```



```

        while queue.pop().is_some() {
            thread::sleep(Duration::from_micros(50));
        }
        producer.join().expect("producer panicked");

        assert!(observed_peak.load(Ordering::SeqCst) <= 4);
    }

    #[test]
    fn close_wakes_a_blocked_consumer() {
        let queue = Arc::new(BoundedQueue::new(1));
        let consumer = {
            let queue = Arc::clone(&queue);
            thread::spawn(move || queue.pop())
        };

        thread::sleep(Duration::from_millis(20));
        queue.close();

        assert_eq!(consumer.join().expect("consumer panicked"), None);
    }

    #[test]
    fn push_after_close_returns_the_item() {
        let queue = BoundedQueue::new(1);
        queue.close();
        assert_eq!(
            queue.push("workload".to_string()),
            Err("workload".to_string())
        );
    }
}

```

The loops in `push` and `pop` test the closed flag on every iteration, not only before the first wait. A thread parked on a condition variable may be woken by `close` rather than by a state change it was waiting for, and the re-check is what turns that wake-up into a return.

`close` sets the flag, releases the lock, and calls `notify_all` on both condition variables. `notify_one` would be wrong here. Closing is a change that every waiter must observe, because every parked producer must fail and every parked consumer must either take a remaining item or return `None`. The method is idempotent: a second call sets a flag that is already set.

`pop` checks for an item before it checks the flag. That order is what drains the queue after `close`: items pushed before shutdown are still delivered, and `None` means that there is no more work, not that shutdown has been requested.

`try_pop` never waits, and it still signals `not_full` when it removes an item, so a producer parked on a full queue is not stranded by a consumer that polls.

`new` asserts that the capacity is positive, because a queue of capacity zero could never accept an item.

The four tests are concurrency tests rather than single-threaded checks. `preserves_order_across_threads` pushes a thousand values from one thread and pops them on another. `never_exceeds_capacity` records the largest length a producer observes while a slow consumer drains. `close_wakes_a_blocked_consumer` parks a consumer on an empty queue and requires `close` to release it with `None`, which is the test that fails if `pop` checks the flag only before waiting. `push_after_close_returns_the_item` checks that the value comes back to the caller.

The `len`, `is_empty`, and `is_closed` accessors each take the lock and return a snapshot. By the time the caller reads the value, another thread may have changed it, so they are useful for metrics and tests and not for deciding whether to call `pop`.

Intuition

`close_wakes_a_blocked_consumer` with capacity 1

step	consumer thread	main thread	state
1	<code>pop</code> locks; no item; not closed; waits on <code>not_empty</code>		<code>[]</code> , open
2	parked, lock released	sleeps 20 ms	<code>[]</code> , open
3		<code>close</code> : sets <code>closed</code> , unlocks, <code>notify_all</code>	<code>[]</code> , closed
4	wakes, reacquires lock; no item; closed, returns <code>None</code>		<code>[]</code> , closed
5	thread ends	<code>join</code> returns <code>None</code>	

Time and Space Complexity

Operation	Cost
<code>push</code> , <code>pop</code> , <code>try_pop</code>	$O(1)$ amortised under the lock, plus waiting time when full or empty
<code>close</code>	$O(1)$ plus waking every parked thread
accessors	one lock acquisition each

Operation	Cost
memory	capacity slots allocated at construction

Limitations

Poisoning panics every caller. Each method uses `expect("queue mutex poisoned")`. A panic in one thread while it holds the lock makes every later call panic as well. Chapter 22 describes the alternatives.

No fairness. A producer that arrives just as a slot frees can take it ahead of one that has waited longer.

No timeouts. `push` and `pop` wait indefinitely. A service that must give up after a deadline needs `Condvar::wait_timeout_while` and a result that distinguishes a timeout from a close.

notify_all on close wakes every waiter at once. For a queue with thousands of parked threads, closing causes a burst of wake-ups that each take the lock in turn. This is the correct behaviour for shutdown, and it is a cost worth knowing about.

Summary

- A closable queue keeps a `closed` flag under the same lock as the items, so waiters see a consistent state.
- `push` returns the rejected item after close, and `pop` drains remaining items before it reports `None`.
- The closed flag is re-checked inside the wait loops, and `close` uses `notify_all`, because every waiter must observe shutdown.
- `try_pop` must signal `not_full` like `pop`, or a polling consumer can strand producers.

References

- Standard library, `std::sync::Condvar`, on `notify_all` and spurious wake-ups.
- The `crossbeam-channel` crate, `bounded`, a production bounded MPMC channel.

A Lock-Free Ring Buffer

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/ring_buffer.rs`. Test it with `cargo test ring_buffer`.

Problem Statement

Implement `SpSCRing<T, N>` with capacity `N`:

- `push(value) -> Result<(), T>` returns the value when the ring is full;
- `pop() -> Option<T>` returns `None` when the ring is empty;
- one thread may push while another pops, with no lock;
- items still in the ring when it is dropped must be dropped too.

Designing a Solution

Two indices, two owners. `tail` is the index of the next slot to write, and only the producer changes it. `head` is the index of the next slot to read, and only the consumer changes it. Neither side ever writes the other's index, so no compare-and-swap is needed; each side reads the other's index to decide whether it may proceed.

Indices that only grow. The indices count total pushes and total pops, and the slot used is `index % N`. The number of queued items is `tail - head`, computed with wrapping subtraction. Keeping full and empty distinct needs no spare slot: empty is `head == tail`, and full is `tail - head == N`.

Publish after the write. The producer writes the value into the slot first and stores the new `tail` second. Until that store, the consumer's view of `tail` does not include the slot, so it cannot read a slot that is still being written. The con-

sumer does the mirror image: it reads the value out, then stores the new `head`, which is when the producer may reuse the slot.

a ring of four slots after six pushes and three pops

`tail = 6, head = 3, len = tail - head = 3`

slot index:	0	1	2	3
	[v4]	[v5]	[free]	[v3]
			^	^
		tail % 4 = 2		head % 4 = 3
		next write		next read

Uninitialized storage. Slots that do not hold a value must not be dropped or read. Each slot is `UnsafeCell<MaybeUninit<T>>`: `MaybeUninit` allows a slot to be uninitialized, and `UnsafeCell` allows the two threads to write slots through a shared reference.

Implementation

```
/// Lock-free single-producer / single-consumer (SPSC) ring buffer.
///
/// The classic producer/consumer exercise, but with no mutex: the producer owns
/// the `tail` index and the consumer owns the `head` index, and each publishes its
/// write with a release store so the other side sees the slot only after it is
/// initialized. This is the shape inside channel implementations, log shippers,
/// and kernel ring buffers.
///
/// Two rules keep this sound:
///
/// - exactly one thread calls [SpscRing::push] and exactly one calls
///   [SpscRing::pop];
/// - a slot is written before `tail` is published, and read only after `head`
///   reaches it.
///
/// If you need many producers or consumers, use a mutex-backed queue such as
/// [crate::problems::bounded_queue::BoundedQueue].

use std::cell::UnsafeCell;
use std::mem::MaybeUninit;
use std::sync::atomic::{AtomicUsize, Ordering};

/// Ring buffer with capacity `N` (the maximum number of queued items).
pub struct SpscRing<T, const N: usize> {
    slots: [UnsafeCell<MaybeUninit<T>>; N],
    /// Next index to read. Written only by the consumer.
    head: AtomicUsize,
    /// Next index to write. Written only by the producer.
    tail: AtomicUsize,
}

// SAFETY: at most one thread mutates `head` and one mutates `tail`. A slot is
// initialized before `tail` is published with a release store, and the consumer
// performs an acquire load of `tail` before reading that slot, so there is no
// data race and no read of uninitialized memory under the SPSC discipline.
unsafe impl<T: Send, const N: usize> Sync for SpscRing<T, N> {}

impl<T, const N: usize> SpscRing<T, N> {
```

```

/// Create an empty ring. Panics if `N` is zero.
pub fn new() -> Self {
    assert!(N > 0, "ring capacity must be > 0");
    Self {
        slots: std::array::from_fn(|_| UnsafeCell::new(MaybeUninit::uninit())),
        head: AtomicUsize::new(0),
        tail: AtomicUsize::new(0),
    }
}

/// Maximum number of items the ring can hold.
pub const fn capacity(&self) -> usize {
    N
}

/// Number of queued items. Only meaningful to the owning side in a race.
pub fn len(&self) -> usize {
    self.tail
        .load(Ordering::Acquire)
        .wrapping_sub(self.head.load(Ordering::Acquire))
}

pub fn is_empty(&self) -> bool {
    self.len() == 0
}

pub fn is_full(&self) -> bool {
    self.len() >= N
}

/// Enqueue a value, returning `Err(value)` when the ring is full.
pub fn push(&self, value: T) -> Result<(), T> {
    let tail = self.tail.load(Ordering::Relaxed);
    let head = self.head.load(Ordering::Acquire);
    if tail.wrapping_sub(head) >= N {
        return Err(value);
    }

    let index = tail % N;
    // SAFETY: the slot at `index` is free because the ring is not full, and
    // only the producer writes it.
    unsafe { (*self.slots[index].get()).write(value) };

    // Publish the slot to the consumer.
    self.tail.store(tail.wrapping_add(1), Ordering::Release);
    Ok(())
}

/// Dequeue a value, or `None` when the ring is empty.
pub fn pop(&self) -> Option<T> {
    let head = self.head.load(Ordering::Relaxed);
    let tail = self.tail.load(Ordering::Acquire);
    if head == tail {
        return None;
    }

    let index = head % N;
    // SAFETY: `head != tail`, so the producer has published this slot and
    // only the consumer reads it.
    let value = unsafe { (*self.slots[index].get()).assume_init_read() };

    // Publish the freed slot to the producer.
    self.head.store(head.wrapping_add(1), Ordering::Release);
    Some(value)
}

```

```

impl<T, const N: usize> Default for SpscRing<T, N> {
    fn default() -> Self {
        Self::new()
    }
}

impl<T, const N: usize> Drop for SpscRing<T, N> {
    fn drop(&mut self) {
        // Drop any items that were never popped.
        while self.pop().is_some() {}
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Arc;
    use std::sync::atomic::AtomicUsize;
    use std::thread;

    #[test]
    fn is_fifo_within_capacity() {
        let ring = SpscRing::<u32, 4>::new();
        assert!(ring.push(1).is_ok());
        assert!(ring.push(2).is_ok());
        assert!(ring.push(3).is_ok());
        assert_eq!(ring.pop(), Some(1));
        assert_eq!(ring.pop(), Some(2));
        assert_eq!(ring.pop(), Some(3));
        assert_eq!(ring.pop(), None);
    }

    #[test]
    fn full_ring_returns_the_value() {
        let ring = SpscRing::<u32, 2>::new();
        assert!(ring.push(1).is_ok());
        assert!(ring.push(2).is_ok());
        assert!(ring.is_full());
        assert_eq!(ring.push(3), Err(3));
    }

    #[test]
    fn producer_and_consumer_run_concurrently_in_order() {
        const ITEMS: u64 = 50_000;
        const SENTINEL: u64 = u64::MAX;

        let ring = Arc::new(SpscRing::<u64, 256>::new());

        let producer = {
            let ring = Arc::clone(&ring);
            thread::spawn(move || {
                for value in 0..ITEMS {
                    let mut pending = value;
                    loop {
                        match ring.push(pending) {
                            Ok(()) => break,
                            Err(returned) => {
                                pending = returned;
                                thread::yield_now();
                            }
                        }
                    }
                }
            })
        };

        let mut pending = SENTINEL;
        loop {
            match ring.push(pending) {
                Ok(()) => break,
            }
        }
    }
}

```

```

        Err(returned) => {
            pending = returned;
            thread::yield_now();
        }
    }
}

});

let consumer = {
    let ring = Arc::clone(&ring);
    thread::spawn(move || {
        let mut received = Vec::with_capacity(ITEMS as usize);
        loop {
            match ring.pop() {
                Some(SENTINEL) => break,
                Some(value) => received.push(value),
                None => thread::yield_now(),
            }
        }
        received
    })
};

producer.join().expect("producer panicked");
let received = consumer.join().expect("consumer panicked");

assert_eq!(received, (0..ITEMS).collect::<Vec<_>>());
}

#[test]
fn dropped_ring_drops_queued_items() {
    struct Tracker(Arc<AtomicUsize>);
    impl Drop for Tracker {
        fn drop(&mut self) {
            self.0.fetch_add(1, Ordering::SeqCst);
        }
    }

    let dropped = Arc::new(AtomicUsize::new(0));
    {
        let ring = SpscRing::<Tracker, 4>::new();
        assert!(ring.push(Tracker(Arc::clone(&dropped))).is_ok());
        assert!(ring.push(Tracker(Arc::clone(&dropped))).is_ok());
        assert!(ring.push(Tracker(Arc::clone(&dropped))).is_ok());
    }
    assert_eq!(dropped.load(Ordering::SeqCst), 3);
}
}

```

Storage

`slots: [UnsafeCell<MaybeUninit<T>>; N]` is an array whose length is the `const` generic parameter, so the ring stores its items inline with no heap allocation. `new` builds it with `std::array::from_fn`, which calls the closure once per element. The obvious `[UnsafeCell::new(MaybeUninit::uninit()); N]` does not compile, because array repeat expressions require the element to be `Copy` or a constant.

`assert!(N > 0)` runs at construction. A zero-capacity ring would make `tail % N` a division by zero.

The producer's side

`push` loads its own `tail` with `Relaxed`, because no other thread writes it, and loads the consumer's `head` with `Acquire`, because it must see the consumer's latest published value. If the ring is full, the value is returned to the caller in `Err`.

The `unsafe` block writes into the slot with `MaybeUninit::write`, which does not drop the previous contents. That is required: the slot either was never initialized or was moved out of by `pop`, and dropping it would be a double free or a read of uninitialized memory. The `SAFETY` comment names the two facts that justify the write: the slot is free because the ring is not full, and only the producer writes slots.

`self.tail.store(tail.wrapping_add(1), Ordering::Release)` publishes the new item.

The consumer's side

`pop` is the mirror of `push`. `assume_init_read` copies the value out of the slot by bitwise read and leaves the slot logically uninitialized. The `SAFETY` comment rests on `head != tail`, which means the producer has published this slot.

Dropping remaining items

`Drop` calls `pop` until the ring is empty, which moves each remaining value out and drops it. `drop` receives `&mut self`, so no other thread can be using the ring, and the atomic operations are merely unnecessary rather than incorrect.

The Sync implementation

`unsafe impl<T: Send, const N: usize> Sync for SpscRing<T, N> {}` lets `&SpscRing` be shared between threads. The `SAFETY` comment conditions it on "at most one thread mutates `head` and one mutates `tail`". The Limitations section examines what happens when that condition is not met.

`producer_and_consumer_run_concurrently_in_order` pushes 50,000 values and a sentinel from one thread and pops on another, retrying with `yield_now` when the ring is full or empty. It requires the consumer to receive every value in order.

`dropped_ring_drops_queued_items` pushes three `Tracker` values that increment a shared counter when dropped, lets the ring go out of scope, and checks that the counter reads three. It would fail if `Drop` were missing, because `MaybeUninit` never drops its contents by itself.

Intuition

`is_fifo_within_capacity` on `SpscRing::<u32, 4>`

call	tail before	head before	slot	tail after	head after	result
<code>push(1)</code>	0	0	write 0	1	0	<code>Ok(())</code>
<code>push(2)</code>	1	0	write 1	2	0	<code>Ok(())</code>
<code>push(3)</code>	2	0	write 2	3	0	<code>Ok(())</code>
<code>pop()</code>	3	0	read 0	3	1	<code>Some(1)</code>
<code>pop()</code>	3	1	read 1	3	2	<code>Some(2)</code>
<code>pop()</code>	3	2	read 2	3	3	<code>Some(3)</code>
<code>pop()</code>	3	3	none	3	3	<code>None</code>

Time and Space Complexity

Operation	Time	Space
push, pop	$O(1)$: two atomic loads, one store, one slot move	none
len, is_empty, is_full	$O(1)$	none
drop	$O(\text{len})$	none
memory		N slots of <code>size_of::<T>()</code> plus two usize indices, inline

The ring never blocks and never allocates after construction. The producer and consumer still share the cache lines holding head and tail, so heavy traffic on both sides causes the false sharing described in chapter 48; production rings pad the two indices onto separate cache lines.

Limitations

The single-producer, single-consumer rule is not enforced, so the safe API admits undefined behaviour. `push` and `pop` take `&self`, and `SpScRing` is `Sync`. Nothing stops two threads holding an `Arc<SpScRing<T, N>>` from both calling `push`. Both can load the same `tail`, write the same slot concurrently, and store the same new `tail`, so one value is lost and the slot write is a data race. That is undefined behaviour reachable from safe code, which means the `unsafe impl Sync` is unsound as written.

The idiomatic repair is to encode the rule in types. A `split` method hands out one `Producer` with `push` and one `Consumer` with `pop`. Each handle is `Send`, so it can be moved to its thread, and neither is `Clone` or `Sync`, so there can be only one of each and neither can be shared:

```
pub struct Producer<'a, T, const N: usize> {
    ring: &'a SpScRing<T, N>,
    _not_sync: PhantomData<Cell<()>>,
}

pub struct Consumer<'a, T, const N: usize> {
    ring: &'a SpScRing<T, N>,
    _not_sync: PhantomData<Cell<()>>,
}

impl<T: Send, const N: usize> SpScRing<T, N> {
    /// Borrowing `self` mutably guarantees this is the only split.
    pub fn split(&mut self) -> (Producer<'_, T, N>, Consumer<'_, T, N>) {
        let ring: &Self = self;
        (
            Producer { ring, _not_sync: PhantomData },
            Consumer { ring, _not_sync: PhantomData },
        )
    }
}
```

`push` and `pop` then move to `Producer` and `Consumer`, and become private on the ring. `PhantomData<Cell<()>>` removes the automatic `Sync` implementation while keeping `Send`.

Index continuity assumes a power-of-two capacity after wrap-around. When `tail` wraps from `usize::MAX` to 0, the slot index jumps from `usize::MAX % N` to 0, which is the next slot only when `N` divides 2^{64} . With any other `N`, the ring would misplace items after 2^{64} operations. The limit is unreachable in practice, and restricting `N` to powers of two would also let `% N` become a mask.

Busy retrying is the caller's job. A full or empty ring returns immediately. The tests retry with `yield_now`, which uses a core while waiting. A

blocking wrapper needs a way to park and wake the other side, such as a `futex` or a condition variable, which reintroduces some of the cost the design removes.

The array is inline. A ring of 4,096 `u64` values is 32 KiB inside the struct, and constructing it on the stack before moving it into an `Arc` can overflow a small thread stack. Large rings are better allocated on the heap.

Summary

- An SPSC ring gives the tail index to the producer and the head index to the consumer, so each side writes only its own atomic and no lock or compare-and-swap is needed.
- Writing the slot before publishing `tail`, and reading it before publishing `head`, guarantees that neither side sees a slot the other is still using.
- `UnsafeCell<MaybeUninit<T>>` slots in a const-generic array hold values without heap allocation; `MaybeUninit::write` and `assume_init_read` move values in and out without dropping uninitialized memory.
- A `Drop` implementation must drain the ring, because `MaybeUninit` never drops its contents.
- Because `push` and `pop` take `&self` on a `Sync` type, two producers can race; split handles that are `Send` and not `Sync` enforce the single-producer rule in the types.

References

- Mara Bos, *Rust Atomics and Locks*, O'Reilly Media, 2023, Chapter 5, "Building Our Own Channels".
- Standard library, `std::mem::MaybeUninit` and `std::array::from_fn`.
- The Rust Reference, [Const generics](#).
- The `crossbeam` project, `ArrayQueue`, a bounded lock-free MPMC queue.

Part XIV: Performance and the Operating Sys- tem

Cache Locality and False Sharing

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/cs_locality.rs` and `src/bin/concurrency_false_sharing.rs`. Run them with `cargo run --release --bin cs_locality` and `cargo run --release --bin concurrency_false_sharing`.

Problem Statement

Two experiments:

1. Sum 4,194,304 `u64` values once in index order and once in an order that jumps a million positions at each step. Both sums visit every element exactly once and must agree. Report both times.
2. Run four threads, each incrementing its own `AtomicUsize` two million times. Store the counters once packed next to each other and once padded to a cache line each. Report both times.

Designing a Solution

The memory hierarchy. A processor reads memory through several levels of cache. On the machine used for this book, a Cortex-A78AE, each core has 512 KiB of level-1 data cache and 2 MiB of level-2 cache. Memory moves between main memory and the caches in cache lines, 64 bytes at a time. A read of an address already in cache costs about a nanosecond; a read that misses every level waits for main memory, which costs tens of nanoseconds.

Sequential access. Walking an array in order touches one new cache line every eight `u64` elements, and the hardware prefetcher recognises the pattern

and loads the next lines before they are needed. Almost every read is a cache hit.

Strided access. Stepping by 1,000,003 positions modulo the array length lands on an unrelated cache line every time, and the prefetcher cannot predict the next address. For an array of 32 MiB, far larger than the caches, most reads miss. Because the length is a power of two and the stride is odd, the stride and the length are coprime, so the walk visits every index exactly once before it repeats.

False sharing. Two threads that write different variables on the same cache line still invalidate each other's cached copy of that line. Each write by one core forces the other core to fetch the line again before its next write. Padding each counter to 64 bytes places each on its own line and removes the interference.

The diagram below shows the two counter layouts.

unpadded, 8 bytes each one 64-byte cache line [c0 c1 c2 c3 ] ^ ^ ^ ^ every write by any thread invalidates the line on the other three cores	padded, #[repr(align(64))], 64 bytes each line 0 [c0] line 1 [c1] line 2 [c2] line 3 [c3] each thread writes only its own line
---	--

Implementation

Sequential and strided sums

```

/// Cache locality: the same data is much faster to walk in order than to jump
/// around, because memory moves in cache lines and the prefetcher helps only the
/// sequential case.
///
/// Run with: cargo run --bin cs_locality

use std::time::{Duration, Instant};

/// A power of two, so a stride must be odd to visit every index.
const ELEMENTS: usize = 4 * 1024 * 1024;

/// Odd, and therefore coprime with `ELEMENTS`: the walk visits every slot once.
const STRIDE: usize = 1_000_003;

fn sequential_sum(data: &[u64]) -> (u64, Duration) {
    let start = Instant::now();
    let sum = data.iter().sum();
    (sum, start.elapsed())
}

fn strided_sum(data: &[u64]) -> (u64, Duration) {

```

```

let start = Instant::now();
let mut index = 0usize;
let mut sum = 0u64;
for _ in 0..data.len() {
    index = (index + STRIDE) % data.len();
    sum = sum.wrapping_add(data[index]);
}
(sum, start.elapsed())
}

fn main() {
    let data: Vec<u64> = (0..ELEMENTS as u64).collect();
    println!(
        "{} elements, {} MiB",
        data.len(),
        data.len() * 8 / (1024 * 1024)
    );

    let (sequential, sequential_time) = sequential_sum(&data);
    let (strided, strided_time) = strided_sum(&data);

    println!("\n sequential: {sequential} in {sequential_time:?}");
    println!(" strided:      {strided} in {strided_time:?}");

    // Both touch every element exactly once, so the sums must match; only the
    // access pattern differs.
    assert_eq!(sequential, strided);
    assert_eq!(sequential, ELEMENTS as u64 * (ELEMENTS as u64 - 1) / 2);

    println!("\nequal work, different pattern; timings vary by machine");
}

```

`sequential_sum` is `data.iter().sum()`. `strided_sum` advances `index` by `STRIDE` modulo the length and adds with `wrapping_add`, which cannot overflow into a panic and produces the same result as ordinary addition for this total.

The assertions check two facts: the two sums are equal, which proves the strided walk visited every element once, and both equal $n(n - 1)/2$ for the values `0..n`.

Padded and unpadded counters

```

///! False sharing: two independent counters that land on the same cache line make
///! each other slow, because every write moves the line between cores.
///!
///! Run with: cargo run --bin concurrency_false_sharing

use std::mem::size_of;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;
use std::time::{Duration, Instant};

const THREADS: usize = 4;
const ITERATIONS: usize = 2_000_000;

///! Packed by default: consecutive counters share a cache line.
struct Unpadded(AtomicUsize);

impl Unpadded {
    fn new() -> Self {
        Self(AtomicUsize::new(0))
    }
}

```



```

    }

    fn increment(&self) {
        self.0.fetch_add(1, Ordering::Relaxed);
    }
}

/// Padded to a cache line, so no two counters share one.
#[repr(align(64))]
struct Padded(AtomicUsize);

impl Padded {
    fn new() -> Self {
        Self(AtomicUsize::new(0))
    }

    fn increment(&self) {
        self.0.fetch_add(1, Ordering::Relaxed);
    }
}

/// Scoped threads borrow the vector directly, so there is no `Arc` and no clone.
fn run_unpadded() -> Duration {
    let counters: Vec<Unpadded> = (0..THREADS).map(|_| Unpadded::new()).collect();
    let start = Instant::now();
    thread::scope(|scope| {
        for counter in &counters {
            scope.spawn(move || {
                for _ in 0..ITERATIONS {
                    counter.increment();
                }
            });
        }
    });
    start.elapsed()
}

fn run_padded() -> Duration {
    let counters: Vec<Padded> = (0..THREADS).map(|_| Padded::new()).collect();
    let start = Instant::now();
    thread::scope(|scope| {
        for counter in &counters {
            scope.spawn(move || {
                for _ in 0..ITERATIONS {
                    counter.increment();
                }
            });
        }
    });
    start.elapsed()
}

fn main() {
    println!("atomic:   {} bytes", size_of::<AtomicUsize>());
    println!(
        "unpadded: {} bytes (several counters per 64 byte line)",
        size_of::<Unpadded>()
    );
    println!(
        "padded:   {} bytes (one counter per 64 byte line)",
        size_of::<Padded>()
    );

    let unpadded = run_unpadded();
    let padded = run_padded();

    println!("\n{n{THREADS} threads x {ITERATIONS} increments, each on its own counter");
}

```

```
println!("  unpadded (shared lines): {unpadded:?}");
println!("  padded   (own line):      {padded:?}");

assert!(size_of::<Padded>() >= 64, "padding must fill a cache line");
assert!(unpadded.as_nanos() > 0 && padded.as_nanos() > 0);

println!("\ntimings vary by machine; the padded run is normally the faster one");
}
```

`#[repr(align(64))]` raises the alignment of `Padded` to 64 bytes. The size of a type is always a multiple of its alignment, so `size_of::<Padded>()` becomes 64 even though the field is 8 bytes. In a `Vec<Padded>`, every element starts on a 64-byte boundary.

Both runs use `thread::scope` so that the threads can borrow the vector. The `move` closure captures `counter`, a `&Unpadded` or `&Padded`, by value.

`Ordering::Relaxed` is correct for independent counters, as chapter 43 explained. The choice of ordering does not affect the false-sharing result, which comes from the writes to memory, not from the atomic protocol.

False sharing inside one struct

A second program isolates the effect with two counters and no vector: `src/bin/false_sharing.rs`. Run it with `cargo run --release --bin false_sharing`.

```
use std::sync::atomic::{AtomicU64, Ordering};
use std::thread;
use std::time::{Duration, Instant};

const ITERS: u64 = 20_000_000;

/// Both counters are guaranteed to live inside one 64-byte cache line.
#[repr(align(64))]
struct SameLine {
    a: AtomicU64,
    b: AtomicU64,
}

/// Each of these occupies its own 64-byte block.
#[repr(align(64))]
struct Padded(AtomicU64);

fn time<F: FnOnce()>(f: F) -> Duration {
    let t = Instant::now();
    f();
    t.elapsed()
}

fn main() {
    // ---- 1. False sharing: two counters on the same cache line ----
    let same = SameLine { a: AtomicU64::new(0), b: AtomicU64::new(0) };

    let d1 = time(|| {
        thread::scope(|s| {
            s.spawn(|| for _ in 0..ITERS { same.a.fetch_add(1, Ordering::Relaxed); });
            s.spawn(|| for _ in 0..ITERS { same.b.fetch_add(1, Ordering::Relaxed); });
        });
    });
```

```

});

// ---- 2. Padded: same work, but counters are on separate cache lines ----
let p0 = Padded(AtomicU64::new(0));
let p1 = Padded(AtomicU64::new(0));

let d2 = time(|| {
  thread::scope(|s| {
    s.spawn(|| for _ in 0..ITERS { p0.0.fetch_add(1, Ordering::Relaxed); });
    s.spawn(|| for _ in 0..ITERS { p1.0.fetch_add(1, Ordering::Relaxed); });
  });
});

println!("same cache line (false sharing): {:?}", d1);
println!("padded (separate lines):          {:?}", d2);
}

```

`SameLine` holds two `AtomicU64` fields, 8 bytes each, and `#[repr(align(64))]` aligns the struct to a 64-byte boundary. Its size is rounded up to 64 bytes, so both fields are guaranteed to start inside the same 64-byte cache line. The first experiment gives one counter to each of two threads. The threads never touch each other's field, yet every `fetch_add` by one thread invalidates the other core's cached copy of the line, and the next write on that core must fetch the line again.

`Padded` wraps a single `AtomicU64` and is also aligned to 64 bytes, so `p0` and `p1` each occupy their own 64-byte block and start on different cache lines. The second experiment performs the same forty million increments with no shared line.

Both experiments use `thread::scope`, so the closures borrow `same`, `p0`, and `p1` from the stack without `Arc`, and the scope joins both threads before `time` reads the clock. `time` accepts any `FnOnce()` and returns the elapsed `Duration`.

A release build on the NVIDIA Jetson board, run three times, printed:

```

same cache line (false sharing): 1.251460811s
padded (separate lines):          149.715657ms

same cache line (false sharing): 1.389555737s
padded (separate lines):          151.341589ms

same cache line (false sharing): 1.189698031s
padded (separate lines):          154.447719ms

```

The padded version was about eight times faster in every run, although both do exactly the same work. The only difference is whether the two counters share a cache line.

The program assumes a 64-byte cache line, which holds for the Cortex-A78AE and most x86 processors. On a processor with 128-byte lines, such as Apple's

M-series, `SameLine` still shares a line, and `Padded` would need `#[repr(align(128))]` to separate the two counters with certainty.

Intuition

Release builds on the Cortex-A78AE printed:

4194304 elements, 32 MiB

```
sequential: 8796090925056 in 2.95291ms
strided:    8796090925056 in 56.982884ms
```

equal work, different pattern; timings vary by machine

```
atomic:    8 bytes
unpadded:  8 bytes (several counters per 64 byte line)
padded:    64 bytes (one counter per 64 byte line)
```

```
4 threads x 2000000 increments, each on its own counter
unpadded (shared lines): 207.037319ms
padded   (own line):    19.317951ms
```

timings vary by machine; the padded run is normally the faster one

The strided walk took about 19 times as long as the sequential walk, 13.6 ns per element against 0.7 ns. The padded counters finished about 10.7 times faster than the packed ones, although the program performs exactly the same eight million increments in both runs.

Time and Space Complexity

Experiment	Time	Space
sequential sum	$O(n)$, about one cache miss per 8 elements before prefetching	$O(1)$
strided sum	$O(n)$, about one cache miss per element	$O(1)$
unpadded counters	$O(\text{threads} \times \text{iterations})$, with cache-line transfers on most writes	8 bytes per counter
padded counters	$O(\text{threads} \times \text{iterations})$	64 bytes per counter

Both pairs have identical asymptotic cost. The measured differences are constant factors, of 19 and 10.7 respectively on this machine.

Limitations

The locality experiment measures more than cache misses.

`data.iter().sum()` is a loop the compiler vectorizes, adding several elements per instruction. The strided loop cannot be vectorized, and it performs an integer division, `% data.len()`, on every iteration because the divisor is not a constant. Part of the 19-times difference is therefore vectorization and division. A control run that uses a stride of 1 through the same code as `strided_sum` would separate the access-pattern cost from the loop's own cost.

Single runs without repetition. Each experiment is timed once. The array is fully written by `collect` before either timing begins, so page faults are not included, but frequency scaling and interference from other processes are.

The cache-line size is assumed. 64 bytes is correct for the Cortex-A78AE and for most x86 processors. Some processors, including Apple's M-series, use 128-byte lines, where `align(64)` would not fully separate the counters. The `crossbeam-utils` crate's `CachePadded` chooses the alignment per target.

The two run functions duplicate each other. `run_unpadded` and `run_padded` differ only in the counter type. A generic function over a small trait with `new` and `increment`, or over a closure that builds the counters, would remove the duplication, as chapter 38 did for its list variants.

Results vary with scheduling. If the operating system places two of the four threads on the same core, they no longer contend for a cache line in the same way. The program does not pin threads to cores.

Summary

- Visiting memory in order lets the prefetcher load cache lines before they are needed; a strided walk over a large array misses the cache on almost every read.
- On a Cortex-A78AE, the strided walk over 32 MiB took about 19 times longer than the sequential walk, a figure that includes vectorization and division as well as cache misses.
- Independent variables on the same cache line slow each other down when different threads write them; this is false sharing.

- `#[repr(align(64))]` puts each value on its own cache line, and made four independent counters about 10.7 times faster.
- Performance comparisons need a control that isolates the effect being measured.

References

- Ulrich Drepper, "What every programmer should know about memory", 2007, Sections 3 and 6.
- Mara Bos, *Rust Atomics and Locks*, O'Reilly Media, 2023, Chapter 2, "Understanding the Processor".
- The Rust Reference, [The align modifier](#).
- The `crossbeam-utils` crate, `CachePadded`.

Amdahl's Law and Deadlock Detection

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/concurrency_amdahl.rs` and `src/bin/concurrency_deadlock.rs`. Run them with `cargo run --bin concurrency_amdahl` and `cargo run --bin concurrency_deadlock`.

Problem Statement

1. Given the fraction `s` of a program's work that must run serially, compute its speedup on `p` processors, tabulate it for several values of each, and print the limit as `p` grows without bound.
2. Given a list of edges `(waiter, holder)`, each meaning that one task is blocked waiting for a resource another task holds, decide whether the tasks are deadlocked.

Designing a Solution

Amdahl's law

Normalise the program's running time on one processor to 1. The serial part takes `s` and the parallel part takes `1 - s`. On `p` processors the parallel part takes `(1 - s) / p`, and the serial part still takes `s`:

$$\text{speedup}(s, p) = 1 / (s + (1 - s) / p)$$

as `p` grows without bound, `(1 - s) / p` approaches 0, so speedup approaches `1 / s`

A program that is half serial can never run more than twice as fast, however many processors it is given.

Deadlock as a cycle

Draw one vertex per task and one edge from each waiting task to the task holding what it waits for. If following edges from some task leads back to that task, every task on the path is waiting for the next, and none can proceed: a deadlock. If the graph has no cycle, some task at the end of every path is not waiting, and it can finish and release what it holds.

A depth-first search finds a cycle by colouring vertices:

```
0  unvisited
1  on the current search path
2  fully explored; no cycle is reachable from it

visiting a vertex marked 1 means the path has returned to itself: a cycle
visiting a vertex marked 2 can be skipped
```

The two non-zero colours must stay distinct. A vertex reached twice through different paths in an acyclic graph, such as vertex 2 in $0 \rightarrow 1 \rightarrow 2$ and $0 \rightarrow 2$, is marked 2 by the time the second path arrives, and it is not a cycle.

Implementation

```
/// Amdahl's law: the serial fraction of the work caps the speedup from adding
/// processors. A program that is 50 percent serial cannot go faster than 2x, no
/// matter how many cores it gets.
///
/// Run with: cargo run --bin concurrency_amdahl

/// Speedup with `processors` when `serial` is the non-parallel fraction.
fn speedup(serial: f64, processors: f64) -> f64 {
    1.0 / (serial + (1.0 - serial) / processors)
}

fn main() {
    let processor_counts = [1.0, 2.0, 4.0, 8.0, 16.0];
    let serial_fractions = [0.0, 0.05, 0.10, 0.25, 0.50];

    print!("{:>8} |", "serial");
    for processors in processor_counts {
        print!(" {processors:>6.0}");
    }
    println!("    <- processors");
    println!("{:~>9}+{}", "", "-".repeat(7 * processor_counts.len()));

    for serial in serial_fractions {
        print!("{serial:>8.2} |");
        for processors in processor_counts {
            print!(" {:>6.2}", speedup(serial, processors));
        }
        println!();
    }
}
```



```

    }

    println!("\nlimit as processors grow: 1 / serial");
    for serial in serial_fractions {
        let limit = if serial == 0.0 {
            f64::INFINITY
        } else {
            1.0 / serial
        };
        println!(" {serial:>4.2} -> {limit:>6.1}");
    }

    assert!((speedup(0.0, 4.0) - 4.0).abs() < 1e-9);
    assert!((speedup(0.5, 4.0) - 1.6).abs() < 1e-9);
    assert!(speedup(0.5, 16.0) < 1.9, "2x is the ceiling for 50% serial");

    println!("\nall checks passed");
}

```

The table uses format specifiers throughout. `{:>8}` right-aligns in eight columns, `{processors:>6.0}` prints a floating-point value with no decimals in six columns, and `{:->9}` fills nine columns with `-`, which produces the rule under the header.

The limit loop checks `serial == 0.0` before dividing, and prints `inf` for a fully parallel program. Dividing by zero would also produce `f64::INFINITY`, so the check documents the case rather than preventing an error.

The assertions compare floating-point results with a tolerance, `(a - b).abs() < 1e-9`, rather than with `==`.

```

/// Deadlock detection: a deadlock exists exactly when the wait-for graph has a
/// cycle. Each edge `(waiter, holder)` means the first task is blocked on the
/// second.
///
/// Run with: cargo run --bin concurrency_deadlock

/// Iterative-friendly depth-first search over a small adjacency list.
fn has_deadlock(waits_for: &[(usize, usize)]) -> bool {
    let Some(highest) = waits_for.iter().flat_map(|(a, b)| [*a, *b]).max() else {
        return false;
    };

    let mut edges = vec![Vec::new(); highest + 1];
    for (waiter, holder) in waits_for {
        edges[*waiter].push(*holder);
    }

    // 1 = on the current path, 2 = fully explored.
    fn visit(node: usize, edges: &[Vec<usize>], state: &mut [u8]) -> bool {
        match state[node] {
            1 => return true,
            2 => return false,
            _ => state[node] = 1,
        }
        for &next in &edges[node] {
            if visit(next, edges, state) {
                return true;
            }
        }
    }
}

```

```

        state[node] = 2;
        false
    }

    let mut state = vec![0u8; edges.len()];
    (0..edges.len()).any(|node| visit(node, &edges, &mut state))
}

fn main() {
    let acyclic = [(0, 1), (1, 2), (0, 2)];
    let cycle = [(0, 1), (1, 2), (2, 0)];
    let self_wait = [(0, 0)];

    for (label, graph) in [
        ("0->1, 1->2, 0->2", &acyclic[..]),
        ("0->1, 1->2, 2->0", &cycle[..]),
        ("0->0", &self_wait[..]),
    ] {
        println!("{label:<20} deadlock: {}", has_deadlock(graph));
    }

    // The four Coffman conditions must all hold for a deadlock; removing any one
    // of them prevents it.
    println!("\nCoffman conditions:");
    for condition in [
        "mutual exclusion",
        "hold and wait",
        "no preemption",
        "circular wait",
    ] {
        println!("  - {condition}");
    }

    assert!(!has_deadlock(&acyclic));
    assert!(has_deadlock(&cycle));
    assert!(has_deadlock(&self_wait));
    assert!(!has_deadlock(&[]));

    println!("\nall checks passed");
}

```

`has_deadlock` finds the largest task number with `flat_map` and `max`. The `let ... else` returns `false` for an empty edge list, where `max` yields `None`.

The adjacency list is `vec![Vec::new(); highest + 1]`. `Vec::new()` is `Clone`, so the `vec!` macro can repeat it.

`visit` is a nested function, so it receives the graph and the colour array as parameters. The `match state[node]` returns early for colours 1 and 2 and marks the node 1 in the catch-all arm.

`(0..edges.len()).any(|node| visit(node, &edges, &mut state))` starts a search from every vertex, so a cycle in a part of the graph unreachable from vertex 0 is still found. `any` stops at the first `true`.

Intuition

The Amdahl program prints:

```

serial |      1      2      4      8     16    <- processors
-----+-----
 0.00 |   1.00   2.00   4.00   8.00  16.00
 0.05 |   1.00   1.90   3.48   5.93   9.14
 0.10 |   1.00   1.82   3.08   4.71   6.40
 0.25 |   1.00   1.60   2.29   2.91   3.37
 0.50 |   1.00   1.33   1.60   1.78   1.88

limit as processors grow: 1 / serial
0.00 ->    inf
0.05 ->   20.0
0.10 ->   10.0
0.25 ->    4.0
0.50 ->    2.0

all checks passed

```

With 5 percent serial work, sixteen processors deliver a speedup of 9.14, and no number of processors can deliver more than 20.

`has_deadlock(&[(0, 1), (1, 2), (2, 0)])`

call	state before	action	returns
visit(0)	[0, 0, 0]	mark 0 as 1, follow edge to 1	
visit(1)	[1, 0, 0]	mark 1 as 1, follow edge to 2	
visit(2)	[1, 1, 0]	mark 2 as 1, follow edge to 0	
visit(0)	[1, 1, 1]	state[0] == 1: on the path	true

The `true` propagates up through each caller's `if visit(...) { return true; }`, and `any` stops. For the acyclic graph `[(0, 1), (1, 2), (0, 2)]`, the search from 0 marks 2 as fully explored before it follows the edge `0 -> 2`, finds colour 2, and continues, so every vertex ends at 2 and the result is `false`. The program prints:

```

0->1, 1->2, 0->2    deadlock: false
0->1, 1->2, 2->0    deadlock: true
0->0                deadlock: true

Coffman conditions:
- mutual exclusion
- hold and wait
- no preemption
- circular wait

all checks passed

```

Time and Space Complexity

Operation	Time	Space
speedup	$O(1)$	none
has_deadlock	$O(V + E)$, each vertex is explored once and each edge followed once	$O(V + E)$ for the adjacency list and colours, plus $O(V)$ stack frames

V is the largest task number plus one, not the number of distinct tasks.

Limitations

Amdahl's law assumes a fixed workload. It answers how much faster the same job becomes. Gustafson's law answers a different question, how much more work fits in the same time, and it is the more relevant model when larger machines are used for larger problems. Amdahl's law also ignores coordination costs, such as lock contention and the false sharing of chapter 48, which can make a program slower as processors are added.

The search recurses. Its documentation comment calls it "iterative-friendly", but `visit` calls itself once per edge on the current path. A wait-for graph shaped like a chain of a few hundred thousand tasks would overflow the stack. An explicit stack of `(node, next_edge_index)` pairs removes the limit.

Task numbers must be small. The adjacency list has `highest + 1` entries, so edges `(0, 4_000_000_000)` allocate billions of empty vectors. A `HashMap<usize, Vec<usize>>` handles sparse identifiers.

Detection is not prevention. The program answers whether a snapshot of the graph has a cycle. Real systems usually prevent deadlock by removing one Coffman condition, most often circular wait by acquiring locks in a fixed global order, or they recover with timeouts.

The Coffman conditions are printed, not used. The list in `main` is documentation for the reader of the output. The detection algorithm tests only circular wait, the one condition visible in a wait-for graph.

Summary

- Amdahl's law, $1 / (s + (1 - s) / p)$, bounds the speedup of a fixed workload by $1 / s$; a program that is 5 percent serial cannot run more than 20 times faster.
- A deadlock among tasks waiting on each other is a cycle in the wait-for graph.
- Depth-first search with three colours distinguishes a back edge to the current path, which is a cycle, from an edge to an already explored vertex, which is not.
- Starting the search from every vertex finds cycles in disconnected parts of the graph.
- Deadlock requires all four Coffman conditions; practical systems prevent one of them, usually circular wait through a lock ordering.

References

- Gene M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities", *AFIPS Conference Proceedings* 30, 1967, pages 483–485.
- John L. Gustafson, "Reevaluating Amdahl's law", *Communications of the ACM* 31(5), 1988, pages 532–533.
- E. G. Coffman, M. Elphick, and A. Shoshani, "System deadlocks", *ACM Computing Surveys* 3(2), 1971, pages 67–78.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 20.3, "Depth-first search".
- Standard library, `std::fmt`, on width, fill, and precision.

Paging and Round-Robin Scheduling

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/os_paging.rs` and `src/bin/os_scheduler.rs`. Run them with `cargo run --bin os_paging` and `cargo run --bin os_scheduler`.

Problem Statement

1. With 4,096-byte pages, compute the page number and offset of an address, the number of pages needed to hold a given number of bytes, and the kind of page fault an access causes given whether the address is mapped, whether the page is in memory, and whether it is writable.
2. Given a list of runnable tasks and a 5 ms quantum, list the task that runs in each time slice under round-robin scheduling, and state the longest a task can wait for its next slice.

Designing a Solution

Addresses. A page size of 4,096 is 2^{12} , so the low 12 bits of an address are the offset within its page and the remaining bits are the page number. Dividing by the page size gives the page number and the remainder gives the offset; with a power of two the compiler emits a shift and a mask.

```
address 0x1234 = 0001 0010 0011 0100 in binary
                | page | offset |
page    = 0x1234 >> 12    = 0x1
offset  = 0x1234 & 0xfff   = 0x234
```

Page counts. A buffer of `b` bytes needs `ceil(b / 4096)` pages. `u64::div_ceil` computes that without the `(b + 4095) / 4096` idiom, which overflows for `b` near `u64::MAX`.

Fault classes. When the processor cannot complete an access through the page tables, it raises a fault and the kernel decides what happened. The program's model has four outcomes:

- **invalid:** the address is not part of any mapping; the process receives `SIGSEGV`;
- **major:** the mapping is valid but the page must be read from disk or swap;
- **protection:** the page is present, but the access is not permitted, such as a write to a read-only page;
- **minor:** the page is present and the access is permitted, and the kernel only has to update the page tables.

Round robin. Tasks form a circular queue. Each runs for at most one quantum and then moves to the back. With `n` tasks and quantum `q`, a task that has just run waits at most `(n - 1) × q` before it runs again, and slice `t` belongs to task `t mod n`.

Implementation

```
/// Virtual memory arithmetic: address to page and offset, page counts, and how a
/// page fault is classified.
///
/// Run with: cargo run --bin os_paging

const PAGE_SIZE: u64 = 4096;

fn split_page(address: u64) -> (u64, u64) {
    (address / PAGE_SIZE, address % PAGE_SIZE)
}

fn pages_needed(bytes: u64) -> u64 {
    bytes.div_ceil(PAGE_SIZE)
}

#[derive(Debug, PartialEq, Eq)]
enum Fault {
    /// The page is resident; a copy-on-write write or permission upgrade.
    Minor,
    /// The mapping is valid but the page must be read from backing store.
    Major,
    /// The mapping is valid and resident, but the access permission is wrong.
    Protection,
    /// No valid mapping at this address.
    Invalid,
}

fn classify_fault(mapped: bool, present: bool, writable: bool) -> Fault {
```

```

match (mapped, present, writable) {
  (false, _, _) => Fault::Invalid,
  (true, false, _) => Fault::Major,
  (true, true, false) => Fault::Protection,
  (true, true, true) => Fault::Minor,
}

fn main() {
  println!("page size: {PAGE_SIZE} bytes");

  println!("\naddress -> page, offset");
  for address in [0u64, 0x1234, 0x1fff, 0x2000] {
    let (page, offset) = split_page(address);
    println!("  0x{address:04x} -> page {page:>2}, offset 0x{offset:03x}");
  }

  println!("\npages needed");
  for bytes in [0u64, 1, PAGE_SIZE, PAGE_SIZE + 1] {
    println!("  {bytes:>5} bytes -> {}", pages_needed(bytes));
  }

  println!("\nfault classification");
  println!(
    "  unmapped          -> {:?}",
    classify_fault(false, false, false)
  );
  println!(
    "  mapped, not in ram -> {:?}",
    classify_fault(true, false, true)
  );
  println!(
    "  resident, read-only-> {:?}",
    classify_fault(true, true, false)
  );
  println!(
    "  resident, writable -> {:?}",
    classify_fault(true, true, true)
  );

  assert_eq!(split_page(0x1234), (1, 0x234));
  assert_eq!(pages_needed(0), 0);
  assert_eq!(pages_needed(PAGE_SIZE + 1), 2);
  assert_eq!(classify_fault(true, false, true), Fault::Major);
  assert_eq!(classify_fault(true, true, false), Fault::Protection);
  assert_eq!(classify_fault(false, true, true), Fault::Invalid);

  println!("\nall checks passed");
}

```

`classify_fault` matches on a tuple `(mapped, present, writable)`. The patterns are ordered from the most general failure to the most specific: `(false, _, _)` covers every unmapped access in one arm, and `(true, false, _)` every non-resident page. The four arms are exhaustive over the eight combinations of three booleans, and the compiler verifies that no combination is missing.

`Fault` derives `PartialEq` and `Eq` so that `assert_eq!` can compare variants, and `Debug` so that `{:?}",` can print them.


```

/// Round-robin scheduling: each runnable task gets one fixed time slice in turn,
/// so no task starves. This is the simplest fair policy a scheduler can use.
///
/// Run with: cargo run --bin os_scheduler

const QUANTUM_MS: u64 = 5;

/// The task chosen on each of `ticks` time slices.
fn schedule<'a>(tasks: &[&'a str], ticks: usize) -> Vec<&'a str> {
    if tasks.is_empty() {
        return Vec::new();
    }
    (0..ticks).map(|tick| tasks[tick % tasks.len()]).collect()
}

fn main() {
    let tasks = ["encoder", "decoder", "gc"];

    println!("{}", runnable tasks, {QUANTUM_MS} ms quantum", tasks.len());
    for (tick, task) in schedule(&tasks, 7).iter().enumerate() {
        let start = tick as u64 * QUANTUM_MS;
        println!(
            "  slice {tick}: {start:>2}-{:>2} ms  {task}",
            start + QUANTUM_MS
        );
    }

    let order = schedule(&tasks, 7);
    assert_eq!(
        order,
        [
            "encoder", "decoder", "gc", "encoder", "decoder", "gc", "encoder"
        ]
    );
    assert!(schedule(&[], 3).is_empty());

    // A long task is preempted, not run to completion, which is what bounds its
// worst-case latency to one quantum times the number of tasks.
    println!(
        "\nworst-case wait for one slice: {} ms",
        QUANTUM_MS * tasks.len() as u64
    );
    println!("all checks passed");
}

```

`schedule` maps each tick to `tasks[tick % tasks.len()]` and collects the results. The early return for an empty task list prevents a division by zero in `%`.

The signature `fn schedule<'a>(tasks: &[&'a str], ticks: usize) -> Vec<&'a str>` ties the returned string slices to the strings themselves, not to the slice that holds them. A caller can therefore drop or rebuild the task list while keeping the schedule, as long as the strings outlive it. Here they are string literals, which live for the whole program.

Intuition

The paging program prints:

```

page size: 4096 bytes

address -> page, offset
0x0000 -> page 0, offset 0x000
0x1234 -> page 1, offset 0x234
0x1fff -> page 1, offset 0xfff
0x2000 -> page 2, offset 0x000

pages needed
0 bytes -> 0
1 bytes -> 1
4096 bytes -> 1
4097 bytes -> 2

fault classification
unmapped -> Invalid
mapped, not in ram -> Major
resident, read-only -> Protection
resident, writable -> Minor

all checks passed

```

`0x1fff` and `0x2000` differ by one byte and lie on different pages, which is why a buffer that crosses a page boundary can fault halfway through a single read.

The scheduler prints:

```

3 runnable tasks, 5 ms quantum
slice 0: 0- 5 ms  encoder
slice 1: 5-10 ms decoder
slice 2: 10-15 ms  gc
slice 3: 15-20 ms encoder
slice 4: 20-25 ms decoder
slice 5: 25-30 ms  gc
slice 6: 30-35 ms encoder

worst-case wait for one slice: 15 ms
all checks passed

```

Time and Space Complexity

Operation	Time	Space
split_page, pages_needed, classify_fault	$O(1)$	none
schedule(tasks, ticks)	$O(\text{ticks})$	one &str per tick

Limitations

The fault model is a teaching simplification. In Linux, a minor fault is one the kernel resolves without I/O, typically because the page is already in the page cache or is a fresh zero page, and it can occur on a read as well as

a write. A write to a read-only page is a protection fault only when the mapping itself is read-only; when the mapping is writable and the page is shared copy-on-write, the same hardware fault is resolved as a minor fault by copying the page. Three booleans cannot represent that distinction.

The worst-case figure is off by one quantum. The program prints `QUANTUM_MS × tasks.len()`, 15 ms, as the wait for one slice. A task that has just finished its slice waits for the other two tasks, 10 ms, before it runs again. The 15 ms figure is the length of one full round, which is the bound on the time from any moment until a given task finishes its next slice, provided every task uses its whole quantum.

Real schedulers are not round robin. Linux's CFS and EEVDF schedulers weight tasks by priority, track the time each has received, and let tasks that block early give up the rest of their quantum. Round robin is the base case those designs refine.

The page size is a constant. 4,096 bytes is the default on x86-64 and on most ARM Linux systems, but AArch64 kernels can be configured for 16 KiB or 64 KiB pages, and huge pages of 2 MiB and 1 GiB coexist with small ones. Portable code asks the system, for example with `sysconf(_SC_PAGESIZE)`.

schedule allocates the whole schedule. A scheduler would compute the next task on demand. An iterator such as `tasks.iter().cycle().take(ticks)` expresses the same sequence lazily.

Summary

- With a page size of 2^{12} , the page number is the address shifted right by 12 and the offset is the low 12 bits.
- `div_ceil` computes the number of whole pages for a byte count without the overflow of the add-then-divide idiom.
- Matching on a tuple of booleans states a decision table whose completeness the compiler checks.
- Round robin gives each of `n` tasks one quantum `q` in turn, so a task waits at most $(n - 1) \times q$ between slices.
- Both programs model the ideas and omit what real kernels add: copy-on-write, page cache, multiple page sizes, and weighted scheduling.

References

- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, 2018, chapters 2 ("Scheduling: Introduction"), 18 ("Paging: Introduction"), and 21 ("Swapping: Mechanisms").
- The Linux kernel documentation, [Memory management concepts](#).
- Standard library, `u64::div_ceil`.

Part XV: Networking and HTTP

IPv4 Addresses and TCP Windows

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/net_ipv4.rs` and `src/bin/net_window.rs`. Run them with `cargo run --bin net_ipv4` and `cargo run --bin net_window`.

Problem Statement

1. Parse text such as `192.168.1.10` into four octets, rejecting anything that is not exactly four values in `0..=255`. Report whether the address is in one of the private ranges, and convert it to and from the 32-bit integer carried in an IP header.
2. Given a receiver window and a congestion window, compute how much unacknowledged data a TCP sender may have in flight. Given a link's bandwidth and round-trip time, compute how many bytes must be in flight to keep the link busy.

Designing a Solution

Parsing. Split the text on `.`, parse each part as `u8`, and require exactly four parts. Parsing as `u8` rejects values above 255 without a separate range check.

Private ranges. RFC 1918 reserves three blocks for private networks:

<code>10.0.0.0/8</code>	first octet 10
<code>172.16.0.0/12</code>	first octet 172, second octet 16 through 31
<code>192.168.0.0/16</code>	first octets 192, 168

Each block is a condition on a prefix of the octets, which a slice pattern expresses directly.

Byte order. Protocol headers carry multi-byte integers most significant byte first, known as big-endian or network byte order. The address `127.0.0.1` is the integer `0x7f000001`, stored in a header as the bytes `7f 00 00 01` whatever the host's own byte order.

Windows. A TCP sender limits unacknowledged data to the smaller of two windows: the receiver's advertised window, which protects the receiver's buffer, and the congestion window, which the sender maintains to avoid overloading the network.

Bandwidth-delay product. A sender can release new data only as acknowledgements return, and they return one round-trip time after the data left. To keep a link of bandwidth `B` bytes per second busy, the sender must have `B × RTT` bytes in flight at all times. If the effective window is smaller than that product, the link sits partly idle regardless of its bandwidth.

Implementation

```

/// IPv4 addresses: parse a dotted quad, check the private ranges, and show the
/// network byte order used on the wire.
///
/// Run with: cargo run --bin net_ipv4

/// Parse `a.b.c.d`, rejecting anything that is not exactly four octets.
fn parse(input: &str) -> Option<[u8; 4]> {
    let mut parts = input.split('.').map(|part| part.parse::<u8>().ok());
    let address = [
        parts.next()?,
        parts.next()?,
        parts.next()?,
        parts.next()?,
    ];
    parts.next().is_none().then_some(address)
}

/// `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16` are not routable on the
/// public internet.
fn is_private(address: [u8; 4]) -> bool {
    matches!(address, [10, ..] | [172, 16..=31, ..] | [192, 168, ..])
}

/// Addresses travel big-endian, so the first octet is the most significant byte.
fn to_network_order(address: [u8; 4]) -> u32 {
    u32::from_be_bytes(address)
}

fn from_network_order(value: u32) -> [u8; 4] {
    value.to_be_bytes()
}

fn main() {
    println!(
        "{:<16} {:<18} {:>8} {:>12}",
        "input", "parsed", "private", "as u32"
    );
}

```

```

for text in [
    "10.0.0.1",
    "172.16.5.4",
    "192.168.1.10",
    "8.8.8.8",
    "256.1.1.1",
    "1.2.3",
] {
    match parse(text) {
        Some(address) => println!(
            "{text:<16} {:<18} {:>8} {:>12}",
            format!("{address:?}"),
            is_private(address),
            to_network_order(address)
        ),
        None => println!("{text:<16} invalid"),
    }
}

assert_eq!(parse("10.0.0.1"), Some([10, 0, 0, 1]));
assert_eq!(parse("1.2.3"), None);
assert_eq!(parse("1.2.3.4.5"), None);
assert_eq!(parse("256.0.0.1"), None);

assert!(is_private([10, 1, 2, 3]));
assert!(is_private([172, 31, 0, 1]));
assert!(!is_private([172, 32, 0, 1]));
assert!(!is_private([8, 8, 8, 8]));

assert_eq!(to_network_order([127, 0, 0, 1]), 0x7f00_0001);
assert_eq!(from_network_order(0x7f00_0001), [127, 0, 0, 1]);

println!("\nall checks passed");
}

```

`parse` maps each part to `part.parse::<u8>().ok()`, so the iterator yields `Option<Option<u8>>`: the outer `Option` says whether a part exists, and the inner one whether it parsed. `parts.next()??` applies `?` twice, returning `None` from the function if either layer is `None`. Four such expressions build the array, and `parts.next().is_none().then_some(address)` rejects a fifth part.

`is_private` uses slice patterns on the array. `[10, ..]` matches any address whose first octet is 10. `[172, 16..=31, ..]` matches a first octet of 172 and a second in the inclusive range, which is exactly the /12 block. The `|` combines the three patterns in one `matches!`.

`u32::from_be_bytes(address)` interprets the array as a big-endian integer, and `value.to_be_bytes()` performs the inverse. Both are `const fn` and compile to a byte swap on little-endian processors and to nothing on big-endian ones.

```

/// TCP window math: the effective window is the smaller of the receiver and
/// congestion windows, and the bandwidth-delay product is how many bytes must be
/// in flight to fill a link.
///
/// Run with: cargo run --bin net_window

```



```

/// The sender may have this much unacknowledged data in flight.
fn effective_window(receive: u32, congestion: u32) -> u32 {
    receive.min(congestion)
}

/// Bytes in flight needed to keep a link busy: bandwidth times round-trip time.
fn bandwidth_delay_product(bytes_per_second: u64, rtt_millis: u64) -> u64 {
    bytes_per_second * rtt_millis / 1_000
}

fn main() {
    println!("effective window = min(receiver window, congestion window)");
    for (receive, congestion) in [(64_000, 16_000), (16_000, 64_000), (32_000,
32_000)] {
        println!(
            " receiver {receive:>7} B, congestion {congestion:>7} B -> {:>7} B",
            effective_window(receive, congestion)
        );
    }

    println!("\nbandwidth-delay product (bytes needed in flight)");
    for (rate, rtt) in [(1_000_000, 1), (10_000_000, 50), (1_000_000_000, 100)] {
        println!(
            " {:>12} B/s x {rtt:>3} ms -> {:>12} B",
            rate,
            bandwidth_delay_product(rate, rtt)
        );
    }

    // A window smaller than the product leaves the link idle, no matter how much
    // bandwidth is available. This is why latency, not only bandwidth, matters.
    let product = bandwidth_delay_product(1_000_000_000, 100);
    let window = u64::from(effective_window(64_000, 1_000_000));
    println!(
        "\nwindow {window} B vs product {product} B: link is {} the window too small",
        if window < product {
            "starved,"
        } else {
            "usable,"
        }
    );

    assert_eq!(effective_window(64_000, 16_000), 16_000);
    assert_eq!(effective_window(16_000, 64_000), 16_000);
    assert_eq!(bandwidth_delay_product(10_000_000, 50), 500_000);
    assert!(window < product);

    println!("\nall checks passed");
}

```

`effective_window` is `receive.min(congestion)`. `bandwidth_delay_product` multiplies before dividing, `bytes_per_second * rtt_millis / 1_000`, so that integer division does not discard the fractional part of the round-trip time in seconds.

`u64::from(effective_window(...))` widens the `u32` window before comparing it with the `u64` product. `From` is used rather than `as` because it cannot truncate.

Intuition

The IPv4 program prints:

```

input      parsed      private      as u32
10.0.0.1   [10, 0, 0, 1]      true        167772161
172.16.5.4 [172, 16, 5, 4] true        2886731012
192.168.1.10 [192, 168, 1, 10] true        3232235786
8.8.8.8     [8, 8, 8, 8]        false       134744072
256.1.1.1   invalid
1.2.3       invalid

```

all checks passed

parse("1.2.3")

expression	value
parts.next() for the first three octets	Some(Some(1)), Some(Some(2)), Some(Some(3))
fourth parts.next()	None
first ? on None	the function returns None

The window program prints:

```

effective window = min(receiver window, congestion window)
receiver  64000 B, congestion  16000 B ->  16000 B
receiver  16000 B, congestion  64000 B ->  16000 B
receiver  32000 B, congestion  32000 B ->  32000 B

bandwidth-delay product (bytes needed in flight)
  1000000 B/s x  1 ms ->  1000 B
  10000000 B/s x 50 ms -> 500000 B
 100000000 B/s x 100 ms -> 10000000 B

window 64000 B vs product 10000000 B: link is starved, the window too small

all checks passed

```

A link of one gigabyte per second with a 100 ms round trip needs 100 MB in flight. A 64 KB window fills 0.064 percent of it. The classic 16-bit TCP window field cannot express more than 65,535 bytes; RFC 7323 added a window scale option to go beyond it.

Time and Space Complexity

Function	Time	Space
parse	$O(\text{len})$	none
is_private, to_network_order, from_network_order	$O(1)$	none
effective_window, bandwidth_delay_product	$O(1)$	none

Limitations

The parser accepts forms that address parsers reject.

`str::parse::<u8>` accepts a leading `+` and leading zeros, so `+1.2.3.4` and `010.0.0.1` parse. The second is especially misleading, because the C function `inet_aton` reads `010` as octal 8. The standard library's `"10.0.0.1".parse::<std::net::Ipv4Addr>()` rejects both forms.

The standard library already provides these functions.

`Ipv4Addr` implements `FromStr`, `Ipv4Addr::is_private` checks the same three blocks, and `u32::from(Ipv4Addr)` and `Ipv4Addr::from(u32)` perform the byte-order conversion. Writing the functions by hand is useful in an interview; production code should use the type.

bandwidth_delay_product can overflow. `bytes_per_second * rtt_millis` is a `u64` product. Values large enough to overflow are physically unrealistic, but the function would panic in a debug build and wrap in a release build. `checked_mul` or `u128` arithmetic would make the limit explicit.

The window model is static. Real congestion windows change on every acknowledgement and every loss, and the receiver window changes as the application reads. The program compares two numbers at one instant.

The final message's wording. The format string produces `link is starved, the window too small`, which reads as a sentence fragment; it would be clearer as two clauses, such as `the window is too small, so the link is starved`.

Summary

- `part.parse::<u8>().ok()` inside an iterator yields `Option<Option<u8>>`, and `??` unwraps both layers or returns `None`.
- Slice patterns with ranges, such as `[172, 16..=31, ..]`, express address blocks directly and are checked by the compiler.
- `u32::from_be_bytes` and `to_be_bytes` convert between octets and network byte order.
- A TCP sender's effective window is the minimum of the receiver and congestion windows.

- A link is fully used only when the window is at least the bandwidth-delay product, which for fast, long links is far larger than 64 KB.
- `std::net::Ipv4Addr` provides stricter parsing and the same classification out of the box.

References

- RFC 1918, *Address Allocation for Private Internets*, 1996.
- RFC 7323, *TCP Extensions for High Performance*, 2014, on window scaling.
- W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff, *UNIX Network Programming, Volume 1*, 3rd edition, Addison-Wesley, 2003, Chapter 35, on byte-ordering functions.
- Standard library, `std::net::Ipv4Addr` and `u32::from_be_bytes`.

A Thread-per-Connection Echo Server

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/tcp_echo_server.rs`. Run it with `cargo run --bin tcp_echo_server`, then connect with `nc 127.0.0.1 <port>`.

Problem Statement

Listen on a TCP address. For every client that connects, send back every byte the client sends, until the client closes its side of the connection. Several clients must be served at the same time, and an error on one connection must not stop the server.

Designing a Solution

The server's lifecycle has a fixed order:

<code>bind</code>	reserve a local address and port
<code>listen</code>	ask the kernel to queue incoming connections (<code>TcpListener::bind</code> does both)
<code>accept</code>	take one queued connection, which yields a new <code>TcpStream</code>
<code>read</code>	block until the client sends bytes, or returns 0 when the client closes
<code>write</code>	send the bytes back
<code>close</code>	drop the <code>TcpStream</code>

`accept` returns a separate socket for each client, while the listening socket keeps accepting. Handling each accepted socket on a new thread lets a slow client block only its own thread. Reads and writes use ordinary blocking calls, and the code reads as a simple loop.

TCP delivers a byte stream, not messages. One `write` from the client can arrive as several `reads` at the server, and several writes can arrive as one read. An

echo server does not care about message boundaries, which is what makes it a clean first exercise.

A `read` that returns `Ok(0)` means the peer has shut down its sending direction, which happens when it closes the socket. It is not an error, and the server should finish the connection normally.

Implementation

```
/// TCP echo server: the canonical socket exercise.
///
/// Demonstrates the server side of the TCP lifecycle: `bind` -> `listen` ->
/// `accept` -> read/write loop -> peer `close` (a `read` of 0). It uses a
/// thread-per-connection model, which is the right default for bounded concurrency
/// and blocking handlers; the event-loop alternative is `src/bin/epoll_echo.rs`.
///
/// `TCP_NODELAY` is set because an echo server must not sit on Nagle's algorithm
/// waiting to coalesce a reply that will never be followed by more data.
///
/// ```bash
/// cargo run --bin tcp_echo_server          # binds 127.0.0.1:0, prints the port
/// cargo run --bin tcp_echo_server 0.0.0.0:9000
/// ```

use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

/// Accept connections forever, handling each on its own thread.
pub fn serve(listener: TcpListener) {
    for incoming in listener.incoming() {
        match incoming {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(error) = handle_connection(stream) {
                        eprintln!("connection error: {error}");
                    }
                });
            }
            Err(error) => eprintln!("accept error: {error}"),
        }
    }
}

/// Echo every byte back until the peer closes the connection.
fn handle_connection(mut stream: TcpStream) -> std::io::Result<()> {
    stream.set_nodelay(true)?;
    if let Ok(peer) = stream.peer_addr() {
        println!("connection from {peer}");
    }

    let mut buffer = [0u8; 4096];
    loop {
        let read = stream.read(&mut buffer)?;
        if read == 0 {
            // A zero-byte read is the peer's FIN: an orderly close.
            return Ok(());
        }
        stream.write_all(&buffer[..read])?;
    }
}
```

```

fn main() {
    let address = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "127.0.0.1:0".to_string());

    let listener = TcpListener::bind(&address).expect("failed to bind");
    let local = listener.local_addr().expect("failed to read local address");
    println!("tcp echo server listening on {local}");

    serve(listener);
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::net::Shutdown;

    #[test]
    fn echoes_bytes_until_the_client_closes() {
        let listener = TcpListener::bind("127.0.0.1:0").expect("bind ephemeral port");
        let address = listener.local_addr().expect("local address");

        // Accept exactly one connection, echo it, then return so the test can join.
        let server = thread::spawn(move || {
            let (stream, _) = listener.accept().expect("accept one connection");
            handle_connection(stream).expect("echo");
        });

        let mut client = TcpStream::connect(address).expect("connect");
        client.write_all(b"hello gpu").expect("write");
        client.shutdown(Shutdown::Write).expect("half close");

        let mut echoed = Vec::new();
        client.read_to_end(&mut echoed).expect("read echo");

        server.join().expect("server thread panicked");
        assert_eq!(echoed, b"hello gpu");
    }
}

```

`serve` iterates over `listener.incoming()`, which calls `accept` in a loop and yields `io::Result<TcpStream>`. An `accept` error, such as running out of file descriptors, is printed and the loop continues. The listener is not closed on a transient error.

`thread::spawn(move || ...)` moves the `TcpStream` into the new thread. The closure reports a connection error to standard error and returns, and the thread ends. The `JoinHandle` is dropped, which detaches the thread.

`handle_connection` returns `std::io::Result<>`, so every fallible call inside it uses `?.` `stream.set_nodelay(true)?` disables Nagle's algorithm. Nagle's algorithm delays a small write while earlier data is unacknowledged, hoping to combine it with the next write. An echo server writes one reply per read, so the delay would add latency without saving packets.

`write_all(&buffer[..read])` writes exactly the bytes received. `write` alone may send fewer bytes than requested, and `write_all` repeats it until every byte is sent or an error occurs.

`main` binds to `127.0.0.1:0` by default. Port 0 asks the kernel to choose a free port, and `listener.local_addr()` reports which one it chose.

The test binds its own listener on port 0, accepts exactly one connection on a helper thread, and calls `handle_connection` directly, so it needs neither a fixed port nor a separate process. The client calls `shutdown(Shutdown::Write)` after writing. That sends a FIN, which makes the server's next `read` return 0, and the client's `read_to_end` then receives the echoed bytes followed by the server's own close.

Intuition

The test `echoes_bytes_until_the_client_closes`

step	client	server thread
1		<code>TcpListener::bind("127.0.0.1:0")</code> , <code>accept</code> blocks
2	<code>TcpStream::connect(address)</code>	<code>accept</code> returns a stream
3	<code>write_all(b"hello gpu")</code>	<code>read</code> returns 9
4	<code>shutdown(Write)</code> sends FIN	<code>write_all</code> sends 9 bytes back
5	<code>read_to_end</code> receives 9 bytes	<code>read</code> returns 0, <code>handle_connection</code> returns <code>Ok(())</code>
6	<code>read_to_end</code> sees the server's close and returns	the stream is dropped, closing the socket
7	<code>server.join()</code> succeeds; <code>echoed == b"hello gpu"</code>	

Time and Space Complexity

Resource	Cost
Per connection	one thread, with its own stack (2 MiB by default for spawned threads) and one 4 KiB buffer
Per byte	

Resource	Cost
	one copy into the buffer and one copy out, plus system calls for read and write
Concurrency	limited by the number of threads the process and the kernel allow

Threads are cheap to write and expensive to have many of. A server with ten thousand idle connections holds ten thousand threads, most of them blocked in read.

Limitations

The number of threads is unbounded. Every connection spawns a thread with no limit, so a burst of connections can exhaust memory or the process's thread limit. A worker pool, or a semaphore around `accept` as in chapter 45, bounds the work.

No timeouts. A client that connects and never sends anything holds a thread forever. `set_read_timeout` and `set_write_timeout` on the stream give each connection an idle limit.

A slow reader stalls its own thread. `write_all` blocks when the client does not read and the socket's send buffer fills. With one thread per connection that affects only that client, which is the main advantage of this design over a naive single-threaded loop.

Detached threads are not shut down. `serve` never returns, and the spawned threads are not tracked. A server that must stop cleanly needs a shutdown signal and a way to join or cancel connection threads.

`serve` is public in a binary crate. The `pub` keyword has no effect in a binary, which has no downstream users. It is harmless and suggests the function was meant to live in the library.

Summary

- `TcpListener::bind` performs `bind` and `listen`, and `incoming` yields one `TcpStream` per accepted connection.

- One thread per connection keeps each handler a simple blocking loop and isolates slow clients from each other.
- A `read` of zero bytes is the peer's orderly close, not an error.
- `write_all` handles partial writes, and `set_nodelay(true)` removes Nagle's delay for request-reply traffic.
- Binding to port 0 and calling the handler directly lets a server be tested in-process with no fixed port.

References

- W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff, *UNIX Network Programming, Volume 1*, 3rd edition, Addison-Wesley, 2003, Chapters 19 and 36.
- RFC 896, *Congestion Control in IP/TCP Internetworks*, 1984, which describes Nagle's algorithm.
- Standard library, `std::net::TcpListener` and `TcpStream::set_nodelay`.
- Standard library, `Write::write_all`.

An epoll Echo Server

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/epoll_echo.rs`. It runs on Linux only. Run it with `cargo run --bin epoll_echo`, optionally followed by a port number, and connect with `nc 127.0.0.1 9000`.

Problem Statement

Serve any number of echo clients from a single thread. No call may block on one client while others are waiting. A client that sends faster than the server can write back to it must not stall the rest.

Designing a Solution

Non-blocking sockets. A socket opened with `SOCK_NONBLOCK` never waits. A read with no data available fails with `EAGAIN` instead of sleeping, and so does a write when the send buffer is full.

Readiness notification. An `epoll` instance holds a set of file descriptors and the events of interest for each. `epoll_wait` sleeps until at least one registered descriptor is ready, then reports which ones, and the loop handles only those.

Level-triggered events. Without the `EPOLLET` flag, `epoll` reports a descriptor as readable for as long as unread data remains. A loop that does not drain a socket in one pass is reminded on the next `epoll_wait`, which makes the code harder to get wrong than the edge-triggered mode.

Per-connection output buffers. When a write accepts only part of the reply, the rest is stored in the connection's buffer and the loop registers interest in `EPOLLOUT`. When the socket becomes writable, the loop writes the remainder

and removes the interest, so `epoll` does not wake the loop continuously for a socket that has nothing to send.

The diagram below shows one iteration of the loop.

```

epoll_wait(epfd, events, 64, 100 ms)
|
+-- listening socket readable ---> accept4 until EAGAIN, register each client for
EPOLLIN
|
+-- client readable -----> read until EAGAIN into connection.out, then
flush
|
|                               read returns 0 -> close the connection
|
+-- client writable -----> flush the rest of connection.out
|
+-- after either -----> EPOLLIN | EPOLLOUT if output is pending, else
EPOLLIN

```

Implementation

The file keeps its Linux code in a module compiled only on Linux, with a fall-back `main` for other platforms. The listings follow the file from top to bottom.

Connection state and the listening socket

```

/// epoll echo server: one thread, many non-blocking sockets.
///
/// Reads run until `EAGAIN`, and a reply that does not fit the socket buffer is
/// queued and finished under `EPOLLOUT`, so one slow client cannot stall the loop.
/// Linux only; kqueue and IOCP are the portable equivalents.

#[cfg(target_os = "linux")]
mod linux {
    use std::collections::HashMap;
    use std::io;
    use std::sync::atomic::{AtomicBool, Ordering};

    const MAX_EVENTS: usize = 64;
    const EPOLL_TIMEOUT_MS: i32 = 100;

    /// Per-connection state. `out`/`out_pos` form a write buffer that survives a
    /// partial write.
    #[derive(Default)]
    struct Connection {
        out: Vec<u8>,
        out_pos: usize,
    }

    impl Connection {
        fn has_pending_output(&self) -> bool {
            self.out_pos < self.out.len()
        }
    }

    /// Bind a non-blocking listener on `127.0.0.1:port`; port 0 asks the kernel for
    /// a free port, which is returned alongside the fd.
    pub fn bind_listener(port: u16) -> io::Result<(i32, u16)> {

```

```

// SAFETY: all pointers are to stack values that outlive the call.
unsafe {
    let fd = libc::socket(libc::AF_INET, libc::SOCK_STREAM | libc::SOCK_NONBLOC
K, 0);
    if fd < 0 {
        return Err(io::Error::last_os_error());
    }

    let enable: libc::c_int = 1;
    if libc::setsockopt(
        fd,
        libc::SOL_SOCKET,
        libc::SO_REUSEADDR,
        &enable as *const _ as *const libc::c_void,
        std::mem::size_of::<libc::c_int>() as libc::socklen_t,
    ) < 0
    {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    let mut address: libc::sockaddr_in = std::mem::zeroed();
    address.sin_family = libc::AF_INET as libc::sa_family_t;
    address.sin_port = port.to_be();
    address.sin_addr.s_addr = libc::INADDR_LOOPBACK.to_be();

    if libc::bind(
        fd,
        &address as *const _ as *const libc::sockaddr,
        std::mem::size_of::<libc::sockaddr_in>() as libc::socklen_t,
    ) < 0
    {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    if libc::listen(fd, 128) < 0 {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    let mut bound: libc::sockaddr_in = std::mem::zeroed();
    let mut length = std::mem::size_of::<libc::sockaddr_in>() as libc::socklen_
t;
    if libc::getsockname(fd, &mut bound as *mut _ as *mut libc::sockaddr, &mut
length) < 0 {
        let error = io::Error::last_os_error();
        libc::close(fd);
        return Err(error);
    }

    Ok((fd, u16::from_be(bound.sin_port)))
}
}

```

Connection holds out, the bytes waiting to be written, and out_pos, how many of them have already been written. Advancing out_pos instead of removing written bytes from the front of the vector avoids moving the remaining bytes after every partial write.

`bind_listener` wraps its whole body in one `unsafe` block, with a comment explaining that every pointer passed to the C functions refers to a stack value that outlives the call. Each step checks the return value, closes the socket on failure, and returns `io::Error::last_os_error()`, which reads `errno`. `address.sin_port = port.to_be()` converts the port to network byte order, as chapter 51 described.

Binding to port 0 and reading the result back with `getsockname` gives the test a free port without guessing.

The event loop

```
pub fn event_loop(epfd: i32, listen_fd: i32, shutdown: &AtomicBool) ->
io::Result<()> {
    epoll_add(epfd, listen_fd, libc::EPOLLIN as u32)?;

    let mut connections: HashMap<i32, Connection> = HashMap::new();
    let mut events = vec![libc::epoll_event { events: 0, u64: 0 }; MAX_EVENTS];

    while !shutdown.load(Ordering::Relaxed) {
        // SAFETY: `events` is a valid, writable buffer of MAX_EVENTS entries.
        let ready = unsafe {
            libc::epoll_wait(
                epfd,
                events.as_mut_ptr(),
                events.len() as i32,
                EPOLL_TIMEOUT_MS,
            )
        };

        if ready < 0 {
            let error = io::Error::last_os_error();
            if error.raw_os_error() == Some(libc::EINTR) {
                continue;
            }
            return Err(error);
        }

        for event in &events[..ready as usize] {
            let fd = event.u64 as i32;
            if fd == listen_fd {
                accept_ready(epfd, listen_fd, &mut connections)?;
                continue;
            }

            let readable = event.events & (libc::EPOLLIN as u32) != 0;
            let writable = event.events & (libc::EPOLLOUT as u32) != 0;

            if readable && !read_into(fd, &mut connections)? {
                continue; // connection closed
            }
            if writable && connections.contains_key(&fd) {
                flush(fd, &mut connections)?;
            }
            if connections.contains_key(&fd) {
                update_interest(epfd, fd, &mut connections)?;
            }
        }
    }
}
```

```
    Ok(())
}
```

The loop registers the listener for `EPOLLIN`, allocates an event array of 64 entries once, and waits with a timeout of 100 ms so that it can check the shutdown flag regularly. `EINTR`, a wait interrupted by a signal, is not an error, so the loop retries.

Each event carries the file descriptor in its `u64` field, stored there when the descriptor was registered. The loop dispatches on whether it is the listener, then handles readability and writability for client sockets. `read_into` returns `false` when it has closed the connection, and the `continue` skips the remaining steps for a descriptor that no longer exists.

Accepting and reading

```
/// `SOCK_NONBLOCK` means a client socket can never block the loop.
fn accept_ready(
    epfd: i32,
    listen_fd: i32,
    connections: &mut HashMap<i32, Connection>,
) -> io::Result<()> {
    loop {
        // SAFETY: a null sockaddr is explicitly allowed for accept.
        let client = unsafe {
            libc::accept4(
                listen_fd,
                std::ptr::null_mut(),
                std::ptr::null_mut(),
                libc::SOCK_NONBLOCK,
            )
        };
        if client < 0 {
            if would_block() {
                return Ok(());
            }
            return Err(io::Error::last_os_error());
        }

        epoll_add(epfd, client, libc::EPOLLIN as u32)?;
        connections.insert(client, Connection::default());
    }
}

/// Drain readable bytes into the connection's output buffer.
///
/// Returns `false` when the peer closed the connection.
fn read_into(fd: i32, connections: &mut HashMap<i32, Connection>) -> io::Result<bool> {
    let mut buffer = [0u8; 4096];
    loop {
        // SAFETY: `buffer` is a valid writable region of `buffer.len()` bytes.
        let read = unsafe { libc::read(fd, buffer.as_mut_ptr() as *mut libc::c_void, buffer.len()) };
    }
}
```

```

    if read == 0 {
        close_connection(fd, connections);
        return Ok(false);
    }
    if read < 0 {
        if would_block() {
            break;
        }
        close_connection(fd, connections);
        return Err(io::Error::last_os_error());
    }

    if let Some(connection) = connections.get_mut(&fd) {
        connection.out.extend_from_slice(&buffer[..read as usize]);
    }
}

flush(fd, connections)?;
Ok(true)
}

```

`accept_ready` accepts in a loop until `EAGAIN`, because one readiness notification can stand for several queued connections. `accept4` with `SOCK_NONBLOCK` creates each client socket already non-blocking, which avoids a separate `fcntl` call and the window in which the socket would still be blocking.

`read_into` also loops until `EAGAIN`, appending each chunk to the connection's output buffer, and then calls `flush` to echo as much as the socket will accept immediately.

Writing and updating interest

```

/// Write as much of the buffered reply as the socket accepts.
fn flush(fd: i32, connections: &mut HashMap<i32, Connection>) -> io::Result<()> {
    let Some(connection) = connections.get_mut(&fd) else {
        return Ok(());
    };

    while connection.has_pending_output() {
        let remaining = connection.out.len() - connection.out_pos;
        // SAFETY: the slice is valid for `remaining` bytes.
        let written = unsafe {
            libc::write(
                fd,
                connection.out[connection.out_pos..].as_ptr() as *const libc::c_void,
                remaining,
            )
        };

        if written < 0 {
            if would_block() {
                // Keep the rest buffered; EPOLLOUT will drive the next attempt.
                return Ok(());
            }
            return Err(io::Error::last_os_error());
        }
        connection.out_pos += written as usize;
    }
}

```



```

        connection.out.clear();
        connection.out_pos = 0;
        Ok(())
    }

    /// Ask for `EPOLLOUT` only while a reply is still buffered.
    fn update_interest(
        epfd: i32,
        fd: i32,
        connections: &mut HashMap<i32, Connection>,
    ) -> io::Result<()> {
        let pending = connections
            .get(&fd)
            .is_some_and(Connection::has_pending_output);
        let events = if pending {
            (libc::EPOLLIN | libc::EPOLLOUT) as u32
        } else {
            libc::EPOLLIN as u32
        };
        epoll_modify(epfd, fd, events)
    }

    fn close_connection(fd: i32, connections: &mut HashMap<i32, Connection>) {
        connections.remove(&fd);
        // SAFETY: `fd` is owned by us and open at this point.
        unsafe {
            libc::close(fd);
        }
    }

    fn epoll_add(epfd: i32, fd: i32, events: u32) -> io::Result<()> {
        let mut event = libc::epoll_event {
            events,
            u64: fd as u64,
        };
        // SAFETY: `event` outlives the call.
        if unsafe { libc::epoll_ctl(epfd, libc::EPOLL_CTL_ADD, fd, &mut event) } < 0 {
            Err(io::Error::last_os_error())
        } else {
            Ok(())
        }
    }

    fn epoll_modify(epfd: i32, fd: i32, events: u32) -> io::Result<()> {
        let mut event = libc::epoll_event {
            events,
            u64: fd as u64,
        };
        // SAFETY: `event` outlives the call.
        if unsafe { libc::epoll_ctl(epfd, libc::EPOLL_CTL_MOD, fd, &mut event) } < 0 {
            Err(io::Error::last_os_error())
        } else {
            Ok(())
        }
    }

    fn would_block() -> bool {
        matches!(
            io::Error::last_os_error().raw_os_error(),
            Some(libc::EAGAIN)
        )
    }
}

```

`flush` writes from `out_pos` until the buffer is empty or the socket returns `EAGAIN`. On `EAGAIN` it returns `Ok(())` with the remainder still buffered. When everything has been written it clears the vector, which keeps its capacity for the next reply.

`update_interest` computes the event mask from `has_pending_output` and calls `EPOLL_CTL_MOD`. Asking for `EPOLLOUT` only while output is pending is the difference between a loop that sleeps and a loop that spins: an idle socket is almost always writable, and a level-triggered `EPOLLOUT` on it would wake `epoll_wait` immediately, forever.

`would_block` compares `errno` with `EAGAIN`. On Linux, `EWouldBlock` has the same value.

Startup and the test

```
#[cfg(target_os = "linux")]
fn main() {
    use std::sync::atomic::AtomicBool;

    let port: u16 = std::env::args()
        .nth(1)
        .and_then(|argument| argument.parse().ok())
        .unwrap_or(9000);

    let (listen_fd, bound_port) = linux::bind_listener(port).expect("failed to bind");
    // SAFETY: epoll_create1 takes flags and returns an fd or -1.
    let epfd = unsafe { libc::epoll_create1(0) };
    assert!(epfd >= 0, "epoll_create1 failed");

    // A write to a peer that vanished raises SIGPIPE, which by default kills the
    // process. Servers ignore it and handle EPIPE from write instead.
    // SAFETY: installing SIG_IGN is a single-threaded startup action.
    unsafe {
        libc::signal(libc::SIGPIPE, libc::SIG_IGN);
    }

    println!("epoll echo server listening on 127.0.0.1:{bound_port} (Ctrl-C to stop)");

    let shutdown = AtomicBool::new(false);
    let result = linux::event_loop(epfd, listen_fd, &shutdown);
    // SAFETY: both fds are owned by this process.
    unsafe {
        libc::close(epfd);
        libc::close(listen_fd);
    }

    if let Err(error) = result {
        eprintln!("event loop error: {error}");
        std::process::exit(1);
    }
}

#[cfg(not(target_os = "linux"))]
fn main() {
    eprintln!("epoll_echo requires Linux; use kqueue or IOCP on this platform.");
}
```

```
#[cfg(all(test, target_os = "linux"))]
mod tests {
    use super::linux;
    use std::io::{Read, Write};
    use std::net::TcpStream;
    use std::sync::Arc;
    use std::sync::atomic::{AtomicBool, Ordering};
    use std::thread;

    #[test]
    fn echoes_a_message_through_the_event_loop() {
        let (listen_fd, port) = linux::bind_listener(0).expect("bind");
        // SAFETY: epoll_create1 takes flags and returns an fd or -1.
        let epfd = unsafe { libc::epoll_create1(0) };
        assert!(epfd >= 0, "epoll_create1 failed");

        let shutdown = Arc::new(AtomicBool::new(false));
        let server_shutdown = Arc::clone(&shutdown);
        let server = thread::spawn(move || {
            let result = linux::event_loop(epfd, listen_fd, &server_shutdown);
            // SAFETY: the test owns both fds.
            unsafe {
                libc::close(epfd);
                libc::close(listen_fd);
            }
            result
        });

        let mut client = TcpStream::connect(("127.0.0.1", port)).expect("connect");
        client.write_all(b"epoll echo").expect("write");
        let mut echoed = [0u8; 10];
        client.read_exact(&mut echoed).expect("read echo");
        assert_eq!(&echoed, b"epoll echo");

        shutdown.store(true, Ordering::Relaxed);
        drop(client);
        let result = server.join().expect("server thread panicked");
        assert!(result.is_ok(), "event loop returned {result:?}");
    }
}
```

`main` ignores `SIGPIPE`. By default, writing to a socket whose peer has closed delivers `SIGPIPE`, which terminates the process. With the signal ignored, the write fails with `EPIPE` instead, which the program can handle.

The test runs the event loop on a thread, connects a standard `TcpStream` client, and checks that ten bytes come back. It then sets the shutdown flag, and the loop exits at its next 100 ms timeout. The two descriptors are closed on the server thread, and the loop's result is checked.

Intuition

Events for one client that sends `epoll echo`

epoll_wait result	handler	effect
listener readable	accept_ready	accept4 returns client fd 5, registered for EPOLLIN; second accept4 returns EAGAIN
fd 5 readable	read_into	read returns 10 bytes into out; next read returns EAGAIN; flush writes 10 bytes and clears out
	update_interest	no pending output, so interest stays EPOLLIN
timeout, no events		shutdown flag checked, still false
fd 5 readable	read_into	read returns 0 after the test drops the client; the connection is closed

If the client stopped reading and the socket's send buffer filled, flush would get EAGAIN with bytes left in out, update_interest would add EPOLLOUT, and the next writable event would call flush again.

Time and Space Complexity

Resource	Cost
Per epoll_wait	O(ready descriptors), independent of the number of idle connections
Per connection	one Connection in a HashMap and one kernel socket; no thread
Per event	a read or write system call per chunk, plus one epoll_ctl per event
Memory for output	proportional to bytes received but not yet written

Limitations

One client's error stops the server. read_into and flush return Err for any failure other than EAGAIN, and event_loop propagates it with ?. A client that resets its connection, which makes write fail with ECONNRESET or EPIPE, therefore ends the loop for every client. Errors on a client socket should close that connection and let the loop continue; only errors on the listener or on epoll itself are fatal.

No backpressure. `read_into` reads until `EAGAIN` and appends everything to `out`, even when `flush` could not write the previous reply. A client that sends continuously and never reads makes the server's memory grow without bound. The loop should stop reading from a connection, by removing `EPOLLIN`, while its output buffer is above a limit.

Raw descriptors have no owner. File descriptors are plain `i32` values. When `event_loop` returns an error, the client descriptors in `connections` are never closed. `std::os::fd::OwnedFd` closes its descriptor on drop, and storing one in `Connection` would make the cleanup automatic.

An `epoll_ctl` call per event. `update_interest` issues `EPOLL_CTL_MOD` after every event even when the mask has not changed. Tracking the registered mask in `Connection` and calling `epoll_ctl` only when it changes removes one system call per event.

`EINTR` is handled only for `epoll_wait`. A `read` or `write` interrupted by a signal also fails with `EINTR`, and the code treats that as a fatal error.

`errno` is read after `close`. In `read_into`, the error path calls `close_connection` before `io::Error::last_os_error()`. If `close` changed `errno`, the reported error would describe the wrong call. Reading the error first avoids the question.

Linux only. macOS and the BSDs provide `kqueue` and Windows provides I/O completion ports. The `mio` crate abstracts over all three, and Tokio is built on it.

Summary

- Non-blocking sockets fail with `EAGAIN` instead of waiting, and `epoll_wait` reports which sockets are ready, so one thread can serve many connections.
- Level-triggered readiness keeps reporting a socket until it is drained, which tolerates handlers that do not read everything at once.
- A per-connection output buffer with a position index survives partial writes, and `EPOLLOUT` should be registered only while output is pending.
- Each `unsafe` call into `libc` states why its pointers are valid, and every return value is checked against `-1` and converted with `io::Error::last_os_error`.

- Production event loops close only the failing connection, apply backpressure, own their descriptors with `OwnedFd`, and are usually built on `mio`.

References

- Linux manual pages `epoll(7)`, `accept4(2)`, and `socket(7)`.
- Michael Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010, Chapter 63, "Alternative I/O Models".
- Standard library, `std::os::fd::OwnedFd`.
- The `mio` crate, [documentation](#).

Parsing HTTP/1.1 Requests

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/http_request.rs`. Test it with `cargo test http_request`.

Problem Statement

Given a byte slice that begins at the start of a request, return a `Request` containing the method, target, version, headers, and body, or an error that says why no request could be produced. The parser must:

- return `Incomplete` when the bytes so far are a valid prefix of a request;
- reject a request line with a missing, extra, or unknown part;
- require exactly one `Host` header for HTTP/1.1;
- reject obsolete header line folding;
- reject a request with both `Content-Length` and `Transfer-Encoding`;
- decode chunked bodies, and bound header and body sizes.

Designing a Solution

An HTTP/1.1 request has three parts:

POST /v1/gpu-workloads HTTP/1.1\r\n	request line
Host: api.example.com\r\n	header lines
Content-Length: 13\r\n	
\r\n	empty line ends the head
{"gpuCount":1}	body, framed by the headers

Find the end of the head first. The head ends at the first `\r\n\r\n`. Until those four bytes arrive, the request is incomplete, unless the bytes already ex-

ceed the header limit, in which case the request is too large. Everything before that point is split into lines.

Parse the head as text only where the grammar is text. The request line and header names and values are converted to `&str` with `from_utf8`, and invalid UTF-8 is a malformed request. The body stays as bytes.

Decide the body length from exactly one source. The body is framed either by `Content-Length`, a decimal byte count, or by `Transfer-Encoding: chunked`, a sequence of hexadecimal-length chunks ending with a zero-length chunk. RFC 9112 requires a server to treat a request carrying both as an error, because a proxy that honours one and a server that honours the other would split the same bytes into different requests. That disagreement is the basis of request smuggling.

Bound everything. A header section larger than 16 KiB, or a body larger than 8 MiB, is rejected before the parser allocates for it.

Implementation

```

/// HTTP/1.1 request parsing: request line, headers, and body framing.
///
/// The framing checks are the point. `Content-Length` together with
/// `Transfer-Encoding` is rejected as ambiguous (request smuggling), as are
/// obsolete line folding and a missing `Host`. Header and body sizes are bounded
/// so a slow client cannot hold a connection open.

use std::fmt;

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Method {
    Get,
    Head,
    Post,
    Put,
    Patch,
    Delete,
    Options,
}

impl Method {
    pub fn parse(token: &str) -> Option<Self> {
        Some(match token {
            "GET" => Self::Get,
            "HEAD" => Self::Head,
            "POST" => Self::Post,
            "PUT" => Self::Put,
            "PATCH" => Self::Patch,
            "DELETE" => Self::Delete,
            "OPTIONS" => Self::Options,
            _ => return None,
        })
    }

    pub fn is_safe(self) -> bool {

```



```

        matches!(self, Self::Get | Self::Head | Self::Options)
    }

    pub fn is_idempotent(self) -> bool {
        matches!(
            self,
            Self::Get | Self::Head | Self::Options | Self::Put | Self::Delete
        )
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Version {
    Http10,
    Http11,
}

#[derive(Debug, Clone, Copy)]
pub struct Limits {
    pub max_header_bytes: usize,
    pub max_body_bytes: usize,
}

impl Default for Limits {
    fn default() -> Self {
        Self {
            max_header_bytes: 16 * 1024,
            max_body_bytes: 8 * 1024 * 1024,
        }
    }
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Request {
    pub method: Method,
    pub target: String,
    pub version: Version,
    pub headers: Vec<(String, String)>,
    pub body: Vec<u8>,
}

impl Request {
    /// First header value matching `name`, case-insensitively.
    pub fn header(&self, name: &str) -> Option<&str> {
        self.headers
            .iter()
            .find(|(key, _)| key.eq_ignore_ascii_case(name))
            .map(|(_, value)| value.as_str())
    }

    /// HTTP/1.1 reuses the connection unless `Connection: close`; HTTP/1.0 only
    /// reuses it with `Connection: keep-alive`.
    pub fn should_keep_alive(&self) -> bool {
        let connection = self.header("connection").unwrap_or("");
        let has = |token: &str| {
            connection
                .split(',')
                .any(|part| part.trim().eq_ignore_ascii_case(token))
        };
        match self.version {
            Version::Http11 => !has("close"),
            Version::Http10 => has("keep-alive"),
        }
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]

```

```

pub enum ParseError {
    /// Not a complete request yet; read more and retry.
    Incomplete,
    Malformed(&'static str),
    MissingHost,
    /// `Content-Length` and `Transfer-Encoding` were both present.
    AmbiguousBodyLength,
    UnsupportedTransferEncoding,
    PayloadTooLarge,
}

impl fmt::Display for ParseError {
    fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
        match self {
            Self::Incomplete => write!(f, "incomplete request"),
            Self::Malformed(reason) => write!(f, "malformed request: {reason}"),
            Self::MissingHost => write!(f, "HTTP/1.1 requires exactly one Host header"),
            Self::AmbiguousBodyLength => {
                write!(
                    f,
                    "Content-Length and Transfer-Encoding are mutually exclusive"
                )
            },
            Self::UnsupportedTransferEncoding => write!(f, "unsupported Transfer-
Encoding"),
            Self::PayloadTooLarge => write!(f, "request exceeds configured limits"),
        }
    }
}

impl std::error::Error for ParseError {}

/// Parse one request from the start of `input`.
pub fn parse_request(input: &[u8], limits: Limits) -> Result<Request, ParseError> {
    let Some(header_end) = input.windows(4).position(|window| window == b"\r\n\r\n") else {
        return Err(if input.len() > limits.max_header_bytes {
            ParseError::PayloadTooLarge
        } else {
            ParseError::Incomplete
        });
    };
    if header_end + 4 > limits.max_header_bytes {
        return Err(ParseError::PayloadTooLarge);
    }

    let head = &input[..header_end];
    let body_start = header_end + 4;

    let mut lines = head
        .split(|byte| *byte == b'\n')
        .map(|line| line.strip_suffix(b"\r").unwrap_or(line));

    let request_line = std::str::from_utf8(lines.next().ok_or(ParseError::Malformed("empty"))?)
        .map_err(|_| ParseError::Malformed("request line"))?;
    let mut parts = request_line.split(' ');
    let method = Method::parse(parts.next().ok_or(ParseError::Malformed("method"))?)
        .ok_or(ParseError::Malformed("unknown method"))?;
    let target = parts.next().ok_or(ParseError::Malformed("target"))?;
    let version = match parts.next() {
        Some("HTTP/1.1") => Version::Http11,
        Some("HTTP/1.0") => Version::Http10,
        _ => return Err(ParseError::Malformed("version")),
    };
    if parts.next().is_some() || target.is_empty() {

```

```

        return Err(ParseError::Malformed("request line"));
    }

    let mut headers = Vec::new();
    for line in lines {
        if line.is_empty() {
            continue;
        }
        if matches!(line.first(), Some(b' ') | Some(b'\t')) {
            return Err(ParseError::Malformed("obsolete line folding"));
        }
        let colon = line
            .iter()
            .position(|byte| *byte == b':')
            .ok_or(ParseError::Malformed("header without colon"))?;
        let name = std::str::from_utf8(&line[..colon])
            .map_err(|_| ParseError::Malformed("header name"))?;
        if name.is_empty() || name.bytes().any(|byte| byte.is_ascii_whitespace()) {
            return Err(ParseError::Malformed("invalid header name"));
        }
        let value = std::str::from_utf8(&line[colon + 1..])
            .map_err(|_| ParseError::Malformed("header value"))?;
        headers.push((name.to_string(), value.trim().to_string()));
    }

    if version == Version::Http11
        && headers
            .iter()
            .filter(|(name, _)| name.eq_ignore_ascii_case("host"))
            .count()
            != 1
    {
        return Err(ParseError::MissingHost);
    }

    let body = parse_body(input, body_start, &headers, limits)?;

    Ok(Request {
        method,
        target: target.to_string(),
        version,
        headers,
        body,
    })
}

fn parse_body(
    input: &[u8],
    body_start: usize,
    headers: &[(String, String)],
    limits: Limits,
) -> Result<Vec<u8>, ParseError> {
    let collect = |name: &str| -> Vec<&str> {
        headers
            .iter()
            .filter(|(key, _)| key.eq_ignore_ascii_case(name))
            .map(|(_, value)| value.as_str())
            .collect()
    };

    let transfer_encoding = collect("transfer-encoding");
    let content_length = collect("content-length");

    // Two framing headers is the classic request-smuggling setup.
    if !transfer_encoding.is_empty() && !content_length.is_empty() {
        return Err(ParseError::AmbiguousBodyLength);
    }
}

```

```

if !transfer_encoding.is_empty() {
    let joined = transfer_encoding.join(",");
    let codings: Vec<&str> = joined
        .split(',')
        .map(str::trim)
        .filter(|token| !token.is_empty())
        .collect();
    let all_chunked = codings
        .iter()
        .all(|token| token.eq_ignore_ascii_case("chunked"));
    if !all_chunked
        || !codings
            .last()
            .is_some_and(|t| t.eq_ignore_ascii_case("chunked"))
    {
        return Err(ParseError::UnsupportedTransferEncoding);
    }
    return decode_chunked(&input[body_start..], limits.max_body_bytes);
}

if content_length.is_empty() {
    return Ok(Vec::new());
}

let mut lengths = content_length.iter().map(|value| {
    value
        .trim()
        .parse::<usize>()
        .map_err(|_| ParseError::Malformed("invalid Content-Length"))
});
let first = lengths.next().expect("non-empty");
if lengths.any(|length| length != Ok(first)) {
    return Err(ParseError::Malformed("conflicting Content-Length"));
}
if first > limits.max_body_bytes {
    return Err(ParseError::PayloadTooLarge);
}
let end = body_start + first;
if input.len() < end {
    return Err(ParseError::Incomplete);
}
Ok(input[body_start..end].to_vec())
}

fn decode_chunked(data: &[u8], max_body: usize) -> Result<Vec<u8>, ParseError> {
    let mut cursor = 0;
    let mut body = Vec::new();

    loop {
        let line_end = find_crlf(data, cursor).ok_or(ParseError::Incomplete)?;
        let size_token = data[cursor..line_end]
            .split(|byte| *byte == b';')
            .next()
            .unwrap_or(&[]);
        let size = usize::from_str_radix(
            std::str::from_utf8(size_token)
                .map_err(|_| ParseError::Malformed("chunk size"))?
                .trim(),
            16,
        )
        .map_err(|_| ParseError::Malformed("chunk size"))?;
        cursor = line_end + 2;

        if size == 0 {
            // Consume trailers up to the final empty line.
            loop {
                let trailer_end = find_crlf(data,

```

```

cursor).ok_or(ParseError::Incomplete)?;
    let is_empty = trailer_end == cursor;
    cursor = trailer_end + 2;
    if is_empty {
        return Ok(body);
    }
}

if body.len() + size > max_body {
    return Err(ParseError::PayloadTooLarge);
}
let chunk_end = cursor
    .checked_add(size)
    .ok_or(ParseError::Malformed("chunk overflow"))?;
if chunk_end + 2 > data.len() {
    return Err(ParseError::Incomplete);
}
body.extend_from_slice(&data[cursor..chunk_end]);
if &data[chunk_end..chunk_end + 2] != b"\r\n" {
    return Err(ParseError::Malformed("chunk not CRLF-terminated"));
}
cursor = chunk_end + 2;
}
}

fn find_crlf(data: &[u8], from: usize) -> Option<usize> {
    data.get(from..)?
        .windows(2)
        .position(|window| window == b"\r\n")
        .map(|offset| offset + from)
}

#[cfg(test)]
mod tests {
    use super::*;

    fn parse(raw: &[u8]) -> Result<Request, ParseError> {
        parse_request(raw, Limits::default())
    }

    #[test]
    fn parses_a_get_and_a_post_with_content_length() {
        let get = parse(b"GET /v1/gpu-workloads HTTP/1.1\r\nHost: api\r\nAccept: */*\r\n\r\n")
            .expect("valid");
        assert_eq!(get.method, Method::Get);
        assert_eq!(get.header("HOST"), Some("api"));
        assert!(get.body.is_empty());
        assert!(get.should_keep_alive());

        let post = parse(b"POST / HTTP/1.1\r\nHost: api\r\nContent-Length: 5\r\n\r\nhello")
            .expect("valid");
        assert_eq!(post.body, b"hello");
        assert!(!post.method.is_idempotent());
    }

    #[test]
    fn decodes_chunked_bodies() {
        let raw = b"POST /u HTTP/1.1\r\nHost: api\r\nTransfer-Encoding:
chunked\r\n\r\n5\r\nhello\r\n6\r\n world\r\n0\r\n\r\n";
        assert_eq!(parse(raw).expect("valid").body, b"hello world");
    }

    #[test]
    fn rejects_ambiguous_framing_and_bad_framing() {

```

```

    let smuggling = b"POST / HTTP/1.1\r\nHost: api\r\nContent-Length: 5\r\nTransfer
-Encoding: chunked\r\n\r\nhello";
    assert_eq!(parse(smuggling), Err(ParseError::AmbiguousBodyLength));

    let gzip = b"POST / HTTP/1.1\r\nHost: api\r\nTransfer-Encoding: gzip\r\n\r\n";
    assert_eq!(parse(gzip), Err(ParseError::UnsupportedTransferEncoding));
}

#[test]
fn rejects_missing_host_and_line_folding() {
    assert_eq!(
        parse(b"GET / HTTP/1.1\r\nAccept: */*\r\n\r\n"),
        Err(ParseError::MissingHost)
    );
    let folded = b"GET / HTTP/1.1\r\nHost: api\r\nX-A: 1\r\n continued\r\n\r\n";
    assert!(matches!(parse(folded), Err(ParseError::Malformed(_))));
}

#[test]
fn reports_incomplete_and_enforces_limits() {
    assert_eq!(
        parse(b"GET / HTTP/1.1\r\nHost: api\r\n"),
        Err(ParseError::Incomplete)
    );
    let limits = Limits {
        max_header_bytes: 1024,
        max_body_bytes: 10,
    };
    let raw = b"POST / HTTP/1.1\r\nHost: api\r\nContent-Length: 999\r\n\r\n";
    assert_eq!(parse_request(raw, limits), Err(ParseError::PayloadTooLarge));
}

#[test]
fn http_1_0_needs_keep_alive_opt_in() {
    let raw = b"GET / HTTP/1.0\r\n\r\n";
    assert!(!parse(raw).expect("valid").should_keep_alive());
    let raw = b"GET / HTTP/1.0\r\nConnection: keep-alive\r\n\r\n";
    assert!(parse(raw).expect("valid").should_keep_alive());
}
}

```

Methods, versions, and the request type

`Method::parse` uses `Some(match token { ... _ => return None })`. Every arm except the last produces a `Method`, and the last returns from the function, so the `match` expression has type `Method` and the whole body fits in one expression.

`is_safe` and `is_idempotent` encode RFC 9110's definitions. `PUT` and `DELETE` are idempotent but not safe, and `POST` and `PATCH` are neither. Chapter 57 uses the same distinction to decide which requests may be retried.

`Request::header` compares names with `eq_ignore_ascii_case`, because HTTP header names are case-insensitive, and returns the first match.

`should_keep_alive` splits the `Connection` header on commas, because the header is a list, and applies the default for each version: HTTP/1.1 keeps connec-

tions open unless told to close, and HTTP/1.0 closes them unless told to keep them alive.

Errors

`ParseError` is `Copy`. `Malformed` carries a `&'static str` reason rather than a `String`, so constructing an error never allocates. A client can make the parser fail on purpose, and failing should cost as little as possible.

The request line and headers

`let Some(header_end) = ... else { ... }` handles the missing terminator first, choosing between `PayloadTooLarge` and `Incomplete` by the input length.

The head is split on `\n` with each line's trailing `\r` removed by `strip_suffix`. The request line is split on single spaces into exactly three parts. `parts.next().is_some()` after the version rejects a fourth part, and `target.is_empty()` rejects the empty target that two consecutive spaces produce.

Header lines beginning with a space or tab are obsolete line folding, which RFC 9112 says a server must reject or replace; this parser rejects them. A header name must be non-empty and free of whitespace, which rejects `Host : api`, a form some servers accept and others do not.

The `Host` check counts matching headers and requires exactly one, for HTTP/1.1 only.

Body framing

`collect` is a closure that returns every value of a header. Collecting all values rather than the first is required for framing: two `Content-Length` headers with different values are rejected as `conflicting Content-Length`, while two with the same value are accepted, as RFC 9112 permits.

For `Transfer-Encoding`, all values are joined and split on commas. The only coding the parser supports is `chunked`, and it must be the final coding. Anything else is `UnsupportedTransferEncoding`.

`lengths.next().expect("non-empty")?` combines two operations. `expect` documents that the iterator cannot be empty, because the function returned earlier for an empty list, and `?` propagates a parse failure of the first value.

Chunked decoding

Each chunk begins with a hexadecimal size, optionally followed by `;` and chunk extensions, which the parser ignores. A size of zero ends the body; the loop then skips trailer lines until an empty line. For a non-zero size, the parser checks the body limit, checks that the chunk and its trailing `\r\n` are fully present, copies the chunk, and verifies the `\r\n`.

`find_crlf` uses `data.get(from..)?`, which returns `None` rather than panicking when `from` is past the end of the data.

Intuition

Parsing a chunked POST from the test `decodes_chunked_bodies`

stage	input examined	result
find head end	up to the first <code>\r\n\r\n</code>	<code>header_end</code> found
request line	<code>POST /u HTTP/1.1</code>	<code>Method::Post</code> , target <code>/u</code> , <code>Http11</code>
headers	<code>Host: api</code> , <code>Transfer-Encoding: chunked</code>	two headers; exactly one <code>Host</code>
framing	<code>transfer-encoding</code> present, no <code>content-length</code>	chunked
chunk 1	<code>5\r\nhello\r\n</code>	size 5, body <code>hello</code>
chunk 2	<code>6\r\n world\r\n</code>	size 6, body <code>hello world</code>
last chunk	<code>0\r\n\r\n</code>	size 0; empty trailer line ends the body
result		<code>Ok(Request { body: b"hello world", .. })</code>

For the smuggling test, both `Content-Length: 5` and `Transfer-Encoding: chunked` are present, and `parse_body` returns `AmbiguousBodyLength` before looking at the body.

Time and Space Complexity

Stage	Time	Space
finding the head	$O(n)$ over the bytes received	none
request line and headers	$O(\text{head length})$	one String pair per header
Content-Length body	$O(\text{body length})$	one copy of the body
chunked body	$O(\text{body length})$ plus $O(\text{chunks})$ line searches	one growing <code>Vec<u8></code>

Limitations

A chunk size near `usize::MAX` panics in a debug build. In `decode_chunked`, the check `body.len() + size > max_body` adds before comparing. A client that sends one valid chunk and then a chunk size of `ffffffffffffffff` makes the addition overflow. With overflow checks enabled, as in a debug build, the parser panics; the following request triggers it:

```
POST / HTTP/1.1\r\nHost: a\r\nTransfer-Encoding:
chunked\r\n\r\n5\r\nhello\r\nffffffffffffffff\r\n
```

In a release build the sum wraps, passes the size check, and the later `checked_add` reports `Malformed`, so the outcome depends on the build profile. Comparing without adding removes the problem:

```
if size > max_body - body.len() {
    return Err(ParseError::PayloadTooLarge);
}
```

`body.len()` never exceeds `max_body`, so the subtraction cannot underflow.

The head is searched again on every call. A server that calls `parse_request` after each read rescans all the bytes received so far for `\r\n\r\n`. For a client that sends a 16 KiB head one byte at a time, that is quadratic work bounded by the header limit. A parser that remembers where it stopped avoids it.

Header names are not validated as tokens. RFC 9110 restricts header names to a set of token characters. The parser rejects only empty names and names containing whitespace, so names such as `X(Y)` are accepted.

Transfer-Encoding is accepted for HTTP/1.0. The chunked coding was introduced in HTTP/1.1, and RFC 9112 advises treating its presence in a 1.0 request as a framing error.

One request per call. The parser does not report how many bytes it consumed, so a caller cannot find the start of a pipelined second request in the same buffer. Returning the consumed length alongside the `Request` would allow it.

Summary

- Parse the head only after the terminating empty line has arrived; report `Incomplete` until then and `PayloadTooLarge` if the limit is exceeded first.
- Convert only the textual parts of a request to `&str`, and keep the body as bytes.
- Reject a request that carries both `Content-Length` and `Transfer-Encoding`, and reject conflicting `Content-Length` values, to close the ambiguities behind request smuggling.
- Bound header and body sizes before allocating, and use allocation-free error values in code that untrusted input can drive to failure.
- Size arithmetic on untrusted numbers must not overflow; compare against the remaining budget instead of adding to the amount used.

References

- RFC 9110, *HTTP Semantics*, 2022, Sections 5.1 and 9.2.
- RFC 9112, *HTTP/1.1*, 2022, Sections 6 and 7, on message body length and chunked transfer coding.
- James Kettle, "HTTP Desync Attacks: Request Smuggling Reborn", PortSwigger Research, 2019.
- The `httparse` crate, [documentation](#), a zero-copy incremental parser.

A Small HTTP Server

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/http_server.rs`. Run it with `cargo run --bin http_server`, then `curl -i http://127.0.0.1:8080/healthz`.

Problem Statement

Serve the following routes over HTTP/1.1:

Method	Path	Response
GET	/healthz	200 plain text
GET	/v1/gpu-workloads	200 JSON list
POST	/v1/gpu-workloads	201 created
GET	/v1/gpu-workloads/{id}	200 one workload
DELETE	/v1/gpu-workloads/{id}	204 no content

A known path with an unsupported method returns `405 Method Not Allowed`, an unknown path returns `404 Not Found`, and a request the parser rejects returns `400 Bad Request`. A connection stays open for further requests unless the client asks to close it.

Designing a Solution

Separate the protocol from the application. `route` takes a parsed `Request` and returns `(status, content_type, body)`. It knows nothing about sockets or bytes on the wire, so its behaviour can be tested with constructed requests.

`build_response` turns that triple into bytes, and `handle_connection` owns the socket.

Read until a request parses. `handle_connection` keeps a growing buffer. It asks the parser from chapter 54 for a request, and on `Incomplete` reads more bytes and tries again. Any other error ends the connection with a 400. After a response, the loop either returns, when the connection should close, or clears the buffer and waits for the next request.

```
loop:
    match parse_request(buffer):
        Ok(request)      -> route, write response, close or clear buffer
        Err(Incomplete)  -> read more bytes; a read of 0 means the client left
        Err(other)       -> write 400 with Connection: close, return
```

404 or 405. The status tells the client what to change. A 404 says the resource does not exist, so a different method will not help; a 405 says the resource exists and the method is wrong. The router therefore matches the path first and the method second.

Implementation

```
/// Minimal HTTP/1.1 server with REST-style routes.
///
/// This is the "write a web server" exercise an SDK/CLI interview often starts
/// with. It uses only `std::net` plus the request parser in
/// [nvidia_rust_interview_lab::problems::http_request], so the interesting parts
/// stay visible: the accept loop, the keep-alive loop, status codes, and the
/// routing decision.
///
/// Routes:
///
/// | Method | Path | Response |
/// |---|---|---|
/// | `GET` | `/healthz` | `200` plain text |
/// | `GET` | `/v1/gpu-workloads` | `200` JSON list |
/// | `POST` | `/v1/gpu-workloads` | `201` created |
/// | `GET` | `/v1/gpu-workloads/{id}` | `200` one workload |
/// | `DELETE` | `/v1/gpu-workloads/{id}` | `204` no content |
///
/// A known path with the wrong method returns `405`, and an unknown path returns
/// `404`. Pipelined requests are not supported: the loop handles one request per
/// read buffer, which matches how most servers treat pipelining in practice.
///
/// ```bash
/// cargo run --bin http_server # binds 127.0.0.1:8080
/// curl -i http://127.0.0.1:8080/healthz
/// curl -i -X POST -d '{"gpuCount":1}' http://127.0.0.1:8080/v1/gpu-workloads
/// ```

use nvidia_rust_interview_lab::problems::http_request::{
    Limits, Method, ParseError, Request, parse_request,
};
use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};
```

```

use std::thread;

/// Map a request to `(status, content_type, body)`.
///
/// Pure, so the routing rules are unit-testable without a socket.
pub fn route(request: &Request) -> (u16, &'static str, Vec<u8>) {
    let target = request.target.as_str();

    if target == "/healthz" {
        return match request.method {
            Method::Get | Method::Head => (200, "text/plain", b"ok\n".to_vec()),
            _ => method_not_allowed(),
        };
    }

    if target == "/v1/gpu-workloads" {
        return match request.method {
            Method::Get | Method::Head => (
                200,
                "application/json",
                br#"{"id":"wl-a100-1","gpuCount":1,"state":"Running"}"#.to_vec(),
            ),
            Method::Post => (
                201,
                "application/json",
                br#"{"id":"wl-a100-2","gpuCount":1,"state":"Pending"}"#.to_vec(),
            ),
            _ => method_not_allowed(),
        };
    }

    if let Some(id) = target.strip_prefix("/v1/gpu-workloads/") {
        if id.is_empty() || id.contains('/') {
            return (404, "text/plain", b"not found\n".to_vec());
        }
        return match request.method {
            Method::Get | Method::Head => (
                200,
                "application/json",
                format!(r#"{"id":"{id}","state":"Running"}"#).into_bytes(),
            ),
            Method::Delete => (204, "", Vec::new()),
            _ => method_not_allowed(),
        };
    }

    (404, "text/plain", b"not found\n".to_vec())
}

fn method_not_allowed() -> (u16, &'static str, Vec<u8>) {
    (405, "text/plain", b"method not allowed\n".to_vec())
}

fn status_reason(status: u16) -> &'static str {
    match status {
        200 => "OK",
        201 => "Created",
        204 => "No Content",
        400 => "Bad Request",
        404 => "Not Found",
        405 => "Method Not Allowed",
        413 => "Payload Too Large",
        _ => "Internal Server Error",
    }
}

fn build_response(status: u16, content_type: &str, body: &[u8], keep_alive: bool) -> Ve

```

```

c<u8> {
    let mut response = format!("HTTP/1.1 {} {}\\r\\n", status, status_reason(status));
    response.push_str(&format!("Content-Length: {}\\r\\n", body.len()));
    if !content_type.is_empty() {
        response.push_str(&format!("Content-Type: {content_type}\\r\\n"));
    }
    response.push_str(if keep_alive {
        "Connection: keep-alive\\r\\n"
    } else {
        "Connection: close\\r\\n"
    });
    response.push_str("\\r\\n");

    let mut bytes = response.into_bytes();
    bytes.extend_from_slice(body);
    bytes
}

/// Serve connections forever, one thread per connection.
pub fn serve(listener: TcpListener) {
    for incoming in listener.incoming() {
        match incoming {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(error) = handle_connection(stream) {
                        eprintln!("connection error: {error}");
                    }
                });
            }
            Err(error) => eprintln!("accept error: {error}"),
        }
    }
}

fn handle_connection(mut stream: TcpStream) -> std::io::Result<()> {
    stream.set_nodelay(true)?;
    let limits = Limits::default();
    let mut buffer = Vec::new();
    let mut chunk = [0u8; 4096];

    loop {
        match parse_request(&buffer, limits) {
            Ok(request) => {
                let keep_alive = request.should_keep_alive();
                let (status, content_type, body) = route(&request);
                let response = build_response(status, content_type, &body, keep_alive);
                stream.write_all(&response)?;

                if !keep_alive {
                    return Ok(());
                }
                buffer.clear();
            }
            Err(ParseError::Incomplete) => {
                let read = stream.read(&mut chunk)?;
                if read == 0 {
                    // Peer closed mid-request.
                    return Ok(());
                }
                buffer.extend_from_slice(&chunk[..read]);
            }
            Err(error) => {
                let response =
                    build_response(400, "text/plain", error.to_string().as_bytes(), false);
                stream.write_all(&response)?;
                return Ok(());
            }
        }
    }
}

```

```

    }
  }
}

fn main() {
  let address = std::env::args()
    .nth(1)
    .unwrap_or_else(|| "127.0.0.1:8080".to_string());

  let listener = TcpListener::bind(&address).expect("failed to bind");
  let local = listener.local_addr().expect("failed to read local address");
  println!("http server listening on http://{local}");

  serve(listener);
}

#[cfg(test)]
mod tests {
  use super::*;
  use nvidia_rust_interview_lab::problems::http_request::Version;

  fn request(method: Method, target: &str) -> Request {
    Request {
      method,
      target: target.to_string(),
      version: Version::Http11,
      headers: vec![("host".to_string(), "localhost".to_string())],
      body: Vec::new(),
    }
  }

  #[test]
  fn healthz_is_ok() {
    let (status, content_type, body) = route(&request(Method::Get, "/healthz"));
    assert_eq!(status, 200);
    assert_eq!(content_type, "text/plain");
    assert_eq!(body, b"ok\n");
  }

  #[test]
  fn lists_and_creates_workloads() {
    let (status, _, _) = route(&request(Method::Get, "/v1/gpu-workloads"));
    assert_eq!(status, 200);

    let (status, content_type, body) = route(&request(Method::Post, "/v1/gpu-
workloads"));
    assert_eq!(status, 201);
    assert_eq!(content_type, "application/json");
    assert!(String::from_utf8(body).expect("json").contains("Pending"));
  }

  #[test]
  fn gets_and_deletes_one_workload() {
    let (status, _, body) = route(&request(Method::Get, "/v1/gpu-workloads/wl-7"));
    assert_eq!(status, 200);
    assert!(String::from_utf8(body).expect("json").contains("wl-7"));

    let (status, _, body) = route(&request(Method::Delete, "/v1/gpu-workloads/
wl-7"));
    assert_eq!(status, 204);
    assert!(body.is_empty());
  }

  #[test]
  fn wrong_method_on_known_path_is_405() {
    let (status, _, _) = route(&request(Method::Put, "/healthz"));
  }
}

```

```

    assert_eq!(status, 405);
    let (status, _, _) = route(&request(Method::Put, "/v1/gpu-workloads"));
    assert_eq!(status, 405);
}

#[test]
fn unknown_path_is_404() {
    let (status, _, _) = route(&request(Method::Get, "/nope"));
    assert_eq!(status, 404);
}
}

```

Routing

`route` matches three kinds of path in order: the exact `/healthz`, the exact collection path, and the collection path followed by an identifier. `target.strip_prefix(...)` returns the remainder only when the prefix matches, so `if let Some(id)` both tests and extracts. An empty identifier, or one containing `/`, returns 404, which rejects paths such as `/v1/gpu-workloads/` and `/v1/gpu-workloads/a/b`.

The return type `(u16, &'static str, Vec<u8>)` uses a static string for the content type, because every content type the server sends is a literal, and a `Vec<u8>` for the body, because one route formats its body at run time.

`br#"..."#` is a raw byte string literal. The `r#` form allows the JSON's double quotes without escaping, and the `b` prefix makes the literal a `&[u8; N]`.

Responses and connections

`build_response` writes the status line, `Content-Length`, an optional `Content-Type`, and a `Connection` header, followed by an empty line and the body. `Content-Length` is what lets a keep-alive client find the end of the response without waiting for the connection to close.

`handle_connection` computes `keep_alive` from the request before routing, writes the response with `write_all`, and returns when the connection should close. On a parse error it writes the error's `Display` text as a 400 body and closes the connection, because after a malformed request the server cannot know where the next request would begin.

`main` accepts an optional address argument and defaults to `127.0.0.1:8080`.

The test helper `request` builds a `Request` value directly. Because `route` is pure, the tests check status codes, content types, and bodies without binding a port.

Intuition

A session with `curl` against the running server:

```
$ curl -i http://127.0.0.1:8080/v1/gpu-workloads/wl-7
HTTP/1.1 200 OK
Content-Length: 31
Content-Type: application/json
Connection: keep-alive

{"id":"wl-7","state":"Running"}

$ curl -i -X DELETE http://127.0.0.1:8080/v1/gpu-workloads/wl-7
HTTP/1.1 204 No Content
Content-Length: 0
Connection: keep-alive
```

How route answers several requests

request	path branch	method arm	response
GET /healthz	exact /healthz	Get	200, ok\n
PUT /healthz	exact /healthz	_	405
POST /v1/gpu-workloads	exact collection	Post	201, JSON
DELETE /v1/gpu-workloads/wl-7	prefix, id = "wl-7"	Delete	204, empty body
GET /v1/gpu-workloads/	prefix, id = ""	not reached	404
GET /nope	none	not reached	404

Time and Space Complexity

Stage	Cost
per request	one parse of the buffered bytes, one route match, one response allocation
per connection	one thread and one growing buffer, bounded by the parser's limits
routing	$O(\text{path length})$ for the prefix comparisons

Limitations

Responses to HEAD include a body. `route` returns the same body for HEAD as for GET, and `build_response` always appends it. RFC 9110 requires that a response to HEAD carry the headers of the GET response and no body. A client that sends HEAD and then a second request on the same connection will read the unexpected body as the start of the next response.

204 responses carry Content-Length: 0. RFC 9110 says a server must not send Content-Length in a 204 response. Most clients tolerate it, and it is still a protocol violation that a strict client or proxy may reject.

The identifier is inserted into JSON without escaping. `format!(r#"{{"id":"{id}","state":"Running"}}"#)` places the path segment inside a JSON string as it arrived. A request for `/v1/gpu-workloads/a"b` produces invalid JSON, and a crafted identifier can add fields to the object. Serializing with `serde_json`, or at least escaping `"` and `\`, prevents both.

All parse errors become 400. `status_reason` includes 413 Payload Too Large, but `handle_connection` sends 400 for every error, including `ParseError::PayloadTooLarge`. Mapping that variant to 413 gives the client the correct reason.

Pipelined requests are discarded. After a request is handled, `buffer.clear()` drops any bytes that arrived after it in the same read. A client that pipelines two requests receives one response and then waits. The module documentation states this, and fixing it requires the parser to report how many bytes it consumed.

Unbounded threads and no timeouts. The limitations of chapter 52 apply unchanged: a thread per connection, no limit on connections, and no idle timeout.

String building allocates repeatedly. `build_response` calls `format!` for each header and appends the result with `push_str`. `std::fmt::Write` allows `write!(response, "Content-Length: {}r\n", body.len())` directly into the `String`, which avoids the temporary allocations.

Summary

- Keeping `route` pure, from `Request` to `(status, content_type, body)`, makes the application logic testable without a socket.
- The connection loop parses what it has, reads more on `Incomplete`, and closes after a malformed request because it cannot find the next request boundary.
- Match the path before the method, so a wrong method on an existing path returns 405 and an unknown path returns 404.
- `Content-Length` lets keep-alive clients find the end of each response.
- A conforming server sends no body for `HEAD`, no `Content-Length` for 204, escapes data it places in JSON, and maps each parse failure to its own status.

References

- RFC 9110, *HTTP Semantics*, 2022, Sections 9.3.2 (`HEAD`), 15.3.5 (204), and 15.5.6 (405).
- RFC 9112, *HTTP/1.1*, 2022, Section 9, on persistent connections.
- The `serde_json` crate, `json!` [macro](#).
- Standard library, `str::strip_prefix` and `std::fmt::Write`.

Part XVI: Async Rust and Cloud Clients

A Minimal Async Runtime

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/problems/async_mini.rs` and `src/bin/async_demo.rs`. Run the tests with `cargo test async_mini` and the demonstration with `cargo run --bin async_demo`.

Problem Statement

Using only the standard library:

1. Implement `block_on(future)`, which runs a future on the current thread and returns its output, without spinning while the future is waiting.
2. Implement two futures by hand: one that yields a given number of times before it completes, and one that completes after a wall-clock delay.
3. Implement an executor that runs several spawned tasks, each polled again only after it has been woken.

Designing a Solution

The contract. A future has one method:

```
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>
```

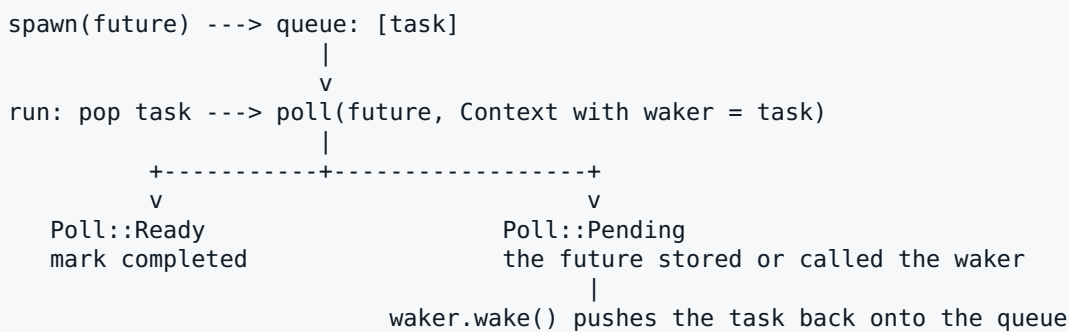
`Poll::Ready(value)` means the future is finished. `Poll::Pending` means it cannot make progress now, and it carries an obligation: before returning `Pending`, the future must arrange for `cx.waker()` to be called when progress becomes possible. A runtime that receives `Pending` does not poll that future again until its waker is called.

Pinning. An `async` block that holds a reference to one of its own local variables across an `.await` becomes a self-referential struct. Moving it would invalidate the reference. `Pin<&mut Self>` promises the future that it will not be moved again, which is why `poll` receives a pinned reference and why runtimes pin futures, typically with `Box::pin`, before polling them.

A waker for a thread. `block_on` has one future and one thread. Its waker can simply unpark that thread. The loop polls, and on `Pending` parks the thread until the waker unparks it.

A waker for a task. An executor with many tasks keeps a queue of tasks ready to be polled. A task's waker pushes the task back onto that queue. The executor pops a task, polls it, and moves on; a task that returned `Pending` is not in the queue until something wakes it.

The diagram below shows the executor's cycle.



Implementation

`block_on`

```

/// A minimal async runtime, built from `Future`, `Waker`, and `Pin`.
///
/// `.await` is not magic. A `Future` is a state machine with one method, `poll`,
/// that returns `Ready(value)` or `Pending`; when it returns `Pending` it must
/// arrange for the `Waker` in its `Context` to be called later. An executor is a
/// loop that polls tasks and parks until a waker fires.
///
/// This module implements that loop twice, from scratch and without a runtime
/// dependency:
///
/// - [`block_on`] drives one future on the current thread, parking on `Pending`;
/// - [`MiniExecutor`] queues independent tasks and re-enqueues them when woken.
///
/// It is deliberately small. What it makes concrete is the part interviewers ask
/// about: why `poll` takes `Pin<&mut Self>`, what `Pending` promises, and where
/// `Send` matters for a multi-threaded runtime.

use std::collections::VecDeque;
use std::future::Future;
use std::pin::Pin;
  
```

```

use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Wake, Waker};
use std::thread;
use std::time::Duration;

/// Block the current thread until `future` completes.
///
/// The future is pinned on the heap so its address is stable across `await`
/// points, and the thread parks whenever the future reports `Pending`. The waker
/// unparks it, so the loop polls again only when there is a reason to.
pub fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = Box::pin(future);
    let waker = Waker::from(Arc::new(ThreadWaker(thread::current())));
    let mut context = Context::from_waker(&waker);

    loop {
        match future.as_mut().poll(&mut context) {
            Poll::Ready(value) => return value,
            // `park` may return spuriously, so the loop re-polls either way.
            Poll::Pending => thread::park(),
        }
    }
}

/// Waker that unparks the thread which is driving the future.
struct ThreadWaker(thread::Thread);

impl Wake for ThreadWaker {
    fn wake(self: Arc<Self>) {
        self.0.unpark();
    }

    fn wake_by_ref(self: &Arc<Self>) {
        self.0.unpark();
    }
}

```

`Box::pin(future)` moves the future to the heap and returns `Pin<Box<F>>`. After that the future's address does not change, so `poll` may be called on it repeatedly. `future.as_mut()` produces the `Pin<&mut F>` that `poll` requires, without moving the box.

`Waker::from(Arc::new(ThreadWaker(thread::current())))` builds a waker from any type that implements the `Wake` trait, stable since Rust 1.51. Before `Wake`, building a waker required a hand-written virtual function table and `unsafe` code.

`thread::park` is safe against a lost wake-up. If the waker calls `unpark` before the thread parks, the thread receives a token, and the next `park` returns immediately. `park` may also return spuriously, which the loop tolerates because it simply polls again.

A future that yields

```

/// A future that yields to the executor `times` times before completing.
///
/// Completes with the number of times it yielded, which makes it easy to assert
/// that a runtime actually suspended and resumed a task.
pub struct YieldTimes {
    remaining: usize,
    yielded: usize,
}

impl YieldTimes {
    pub fn new(times: usize) -> Self {
        Self {
            remaining: times,
            yielded: 0,
        }
    }
}

impl Future for YieldTimes {
    type Output = usize;

    fn poll(self: Pin<&mut Self>, context: &mut Context<'_>) -> Poll<usize> {
        // `YieldTimes` is `Unpin`, so a mutable reference is safe to take.
        let this = self.get_mut();
        if this.remaining == 0 {
            return Poll::Ready(this.yielded);
        }

        this.remaining -= 1;
        this.yielded += 1;
        // Ask to be polled again; an executor that ignores this hangs.
        context.waker().wake_by_ref();
        Poll::Pending
    }
}

```

`YieldTimes` contains only two `usize` fields, so it implements `Unpin`, and `self.get_mut()` can turn `Pin<&mut Self>` back into `&mut Self` without unsafe code.

Each `Pending` result is preceded by `context.waker().wake_by_ref()`. The future asks to be polled again immediately. Returning `Pending` without arranging a wake-up would leave the future suspended forever, which is the most common bug in hand-written futures.

A future that waits for time

```

struct DelayState {
    ready: bool,
    waker: Option<Waker>,
}

/// A future that completes after a wall-clock duration.
///
/// A background thread sleeps and then wakes the task. Storing and waking the
/// [Waker] – rather than spinning in `poll` – is the whole point: `poll` never

```



```

/// blocks the executor thread.
pub struct Delay {
    state: Arc<Mutex<DelayState>>,
}

impl Delay {
    pub fn new(duration: Duration) -> Self {
        let state = Arc::new(Mutex::new(DelayState {
            ready: false,
            waker: None,
        }));

        let thread_state = Arc::clone(&state);
        thread::spawn(move || {
            thread::sleep(duration);
            let waker = {
                let mut guard = thread_state.lock().expect("delay mutex poisoned");
                guard.ready = true;
                guard.waker.take()
            };
            if let Some(waker) = waker {
                waker.wake();
            }
        });

        Self { state }
    }
}

impl Future for Delay {
    type Output = ();

    fn poll(self: Pin<&mut Self>, context: &mut Context<'_>) -> Poll<()> {
        let mut guard = self.state.lock().expect("delay mutex poisoned");
        if guard.ready {
            Poll::Ready(())
        } else {
            guard.waker = Some(context.waker().clone());
            Poll::Pending
        }
    }
}

```

`Delay::new` starts a thread that sleeps and then sets `ready` and takes the stored waker, both under the mutex. `poll` checks `ready` and stores a clone of the current waker, also under the mutex. Because both sides hold the lock while they read or write the shared state, there is no moment at which the timer thread can see "no waker yet" after `poll` has decided to return `Pending`. Without the lock, a wake-up could be lost.

`poll` replaces the stored waker on every call. A future may be polled with a different waker each time, for example after being moved to another task, and it must wake the most recent one.

The timer thread calls `waker.wake()` after releasing the lock. Calling a waker while holding a lock the woken task might also need is a common source of deadlock.

The executor

```

struct Task {
    future: Mutex<Pin<Box<dyn Future<Output = ()> + Send>>>,
    queue: Arc<Mutex<VecDeque<Arc<Task>>>>,
    completed: AtomicBool,
}

impl Wake for Task {
    fn wake(self: Arc<Self>) {
        // Re-enqueue the task. Cloning the `Arc` is a refcount bump, not a deep copy.
        self.queue
            .lock()
            .expect("executor queue poisoned")
            .push_back(Arc::clone(&self));
    }

    fn wake_by_ref(self: &Arc<Self>) {
        self.queue
            .lock()
            .expect("executor queue poisoned")
            .push_back(Arc::clone(self));
    }
}

/// A single-threaded, `Send`-only executor.
///
/// Real runtimes add work stealing, timers, and I/O readiness. This keeps only the
/// part that matters for explaining how `.await` makes progress: a queue of tasks
/// where waking a task puts it back on the queue.
#[derive(Default)]
pub struct MiniExecutor {
    queue: Arc<Mutex<VecDeque<Arc<Task>>>>,
}

impl MiniExecutor {
    pub fn new() -> Self {
        Self::default()
    }

    /// Add a task to the executor.
    pub fn spawn<F>(&self, future: F)
    where
        F: Future<Output = ()> + Send + 'static,
    {
        let task = Arc::new(Task {
            future: Mutex::new(Box::pin(future)),
            queue: Arc::clone(&self.queue),
            completed: AtomicBool::new(false),
        });
        self.queue
            .lock()
            .expect("executor queue poisoned")
            .push_back(task);
    }

    /// Run until the queue is empty.
    ///
    /// A task that returns `Pending` without waking stays suspended, which is why
    /// this returns rather than blocks: a runtime with timers or I/O would also
    /// park on an event source.
    pub fn run(&self) {
        loop {
            let Some(task) = self
                .queue
                .lock()

```

```

        .expect("executor queue poisoned")
        .pop_front()
    else {
        return;
    };

    if task.completed.load(Ordering::Acquire) {
        continue;
    }

    let waker = Waker::from(Arc::clone(&task));
    let mut context = Context::from_waker(&waker);
    let mut future = task.future.lock().expect("task mutex poisoned");
    if future.as_mut().poll(&mut context).is_ready() {
        task.completed.store(true, Ordering::Release);
    }
}
}
}

```

A `Task` holds its future as `Mutex<Pin<Box<dyn Future<Output = ()> + Send>>>`. The `Wake` trait requires `Arc<Task>` to be `Send` and `Sync`, because a waker may be called from any thread. The `Mutex` makes the task `Sync`; the `Send` bound on the future makes it safe to poll the future from whichever thread holds the lock.

`spawn` requires `F: Future<Output = ()> + Send + 'static`. The future is stored in the task and outlives the call to `spawn`, so it cannot borrow from the caller, and it may be polled after a wake from another thread, so it must be `Send`. These are the same bounds that `tokio::spawn` imposes, for the same reasons.

`run` pops tasks until the queue is empty. `let Some(task) = ... else { return; }` takes the queue lock only for the `pop_front`, because the temporary guard is dropped at the end of the `let` statement. The lock must not be held while polling, because a future that wakes itself during `poll`, as `YieldTimes` does, pushes onto the same queue.

The `completed` flag handles a task that is woken after it has finished. Its waker may still exist, held by a timer or a clone, and pushing it onto the queue must not poll a completed future again.

```

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::atomic::AtomicUsize;
    use std::time::Instant;

    #[test]
    fn block_on_drives_a_hand_written_future() {
        assert_eq!(block_on(YieldTimes::new(3)), 3);
    }

    #[test]
    fn block_on_polls_an_async_block() {

```

```

    let total = block_on(async {
        let a = YieldTimes::new(1).await;
        let b = YieldTimes::new(2).await;
        a + b
    });
    assert_eq!(total, 3);
}

#[test]
fn block_on_waits_for_a_delay() {
    let start = Instant::now();
    block_on(Delay::new(Duration::from_millis(30)));
    assert!(start.elapsed() >= Duration::from_millis(20));
}

#[test]
fn executor_runs_spawned_tasks_to_completion() {
    let executor = MiniExecutor::new();
    let counter = Arc::new(AtomicUsize::new(0));

    for _ in 0..4 {
        let counter = Arc::clone(&counter);
        executor.spawn(async move {
            YieldTimes::new(2).await;
            counter.fetch_add(1, Ordering::SeqCst);
        });
    }

    executor.run();
    assert_eq!(counter.load(Ordering::SeqCst), 4);
}

#[test]
fn executor_polls_an_immediately_ready_task() {
    let executor = MiniExecutor::new();
    let flag = Arc::new(AtomicBool::new(false));

    let task_flag = Arc::clone(&flag);
    executor.spawn(async move {
        task_flag.store(true, Ordering::SeqCst);
    });

    executor.run();
    assert!(flag.load(Ordering::SeqCst));
}
}

```

```

///! Runs the hand-written async runtime in
['nvidia_rust_interview_lab::problems::async_mini`].
///!
///! There is no Tokio here on purpose: the point is to show that `Future`, `Waker`,
///! `Poll`, and `Pin` are enough to build an executor, and that `.await` is just
///! sugar over a state machine that is polled.
///!
///! ```bash
///! cargo run --bin async_demo
///! ```

use nvidia_rust_interview_lab::problems::async_mini::{Delay, MiniExecutor, YieldTimes,
block_on};
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::time::{Duration, Instant};

fn main() {
    println!("== block_on drives a hand-written future ==");
}

```

```

let yields = block_on(YieldTimes::new(3));
println!("YieldTimes suspended and resumed {yields} time(s)");

println!("\n== block_on drives an async block ==");
let total = block_on(async {
    let first = YieldTimes::new(1).await;
    let second = YieldTimes::new(2).await;
    first + second
});
println!("async block finished with {total}");

println!("\n== block_on parks the thread until the waker fires ==");
let start = Instant::now();
block_on(Delay::new(Duration::from_millis(50)));
println!("Delay completed after {:?}", start.elapsed());

println!("\n== MiniExecutor runs independent tasks ==");
let executor = MiniExecutor::new();
let completed = Arc::new(AtomicUsize::new(0));

for id in 1..=3 {
    let completed = Arc::clone(&completed);
    executor.spawn(async move {
        YieldTimes::new(id).await;
        completed.fetch_add(1, Ordering::SeqCst);
        println!("task {id} completed after {id} yield(s)");
    });
}

executor.run();
println!("{}", task(s) completed", completed.load(Ordering::SeqCst));
}

```

Intuition

The demonstration program printed:

```

== block_on drives a hand-written future ==
YieldTimes suspended and resumed 3 time(s)

== block_on drives an async block ==
async block finished with 3

== block_on parks the thread until the waker fires ==
Delay completed after 50.291507ms

== MiniExecutor runs independent tasks ==
task 1 completed after 1 yield(s)
task 2 completed after 2 yield(s)
task 3 completed after 3 yield(s)
3 task(s) completed

```

The executor queue while running the three tasks

pop	task	poll result	wake	queue after
1	task 1	Pending, no yields remain	re-enqueue 1	[2, 3, 1]
2	task 2	Pending	re-enqueue 2	[3, 1, 2]

pop	task	poll result	wake	queue after
3	task 3	Pending	re-enqueue 3	[1, 2, 3]
4	task 1	Ready, prints	none	[2, 3]
5	task 2	Pending	re-enqueue 2	[3, 2]
6	task 3	Pending	re-enqueue 3	[2, 3]
7	task 2	Ready, prints	none	[3]
8	task 3	Pending	re-enqueue 3	[3]
9	task 3	Ready, prints	none	[]

The round-robin order in the table explains why task 1 finishes first even though all three started together.

Time and Space Complexity

Operation	Cost
<code>block_on</code>	one poll per wake-up; the thread sleeps between them
<code>YieldTimes</code>	<code>times + 1</code> polls
<code>Delay::new</code>	one operating-system thread per delay
<code>MiniExecutor::run</code>	one poll and two queue lock acquisitions per wake-up
memory per task	one <code>Arc<Task></code> holding the boxed future, a mutex, and a flag

Limitations

MiniExecutor::run returns while tasks are still waiting. If a spawned task awaits a `Delay`, its first poll returns `Pending` and the queue empties before the timer fires, so `run` returns and the task never completes. `block_on` parks instead, and the `Delay` test therefore uses `block_on`. A complete executor parks on an event source when the queue is empty and has live tasks.

One thread per timer. `Delay` costs an operating-system thread, which defeats the purpose of an async runtime. Real runtimes keep one timer structure, such as a hashed wheel, and fire wakers from the event loop.

A reference cycle can leak tasks. Each `Task` holds an `Arc` of the queue, and the queue holds `Arc<Task>` values. If a `MiniExecutor` is dropped while tasks are still queued, the queue and those tasks keep each other alive and are never freed. Storing a `Weak` reference to the queue in each task breaks the cycle.

No `JoinHandle` and no cancellation. Spawned tasks return `()`, and the caller cannot wait for one or cancel it. Results must be communicated through shared state, as the tests do with atomics.

Single-threaded execution. `run` polls on the calling thread. The `Send` bounds are in place for a multi-threaded version, which would need several threads popping from the queue and, for good performance, per-thread queues with work stealing.

Summary

- A future returns `Poll::Ready` when done or `Poll::Pending` after arranging for its waker to be called; a runtime polls it again only after that call.
- `Pin` guarantees a future will not move, which self-referential `async` state machines require, and `Box::pin` provides it.
- `block_on` pairs a poll loop with a waker that unparks the thread, and `thread::park` cannot lose an early wake-up.
- A hand-written future that returns `Pending` must store or call the waker, under the same lock that the waking side uses.
- An executor is a queue of tasks whose wakers re-enqueue them; `Send + 'static` bounds on `spawn` follow from storing tasks and waking them from other threads.

References

- The Rust Async Book, [Under the hood: executing futures and tasks](#).
- Standard library, `std::future::Future`, `std::task::Wake`, and `std::pin`.
- Standard library, `std::thread::park`, on the unpark token.
- Mara Bos, *Rust Atomics and Locks*, O'Reilly Media, 2023, Chapter 1, on thread parking.

Retry with Backoff and Jitter

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/retry.rs`. Test it with `cargo test retry`.

Problem Statement

Write `retry(policy, operation, unit_random)` that calls `operation` until one of the following happens:

- it succeeds, and the value is returned;
- it fails with an error that retrying cannot fix, and the error is returned at once;
- the policy's maximum number of attempts is reached, and the last error is returned.

Between attempts, wait for a delay that grows exponentially up to a cap, is randomised according to the policy, and is never shorter than a delay the server requested.

Designing a Solution

Classify errors by type. A `Retryable` trait with `is_retryable` and an optional `retry_after` lets the loop ask each error what to do. `ApiError` implements it: rate limiting and 5xx server errors are retryable, while `NotFound`, `Unauthorized`, and `InvalidRequest` are not, because waiting does not change the outcome.

Exponential backoff with a cap. The delay before retry `k`, counting from zero, is `base × 2k`, limited to `max_delay`:


```
base = 100 ms, max = 1 s

retry k    base × 2k    capped
0          100 ms      100 ms
1          200 ms      200 ms
2          400 ms      400 ms
3          800 ms      800 ms
4         1600 ms     1000 ms
```

Full jitter. With no randomness, clients that failed at the same moment retry at the same moments. Full jitter chooses the delay uniformly between zero and the capped backoff, which spreads retries across the whole interval. Analysis published by AWS found that full jitter completed the same work with fewer calls than equal jitter or no jitter.

Inject the random source. The loop takes `unit_random: impl FnMut() -> f64`. Tests pass `|| 0.0` and get deterministic delays; production code passes a real generator. The crate has no dependency on a random-number library.

Retry-After is a floor. When a server sends `429 Too Many Requests` with a `Retry-After` value, retrying sooner only earns another 429. The loop waits for the larger of the server's value and its own backoff.

Implementation

```
/// Retry with exponential backoff, full jitter, and `Retry-After`.
///
/// Every cloud SDK retries. Done badly it makes an outage worse: synchronized
/// retries from thousands of clients form a thundering herd, and retrying a
/// non-idempotent `POST` can create duplicate workloads. This module models the
/// policy explicitly.
///
/// Two details that interviews probe:
///
/// - Jitter. Full jitter picks a random delay in `[0, capped_backoff]` so
///   clients spread out instead of retrying in lockstep.
/// - Retry-After. When the server tells you when to come back, wait at
///   least that long; never retry sooner than the server asked.
///
/// The random source is injected as a closure so the delay math is deterministic
/// under test.

use crate::problems::state_machine::ApiError;
use std::time::Duration;

/// An error that knows whether retrying could help.
pub trait Retryable {
    /// `true` if the operation is worth retrying.
    fn is_retryable(&self) -> bool;

    /// A server-provided minimum delay, such as HTTP `Retry-After`.
    fn retry_after(&self) -> Option<Duration> {
        None
    }
}
```

```
impl Retryable for ApiError {
    fn is_retryable(&self) -> bool {
        ApiError::is_retryable(self)
    }

    fn retry_after(&self) -> Option<Duration> {
        match self {
            ApiError::RateLimited { retry_after } => Some(*retry_after),
            _ => None,
        }
    }
}
```

`retry_after` has a default body returning `None`, so an error type that has no server-provided delay implements only `is_retryable`.

`impl Retryable for ApiError` calls `ApiError::is_retryable(self)`, the inherent method from chapter 19, which has the same name as the trait method being defined. Method-call syntax would reach the same function, because inherent methods take priority over trait methods, but a reader would have to know that rule to see that `self.is_retryable()` does not recurse. The path form states the target directly.

```
/// How randomness is applied to the backoff.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Jitter {
    /// Delay is exactly the capped exponential backoff.
    None,
    /// Delay is `unit_random * capped_backoff` (AWS "full jitter").
    Full,
}

/// Backoff configuration.
#[derive(Debug, Clone, Copy)]
pub struct RetryPolicy {
    /// Total attempts, including the first one. Must be at least 1.
    pub max_attempts: u32,
    /// Delay before the first retry.
    pub base_delay: Duration,
    /// Upper bound on a single delay.
    pub max_delay: Duration,
    pub jitter: Jitter,
}

impl RetryPolicy {
    pub fn new(
        max_attempts: u32,
        base_delay: Duration,
        max_delay: Duration,
        jitter: Jitter,
    ) -> Self {
        assert!(max_attempts > 0, "max_attempts must be > 0");
        Self {
            max_attempts,
            base_delay,
            max_delay,
            jitter,
        }
    }
}
```

```

/// Capped exponential backoff for a zero-based retry attempt:
/// `min(base * 2^attempt, max_delay)`.
pub fn exponential_delay(&self, attempt: u32) -> Duration {
    let shift = attempt.min(31);
    let factor = 1u32 << shift;
    self.base_delay.saturating_mul(factor).min(self.max_delay)
}

/// Backoff for `attempt` with `unit_random` in `[0, 1)` applied when jitter
/// is enabled.
pub fn delay_for(&self, attempt: u32, unit_random: f64) -> Duration {
    let capped = self.exponential_delay(attempt);
    match self.jitter {
        Jitter::None => capped,
        Jitter::Full => capped.mul_f64(unit_random.clamp(0.0, 1.0)),
    }
}
}

```

`exponential_delay` computes 2^{attempt} as `1u32 << attempt.min(31)`. The `min` keeps the shift below 32, where shifting a `u32` would overflow. `Duration::saturating_mul` returns `Duration::MAX` instead of panicking when the product is too large, and `.min` then applies the cap. Attempt 50 therefore produces `max_delay`, as the test checks.

`delay_for` clamps `unit_random` to `[0.0, 1.0]` before `Duration::mul_f64`, which panics on a negative or non-finite factor. A faulty random source cannot crash the retry loop.

`RetryPolicy::new` asserts `max_attempts > 0`. The fields are also `pub`, so a caller can build a policy with a struct literal and bypass the assertion.

```

/// Retry `operation` until it succeeds, fails permanently, or the policy is
/// exhausted.
///
/// `unit_random` must return values in `[0, 1)`, pass `|| 0.0` for deterministic
/// tests or a real RNG in production.
pub fn retry<T, E, F, R>(policy: &RetryPolicy, mut operation: F, mut unit_random: R) -
> Result<T, E>
where
    F: FnMut(u32) -> Result<T, E>,
    E: Retryable,
    R: FnMut() -> f64,
{
    let mut attempt = 0;
    loop {
        match operation(attempt) {
            Ok(value) => return Ok(value),
            Err(error) => {
                let exhausted = attempt + 1 >= policy.max_attempts;
                if exhausted || !error.is_retryable() {
                    return Err(error);
                }

                let backoff = policy.delay_for(attempt, unit_random());
                // Honor the server's hint, but never retry sooner than our own
                // backoff says is healthy.
                let delay = error

```

```

        .retry_after()
        .map_or(backoff, |hint| hint.max(backoff));

    std::thread::sleep(delay);
    attempt += 1;
}
}
}
}

```

The operation is `FnMut(u32) -> Result<T, E>`. `FnMut` allows it to update state between calls, such as a counter in a test, and the `u32` argument tells it which attempt it is running, which is useful for logging.

`attempt + 1 >= policy.max_attempts` checks whether the attempt that just failed was the last one. With `max_attempts = 3`, attempts 0, 1, and 2 run, and the error from attempt 2 is returned without a final sleep.

`error.retry_after().map_or(backoff, |hint| hint.max(backoff))` returns the backoff when there is no hint and the larger of the two when there is.

```

#[cfg(test)]
mod tests {
    use super::*;

    fn policy(max_attempts: u32, jitter: Jitter) -> RetryPolicy {
        RetryPolicy::new(
            max_attempts,
            Duration::from_millis(100),
            Duration::from_millis(1_000),
            jitter,
        )
    }

    #[test]
    fn exponential_backoff_doubles_until_capped() {
        let policy = policy(6, Jitter::None);
        assert_eq!(policy.exponential_delay(0), Duration::from_millis(100));
        assert_eq!(policy.exponential_delay(1), Duration::from_millis(200));
        assert_eq!(policy.exponential_delay(2), Duration::from_millis(400));
        assert_eq!(policy.exponential_delay(3), Duration::from_millis(800));
        // Capped from here on.
        assert_eq!(policy.exponential_delay(4), Duration::from_millis(1_000));
        assert_eq!(policy.exponential_delay(50), Duration::from_millis(1_000));
    }

    #[test]
    fn full_jitter_scales_the_delay() {
        let policy = policy(6, Jitter::Full);
        assert_eq!(policy.delay_for(2, 0.0), Duration::ZERO);
        assert_eq!(policy.delay_for(2, 0.5), Duration::from_millis(200));
        // Values above 1.0 are clamped rather than overflowing.
        assert_eq!(policy.delay_for(2, 9.9), Duration::from_millis(400));
    }

    #[test]
    fn retries_then_succeeds() {
        let policy = RetryPolicy::new(5, Duration::ZERO, Duration::ZERO, Jitter::None);
        let mut attempts = 0;
        let result: Result<u32, ApiError> = retry(

```

```

        &policy,
        |_| {
            attempts += 1;
            if attempts < 3 {
                Err(ApiError::Server {
                    status: 503,
                    message: "temporarily unavailable".into(),
                })
            } else {
                Ok(42)
            }
        },
        || 0.0,
    );

    assert_eq!(result, Ok(42));
    assert_eq!(attempts, 3);
}

#[test]
fn non_retryable_errors_return_immediately() {
    let policy = RetryPolicy::new(5, Duration::ZERO, Duration::ZERO, Jitter::None);
    let mut attempts = 0;
    let result: Result<(), ApiError> = retry(
        &policy,
        |_| {
            attempts += 1;
            Err(ApiError::NotFound {
                resource_id: "gpu-0".into(),
            })
        },
        || 0.0,
    );

    assert!(result.is_err());
    assert_eq!(attempts, 1);
}

#[test]
fn exhausted_policy_returns_the_last_error() {
    let policy = RetryPolicy::new(3, Duration::ZERO, Duration::ZERO, Jitter::None);
    let mut attempts = 0;
    let result: Result<(), ApiError> = retry(
        &policy,
        |_| {
            attempts += 1;
            Err(ApiError::Server {
                status: 500,
                message: "boom".into(),
            })
        },
        || 0.0,
    );

    assert!(result.is_err());
    assert_eq!(attempts, 3);
}

#[test]
fn retry_after_is_honored_as_a_floor() {
    let policy = policy(2, Jitter::None);
    let error = ApiError::RateLimited {
        retry_after: Duration::from_secs(5),
    };
    let backoff = policy.delay_for(0, 0.0);
    let chosen = error
        .retry_after()

```

```

        .map_or(backoff, |hint| hint.max(backoff));
    assert_eq!(chosen, Duration::from_secs(5));
}
}

```

The loop tests use a policy with zero delays, so they run without sleeping.

Intuition

retries_then_succeeds with max_attempts = 5 and zero delays

at-tempt	operation result	exhausted?	retry-able?	action
0	Err(Server { status: 503, .. })	1 >= 5 is false	yes	sleep 0, attempt = 1
1	Err(Server { status: 503, .. })	2 >= 5 is false	yes	sleep 0, attempt = 2
2	Ok(42)			return Ok(42)

Delays for a policy of base 100 ms and cap 1 s with full jitter

attempt	capped backoff	unit_random	delay	server Retry-After	chosen
0	100 ms	0.37	37 ms	none	37 ms
1	200 ms	0.90	180 ms	none	180 ms
2	400 ms	0.10	40 ms	5 s	5 s

Time and Space Complexity

Operation	Cost
exponential_delay, delay_for	O(1)
retry	at most max_attempts calls to operation, and the sum of the chosen delays in wall-clock time
memory	none beyond what operation uses

With full jitter the expected total delay is half the sum of the capped back-offs.

Limitations

retry blocks the thread. The loop calls `std::thread::sleep`. In an async program that would stall an executor thread; an async version awaits a timer such as `tokio::time::sleep` instead, as chapter 58 does.

The `Retry-After` test does not call `retry`. `retry_after_is_honored_as_a_floor` repeats the expression from the loop and checks its value, because calling `retry` with a five-second hint would make the test sleep for five seconds. The loop's use of the hint is therefore untested. Injecting the sleep function, as the random source already is, would let a test record the requested delays without waiting.

`Retry-After` is not capped. A server that answers `Retry-After: 3600` makes the loop sleep for an hour. A client usually caps the hint, or gives up and reports it to the caller, rather than blocking that long.

Retryability ignores the request. A 503 is retryable for a `GET` and dangerous for a `POST` that may already have created a resource on the server. The policy knows only the error. Chapter 54's `Method::is_idempotent`, or an idempotency key as in chapter 34, supplies the missing information.

No overall deadline. A caller that needs an answer within two seconds cannot say so; the policy bounds the number of attempts, not the elapsed time.

Summary

- A `Retryable` trait lets the loop ask each error whether waiting can help and whether the server requested a delay.
- Capped exponential backoff is `min(base × 2^attempt, max)`; `saturating_mul` and a bounded shift keep the arithmetic from overflowing.
- Full jitter chooses a delay uniformly in `[0, capped]`, which spreads retries from many clients across time.
- Injecting the random source makes the delay arithmetic deterministic under test; the sleep function deserves the same treatment.
- A server's `Retry-After` is a lower bound, and retries of non-idempotent requests need information the error type does not carry.

References

- Marc Brooker, "Exponential Backoff And Jitter", AWS Architecture Blog, 2015.
- RFC 9110, *HTTP Semantics*, 2022, Section 10.2.3, `Retry-After`.
- Google Cloud, "Retry strategy", Cloud Storage documentation, on truncated exponential backoff.
- Standard library, `Duration::saturating_mul` and `Duration::mul_f64`.

An HTTP Client with Retries and Caching

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/fun_network_call.rs` and `src/bin/request_and_tokio.rs`. Both send a request to `httpbin.org` and need network access. Run them with `cargo run --bin fun_network_call` and `cargo run --bin request_and_tokio`.

Problem Statement

Create an item on a REST API with `POST /post` and a JSON body. The client must:

- retry a failed attempt with exponential backoff, up to three attempts;
- return the stored response, without sending the request again, when the same idempotency key is used a second time;
- report a final failure to the caller.

The chapter reads two versions of the same client. The first uses only the standard library and writes the HTTP request by hand. The second uses `request` on the `tokio` runtime. Reading them side by side shows which concerns a library absorbs and which remain the caller's responsibility.

Designing a Solution

Both clients layer the same three functions:

```
execute(key, request)
|
+-- cache: return the stored body if key was seen
|
```

```

+-- retry: call send up to max_attempts times with growing delays
    |
    +-- send: one HTTP exchange

```

The order of the layers is a design decision. The cache is outside the retry loop, so a successful response is stored once, however many attempts it took, and a cached key never reaches the network.

A client-side cache protects against the caller repeating a call. It cannot protect against the case chapter 34 began with, where the server performed the work and the response was lost, because the client never received a response to cache. That case needs the server's help: the client sends the key in an `Idempotency-Key` header, and the server stores the result under that key. The `request` version sends the header; the standard-library version does not.

Implementation

The standard-library client

```

use std::collections::HashMap;
use std::io::{Read, Write};
use std::net::TcpStream;
use std::sync::Mutex;
use std::time::Duration;

type RuntimeError = Box<dyn std::error::Error>;

enum RequestType {
    GET    { path: String, headers: Vec<(String, String)> },
    OPTION { path: String, headers: Vec<(String, String)> },
    POST   { path: String, headers: Vec<(String, String)>, body: String },
    PUT    { path: String, headers: Vec<(String, String)>, body: String },
}

struct IdemCache {
    store: Mutex<HashMap<String, String>>,
}

impl IdemCache {
    fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    fn get_or_insert<F>(&self, key: &str, f: F) -> Result<String, RuntimeError>
    where
        F: FnOnce() -> Result<String, RuntimeError>,
    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned"?;
        if let Some(v) = store.get(key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key.to_string(), v.clone());
        Ok(v)
    }
}

```

```

struct RetryPolicy {
    max_attempts: u32,
    base_delay: Duration,
    max_delay: Duration,
}

impl RetryPolicy {
    fn new() -> Self {
        Self {
            max_attempts: 3,
            base_delay: Duration::from_millis(100),
            max_delay: Duration::from_secs(5),
        }
    }

    fn execute<F, T>(&self, mut f: F) -> Result<T, RuntimeError>
    where
        F: FnMut() -> Result<T, RuntimeError>,
    {
        let mut attempt = 0;
        loop {
            attempt += 1;
            match f() {
                Ok(v) => return Ok(v),
                Err(e) if attempt >= self.max_attempts => return Err(e),
                Err(_) => {
                    let delay = (self.base_delay * 2u32.pow(attempt - 1)).min(self.max_
delay);
                    std::thread::sleep(delay);
                }
            }
        }
    }
}

/// -- Build a HTTP client with those strategies

struct HttpClient {
    host: String,
    port: u16,
    cache: IdemCache,
    retry: RetryPolicy,
}

impl HttpClient {
    fn new(host: &str, port: u16) -> Self {
        Self {
            host: host.to_string(),
            port,
            cache: IdemCache::new(),
            retry: RetryPolicy::new(),
        }
    }

    fn execute(&self, key: &str, req: &RequestType) -> Result<String, RuntimeError> {
        self.cache.get_or_insert(key, || self.retry.execute(|| self.send(req)))
    }

    fn send(&self, req: &RequestType) -> Result<String, RuntimeError> {
        let (method, path, headers, body) = match req {
            RequestType::GET { path, headers } => ("GET", path, headers, N
one),
            RequestType::OPTION { path, headers } => ("OPTIONS", path, headers, N
one),
            RequestType::POST { path, headers, body } => ("POST", path, headers, S
ome(body)),

```

```

        RequestType::PUT    { path, headers, body } => ("PUT",    path, headers, S
ome(body)),
    };

    let mut raw = format!(
        "{method} {path} HTTP/1.1\r\nHost: {}\r\nConnection: close\r\n",
        self.host
    );
    for (k, v) in headers { raw += &format!("{k}: {v}\r\n"); }
    if let Some(b) = body { raw += &format!("Content-Length: {}\r\n", b.len()); }
    raw += "\r\n";
    if let Some(b) = body { raw += b; }

    let mut stream = TcpStream::connect((self.host.as_str(), self.port))?;
    stream.write_all(raw.as_bytes())?;
    let mut resp = String::new();
    stream.read_to_string(&mut resp)?;
    Ok(resp.split_once("\r\n\r\n").map(|(_, b)| b).unwrap_or("").to_string())
}

// --- The actual REST call ---

fn create_item(client: &HttpClient, item_name: &str) -> Result<String, RuntimeError> {
    client.execute("create-demo-001", &RequestType::POST {
        path: "/post".into(),
        headers: vec![
            ("Content-Type".into(), "application/json".into()),
            ("Accept".into(), "application/json".into()),
        ],
        body: format!(r#"{"name":"{item_name}"}"#),
    })
}

fn main() -> Result<(), RuntimeError> {
    let client = HttpClient::new("httpbin.org", 80);
    println!("created:\n{}", create_item(&client, "demo")?);
    Ok(())
}

```

`RuntimeError` is `Box<dyn std::error::Error>`, the conventional error type for a program's top level. Any error type, and any `&str` or `String`, converts into it with `?`, which is how `map_err(|_| "lock poisoned")?` turns a poisoned lock into an error.

`IdemCache::get_or_insert` holds the mutex for its whole body, including the call to `f`. The same design and the same trade-off appear in chapter 34: a second caller with the same key waits and then sees the stored value, and every caller, whatever its key, waits while one request is in flight.

`RetryPolicy::execute` retries every error. The guard `Err(e) if attempt >= self.max_attempts` returns the last error, and the final arm sleeps for `base_delay × 2(attempt - 1)`, capped at `max_delay`.

`execute` composes the layers in one expression: `self.cache.get_or_insert(key, || self.retry.execute(|| self.send(req)))`.

`send` deconstructs the request enum into a method string, a path, headers, and an optional body with a single `match`. It writes the request line, a `Host` header, and `Connection: close`, which tells the server to close the connection after the response. That lets the client read the whole response with `read_to_string`, which returns when the server closes, instead of parsing `Content-Length` or chunked framing. The body follows the empty line, with a `Content-Length` header computed from its byte length.

`resp.split_once("\r\n\r\n")` separates the response head from the body, and the client returns the body.

The request and tokio client

```
use std::{collections::HashMap, sync::Mutex, time::Duration};
use request::{Client, Method, StatusCode};
use serde_json::Value;

type Error = Box<dyn std::error::Error + Send + Sync>;

const RETRY: [StatusCode; 5] = [
    StatusCode::TOO_MANY_REQUESTS,
    StatusCode::INTERNAL_SERVER_ERROR,
    StatusCode::BAD_GATEWAY,
    StatusCode::SERVICE_UNAVAILABLE,
    StatusCode::GATEWAY_TIMEOUT,
];

struct Config {
    base: String,
    max_attempts: u32,
    base_delay: Duration,
    max_delay: Duration,
    timeout: Duration,
}

struct Api {
    http: Client,
    cache: Mutex<HashMap<String, String>>,
    cfg: Config,
}

impl Api {
    fn new(cfg: Config) -> Result<Self, Error> {
        Ok(Self {
            http: Client::builder().timeout(cfg.timeout).build()?,
            cache: Mutex::new(HashMap::new()),
            cfg,
        })
    }

    fn cached(&self, key: &str) -> Option<String> {
        self.cache
            .lock()
            .unwrap_or_else(|e| e.into_inner())
            .get(key)
            .cloned()
    }

    fn store(&self, key: &str, text: &str) {
```

```

        self.cache
            .lock()
            .unwrap_or_else(|e| e.into_inner())
            .insert(key.into(), text.into());
    }

    async fn call(
        &self,
        key: &str,
        method: Method,
        path: &str,
        body: Option<&Value>,
    ) -> Result<String, Error> {
        match self.cached(key) {
            Some(hit) => return Ok(hit),
            None => {}
        }

        let url = format!("{}", self.cfg.base, path);
        let mut delay = self.cfg.base_delay;
        let mut attempt = 0;

        loop {
            attempt += 1;

            let req = self
                .http
                .request(method.clone(), &url)
                .header("Idempotency-Key", key)
                .header("Accept", "application/json");
            let req = match body {
                Some(b) => req.json(b),
                None => req,
            };

            let resp = req.send().await?;
            let status = resp.status();
            let text = resp.text().await?;

            match status {
                s if s.is_success() => {
                    self.store(key, &text);
                    return Ok(text);
                }
                s if RETRY.contains(&s) && attempt < self.cfg.max_attempts => {
                    tokio::time::sleep(delay).await;
                    delay = (delay * 2).min(self.cfg.max_delay);
                }
                _ => return Err(format!("HTTP {s} after {attempt} attempts: {text}").into())
            }
        }
    }

    to()),
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let api = Api::new(Config {
        base: "https://httpbin.org".into(),
        max_attempts: 3,
        base_delay: Duration::from_millis(100),
        max_delay: Duration::from_secs(5),
        timeout: Duration::from_secs(15),
    })?;

    let body = serde_json::json!({ "name": "demo" });
    let out = api

```

```

        .call("create-demo-001", Method::POST, "/post", Some(&body))
        .await?;
    println!("created:\n{out}");
    Ok(())
}

```

Error is `Box<dyn std::error::Error + Send + Sync>`. The additional bounds let the error cross threads, which `tokio`'s multi-threaded runtime can require of values held across an `.await`.

`Client::builder().timeout(cfg.timeout).build()? sets a total timeout for each request. Building one Client and reusing it keeps a connection pool, so repeated calls to the same host reuse TCP and TLS connections.`

The cache uses `std::sync::Mutex` in async code. That is correct here, because `cached` and `store` take and release the lock without awaiting in between. A standard mutex held across an `.await` would block the executor thread. `lock().unwrap_or_else(|e| e.into_inner())` recovers the guard from a poisoned lock, which is reasonable for a cache whose entries are independent strings.

The retry loop sends the `Idempotency-Key` header on every attempt, so a server that supports idempotency keys recognises a retried request after a lost response. It reads the status and body, returns and caches a success, sleeps and doubles the delay for a status in `RETRY` while attempts remain, and otherwise returns an error that includes the status, the attempt count, and the response text. `tokio::time::sleep(delay).await` yields to the runtime rather than blocking a thread.

`#[tokio::main]` wraps `main` in a runtime, and `serde_json::json!` builds the body from JSON-like syntax with correct escaping.

Intuition

`create_item` in the standard-library client, when the first attempt fails to connect

layer	action
<code>execute("create-demo-001", ..)</code>	locks the cache; key not present
<code>retry.execute</code> , attempt 1	<code>send</code> fails in <code>TcpStream::connect</code> ; attempt 1 < 3, sleep 100 ms

layer	action
<code>retry.execute</code> , attempt 2	send connects, writes the request, reads to close; returns the body
cache	stores the body under <code>create-demo-001</code> , releases the lock
second <code>execute</code> with the same key	returns the stored body without connecting

Api::call in the request client, for a 503 followed by a 200

attempt	status	action
1	503	in <code>RETRY</code> , $1 < 3$: sleep 100 ms, delay becomes 200 ms
2	200	success: store the text under the key and return it

Time and Space Complexity

Operation	Cost
cache hit	one lock acquisition and one <code>String</code> clone
cache miss	up to three HTTP exchanges plus up to 100 ms + 200 ms of backoff
memory	one stored body per distinct key, never evicted

Limitations

The standard-library client treats every HTTP response as success. `send` returns the body of any response, including `500 Internal Server Error`. A server error therefore reaches the caller as `Ok`, is never retried, and is stored in the cache, so every later call with the same key returns the error page. Checking the status line before returning closes the gap:

```
let (head, body) = resp.split_once("\r\n\r\n").ok_or("malformed response")?;
let status: u16 = head
    .split(' ')
    .nth(1)
    .and_then(|code| code.parse().ok())
    .ok_or("malformed status line")?;
if !(200..300).contains(&status) {
    return Err(format!("HTTP {status}: {body}").into());
}
Ok(body.to_string())
```


The request client does not retry transport failures. `req.send().await?` returns at once when the connection fails or the request times out, which are the failures a retry is most likely to fix. Only HTTP status codes in `RETRY` are retried. Matching on the send result retries timeouts and connection errors as well:

```
let resp = match req.send().await {
  Ok(resp) => resp,
  Err(error)
    if (error.is_timeout() || error.is_connect()) && attempt < self.cfg.max_at
    tempts =>
    {
      tokio::time::sleep(delay).await;
      delay = (delay * 2).min(self.cfg.max_delay);
      continue;
    }
  Err(error) => return Err(error.into()),
};
```

Neither client adds jitter or honours `Retry-After`. Both double a fixed delay, so many clients that fail together retry together, and a 429 response's `Retry-After` header is ignored. Chapter 57's policy provides both.

The standard-library client retries non-idempotent requests without a server-side key. A `POST` whose response is lost is sent again with nothing that lets the server recognise it, which can create a duplicate item. The request client's `Idempotency-Key` header addresses this when the server supports it.

The cache check and the request are not atomic in the async client. Two concurrent calls with the same key can both miss the cache and both send the request. The standard-library client avoids this by holding its lock across the request, at the cost of serializing every request.

No timeouts in the standard-library client. `TcpStream::connect`, `write_all`, and `read_to_string` can each block indefinitely. `TcpStream::connect_timeout`, `set_read_timeout`, and `set_write_timeout` bound them.

The JSON body is built with `format!`. In `create_item`, a name containing `"` or `\` produces invalid JSON. The request version's `json!` macro escapes correctly.

Compiler and Clippy warnings. The variants `GET`, `OPTION`, and `PUT` are never constructed, and variant names in all capitals trigger Clippy's `upper_case_acronyms`; the idiomatic names are `Get`, `Options`, `Post`, and `Put`. In the async client, `match self.cached(key) { Some(hit) => return`

`Ok(hit), None => {} }` is clearer as `if let Some(hit) = self.cached(key) { return Ok(hit); }`, as Clippy's `single_match` lint suggests.

No response framing. Reading until `close` works only because the request asks for `Connection: close`. The client cannot reuse connections, and it would return the raw chunk-size lines of a chunked response as part of the body.

Summary

- A client composes a cache, a retry policy, and a single-exchange function; placing the cache outside the retry loop stores each result once.
- A hand-written HTTP/1.1 client writes the request line, `Host`, framing headers, an empty line, and the body, and `Connection: close` lets it read the response to the end of the stream.
- A client-side cache handles repeated calls; only a server-side idempotency key handles a response that was lost after the server did the work.
- `request` provides connection pooling, timeouts, TLS, and JSON encoding; the caller still decides which statuses and transport errors to retry.
- A client must check the status before caching or returning a body, and must retry the transport failures that retries exist for.

References

- The `request` crate, `Client` and `Error`.
- The Tokio tutorial, [Shared state](#), on using `std::sync::Mutex` in async code.
- IETF HTTPAPI working group, *The Idempotency-Key HTTP Header Field*, Internet-Draft.
- RFC 9112, *HTTP/1.1*, 2022, Section 9.6, on `Connection: close`.

Part XVII: Interview Drills

Interview Drills

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/drills.rs`. Test it with `cargo test drills`.

Problem Statement

The other chapters read complete, tested modules. In an interview there is time for perhaps forty lines. The file `drills.rs` collects the version of ten exercises that fits that budget, written to be typed from memory:

Drill	Full treatment
Thread-safe counter with <code>Arc<Mutex></code>	Chapters 36 and 43
Scoped parallel sum	Chapters 31 and 43
Spin lock	Chapter 44
Semaphore	Chapter 45
Bounded blocking queue with <code>close</code>	Chapters 30 and 46
Thread pool	Chapter 21
Retry with exponential backoff	Chapter 57
Consistent hashing ring	Chapter 42
HTTP request head parser	Chapter 54
Newtype and routing enumeration	Chapters 35 and 55

Designing a Solution

Each drill should be written in 30 to 45 minutes without looking at the file, and then compared with it. After writing it, state the failure modes and how you would test them. The short versions keep the core mechanism of each design and drop everything a caller could live without in an interview: accessors, timeouts, typed errors, and most documentation.

Implementation

```

/// Short answers to the common prompts, sized for a live round.
///
/// Each item is a standalone implementation of roughly the length you would write
/// in an interview. The modules next to this one are reference versions with the
/// long explanations and full test suites; this file is what you type.

use std::cell::UnsafeCell;
use std::collections::VecDeque;
use std::collections::hash_map::DefaultHasher;
use std::hash::Hasher;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::{Arc, Condvar, Mutex, mpsc};
use std::thread::{self, JoinHandle};
use std::time::Duration;

// --- Thread-safe counter (Arc + Mutex) ---

pub fn count_parallel(threads: usize, per_thread: usize) -> usize {
    let counter = Arc::new(Mutex::new(0usize));
    let handles: Vec<_> = (0..threads)
        .map(|_| {
            let counter = Arc::clone(&counter);
            thread::spawn(move || {
                for _ in 0..per_thread {
                    *counter.lock().expect("poisoned") += 1;
                }
            })
        })
        .collect();

    for handle in handles {
        handle.join().expect("worker panicked");
    }
    *counter.lock().expect("poisoned")
}

// --- Scoped threads (borrow instead of Arc) ---

pub fn parallel_sum(values: &[i32]) -> i64 {
    let chunk_size = values.len().div_ceil(4).max(1);
    thread::scope(|scope| {
        values
            .chunks(chunk_size)
            .map(|chunk| {
                scope.spawn(move || chunk.iter().map(|value| i64::from(*value)).sum::<i
64>())
            })
            .map(|handle| handle.join().expect("worker panicked"))
    })
}

```

```

        .sum()
    })
}

// --- Spin lock from an atomic ---

pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}

unsafe impl<T: Send> Sync for SpinLock<T> {}

impl<T> SpinLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            value: UnsafeCell::new(value),
        }
    }

    pub fn lock(&self) -> SpinGuard<'_, T> {
        while self.locked.swap(true, Ordering::Acquire) {}
        std::hint::spin_loop();
    }
    SpinGuard { lock: self }
}

pub struct SpinGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

impl<T> Drop for SpinGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Ordering::Release);
    }
}

impl<T> Deref for SpinGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.lock.value.get() }
    }
}

impl<T> DerefMut for SpinGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.value.get() }
    }
}

// --- Semaphore (acquire blocks) ---

pub struct Semaphore {
    permits: Mutex<usize>,
    released: Condvar,
}

impl Semaphore {
    pub fn new(permits: usize) -> Self {
        Self {
            permits: Mutex::new(permits),
            released: Condvar::new(),
        }
    }
}

```

```

pub fn acquire(&self) {
    let mut permits = self.permits.lock().expect("poisoned");
    while *permits == 0 {
        permits = self.released.wait(permits).expect("poisoned");
    }
    *permits -= 1;
}

pub fn try_acquire(&self) -> bool {
    let mut permits = self.permits.lock().expect("poisoned");
    if *permits == 0 {
        false
    } else {
        *permits -= 1;
        true
    }
}

pub fn release(&self) {
    *self.permits.lock().expect("poisoned") += 1;
    self.released.notify_one();
}
}

// --- Bounded blocking queue (backpressure + close) ---

pub struct BlockingQueue<T> {
    inner: Mutex<Inner<T>>,
    not_empty: Condvar,
    not_full: Condvar,
}

struct Inner<T> {
    items: VecDeque<T>,
    capacity: usize,
    closed: bool,
}

impl<T> BlockingQueue<T> {
    pub fn new(capacity: usize) -> Self {
        Self {
            inner: Mutex::new(Inner {
                items: VecDeque::new(),
                capacity,
                closed: false,
            }),
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    pub fn push(&self, item: T) -> Result<(), T> {
        let mut inner = self.inner.lock().expect("poisoned");
        loop {
            if inner.closed {
                return Err(item);
            }
            if inner.items.len() < inner.capacity {
                inner.items.push_back(item);
                self.not_empty.notify_one();
                return Ok(());
            }
            inner = self.not_full.wait(inner).expect("poisoned");
        }
    }

    pub fn pop(&self) -> Option<T> {

```

```

        let mut inner = self.inner.lock().expect("poisoned");
        loop {
            if let Some(item) = inner.items.pop_front() {
                self.not_full.notify_one();
                return Some(item);
            }
            if inner.closed {
                return None;
            }
            inner = self.not_empty.wait(inner).expect("poisoned");
        }
    }

    pub fn close(&self) {
        let mut inner = self.inner.lock().expect("poisoned");
        inner.closed = true;
        drop(inner);
        self.not_empty.notify_all();
        self.not_full.notify_all();
    }
}

// --- Thread pool (fixed workers + channel) ---

pub struct ThreadPool {
    sender: Option<mpsc::Sender<Box<dyn FnOnce() + Send + 'static>>>,
    workers: Vec<JoinHandle<>>,
}

impl ThreadPool {
    pub fn new(size: usize) -> Self {
        let (sender, receiver) = mpsc::channel::<Box<dyn FnOnce() + Send + 'static>>>();
        let receiver = Arc::new(Mutex::new(receiver));
        let workers = (0..size)
            .map(|_| {
                let receiver = Arc::clone(&receiver);
                thread::spawn(move || {
                    loop {
                        let Ok(job) = receiver.lock().expect("poisoned").recv() else {
                            return;
                        };
                        job();
                    }
                })
            })
            .collect();
        Self {
            sender: Some(sender),
            workers,
        }
    }

    pub fn execute(&self, job: impl FnOnce() + Send + 'static) {
        self.sender
            .as_ref()
            .expect("pool dropped")
            .send(Box::new(job))
            .expect("worker gone");
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.sender.take());
        for worker in self.workers.drain(..) {
            let _ = worker.join();
        }
    }
}

```



```

    }
}

// --- Retry with exponential backoff ---

pub fn retry<T, E>(
    max_attempts: u32,
    mut operation: impl FnMut() -> Result<T, E>,
    retryable: impl Fn(&E) -> bool,
) -> Result<T, E> {
    let mut delay = Duration::from_millis(25);
    let mut attempt = 0;
    loop {
        match operation() {
            Ok(value) => return Ok(value),
            Err(error) => {
                attempt += 1;
                if attempt >= max_attempts || !retryable(&error) {
                    return Err(error);
                }
                thread::sleep(delay);
                delay = (delay * 2).min(Duration::from_secs(1));
            }
        }
    }
}

// --- Consistent hashing ring ---

#[derive(Default)]
pub struct HashRing {
    points: Vec<(u64, String)>,
}

impl HashRing {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn add(&mut self, node: &str, replicas: usize) {
        for replica in 0..replicas {
            self.points.push((hash(node, replica), node.to_string()));
        }
        self.points.sort_unstable_by_key(|(point, _)| *point);
    }

    pub fn get(&self, key: &str) -> Option<&str> {
        if self.points.is_empty() {
            return None;
        }
        let hash = hash(key, 0);
        let index = self.points.partition_point(|(point, _)| *point <= hash) % self.points.len();
        Some(self.points[index].1.as_str())
    }
}

fn hash(input: &str, replica: usize) -> u64 {
    let mut hasher = DefaultHasher::new();
    hasher.write(input.as_bytes());
    hasher.write_usize(replica);
    hasher.finish()
}

// --- HTTP request head ---

pub struct RequestHead {

```

```

    pub method: String,
    pub target: String,
    pub headers: Vec<(String, String)>,
}

pub fn parse_request_head(input: &str) -> Option<RequestHead> {
    let mut lines = input.split("\r\n");
    let mut request_line = lines.next()?.split(' ');
    let method = request_line.next()?.to_string();
    let target = request_line.next()?.to_string();
    if request_line.next()? != "HTTP/1.1" {
        return None;
    }

    let mut headers = Vec::new();
    for line in lines {
        if line.is_empty() {
            break;
        }
        let (name, value) = line.split_once(':')?;
        headers.push((name.trim().to_ascii_lowercase(), value.trim().to_string()));
    }
    Some(RequestHead {
        method,
        target,
        headers,
    })
}

// --- Newtype and enum ADT ---

pub struct Port(u16);

impl Port {
    pub fn new(value: u16) -> Result<Self, &'static str> {
        (1..=u16::MAX)
            .contains(&value)
            .then_some(Self(value))
            .ok_or("port must be non-zero")
    }

    pub fn get(self) -> u16 {
        self.0
    }
}

pub enum Route {
    Health,
    List,
    ById(String),
}

impl Route {
    pub fn parse(path: &str) -> Option<Self> {
        match path {
            "/healthz" => Some(Self::Health),
            "/v1/gpu-workloads" => Some(Self::List),
            _ => path
                .strip_prefix("/v1/gpu-workloads/")
                .map(|id| Self::ById(id.to_string())),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

```

```

use std::sync::atomic::AtomicUsize;

#[test]
fn counter_scoped_sum_and_spin_lock() {
    assert_eq!(count_parallel(4, 100), 400);
    assert_eq!(parallel_sum(&[1, 2, 3, 4]), 10);

    let lock = Arc::new(SpinLock::new(0usize));
    let handles: Vec<_> = (0..4)
        .map(|_| {
            let lock = Arc::clone(&lock);
            thread::spawn(move || {
                for _ in 0..100 {
                    *lock.lock() += 1;
                }
            })
        })
        .collect();
    for handle in handles {
        handle.join().expect("worker panicked");
    }
    assert_eq!(*lock.lock(), 400);
}

#[test]
fn semaphore_queue_and_pool() {
    let semaphore = Semaphore::new(1);
    assert!(semaphore.try_acquire());
    assert!(!semaphore.try_acquire());
    semaphore.release();
    assert!(semaphore.try_acquire());

    let queue = BlockingQueue::new(1);
    assert_eq!(queue.push(1), Ok(()));
    queue.close();
    assert_eq!(queue.pop(), Some(1));
    assert_eq!(queue.pop(), None);
    assert_eq!(queue.push(2), Err(2));

    let pool = ThreadPool::new(3);
    let counter = Arc::new(AtomicUsize::new(0));
    for _ in 0..100 {
        let counter = Arc::clone(&counter);
        pool.execute(move || {
            counter.fetch_add(1, Ordering::SeqCst);
        });
    }
    drop(pool);
    assert_eq!(counter.load(Ordering::SeqCst), 100);
}

#[test]
fn retry_ring_http_and_adt() {
    let mut attempts = 0;
    let ok = retry(
        3,
        || {
            attempts += 1;
            if attempts < 3 { Err("transient") } else { Ok(7) }
        },
        |_| true,
    );
    assert_eq!(ok, Ok(7));

    let mut attempts = 0;
    let bad: Result<(), &str> = retry(
        3,

```

```

        || {
            attempts += 1;
            Err("permanent")
        },
        |_| false,
    );
    assert!(bad.is_err());
    assert_eq!(attempts, 1);

    let mut ring = HashRing::new();
    ring.add("a", 64);
    ring.add("b", 64);
    let before = ring.get("job-1").map(str::to_string);
    ring.add("c", 64);
    let after = ring.get("job-1").map(str::to_string);
    assert!(after == before || after.as_deref() == Some("c"));

    let head =
        parse_request_head("GET /v1/gpu-workloads HTTP/1.1\r\nHost: api\r\n\r\n").e
        xpect("valid");
    assert_eq!(
        (head.method.as_str(), head.target.as_str()),
        ("GET", "/v1/gpu-workloads")
    );
    assert_eq!(head.headers[0], ("host".to_string(), "api".to_string()));
    assert!(parse_request_head("GET / HTTP/2\r\n\r\n").is_none());

    assert!(Port::new(8080).is_ok());
    assert!(Port::new(0).is_err());
    assert!(matches!(Route::parse("/healthz"), Some(Route::Health)));
    assert!(matches!(
        Route::parse("/v1/gpu-workloads/wl-1"),
        Some(Route::ById(id)) if id == "wl-1"
    ));
}
}

```

Counter and parallel sum

`count_parallel` is the `Arc<Mutex<usize>>` counter from chapter 36, with the lock taken once per increment. The final `*counter.lock().expect("poisoned")` reads the total after every thread has been joined.

`parallel_sum` is shorter than chapter 43's version, and it is not parallel. The iterator `chain .map(|chunk| scope.spawn(...)).map(|handle| handle.join()...).sum()` is lazy: `sum` pulls the first element, which spawns the first thread and immediately joins it, and only then pulls the second. The four chunks are summed one after another on four threads. The result is correct and the test passes, so the problem is easy to miss. Collecting the handles into a `Vec` before joining, as chapter 43's version does, starts every thread before waiting for any.

Spin lock and semaphore

The spin lock uses `swap(true, Ordering::Acquire)` in a loop rather than `compare_exchange_weak`. `swap` stores `true` whether or not the lock was free and

returns the previous value, so the loop exits when the previous value was `false`. It is shorter to write and performs a write on every attempt, which chapter 44's version avoids while waiting. The `unsafe` blocks have no `SAFETY` comments, and `SpinGuard` has the same automatically derived `Sync` implementation that chapter 44 shows to be unsound.

The semaphore has no guard. `acquire` and `release` are separate calls, so a caller that returns early between them leaks a permit. Chapter 45's version returns a guard.

Queue and thread pool

`BlockingQueue` is chapter 46's closable queue without its accessors. `new` does not check the capacity, so `BlockingQueue::new(0)` produces a queue on which every `push` waits forever.

`ThreadPool` sends boxed closures, `Box<dyn FnOnce() + Send + 'static>`, over one channel. Each worker runs `let Ok(job) = receiver.lock().expect("poisoned").recv() else { return; };`. The temporary `MutexGuard` in that statement is dropped at the end of the `let` statement, before `job()` runs, so workers execute jobs in parallel. `Drop` for the pool takes the sender out of its `Option` and drops it, which makes every worker's `recv` fail and its loop end, and then joins the workers. A job that panics ends its worker thread, and `let _ = worker.join()` discards the panic.

Retry, hash ring, HTTP head, and small types

`retry` takes the retryability test as a closure instead of a trait, doubles a delay that starts at 25 ms, and caps it at one second. It has no jitter.

`HashRing::get` computes the owner index as `partition_point(...) % self.points.len()`, which wraps an index equal to the length to zero in one expression. Keys are hashed with the same function as virtual nodes, using replica number 0.

`parse_request_head` returns `Option`, so a caller cannot tell why a head was rejected. It lowercases header names on the way in, which makes later lookups case-insensitive without `eq_ignore_ascii_case`.

`Port::new` accepts any non-zero `u16`. The standard library's `NonZeroU16` expresses the same invariant with no custom type, and `NonZeroU16::new(value)` returns `Option`. `Route::parse` combines exact matches with `strip_prefix` in the catch-all arm.

The tests are grouped three to a function. `retry_ring_http_and_adt` checks the hash ring with the same property as chapter 42: after adding node `c`, a key either keeps its owner or moves to `c`.

Intuition

`parallel_sum(&[1, 2, 3, 4])` as the lazy chain runs it

sum pulls	action	threads alive
element 1	spawn a thread for <code>[1]</code> , then join it: 1	0
element 2	spawn a thread for <code>[2]</code> , then join it: 2	0
element 3	spawn a thread for <code>[3]</code> , then join it: 3	0
element 4	spawn a thread for <code>[4]</code> , then join it: 4	0
end	return 10	0

No two threads are ever alive at the same time.

Time and Space Complexity

Each drill has the same asymptotic cost as its full version; the chapters listed in the table in the Problem Statement give the details. `parallel_sum` costs $O(n)$ work as intended, but its wall-clock time is that of a sequential sum plus four thread creations.

Limitations

What the short versions leave out

Drill	Omitted, compared with the full version
<code>count_parallel</code>	nothing essential
<code>parallel_sum</code>	the lazy chain joins each thread before spawning the next
<code>SpinLock</code>	<code>try_lock</code> , SAFETY comments; the guard is <code>Sync</code> for <code>T: Send</code>
<code>Semaphore</code>	RAII guard, timeout
<code>BlockingQueue</code>	capacity check, <code>try_pop</code> , accessors

Drill	Omitted, compared with the full version
ThreadPool	reporting of panicked jobs
retry	jitter, Retry-After
HashRing	removal, duplicate detection, shared name storage
parse_request_head	body framing, limits, typed errors
Port, Route	typed errors; NonZeroU16 would suffice for Port

Summary

- A drill is the version of an exercise that fits an interview's time budget; the full chapters show what a production version adds.
- Iterator chains are lazy. Spawning and joining threads in one chain runs them sequentially, and the result is still correct, so only timing reveals it.
- Shortened `unsafe` code keeps the soundness obligations of the full version, including the `Send` and `Sync` implementations of guard types.
- Guards that release in `Drop` are worth their few extra lines, even in a drill.

References

- Standard library, `Iterator`, on laziness.
- Standard library, `std::num::NonZeroU16`.

Part XVIII: Graphs and Lists, Revisited

Graph Traversal: BFS and DFS

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/problems/graph_bfs.rs` and `src/problems/graph_dfs.rs`. Test them with `cargo test graph_bfs` and `cargo test graph_dfs`.

Problem Statement

Given a directed graph and a start node, return the nodes reachable from the start in the order a traversal visits them, once in breadth-first order and once in depth-first order. Each node must appear at most once, and the traversal must terminate on graphs with cycles.

Each algorithm is written twice. The vector variant numbers the nodes `0..n` and stores the graph as `Vec<Vec<(usize, u32)>>`. The map variant accepts any node type that is `Copy + Eq + Hash`, such as `&str`, and stores the graph as a `HashMap` from a node to a slice of `(neighbor, weight)` edges. The weights are unused by the traversals and are kept so that the same graph can be passed to Dijkstra's algorithm in chapter 61.

Designing a Solution

Both traversals keep a frontier of nodes still to explore and a visited set. They differ only in the frontier.

BFS: frontier is a queue (`VecDeque`)
`pop_front`, `push_back`

explores all nodes at distance 1,
 then all at distance 2, ...

DFS: frontier is a stack (`Vec`)
`pop`, `push`

follows one path as deep as it goes,
 then backtracks

The sample graph used in the tests:

```

A --1--> B --5--> D
|         |       ^
4         2       |
v         v       |
C -----1-----+
(B -> C with weight 2, C -> D with weight 1)

vector variant: A = 0, B = 1, C = 2, D = 3

```

When a node is marked visited differs between the two. BFS marks a node when it is enqueued. Every node enters the queue once, and the queue never holds a duplicate. The iterative DFS marks a node when it is popped. A node can be pushed several times, once by each neighbor that reaches it before it is explored, and the later copies are skipped when they are popped. Marking on pop is what makes the order depth-first: a node pushed recently is explored before an older copy of the same node.

The vector variant is the one to write first. With nodes as indices, the visited set is a `Vec<bool>` indexed directly, with no hashing, and `graph[node]` is a bounds-checked index into a vector. The map variant is the same algorithm with `HashSet` membership and `HashMap` lookup, which is what a problem with named nodes needs.

Implementation

```

///! Breadth-first search.
///!
///! Two variants of the same algorithm:
///!
///! - `bfs_vec`: nodes are `usize` indices and the graph is `Vec<Vec<(usize, u32)>>`,
///!   so `visited` is a plain `Vec<bool>`. This is the version to write first.
///! - `bfs`: nodes are any `Copy + Eq + Hash` type and the graph is a `HashMap`
///!   from a node to a slice of `(neighbor, weight)` edges.

use std::collections::{HashMap, HashSet, VecDeque};
use std::hash::Hash;

/// Adjacency list keyed by node: each node maps to its `(neighbor, weight)` edges.
pub type GraphAdjList<'a, Node> = HashMap<Node, &'a [(Node, u32)]>;

/// BFS over an index-based adjacency list. Returns nodes in visiting order.
pub fn bfs_vec(graph: &Vec<(usize, u32)>, start_node: usize) -> Vec<usize> {
    let mut visited = vec![false; graph.len()];
    let mut queue = VecDeque::from([start_node]);
    let mut result = vec![];

    visited[start_node] = true;

    while let Some(current_node) = queue.pop_front() {
        result.push(current_node);
        for &(neighbor, _) in &graph[current_node] {
            if !visited[neighbor] {
                visited[neighbor] = true;
                queue.push_back(neighbor);
            }
        }
    }

    result
}

```

```

    }
  }
}
result
}

/// BFS over a `HashMap` adjacency list. Returns nodes in visiting order.
pub fn bfs<Node>(graph: &GraphAdjList<Node>, start_node: Node) -> Vec<Node>
where
  Node: Copy + Eq + Hash,
{
  let mut visited = HashSet::new();
  let mut queue = VecDeque::from([start_node]);
  let mut result = vec![];

  visited.insert(start_node);

  while let Some(current_node) = queue.pop_front() {
    result.push(current_node);
    // slice iter yields &(Node, u32); `&` deconstructs it
    for &(neighbor, _) in graph[&current_node] {
      if visited.insert(neighbor) {
        queue.push_back(neighbor);
      }
    }
  }
  result
}

#[cfg(test)]
mod tests {
  use super::*;

  /// A -> B (1), A -> C (4), B -> C (2), B -> D (5), C -> D (1)
  fn sample_vec() -> Vec<Vec<(usize, u32)>> {
    vec![
      vec![(1, 1), (2, 4)],
      vec![(2, 2), (3, 5)],
      vec![(3, 1)],
      vec![],
    ]
  }

  fn sample_map() -> HashMap<'static str, &'static [(&'static str, u32)]> {
    HashMap::from([
      ("A", &[("B", 1), ("C", 4)][..]),
      ("B", &[("C", 2), ("D", 5)][..]),
      ("C", &[("D", 1)][..]),
      ("D", &[][..]),
    ])
  }

  #[test]
  fn vec_variant_visits_level_by_level() {
    assert_eq!(bfs_vec(&sample_vec(), 0), vec![0, 1, 2, 3]);
  }

  #[test]
  fn vec_variant_skips_unreachable_nodes() {
    let graph = vec![vec![(1, 1)], vec![], vec![(0, 1)]];
    assert_eq!(bfs_vec(&graph, 0), vec![0, 1]);
  }

  #[test]
  fn vec_variant_handles_cycles() {
    let graph = vec![vec![(1, 1)], vec![(0, 1)]];
    assert_eq!(bfs_vec(&graph, 0), vec![0, 1]);
  }
}

```

```

}

#[test]
fn map_variant_visits_level_by_level() {
    assert_eq!(bfs(&sample_map(), "A"), vec!["A", "B", "C", "D"]);
}

#[test]
fn both_variants_agree() {
    let names = ["A", "B", "C", "D"];
    let from_vec: Vec<&str> = bfs_vec(&sample_vec(), 0)
        .into_iter()
        .map(|index| names[index])
        .collect();
    assert_eq!(from_vec, bfs(&sample_map(), "A"));
}
}

```

```

/// Iterative depth-first search with an explicit stack.
///
/// Two variants of the same algorithm:
///
/// - `dfs_vec`: nodes are `usize` indices and the graph is `Vec<Vec<usize, u32>>`.
/// - `dfs`: nodes are any `Copy + Eq + Hash` type and the graph is a `HashMap`.
///
/// A node is marked visited when it is popped, not when it is pushed, so it may
/// sit on the stack more than once. Neighbors are pushed in order, which means the
/// last neighbor is explored first.

use crate::problems::graph_bfs::GraphAdjList;
use std::collections::HashSet;
use std::hash::Hash;

/// DFS over an index-based adjacency list. Returns nodes in visiting order.
pub fn dfs_vec(graph: &Vec<usize, u32>, start_node: usize) -> Vec<usize> {
    let mut visited = vec![false; graph.len()];
    let mut stack = vec![start_node];
    let mut result = vec![];

    while let Some(current_node) = stack.pop() {
        if visited[current_node] {
            continue;
        }
        visited[current_node] = true;
        result.push(current_node);
        for &(neighbor, _) in &graph[current_node] {
            stack.push(neighbor);
        }
    }
    result
}

/// DFS over a `HashMap` adjacency list. Returns nodes in visiting order.
pub fn dfs<Node>(graph: &GraphAdjList<Node>, start_node: Node) -> Vec<Node>
where
    Node: Copy + Eq + Hash,
{
    let mut visited = HashSet::new();
    let mut stack = vec![start_node];
    let mut result = vec![];

    while let Some(current_node) = stack.pop() {
        if !visited.insert(current_node) {
            continue;
        }
        result.push(current_node);
    }
}

```

```

        for &(neighbor, _) in graph[&current_node] {
            stack.push(neighbor);
        }
    }
    result
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::collections::HashMap;

    /// A -> B (1), A -> C (4), B -> C (2), B -> D (5), C -> D (1)
    fn sample_vec() -> Vec<Vec<(usize, u32)>> {
        vec![
            vec![(1, 1), (2, 4)],
            vec![(2, 2), (3, 5)],
            vec![(3, 1)],
            vec![],
        ]
    }

    fn sample_map() -> HashMap<&'static str, &'static [(&'static str, u32)]> {
        HashMap::from([
            ("A", &[( "B", 1), ("C", 4)][..]),
            ("B", &[( "C", 2), ("D", 5)][..]),
            ("C", &[( "D", 1)][..]),
            ("D", &[][..]),
        ])
    }

    #[test]
    fn vec_variant_explores_the_last_neighbor_first() {
        // Pop 0, push 1 and 2; pop 2, push 3; pop 3; pop 1.
        assert_eq!(dfs_vec(&sample_vec(), 0), vec![0, 2, 3, 1]);
    }

    #[test]
    fn vec_variant_handles_cycles() {
        let graph = vec![vec![(1, 1)], vec![(2, 1)], vec![(0, 1)]];
        assert_eq!(dfs_vec(&graph, 0), vec![0, 1, 2]);
    }

    #[test]
    fn map_variant_explores_the_last_neighbor_first() {
        assert_eq!(dfs(&sample_map(), "A"), vec!["A", "C", "D", "B"]);
    }

    #[test]
    fn both_variants_agree() {
        let names = ["A", "B", "C", "D"];
        let from_vec: Vec<&str> = dfs_vec(&sample_vec(), 0)
            .into_iter()
            .map(|index| names[index])
            .collect();
        assert_eq!(from_vec, dfs(&sample_map(), "A"));
    }
}

```

`bfs_vec` sets `visited[start_node] = true` before the loop and marks each neighbor as it is enqueued, so the check `!visited[neighbor]` and the mark happen together. The map variant does the same in one call:

`visited.insert(neighbor)` returns `true` only when the node was not already present, so the condition both tests and records membership.

`for &(neighbor, _) in &graph[current_node]` iterates over `&(usize, u32)` values, and the pattern `&(neighbor, _)` copies the neighbor out and ignores the weight. In the map variant, `graph[¤t_node]` already yields a slice reference, `&[(Node, u32)]`, so the loop iterates over it without another `&`.

`dfs_vec` pushes every neighbor, visited or not, and filters at pop time with `if visited[current_node] { continue; }`. The map variant writes the same test as `if !visited.insert(current_node) { continue; }`.

`GraphAdjList<'a, Node>` is `HashMap<Node, &'a [(Node, u32)]>`. The lifetime says the map borrows its edge slices rather than owning them, which is why the tests build the graph from slice literals such as `&[("B", 1), ("C", 4)][..]`. The `[..]` turns an array reference into a slice reference, so every entry of the map has the same type. The alias is defined in `graph_bfs.rs` and imported by the other two graph modules.

The `both_variants_agree` tests map the indices from the vector variant to names and compare the result with the map variant, so the two implementations cannot drift apart.

Intuition

```
bfs_vec(sample, start = 0)
```

pop	queue after pushes	visited	result
-	[0]	{0}	[]
0	[1, 2]	{0, 1, 2}	[0]
1	[2, 3]	{0, 1, 2, 3}	[0, 1] 2 already visited
2	[3]	{0, 1, 2, 3}	[0, 1, 2] 3 already visited
3	[]	{0, 1, 2, 3}	[0, 1, 2, 3]

```
dfs_vec(sample, start = 0)
```

pop	visited before?	stack after pushes	result
0	no	[1, 2]	[0]
2	no	[1, 3]	[0, 2]
3	no	[1]	[0, 2, 3]
1	no	[2, 3]	[0, 2, 3, 1]
3	yes, skip	[2]	
2	yes, skip	[]	

The `tests` `vec_variant_visits_level_by_level` and `vec_variant_explores_the_last_neighbor_first` assert exactly these two orders, and the map variants return `["A", "B", "C", "D"]` and `["A", "C", "D", "B"]`.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(V + E)$	every reachable node is expanded once and each of its edges is examined once
Space, BFS	$O(V)$	the queue and the visited set each hold at most one entry per node
Space, DFS	$O(V + E)$	a node can be pushed once per incoming edge before it is popped
Map variant	the same bounds, expected	HashSet and HashMap operations are expected constant time

The vector variant allocates `visited` for every node in the graph, reachable or not. The map variant grows its set only as nodes are visited.

Limitations

A missing node panics. `graph[current_node]` in the vector variant panics when an edge names an index outside the vector, and `graph[¤t_node]` in the map variant panics when a neighbor has no entry of its own, even an empty one. The tests give every node an entry. A graph read from input should be validated first, or the loop should use `graph.get(node)` and treat a missing entry as a node with no edges.

The DFS order is not the recursive order. A recursive DFS visits the first neighbor first. This iterative version pushes neighbors in order and pops the last one first, so it visits `C` before `B`. Both are valid depth-first orders. Pushing the neighbors in reverse, `for &(neighbor, _) in graph[node].iter().rev()`, reproduces the recursive order.

The DFS stack can grow beyond `V`. Marking on pop allows duplicates on the stack, up to one entry per edge. Marking on push keeps the stack at `V` entries but no longer produces a depth-first order in all cases.

Only reachable nodes are returned. To traverse a graph with several components, call the function from every node that has not been visited yet, as chapter 23 does for grid islands.

The weights are carried but unused. A traversal that only needs connectivity can store `Vec<Vec<usize>>`, which halves the memory per edge.

Summary

- BFS and DFS share one loop and differ only in the frontier: a `VecDeque` used as a queue or a `Vec` used as a stack.
- BFS marks a node when it enters the queue; this iterative DFS marks a node when it is popped, which allows duplicates on the stack and gives a depth-first order.
- The vector variant uses indices and a `Vec<bool>`; the map variant uses a `HashSet`, whose `insert` returns whether the node was new.
- Both run in $O(V + E)$ time, and tests that compare the two variants keep them consistent.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Sections 20.2, "Breadth-first search", and 20.3, "Depth-first search".
- Standard library, `VecDeque`.
- Standard library, `HashSet::insert`, on its return value.

Dijkstra's Shortest Paths

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/graph_dijkstra.rs`. Test it with `cargo test graph_dijkstra`.

Problem Statement

Given a directed graph with non-negative edge weights and a start node, return the length of the shortest path from the start to every node. A node that cannot be reached has no distance.

As in chapter 60, the algorithm is written twice. The vector variant numbers the nodes `0..n` and returns `Vec<Option<u32>>`, with `None` for an unreachable node. The map variant accepts any node type that is `Copy + Eq + Hash + Ord` and returns a `HashMap` that contains only the reachable nodes.

Designing a Solution

BFS finds shortest paths when every edge has the same length, because it settles nodes in order of hop count. Dijkstra's algorithm generalizes that order to weighted edges: it always expands the unexpanded node with the smallest known distance. With non-negative weights, no later path can make that node's distance smaller, so its distance is final once it is expanded.

A priority queue supplies the smallest known distance in $O(\log n)$. Rust's `BinaryHeap` is a max-heap, so each entry is wrapped in `std::cmp::Reverse`, which inverts the ordering and makes the smallest `(cost, node)` pair come out first.

`BinaryHeap` has no operation to lower the priority of an entry already in the heap. Instead, a shorter distance pushes a second entry for the same node. The older entry stays in the heap and is recognised as stale when it is popped, because its cost is larger than the distance recorded for the node.

the sample graph from chapter 60

```
A --1--> B --5--> D
|         |       ^
4         2       |
v         v       |
C -----1-----+
```

shortest distances from A: A 0, B 1, C 3 (via B), D 4 (via B and C)

Implementation

```
/// Dijkstra's shortest paths with a binary min-heap.
///
/// Two variants of the same algorithm:
///
/// - `dijkstra_vec`: nodes are `usize` indices and distances are a
///   `Vec<Option<u32>>`, where `None` means unreachable.
/// - `dijkstra`: nodes are any `Copy + Eq + Hash + Ord` type and distances are a
///   `HashMap` containing only the reachable nodes.
///
/// Edge weights must be non-negative, which `u32` guarantees. `BinaryHeap` is a
/// max-heap, so entries are wrapped in `Reverse` to pop the smallest cost first.
/// A node can be in the heap several times; an entry whose cost is larger than the
/// recorded distance is stale and skipped.

use crate::problems::graph_bfs::GraphAdjList;
use std::cmp::Reverse;
use std::collections::{BinaryHeap, HashMap};
use std::hash::Hash;

/// Shortest distances from `start_node` over an index-based adjacency list.
pub fn dijkstra_vec(graph: &[Vec<(usize, u32)>], start_node: usize) ->
Vec<Option<u32>> {
    let mut distances = vec![None; graph.len()];
    let mut min_heap = BinaryHeap::from([Reverse((0, start_node))]);

    distances[start_node] = Some(0);

    while let Some(Reverse((current_cost, current_node))) = min_heap.pop() {
        // stale heap entry, a shorter path was already recorded
        if distances[current_node].is_some_and(|best| current_cost > best) {
            continue;
        }
        for &(neighbor, weight) in &graph[current_node] {
            let new_distance = current_cost + weight;
            if distances[neighbor].is_none_or(|best| new_distance < best) {
                distances[neighbor] = Some(new_distance);
                min_heap.push(Reverse((new_distance, neighbor)));
            }
        }
    }
    distances
}
```

```

/// Shortest distances from `start_node` over a `HashMap` adjacency list.
pub fn dijkstra<Node>(graph: &GraphAdjList<Node>, start_node: Node) -> HashMap<Node, u32>
where
    Node: Copy + Eq + Hash + Ord,
{
    let mut distances = HashMap::from([(start_node, 0)]);
    let mut min_heap = BinaryHeap::from([Reverse((0, start_node))]);

    while let Some(Reverse((current_cost, current_node))) = min_heap.pop() {
        // stale heap entry, a shorter path was already recorded
        if current_cost > distances[&current_node] {
            continue;
        }
        for &(neighbor, weight) in graph[&current_node] {
            let new_distance = current_cost + weight;
            if new_distance < *distances.get(&neighbor).unwrap_or(&u32::MAX) {
                distances.insert(neighbor, new_distance);
                min_heap.push(Reverse((new_distance, neighbor)));
            }
        }
    }
    distances
}

#[cfg(test)]
mod tests {
    use super::*;

    /// A -> B (1), A -> C (4), B -> C (2), B -> D (5), C -> D (1)
    fn sample_vec() -> Vec<Vec<(usize, u32)>> {
        vec![
            vec![(1, 1), (2, 4)],
            vec![(2, 2), (3, 5)],
            vec![(3, 1)],
            vec![],
        ]
    }

    fn sample_map() -> HashMap<&'static str, &'static [(&'static str, u32)]> {
        HashMap::from([
            ("A", &[( "B", 1), ( "C", 4)][..]),
            ("B", &[( "C", 2), ( "D", 5)][..]),
            ("C", &[( "D", 1)][..]),
            ("D", &[][..]),
        ])
    }

    #[test]
    fn vec_variant_finds_shorter_multi_hop_paths() {
        // A -> C directly costs 4, A -> B -> C costs 3; D is reached via C for 4.
        assert_eq!(
            dijkstra_vec(&sample_vec(), 0),
            vec![Some(0), Some(1), Some(3), Some(4)]
        );
    }

    #[test]
    fn vec_variant_marks_unreachable_nodes() {
        let graph = vec![vec![(1, 7)], vec![], vec![(0, 1)]];
        assert_eq!(dijkstra_vec(&graph, 0), vec![Some(0), Some(7), None]);
    }

    #[test]
    fn map_variant_finds_shorter_multi_hop_paths() {
        let distances = dijkstra(&sample_map(), "A");
        assert_eq!(

```

```

        distances,
        HashMap::from([("A", 0), ("B", 1), ("C", 3), ("D", 4)])
    );
}

#[test]
fn map_variant_omits_unreachable_nodes() {
    let graph: HashMap<u8, &[(u8, u32)]> =
        HashMap::from([(0, &[(1, 2)][..]), (1, &[][..]), (2, &[(0, 1)][..])]);
    assert_eq!(dijkstra(&graph, 0), HashMap::from([(0, 0), (1, 2)]));
}

```

`dijkstra_vec` stores distances as `Vec<Option<u32>>`, so "not reached yet" is `None` rather than a sentinel. The relaxation test `distances[neighbor].is_none_or(|best| new_distance < best)` accepts an edge when the neighbor has no distance yet or when the new path is shorter. `Option::is_none_or` was stabilised in Rust 1.82. The stale-entry test `distances[current_node].is_some_and(|best| current_cost > best)` skips an entry whose cost is larger than the distance already recorded.

The map variant, `dijkstra`, keeps the same logic with a `HashMap`. It uses `*distances.get(&neighbor).unwrap_or(&u32::MAX)` as the current best, so a node without an entry compares as infinitely far. `distances[¤t_node]` cannot panic in the stale-entry test, because a node is pushed onto the heap only after its distance is inserted.

`Node: Ord` is required by the map variant because the heap orders `(cost, node)` tuples, and tuples compare element by element: when two costs are equal, the nodes decide the order. The vector variant gets that ordering for free from `usize`.

The tests cover the multi-hop case, where the direct edge `A -> C` of weight 4 loses to `A -> B -> C` of weight 3, and a graph with an unreachable node in each variant.

Intuition

```

dijkstra_vec(sample, start = 0)

pop (cost, node)   stale?   relaxations                               distances [A, B, C,
D]   heap after
-
-]               [(0,0)]
(0, 0)           no       B: 1 < none, push; C: 4 < none, push [0, 1, 4,
-]               [(1,1), (4,2)]
(1, 1)           no       C: 3 < 4, push; D: 6 < none, push   [0, 1, 3,
6]               [(3,2), (4,2), (6,3)]
(3, 2)           no       D: 4 < 6, push                       [0, 1, 3,
4]               [(4,2), (4,3), (6,3)]

```

```

(4, 2)          yes, 4 > 3, skip          [0, 1, 3,
4]             [(4,3), (6,3)]
(4, 3)          no          D has no edges [0, 1, 3,
4]             [(6,3)]
(6, 3)          yes, 6 > 4, skip          [0, 1, 3,
4]             []

```

The result is `[Some(0), Some(1), Some(3), Some(4)]`. Two of the six pops were stale entries, one for `C` and one for `D`.

Time and Space Complexity

Resource	Cost	Condition
Time	$O((V + E) \log E)$	each edge can push one heap entry, and each push and pop costs $O(\log E)$
Space	$O(V + E)$	the distance table holds one entry per node, and the heap up to one entry per edge
Map variant	the same bounds, expected	HashMap operations are expected constant time

Since E is at most V^2 , $\log E$ is at most $2 \log V$, and the bound is usually written $O((V + E) \log V)$.

Limitations

Negative weights are not supported. The algorithm relies on a node's distance being final when it is expanded. The `u32` weight type rules out negative edges. A graph with negative weights needs the Bellman-Ford algorithm.

Distances can overflow. `current_cost + weight` is a `u32` addition. A path whose total exceeds `u32::MAX` panics in a debug build and wraps in a release build, which could make a long path look short. `checked_add`, or `u64` distances, removes the risk.

Paths are not returned. The functions return distances only. Recording the predecessor of each node when its distance improves, in a `Vec<Option<usize>>` or a `HashMap<Node, Node>`, allows the path to be rebuilt by walking back from the target.

The whole reachable graph is explored. A search for one target can stop as soon as that target is popped, because its distance is final at that point. Neither function takes a target.

A missing node panics. As in chapter 60, an edge to an index outside the vector, or to a node without its own map entry, panics on indexing.

Summary

- Dijkstra's algorithm expands nodes in order of distance, which is final on expansion when every weight is non-negative.
- `BinaryHeap` is a max-heap; `Reverse((cost, node))` turns it into a min-heap.
- Instead of decreasing a key, the algorithm pushes a new entry and skips stale entries whose cost exceeds the recorded distance.
- The vector variant represents "unreachable" as `None` and uses `is_none_or` and `is_some_and` for the two tests; the map variant omits unreachable nodes.
- The cost is $O((V + E) \log V)$ time and $O(V + E)$ space.

References

- Edsger W. Dijkstra, "A note on two problems in connexion with graphs", *Numerische Mathematik* 1, 1959, pages 269–271.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 22.3, "Dijkstra's algorithm".
- Standard library, `BinaryHeap`, on using `Reverse` for a min-heap.
- Standard library, `Option::is_none_or`.

Merge K Sorted Lists

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files, all under `src/bin/`: `merge_k_sorted_lists_divide.rs`, `merge_k_sorted_lists_swap.rs`, `merge_k_sorted_lists_simple.rs`, `merge_k_sorted_lists_pairs.rs`, `merge_k_sorted_lists_heap.rs`, `merge_k_sorted_lists_enum.rs`, and `merge_k_sorted_list_easy.rs`. Run any of them with `cargo run --bin <name>`, and test the two that carry tests with `cargo test --bin merge_k_sorted_lists_divide` and `cargo test --bin merge_k_sorted_lists_simple`.

Problem Statement

Given k singly linked lists, each sorted in ascending order, merge them into one sorted list. The nodes must be relinked rather than copied, and the result must contain every node of every input list.

The chapter solves the problem seven times. The divide-and-conquer version comes first and is read in full; the six that follow change one decision each and are read against it:

Version	What it changes
divide	merges in place inside the input vector, using <code>split_at_mut</code> for two borrows
swap	takes one list out of the vector first, so only one borrow is needed
simple	passes lists by value and appends through a <code>&mut Link</code> , with no dummy node
pairs	merges the front two lists of a <code>VecDeque</code> and pushes the result on the back
heap	keeps the head of every list in a <code>BinaryHeap</code> and pops the smallest
enum	replaces <code>Option<Box<ListNode>></code> with an enum carrying an <code>Empty</code> variant

Version	What it changes
easy	merges two lists recursively, and builds lists through <code>From<Vec<i32>></code>

Designing a Solution

Merging two sorted lists is the building block: compare the heads, move the smaller node to the output, and repeat. Merging k lists one after another into a growing result is simple and slow, because the result is walked again for every list, which costs $O(kN)$ for N nodes in total.

Divide and conquer merges the lists in pairs, like the merge step of merge sort. In the first round, list 0 is merged with list 1, list 2 with list 3, and so on. In the next round, the survivors are merged in pairs again, until one list remains. Each node takes part in one merge per round, and there are $\log k$ rounds, so the total is $O(N \log k)$.

The implementation merges in place inside the input vector. `interval` is the distance between the two lists being merged, and it doubles every round:

```
lists: [L0, L1, L2, L3, L4]      n = 5

interval 1:  L0 = merge(L0, L1)   L2 = merge(L2, L3)   L4 untouched
interval 2:  L0 = merge(L0, L2)                    L4 untouched
interval 4:  L0 = merge(L0, L4)
interval 8:  8 >= 5, stop; the answer is lists[0]
```

`merge_two` uses a dummy head node, so appending the first node of the result needs no special case, and a `tail` reference that always points at the last node of the output.

Implementation

```
struct MergeKSorted { }

#[derive(Default)]
struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>
}

impl ListNode {
    fn new(data: u32) -> Self {
        Self {
            data,
            next: None
        }
    }
}
```



```

type OptionalLink<T> = Option<Box<T>>;

impl MergeKSorted {
    pub fn merge_k_lists(mut lists: Vec<OptionalLink<ListNode>>) -> OptionalLink<ListNode> {
        if lists.is_empty() { return None; }

        let (n, mut interval) = (lists.len(), 1);
        while interval < n {
            let mut i = 0;
            while i + interval < n {
                let (left, right) = lists.split_at_mut(i + interval);
                left[i] = Self::merge_two(&mut left[i], &mut right[0]);
                i += interval * 2;
            }
            interval *= 2;
        }

        lists[0].take()
    }

    fn merge_two(left: &mut OptionalLink<ListNode>, right: &mut OptionalLink<ListNode>) -> OptionalLink<ListNode> {
        let mut dummy = Box::new(ListNode::new(0));
        let mut tail = &mut dummy;

        while let (Some(l), Some(r)) = (left.as_ref(), right.as_ref()) {
            let smaller = if l.data < r.data { &mut *left } else { &mut *right };
            let mut head = smaller.take().unwrap();
            *smaller = head.next.take();
            tail.next = Some(head);
            tail = tail.next.as_mut().unwrap();
        }

        tail.next = left.take().or(right.take());
        dummy.next
    }
}

/// Build a list from a slice, front to back.
fn from_slice(values: &[u32]) -> OptionalLink<ListNode> {
    let mut head = None;
    for &value in values.iter().rev() {
        let mut node = Box::new(ListNode::new(value));
        node.next = head;
        head = Some(node);
    }
    head
}

/// Collect a list's values into a vector.
fn to_vec(mut list: &OptionalLink<ListNode>) -> Vec<u32> {
    let mut values = Vec::new();
    while let Some(node) = list {
        values.push(node.data);
        list = &node.next;
    }
    values
}

fn main() {
    let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2, 6])];
    let merged = MergeKSorted::merge_k_lists(lists);
    println!("{:?}", to_vec(&merged));
}

```

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_three_lists() {
        let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]),
from_slice(&[2, 6])];
        let merged = MergeKSorted::merge_k_lists(lists);
        assert_eq!(to_vec(&merged), vec![1, 1, 2, 3, 4, 4, 5, 6]);
    }

    #[test]
    fn handles_empty_input_and_empty_lists() {
        assert!(MergeKSorted::merge_k_lists(vec![]).is_none());
        let merged = MergeKSorted::merge_k_lists(vec![None, from_slice(&[2]), None]);
        assert_eq!(to_vec(&merged), vec![2]);
    }

    #[test]
    fn handles_an_odd_number_of_lists() {
        let lists: Vec<_> = (0..5).map(|i| from_slice(&[i, i + 10])).collect();
        let merged = MergeKSorted::merge_k_lists(lists);
        assert_eq!(to_vec(&merged), vec![0, 1, 2, 3, 4, 10, 11, 12, 13, 14]);
    }
}
```

`lists.split_at_mut(i + interval)` is the step that makes the in-place merge legal. The merge needs mutable access to two different elements of the same vector, `lists[i]` and `lists[i + interval]`, and the borrow checker does not allow two `&mut` borrows of one vector at the same time. `split_at_mut` divides the vector into two non-overlapping mutable slices, so `left[i]` and `right[0]` are two separate borrows.

`merge_two` takes both lists as `&mut OptionalLink<ListNode>` and consumes them with `take`. The loop condition `while let (Some(l), Some(r)) = (left.as_ref(), right.as_ref())` peeks at both heads without moving them. Inside the loop, `smaller` is a mutable reference to whichever list has the smaller head, `smaller.take().unwrap()` detaches that node, and `*smaller = head.next.take()` advances the list past it.

`tail` is `&mut Box<ListNode>`. After `tail.next = Some(head)`, the statement `tail = tail.next.as_mut().unwrap()` moves the reference to the node just appended. When either input runs out, `tail.next = left.take().or(right.take())` attaches the rest of the other list in one step, with no further comparisons.

`dummy.next` is the merged list. The dummy node itself is freed when the function returns.

The helpers `from_slice` and `to_vec` build lists from slices and read them back, so `main` and the tests can express lists as arrays. The tests cover three lists, an

empty input and empty lists mixed with non-empty ones, and an odd number of lists, where the last list waits until the final round.

The variant that swaps instead of splitting

The two mutable borrows can be avoided rather than legalised. Taking `lists[i + interval]` out of the vector with `take` leaves `None` behind at that index and hands the caller an owned list, so the merge that follows borrows only `lists[i]`, and a single borrow needs no help from `split_at_mut`.

Source file: `src/bin/merge_k_sorted_lists_swap.rs`. Run it with `cargo run --bin merge_k_sorted_lists_swap`.

```
struct MergeKSorted { }

#[derive(Default)]
struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>
}

impl ListNode {
    fn new(data: u32) -> Self {
        Self {
            data,
            next: None
        }
    }
}

type OptionalLink<T> = Option<Box<T>>;

impl MergeKSorted {
    pub fn merge_k_lists(mut lists: Vec<OptionalLink<ListNode>>) -> OptionalLink<ListNode> {
        if lists.is_empty() { return None; }

        let (n, mut interval) = (lists.len(), 1);
        while interval < n {
            let mut i = 0;
            while i + interval < n {
                let mut right = lists[i + interval].take();
                lists[i] = Self::merge_two(&mut lists[i], &mut right);
                i += interval * 2;
            }
            interval *= 2;
        }

        lists[0].take()
    }

    fn merge_two(left: &mut OptionalLink<ListNode>, right: &mut OptionalLink<ListNode>) -> OptionalLink<ListNode> {
        let mut dummy = Box::new(ListNode::new(0));
        let mut tail = &mut dummy;

        while let (Some(l), Some(r)) = (&left, &right) {
            if r.data < l.data {
                std::mem::swap(left, right);
            }
            let mut smallest = left.take().unwrap();
            *left = smallest.next.take();
        }
    }
}
```

```

        tail.next = Some(smallest);
        tail = tail.next.as_mut().unwrap();
    }

    tail.next = left.take().or(right.take());
    dummy.next
}

}

/// Build a list from a slice, front to back.
fn from_slice(values: &[u32]) -> OptionalLink<ListNode> {
    let mut head = None;
    for &value in values.iter().rev() {
        let mut node = Box::new(ListNode::new(value));
        node.next = head;
        head = Some(node);
    }
    head
}

/// Collect a list's values into a vector.
fn to_vec(mut list: &OptionalLink<ListNode>) -> Vec<u32> {
    let mut values = Vec::new();
    while let Some(node) = list {
        values.push(node.data);
        list = &node.next;
    }
    values
}

fn main() {
    let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2, 6]
)];
    let merged = MergeKSorted::merge_k_lists(lists);
    println!("{:?}", to_vec(&merged));
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_three_lists() {
        let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]),
from_slice(&[2, 6])];
        let merged = MergeKSorted::merge_k_lists(lists);
        assert_eq!(to_vec(&merged), vec![1, 1, 2, 3, 4, 4, 5, 6]);
    }

    #[test]
    fn handles_empty_input_and_empty_lists() {
        assert!(MergeKSorted::merge_k_lists(vec![]).is_none());
        let merged = MergeKSorted::merge_k_lists(vec![None, from_slice(&[2]), None]);
        assert_eq!(to_vec(&merged), vec![2]);
    }

    #[test]
    fn handles_an_odd_number_of_lists() {
        let lists: Vec<_> = (0..5).map(|i| from_slice(&[i, i + 10])).collect();
        let merged = MergeKSorted::merge_k_lists(lists);
        assert_eq!(to_vec(&merged), vec![0, 1, 2, 3, 4, 10, 11, 12, 13, 14]);
    }
}

```

The merge changes shape as well. Instead of choosing a reference to whichever list has the smaller head, it swaps the two lists whenever the right head is the smaller one, so after the swap `left` always names the list the next node comes from. Everything below the swap then has one path through it: detach the head of `left`, advance `left` past it, append the node to the tail.

The comparison is `r.data < l.data`, and a tie leaves the two lists as they are, so the node comes from `left`. This version is therefore stable where the divide-and-conquer version above is not.

The variant that moves the lists by value

References are not needed at all if the merge owns what it merges. Taking both lists by value and returning the merged list makes the ownership of every node explicit in the signature, and the caller puts the result back into the vector itself.

Source file: `src/bin/merge_k_sorted_lists_simple.rs`. Run it with `cargo run --bin merge_k_sorted_lists_simple`.

```
struct ListNode {
    data: u32,
    next: Link,
}

type Link = Option<Box<ListNode>>;

fn merge_k_lists(mut lists: Vec<Link>) -> Link {
    if lists.is_empty() { return None; }

    let (n, mut interval) = (lists.len(), 1);
    while interval < n {
        let mut i = 0;
        while i + interval < n {
            let left = lists[i].take();
            let right = lists[i + interval].take();
            lists[i] = merge_two(left, right);
            i += interval * 2;
        }
        interval *= 2;
    }

    lists[0].take()
}

fn merge_two(mut left: Link, mut right: Link) -> Link {
    let mut merged = None;
    let mut tail = &mut merged;

    while let (Some(l), Some(r)) = (&left, &right) {
        if r.data < l.data {
            std::mem::swap(&mut left, &mut right);
        }
        *tail = left;
        tail = &mut tail.as_mut().unwrap().next; // hang left list on the tail
        left = tail.take();                       // step past its first node
        // cut the rest off again
    }
}
```

```

    }

    *tail = left.or(right);
    merged
}

fn from_slice(values: &[u32]) -> Link {
    let mut head = None;
    for &data in values.iter().rev() {
        head = Some(Box::new(ListNode { data, next: head }));
    }
    head
}

fn to_vec(mut list: &Link) -> Vec<u32> {
    let mut values = Vec::new();
    while let Some(node) = list {
        values.push(node.data);
        list = &node.next;
    }
    values
}

fn main() {
    let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]), from_slice(&[2, 6]
)];
    println!("{:?}", to_vec(&merge_k_lists(lists)));
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_three_lists() {
        let lists = vec![from_slice(&[1, 4, 5]), from_slice(&[1, 3, 4]),
from_slice(&[2, 6])];
        assert_eq!(to_vec(&merge_k_lists(lists)), vec![1, 1, 2, 3, 4, 4, 5, 6]);
    }

    #[test]
    fn handles_empty_input_and_empty_lists() {
        assert!(merge_k_lists(vec![]).is_none());
        let merged = merge_k_lists(vec![None, from_slice(&[2]), None]);
        assert_eq!(to_vec(&merged), vec![2]);
    }

    #[test]
    fn handles_an_odd_number_of_lists() {
        let lists: Vec<_> = (0..5).map(|i| from_slice(&[i, i + 10])).collect();
        assert_eq!(to_vec(&merge_k_lists(lists)), vec![0, 1, 2, 3, 4, 10, 11, 12, 13, 1
4]);
    }
}

```

The output is built without a dummy node. `tail` is a `&mut Link`, a mutable reference to the empty `Option` that sits at the end of the merged list, and writing through it appends. At the start it points at `merged` itself, which is why the first node needs no special case.

Three statements do the appending, and they are worth reading slowly. `*tail = left` hangs the whole of the left list on the tail rather than one node. `tail =`

`&mut tail.as_mut().unwrap().next` steps the tail reference past the node that has just been attached. `left = tail.take()` then cuts everything after that node off again and gives it back to `left`. The net effect is the same as moving one node, and no `Box` is allocated to hold a dummy.

The variant that pairs the lists in a queue

Doubling an `interval` is bookkeeping in service of a simple idea: merge the lists two at a time until one is left. A queue expresses the idea directly. Two lists come off the front, their merge goes on the back, and the rounds appear on their own without being counted.

Source file: `src/bin/merge_k_sorted_lists_pairs.rs`. Run it with `cargo run --bin merge_k_sorted_lists_pairs`.

```
use std::collections::VecDeque;

struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>,
}

type Link = Option<Box<ListNode>>;

fn merge_k_lists(lists: Vec<Link>) -> Link {
    let mut queue: VecDeque<Link> = lists.into();
    while queue.len() > 1 {
        let a = queue.pop_front().unwrap();
        let b = queue.pop_front().unwrap();
        queue.push_back(merge_two(a, b));
    }
    queue.pop_front().flatten()
}

fn merge_two(mut a: Link, mut b: Link) -> Link {
    let mut dummy = Box::new(ListNode { data: 0, next: None });
    let mut tail = &mut dummy;

    while let (Some(x), Some(y)) = (&a, &b) {
        let pick = if x.data <= y.data { &mut a } else { &mut b };
        let mut node = pick.take().unwrap();
        *pick = node.next.take();
        tail = tail.next.insert(node);
    }
    tail.next = a.or(b);
    dummy.next
}

fn main() {
    let mut lists = Vec::new();
    for values in [vec![1, 4, 5], vec![1, 3, 4], vec![2, 6]] {
        let mut head = None;
        for data in values.into_iter().rev() {
            head = Some(Box::new(ListNode { data, next: head }));
        }
        lists.push(head);
    }

    let mut cur = merge_k_lists(lists);
```

```

    while let Some(node) = cur {
        print!("{}", node.data);
        cur = node.next;
    }
    println();
}

```

Each pass over the queue halves the number of lists it holds, so a list takes part in $\log k$ merges and the total is $O(N \log k)$, the same as the interval version. The cost is one `VecDeque` of k links, against the $O(1)$ extra space of merging inside the input vector.

`tail.next.insert(node)` replaces the `tail.next = Some(node)` and `tail = tail.next.as_mut().unwrap()` pair used earlier. `Option::insert` stores the value and returns a mutable reference to it, which is exactly the two steps in one. `queue.pop_front().flatten()` turns the `Option<Link>` returned by the last pop into a `Link`, since a queue of one list and an empty queue must both answer `None`.

The variant that keeps the heads in a heap

The complexity table above names a min-heap as the other standard solution, and it is worth writing out. Rather than merging lists against each other, keep the current head of every list in a heap and repeatedly move the smallest one to the output.

Source file: `src/bin/merge_k_sorted_lists_heap.rs`. Run it with `cargo run --bin merge_k_sorted_lists_heap`.

```

use std::cmp::Reverse;
use std::collections::BinaryHeap;

struct ListNode {
    data: u32,
    next: Option<Box<ListNode>>,
}

type Link = Option<Box<ListNode>>;

fn merge_k_lists(mut lists: Vec<Link>) -> Link {
    let mut heap = BinaryHeap::new();
    for (i, list) in lists.iter().enumerate() {
        if let Some(node) = list {
            heap.push(Reverse((node.data, i)));
        }
    }

    let mut dummy = Box::new(ListNode { data: 0, next: None });
    let mut tail = &mut dummy;

    while let Some(Reverse((_, i))) = heap.pop() {
        let mut node = lists[i].take().unwrap();
        lists[i] = node.next.take();
        if let Some(next) = &lists[i] {

```



```

        heap.push(Reverse((next.data, i)));
    }
    tail = tail.next.insert(node);
}
dummy.next
}

fn main() {
    let mut lists = Vec::new();
    for values in [vec![1, 4, 5], vec![1, 3, 4], vec![2, 6]] {
        let mut head = None;
        for data in values.into_iter().rev() {
            head = Some(Box::new(ListNode { data, next: head }));
        }
        lists.push(head);
    }

    let mut cur = merge_k_lists(lists);
    while let Some(node) = cur {
        print!("{}", node.data);
        cur = node.next;
    }
    println();
}

```

`BinaryHeap` is a max-heap, so `Reverse` inverts every comparison and turns it into a min-heap. What the heap stores is a `(data, index)` pair rather than a node: the node itself stays in `lists[i]`, and only its value and the list it came from are needed to decide what to pop next. Storing nodes in the heap would mean moving them out of the lists and back, which is the fight with ownership this arrangement avoids entirely.

Each pop names the list holding the smallest head. That node is taken out of `lists[i]`, the list is advanced past it, and the list's new head, if there is one, goes back into the heap. The heap therefore holds at most `k` entries at any moment, one per non-empty list, and each of the `N` nodes is pushed and popped once, for $O(N \log k)$ time and $O(k)$ space.

Ties are broken by the second field of the pair, the list index, so equal values leave the lists in index order. The merge is stable.

The variant that carries emptiness in the node

`Option<Box<ListNode>>` makes emptiness a property of the link. An enum with an `Empty` variant makes it a property of the node, which is the shape a reader coming from a functional language expects, and the shape an interviewer sometimes asks for by name.

Source file: `src/bin/merge_k_sorted_lists_enum.rs`. Run it with `cargo run --bin merge_k_sorted_lists_enum`.

```

use ListNode::{Empty, Node};

type LinkTo<T> = Box<T>;

enum ListNode {
    Node { data: u32, next: LinkTo<ListNode> },
    Empty,
}

impl ListNode {
    fn take(&mut self) -> ListNode {
        std::mem::replace(self, Empty)
    }
}

struct MergeKSorted {}

impl MergeKSorted {
    pub fn merge_k_lists(mut lists: Vec<ListNode>) -> ListNode {
        if lists.is_empty() { return Empty; }

        let (n, mut interval) = (lists.len(), 1);
        while interval < n {
            let mut i = 0;
            while i + interval < n {
                let (left, right) = lists.split_at_mut(i + interval);
                left[i] = Self::merge_two(&mut left[i], &mut right[0]);
                i += interval * 2;
            }
            interval *= 2;
        }

        lists[0].take()
    }

    fn merge_two(left: &mut ListNode, right: &mut ListNode) -> ListNode {
        let mut left = left.take();
        let mut right = right.take();
        let mut merged = Empty;
        let mut tail = &mut merged;

        loop {
            let smaller = match (&left, &right) {
                (Empty, _) => { *tail = right; break; }
                (_, Empty) => { *tail = left; break; }
                (Node { data: l, .. }, Node { data: r, .. }) => {
                    if l <= r { &mut left } else { &mut right }
                }
            };
            if let Node { data, next } = smaller.take() {
                *smaller = *next;
                *tail = Node { data, next: LinkTo::new(Empty) };
                if let Node { next, .. } = tail {
                    tail = next;
                }
            }
        }

        merged
    }
}

fn main() {
    let mut lists = Vec::new();
    for values in [vec![1, 4, 5], vec![1, 3, 4], vec![2, 6]] {
        let mut head = Empty;

```

```

        for data in values.into_iter().rev() {
            head = Node { data, next: LinkTo::new(head) };
        }
        lists.push(head);
    }

    let merged = MergeKSorted::merge_k_lists(lists);

    let mut current = &merged;
    while let Node { data, next } = current {
        print!("{data} ");
        current = next;
    }
    println!();
}

```

`ListNode::take` is the enum's own version of `Option::take`. `mem::replace(self, Empty)` moves the node out and leaves an empty one in its place, which is what makes it legal to take a node out of a list that is still borrowed.

There is a cost to the change, and it is visible in `*tail = Node { data, next: LinkTo::new(Empty) }`. Every appended node allocates a box to hold the `Empty` that terminates the list, where `Option<Box<ListNode>>` stores the same information as a null pointer inside the box's niche and allocates nothing. The enum form is one allocation per node more expensive than the form printed at the head of this chapter.

The loop is a `loop` rather than a `while let`, because the two exhausted cases now have to be matched explicitly. `(Empty, _)` and `(_, Empty)` attach the remaining list to the tail and break; the third arm takes a mutable reference to whichever list has the smaller head and falls through to the move below it.

The variant that merges recursively

The merge of two sorted lists has a two-line recursive definition: the smaller head, followed by the merge of what remains. Writing it that way removes the tail pointer and the dummy node together.

Source file: `src/bin/merge_k_sorted_list_easy.rs`. Run it with `cargo run --bin merge_k_sorted_list_easy`.

```

pub type NodeLink = Box<ListNode>;

#[derive(Debug, PartialEq, Eq, Default)]
pub enum ListNode {
    #[default]
    Empty,
    Node(i32, NodeLink),
}

impl From<Vec<i32>> for NodeLink {
    fn from(vec: Vec<i32>) -> Self {
        vec.into_iter().rev().fold(NodeLink::default(), |acc, v|

```

```

NodeLink::new(ListNode::Node(v, acc)))
    }
}

impl From<Vec<i32>> for ListNode {
    fn from(vec: Vec<i32>) -> Self {
        *NodeLink::from(vec)
    }
}

impl ListNode {
    pub fn merge_k_lists(mut lists: Vec<NodeLink>) -> NodeLink {
        if lists.is_empty() {
            return NodeLink::default();
        }
        let mut interval = 1;
        while interval < lists.len() {
            for i in (0..lists.len()).step_by(interval * 2) {
                if i + interval < lists.len() {
                    let right = std::mem::take(&mut lists[i + interval]);
                    let left = std::mem::take(&mut lists[i]);
                    lists[i] = Self::merge(left, right);
                }
            }
            interval *= 2;
        }
        lists.swap_remove(0)
    }

    fn merge(l1: NodeLink, l2: NodeLink) -> NodeLink {
        match (*l1, *l2) {
            (ListNode::Empty, l) | (l, ListNode::Empty) => NodeLink::new(l),
            (ListNode::Node(v1, n1), ListNode::Node(v2, n2)) => {
                if v1 <= v2 {
                    NodeLink::new(ListNode::Node(v1, Self::merge(n1, NodeLink::new(ListNode::Node(v2, n2)))))
                } else {
                    NodeLink::new(ListNode::Node(v2, Self::merge(NodeLink::new(ListNode::Node(v1, n1)), n2)))
                }
            }
        }
    }
}

fn main() {
    let lists: Vec<NodeLink> = vec![
        vec![1, 4, 5].into(),
        vec![1, 3, 4].into(),
        vec![2, 6].into(),
    ];

    let merged = ListNode::merge_k_lists(lists);

    println!("{:?}", merged);
}

```

`From<Vec<i32>>` is what lets `main` write `vec![1, 4, 5].into()`. The fold walks the values backwards and hangs each one in front of the list built so far, starting from `NodeLink::default()`, which the `#[default]` attribute on `Empty` defines as the empty list.

`match (*l1, *l2)` dereferences both boxes, which moves the nodes out of them, so each arm owns the data it matched and can rebuild a list from it without cloning. The first arm uses an or-pattern: whichever side is empty, the other is the answer.

The recursion costs a stack frame per node of the merged list, which is the limitation this version trades for its brevity. A merge of a few hundred thousand nodes overflows the stack, as chapters 14 and 38 describe for the recursive destructor. `v1 <= v2` keeps the node from the first list ahead on ties, so this merge is stable.

Intuition

```
merge_two([1, 4, 5], [1, 3, 4])

left   right   taken   output so far
  1     1       right   [1]                1 < 1 is false, so right is taken on ties
  1     3       left    [1, 1]
  4     3       right   [1, 1, 3]
  4     4       right   [1, 1, 3, 4]
  4     -       loop ends: right is empty
                        tail.next = rest of left = [4, 5]
result: [1, 1, 3, 4, 4, 5]
```

For the three lists in `main`, the first round merges `[1, 4, 5]` with `[1, 3, 4]`, and the second round merges the result with `[2, 6]`. The program prints:

```
[1, 1, 2, 3, 4, 4, 5, 6]
```

Time and Space Complexity

Resource	Cost	Condition
Time	$O(N \log k)$	N nodes in total; each of the $\log k$ rounds touches every node at most once
Space	$O(1)$ extra	nodes are relinked in place; one dummy node per merge
Allocations	one per <code>merge_two</code> call	the dummy head is boxed

A min-heap of the k current heads is the other common solution. It also costs $O(N \log k)$, and it needs $O(k)$ extra space for the heap.

Limitations

Ties take the node from the second list. The comparison is `l.data < r.data`, so equal values come from `right` first. The merge is correct, but it is not stable: equal elements do not keep the order of their input lists. Using `<=` would keep elements from the first list ahead on ties.

The dummy node allocates. `Box::new(ListNode::new(0))` allocates for every merge, `k - 1` allocations in total. A local, unboxed dummy avoids the allocation, at the cost of a slightly different type for `tail`.

MergeKSorted is an empty struct used as a namespace. `merge_k_lists` and `merge_two` could be free functions; the struct groups them the way many online judges expect, and it carries no state.

The derived Default is unused. `ListNode` derives `Default`, and nothing calls it.

Dropping a very long result list is recursive. As chapters 14 and 38 showed, the default destructor of `Option<Box<ListNode>>` frees one node per stack frame, so a merged list of a few hundred thousand nodes can overflow the stack when it is dropped.

The values are u32. The algorithm needs only `Ord`; a generic `ListNode<T: Ord>` would merge any ordered type.

Summary

- Merging `k` sorted lists in pairs, doubling the interval each round, costs `O(N log k)` and relinks nodes without copying.
- `split_at_mut` provides two mutable borrows of different vector elements at once.
- A dummy head and a `tail` reference remove the special case for the first node, and `left.take().or(right.take())` attaches the remainder in one step.
- The merge is not stable on ties, and each merge allocates one dummy node. The swap, by-value, queue, heap and recursive versions are all stable, because each of them takes the node from the earlier list when two values are equal.

- The queue and the heap reach the same $O(N \log k)$ by different routes: the queue halves the number of lists on every pass, and the heap holds one entry per list and pops N times.
- An enum with an `Empty` variant reads well and allocates one box per node more than `Option<Box<ListNode>>`, which keeps its terminator in the pointer's niche.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 2.3, on merging.
- Standard library, `slice::split_at_mut`.
- Standard library, `Option::take`.

Kruskal's Minimum Spanning Tree

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/kruskals_algorithm.rs`. Run it with `cargo run --bin kruskals_algorithm`.

Problem Statement

Given an undirected graph on n vertices, presented as a list of weighted edges, return a set of edges that connects every vertex and whose total weight is as small as possible. The graph may contain edges in any order, and the answer has $n - 1$ edges when the graph is connected.

Designing a Solution

The greedy rule is short enough to state in one line: sort the edges by weight and keep each one that joins two vertices not already connected. The rule is correct because of the cut property. For any way of splitting the vertices into two groups, the lightest edge crossing the split belongs to some minimum spanning tree, and an edge rejected by the rule is one whose endpoints already sit on the same side of every split made so far.

What the rule needs is a fast answer to one question: are these two vertices already connected? A disjoint set, also called union-find, answers it. Each component is a tree of vertices, each vertex points at a parent, and the root of the tree is the name of the component. Two vertices are connected when they reach the same root.

Two refinements keep those trees flat. Path compression makes every vertex visited during a lookup point straight at the root, so the next lookup on that

path is one step. Union by rank hangs the shorter tree under the taller one, so the height grows only when two trees of equal height meet. Together they bring the amortised cost of a lookup down to the inverse Ackermann function of n , which is below five for any graph that fits in memory.

`union` returns a `bool`, and that return value is what drives the main loop. It answers "were these two vertices in different components", which is the question the greedy rule asks, and it performs the merge in the same call, so the algorithm never has to ask twice.

Implementation

```
use std::cmp::Ordering::{Equal, Greater, Less};

#[derive(Debug)]
struct Edge { u: usize, v: usize, w: u32 }

struct DisjointSet { parent: Vec<usize>, rank: Vec<usize> }

impl DisjointSet {
    fn new(n: usize) -> Self {
        Self {
            parent: (0..n).collect(),
            rank: vec![0; n]
        }
    }

    fn find(&mut self, elem: usize) -> usize {
        if self.parent[elem] != elem {
            self.parent[elem] = self.find(self.parent[elem]);
        }

        self.parent[elem]
    }

    fn union(&mut self, x: usize, y: usize) -> bool {
        let (x, y) = (self.find(x), self.find(y));

        if x == y { return false; }

        // Note for self : Comparator requires a reference
        match self.rank[x].cmp(&self.rank[y]) {
            Less => self.parent[x] = y,
            Greater => self.parent[y] = x,
            Equal => {
                self.parent[x] = y;
                self.rank[y] += 1;
            }
        }

        true
    }
}

fn kruskals(n: usize, mut edges: Vec<Edge>) -> Vec<Edge> {
    // Greedy algorithm, sort the edges by weight
    edges.sort_by_key(|e| e.w);

    let mut uf = DisjointSet::new(n);
```

```

let mut mst = Vec::new();

for edge in edges {
    if uf.union(edge.u, edge.v) {
        mst.push(edge);
    }
}

mst
}

fn main() {
    let edges = vec![
        Edge { u: 0, v: 1, w: 4 },
        Edge { u: 0, v: 2, w: 3 },
        Edge { u: 1, v: 2, w: 1 },
        Edge { u: 1, v: 3, w: 2 },
        Edge { u: 2, v: 3, w: 4 },
        Edge { u: 3, v: 4, w: 2 },
    ];

    let mst = kruskals(5, edges);
    let total: u32 = mst.iter().map(|e| e.w).sum();

    for e in &mst {
        println!("{}", e.u, e.v, e.w);
    }
    println!("total: {}", total);
}

```

`DisjointSet::new` starts with every vertex as its own component. `(0..n).collect()` fills `parent` so that vertex `i` is its own parent, and every rank starts at zero.

`find` walks to the root and rewrites the path on the way back out. The recursive call returns the root, and the assignment `self.parent[elem] = self.find(self.parent[elem])` stores it directly in the vertex being visited, so a chain of any length collapses into a set of direct links to the root in a single traversal. Writing the compression as a loop takes two passes, one to find the root and one to rewrite; the recursion gets both from the way the call stack unwinds.

`union` looks up both roots first, and an equal pair means the edge would close a cycle, so it returns `false` without touching anything. Otherwise the ranks decide the direction. `self.rank[x].cmp(&self.rank[y])` compares them, and the comparison needs a reference because `Ord::cmp` takes `&self` and `&Self`. The taller tree becomes the parent in the `Less` and `Greater` arms, and neither rank changes, because hanging a shorter tree under a taller one cannot make it taller. Only the `Equal` arm increments a rank, which is the one case where the height genuinely grows.

`kruskals` sorts, then folds the edge list into the answer. `edges.sort_by_key(|e| e.w)` is a stable sort, so edges of equal weight stay in the order they were given, and the tree this function returns is reproducible for a given input rather than merely minimal. The loop consumes `edges`, which is what allows `mst.push(edge)` to move the edge into the result without cloning it.

Intuition

The graph in `main` has five vertices and six edges. Sorted by weight, and with the state of the disjoint set after each decision:

edge	weight	roots	decision	components after
1 - 2	1	1, 2	keep	{0} {1,2} {3} {4}
1 - 3	2	2, 3	keep	{0} {1,2,3} {4}
3 - 4	2	2, 4	keep	{0} {1,2,3,4}
0 - 2	3	0, 2	keep	{0,1,2,3,4}
0 - 1	4	2, 2	skip	already connected
2 - 3	4	2, 2	skip	already connected

The two edges of weight 2 are considered in the order they appear in the input, which the stable sort preserves. The program prints the four kept edges and their total:

```
1 - 2 (1)
1 - 3 (2)
3 - 4 (2)
0 - 2 (3)
total: 8
```

Four edges for five vertices is the expected count. Each kept edge reduces the number of components by exactly one, from `n` to one, so a connected graph always yields `n - 1`.

Time and Space Complexity

Resource	Cost	Condition
Sort	$O(E \log E)$	dominates the whole algorithm
find, union	$O(\alpha(n))$ amortised	path compression and union by rank together; $\alpha(n) < 5$ in practice
Total time	$O(E \log E)$	equivalently $O(E \log V)$, since $E < V^2$
Space	$O(V + E)$	two vectors of length V , plus the edge list and the result

Resource	Cost	Condition
Allocation	one per Vec	<code>sort_by_key</code> sorts in place and allocates only for large inputs

Prim's algorithm is the other standard answer, and it costs $O(E \log V)$ with a binary heap. Kruskal suits an edge list and a sparse graph; Prim suits an adjacency list and a dense one.

Limitations

A disconnected graph returns a forest, and says nothing about it. The loop keeps every edge that joins two components, and when the graph falls into several pieces the result is a minimum spanning forest with fewer than $n - 1$ edges. Nothing in the signature reports this, and a caller that needs a spanning tree has to check `mst.len() == n - 1` itself.

Vertex indices are not validated. `union(edge.u, edge.v)` indexes `parent` directly, so an edge naming a vertex at or beyond `n` panics with an out-of-bounds access rather than returning an error. A function taking untrusted input would check the bounds while sorting.

find is recursive. The depth is the height of the tree, which path compression and union by rank hold at $O(\log n)$, so the stack is safe for any realistic graph. It is still recursion on data supplied by the caller, and an iterative two-pass version removes the question entirely.

The total can overflow. Weights are `u32` and `main` sums them into a `u32`. A graph with many heavy edges overflows, and in release mode that wraps silently. Summing into a `u64` costs nothing here.

There are no tests. The example in `main` is checked by reading the output. A test that asserts the total weight, and a second one on a disconnected graph, would pin both the result and the forest behaviour described above.

Summary

- Sorting the edges by weight and keeping each edge that joins two components yields a minimum spanning tree, by the cut property.

- A disjoint set answers "are these connected" in near-constant amortised time when it uses both path compression and union by rank.
- Returning `bool` from `union` merges the membership test and the merge into one call, which is exactly the question the greedy rule asks.
- The sort dominates the cost at $O(E \log E)$; the disjoint set is effectively free.
- A disconnected graph yields a forest, and the function does not distinguish that case from a spanning tree.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Chapter 21, on disjoint sets, and Section 23.2, on Kruskal's and Prim's algorithms.
- Robert E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm", *Journal of the ACM*, 22(2), 1975, for the inverse Ackermann bound.
- Standard library, `Ord::cmp`.
- Standard library, `slice::sort_by_key`.

Part XIX: Two Pointers, Revisited

Three Sum

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/three_sum.rs`. Test it with `cargo test three_sum`.

Problem Statement

Given a list of integers and a target, return every unique triplet of values that adds up to the target. Two triplets with the same three values count once, whatever positions they came from, and the triplets are returned with their values in ascending order.

Checking every triple of positions costs $n(n - 1)(n - 2) / 6$ sums, which is $O(n^3)$, and removing duplicate triplets afterwards needs extra storage.

Designing a Solution

Sort the values first. Sorting makes two things cheap: duplicates sit next to each other, and for a fixed first value the remaining pair can be found with two pointers instead of a nested loop.

For each position `i`, the pair must add up to `target - numbers[i]` among the values to the right of `i`. Start `left` just after `i` and `right` at the end:

```
sum < target    the smallest candidate is too small: move left one step right
sum > target    the largest candidate is too large: move right one step left
sum == target   record the triplet, skip equal neighbors on both sides, move both
```

Each step discards one candidate that can no longer be part of a triplet with `numbers[i]`, so the inner loop is linear and the whole algorithm is $O(n^2)$.

The three-way decision maps directly onto `Ord::cmp`, which returns `Ordering::Less`, `Greater`, or `Equal`. Importing the variants with `use std::cmp::Ordering::{Equal, Greater, Less};` lets the `match` name them without the `Ordering::` prefix, and the compiler checks that all three cases are handled.

Duplicates are skipped in two places. A first value equal to the previous one is skipped, because it would find the same pairs again. After a match, `left` and `right` step past values equal to the ones just used, so the same pair is not recorded twice for this first value.

Implementation

```

/// Three Sum: every unique triplet of values that adds up to a target.
///
/// Sort, fix the first value, then close in on the other two with two pointers.
/// Sorts the caller's slice in place: O(n^2) time, no extra space beyond the result.

// Import the variants directly into scope
use std::cmp::Ordering::{Equal, Greater, Less};

pub fn three_sum(numbers: &mut [i32], target: i32) -> Vec<(i32, i32, i32)> {
    numbers.sort_unstable();

    let mut results = Vec::new();
    let target = i64::from(target);

    for i in 0..numbers.len() {
        // Skip a first value that was already used, so triplets stay unique.
        if i > 0 && numbers[i] == numbers[i - 1] {
            continue;
        }

        let (mut left, mut right) = (i + 1, numbers.len().saturating_sub(1));
        while left < right {
            // Widen to i64 so the sum of three i32 values cannot overflow.
            let current_sum = numbers[i] as i64 + numbers[left] as i64 +
numbers[right] as i64;

            match current_sum.cmp(&target) {
                Less => left += 1,
                Greater => right -= 1,
                Equal => {
                    results.push((numbers[i], numbers[left], numbers[right]));

                    while left < right && numbers[left] == numbers[left + 1] {
                        left += 1;
                    }
                    while left < right && numbers[right] == numbers[right - 1] {
                        right -= 1;
                    }

                    left += 1;
                    right -= 1;
                }
            }
        }
    }
}

```



```

    results
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn finds_all_unique_triplets() {
        let result = three_sum(&mut [-1, 0, 1, 2, -1, -4], 0);
        assert_eq!(result, vec![(-1, -1, 2), (-1, 0, 1)]);
    }

    #[test]
    fn skips_duplicates_on_every_position() {
        assert_eq!(three_sum(&mut [0, 0, 0, 0], 0), vec![(0, 0, 0)]);
        assert_eq!(three_sum(&mut [-2, 0, 0, 2, 2], 0), vec![(-2, 0, 2)]);
    }

    #[test]
    fn supports_a_non_zero_target() {
        assert_eq!(
            three_sum(&mut [1, 2, 3, 4, 5], 9),
            vec![(1, 3, 5), (2, 3, 4)]
        );
    }

    #[test]
    fn returns_nothing_when_no_triplet_exists() {
        assert!(three_sum(&mut [1, 2, 3], 100).is_empty());
        assert!(three_sum(&mut [1, 2], 3).is_empty());
        assert!(three_sum(&mut [], 0).is_empty());
    }

    #[test]
    fn does_not_overflow_on_extreme_values() {
        assert_eq!(
            three_sum(&mut [i32::MAX, i32::MAX, i32::MIN], i32::MAX - 1),
            vec![(i32::MIN, i32::MAX, i32::MAX)]
        );
    }
}

```

`three_sum` takes `numbers: &mut [i32]` and sorts it in place with `sort_unstable`. A mutable slice is enough: sorting only rearranges elements, so the function needs neither ownership nor a copy, and it accepts a `Vec`, an array, or part of either. The `&mut` at the call site, `three_sum(&mut values, 0)`, shows that the caller's data comes back sorted. `sort_unstable` is faster than `sort` and allocates nothing; stability does not matter for integers.

`numbers.len().saturating_sub(1)` gives the last index without underflowing when the slice is empty. For every `i`, `left` starts at `i + 1`, so for the last two positions `left >= right` and the inner loop does not run.

The sum is computed in `i64`. Three `i32` values can add up to about three times `i32::MAX`, which overflows an `i32`: a debug build would panic and a release

build would wrap to a wrong answer. `i64::from` widens each value losslessly, and the target is widened once before the loop.

Inside the `Equal` arm, `results.push((numbers[i], numbers[left], numbers[right]))` records the values. The two `while` loops check `left < right` before indexing `left + 1` and `right - 1`, so neither can step past the other or read outside the pair's range. The final `left += 1; right -= 1;` moves both pointers off the values just used.

The tests cover the standard example, inputs that are all duplicates, a non-zero target, inputs with no answer or fewer than three values, and extreme values whose sum would overflow `i32`.

Intuition

```
three_sum(&mut [-1, 0, 1, 2, -1, -4], 0)

sorted: [-4, -1, -1, 0, 1, 2]
index:   0  1  2  3  4  5

i first left right sum cmp      action
0  -4   1   5   -3 Less     left = 2
0  -4   2   5   -3 Less     left = 3
0  -4   3   5   -2 Less     left = 4
0  -4   4   5   -1 Less     left = 5, loop ends
1  -1   2   5    0 Equal    push (-1, -1, 2); no equal neighbors; left = 3,
right = 4
1  -1   3   4    0 Equal    push (-1, 0, 1); left = 4, right = 3, loop ends
2  -1           same as numbers[1], skip
3   0   4   5    3 Greater  right = 4, loop ends
4   1   5   5           left == right, loop does not run
5   2   6   5           left > right, loop does not run

result: [(-1, -1, 2), (-1, 0, 1)]
```

Time and Space Complexity

Re-source	Cost	Condition
Time	$O(n^2)$	the sort is $O(n \log n)$; each of the n first values runs a linear two-pointer scan
Space	$O(1)$ extra	the slice is sorted in place; the result holds one tuple per unique triplet

`sort_unstable` sorts in place without allocating, so the only allocation is the result.

Limitations

The caller's order is lost. The slice comes back sorted. A caller that needs the original order must clone the data before the call.

Values, not indices, are returned. After sorting, positions no longer refer to the caller's data, so the function returns the values. A problem that asks for the original indices needs the positions carried through the sort, for example by sorting `(value, index)` pairs.

$O(n^2)$ is the expected bound for this problem. A hash-set approach has the same asymptotic cost and more overhead, and makes duplicate handling harder. For very large inputs with a small value range, counting values can do better.

The skip loops are subtle. Removing the `left < right` guard lets `left + 1` or `right - 1` read past the pair's range and, in an input such as `[0, 0, 0]`, index out of bounds. The guard must come before the comparison, which `&&` guarantees by evaluating left to right.

Summary

- Sorting turns Three Sum into `n` two-pointer scans, for $O(n^2)$ time.
- `match current_sum.cmp(&target)` with `Less`, `Greater`, and `Equal` imported into scope states the three moves directly, and the match is checked for exhaustiveness.
- Skipping an equal first value, and equal neighbors after a match, keeps each triplet unique without a set.
- Taking `&mut [i32]` and sorting it in place avoids a copy and ownership; widening to `i64` keeps the sum from overflowing.

References

- Standard library, `Ord::cmp` and `std::cmp::Ordering`.
- Standard library, `slice::sort_unstable`.
- Standard library, `usize::saturating_sub`.

Part XX: Thread Pools

Thread Pool - Workers of the Pool Assemble

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/thread_pool.rs`, `src/problems/thread_pool_v2.rs`, `src/problems/thread_pool_v3.rs`, and `src/problems/thread_pool_v4.rs`. Run the first with `cargo run --bin thread_pool`, and test the others with `cargo test thread_pool`.

Problem Statement

Build a pool of worker threads that runs submitted closures. A caller creates the pool with a fixed number of workers, submits jobs with `execute`, and eventually shuts the pool down, at which point every job already submitted must still run.

This chapter builds the pool four times. The first version is short enough to write in a few minutes. Each later version fixes one problem with the one before it, and adds the tests that prove the fix:

Version	What it adds
1	a working pool in under fifty lines
2	jobs that really run in parallel, and closures without <code>Box::new</code>
3	errors instead of panics, and shutdown when the pool is dropped
4	workers that survive panicking jobs, named threads, spawn errors, and a report

Designing a Solution

All four versions share one design. A channel carries jobs from the pool to the workers, and each job is a boxed closure:

```
type Job = Box<dyn FnOnce() + Send + 'static>;

caller --execute--> Sender<Job> ===channel===> Receiver<Job>
                                     |
                               Arc<Mutex<Receiver<Job>>>
                             /      |      \
worker 0 /      worker 1 |      worker 2 \      worker 3
loop: take a job, run it
```

`FnOnce` lets a job consume what it captures. `Send` lets it move to a worker thread. `'static` means it borrows nothing from the caller's stack, because the worker may run it after the caller's function has returned.

An `mpsc::Receiver` has a single owner, so the workers share it through `Arc<Mutex<Receiver<Job>>>`: `Arc` gives each worker a handle, and the `Mutex` lets one worker at a time wait in `recv`.

Shutdown uses the channel itself. When the last `Sender` is dropped, `recv` returns `Err` once the queue is empty, and each worker's loop ends. Joining the workers then waits until every queued job has run.

Implementation

Version 1: the pool in a few lines

```
use std::sync::{mpsc, Arc, Mutex};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

pub struct ThreadPool {
    sender: mpsc::Sender<Job>,
    workers: Vec<thread::JoinHandle<()>>,
}

impl ThreadPool {
    pub fn new(size: usize) -> Self {
        let (sender, receiver) = mpsc::channel::<Job>();
        let receiver = Arc::new(Mutex::new(receiver));

        let workers = (0..size)
            .map(|_| {
                let rx = Arc::clone(&receiver);
                thread::spawn(move || {
                    while let Ok(job) = rx.lock().unwrap().recv() {
                        job();
                    }
                })
            })
    }
}
```

```

        })
        .collect();

    Self { sender, workers }
}

pub fn execute(&self, job: Job) {
    self.sender.send(job).unwrap();
}

pub fn join(self) {
    drop(self.sender);
    for w in self.workers {
        let _ = w.join();
    }
}
}

fn main() {
    let pool = ThreadPool::new(4);
    for i in 0..100 {
        pool.execute(Box::new(move || println!("task {i}")));
    }

    pool.join();
}

```

`new` creates the channel, wraps the receiver, and spawns `size` workers with an iterator that collects their `JoinHandles`. Each worker loops with `while let Ok(job) = rx.lock().unwrap().recv()` and runs the job. `execute` sends a boxed job. `join` takes `self` by value, drops the sender to close the channel, and joins every worker.

The program queues 100 jobs that print their number, and prints all 100 lines before it exits.

The version works, and it has one surprising property: **the jobs run one at a time**. The condition of a `while let` is a single expression, and temporaries created in it live until the end of the loop body. The `MutexGuard` returned by `rx.lock().unwrap()` is such a temporary, so the worker keeps the receiver locked while `job()` runs. Every other worker waits for that lock, and four workers behave like one.

A measurement makes it visible. Submitting eight jobs that each sleep for 100 ms to a pool of four workers should take about 200 ms. With version 1 it took **801 ms** on the NVIDIA Jetson board used for this book, which is eight jobs in sequence.

The other issues are smaller:

- `ThreadPool::new(0)` creates a pool that accepts jobs and never runs them.
- Callers must write `Box::new` around every closure.

- `execute` and every worker call `unwrap`, so a failure becomes a panic.
- A panicking job kills its worker, and the pool silently shrinks.
- If the caller forgets `join`, the program can exit before queued jobs run.

Version 2: parallel workers

```

///! Thread pool, version 2: workers really run in parallel.
///!
///! Version 1 (`src/bin/thread_pool.rs`) holds the receiver's lock while a job runs,
///! because the `MutexGuard` in `while let Ok(job) = rx.lock().unwrap().recv()` lives
///! until the end of the loop body. This version takes the job in its own `let`
///! statement, so the guard is dropped before the job starts.
///!
///! It also accepts any closure in `execute`, so callers no longer write `Box::new`.

use std::sync::{Arc, Mutex, mpsc};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

pub struct ThreadPool {
    sender: mpsc::Sender<Job>,
    workers: Vec<thread::JoinHandle<()>>,
}

impl ThreadPool {
    /// Start `size` workers. Panics if `size` is zero.
    pub fn new(size: usize) -> Self {
        assert!(size > 0, "a thread pool needs at least one worker");

        let (sender, receiver) = mpsc::channel::<Job>();
        let receiver = Arc::new(Mutex::new(receiver));

        let workers = (0..size)
            .map(|_| {
                let receiver = Arc::clone(&receiver);
                thread::spawn(move || {
                    loop {
                        // The guard is a temporary of this `let` statement, so the
                        // lock is released here, before the job runs.
                        let job = receiver.lock().unwrap().recv();
                        match job {
                            Ok(job) => job(),
                            Err(_) => break, // every sender is gone: shut down
                        }
                    }
                })
            })
            .collect();

        Self { sender, workers }
    }

    /// Queue a closure to run on the next free worker.
    pub fn execute<F>(&self, job: F)
    where
        F: FnOnce() + Send + 'static,
    {
        self.sender.send(Box::new(job)).unwrap();
    }

    /// Stop accepting jobs, let the workers drain the queue, and wait for them.
    pub fn join(self) {

```



```

        drop(self.sender);
        for worker in self.workers {
            let _ = worker.join();
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::Barrier;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::time::Duration;

    #[test]
    fn runs_every_job() {
        let pool = ThreadPool::new(4);
        let counter = Arc::new(AtomicUsize::new(0));
        for _ in 0..100 {
            let counter = Arc::clone(&counter);
            pool.execute(move || {
                counter.fetch_add(1, Ordering::Relaxed);
            });
        }
        pool.join();
        assert_eq!(counter.load(Ordering::Relaxed), 100);
    }

    #[test]
    fn jobs_run_at_the_same_time() {
        // Four jobs wait on a barrier for four parties. They can only all get
        // through if four workers run them at once; version 1 would never finish.
        let pool = ThreadPool::new(4);
        let barrier = Arc::new(Barrier::new(4));
        let (done_tx, done_rx) = mpsc::channel();

        for _ in 0..4 {
            let barrier = Arc::clone(&barrier);
            let done_tx = done_tx.clone();
            pool.execute(move || {
                barrier.wait();
                done_tx.send(()).unwrap();
            });
        }

        for _ in 0..4 {
            done_rx
                .recv_timeout(Duration::from_secs(5))
                .expect("jobs did not run in parallel");
        }
        pool.join();
    }

    #[test]
    #[should_panic(expected = "at least one worker")]
    fn zero_workers_is_rejected() {
        let _ = ThreadPool::new(0);
    }
}

```

The fix for the lock is one line moved into its own statement:

```
let job = receiver.lock().unwrap().recv();
match job {
    Ok(job) => job(),
    Err(_) => break,
}
```

Temporaries in a `let` statement are dropped at the end of that statement, so the guard is released as soon as `recv` returns, before `job()` runs. The same eight 100 ms jobs now take **201 ms**, four times faster, with no other change to the design.

`execute` is now generic over `F: FnOnce() + Send + 'static` and boxes the closure itself, so callers write `pool.execute(move || ...)`. `new` asserts that `size` is positive.

The test `jobs_run_at_the_same_time` proves parallelism without timing. Four jobs wait on a `Barrier` for four parties, which only opens when all four are waiting at once. With version 2 they pass the barrier and report back; with version 1 the first job would wait forever, so the test uses `recv_timeout` to fail instead of hanging.

Version 3: errors and shutdown on drop

```
///! Thread pool, version 3: errors instead of panics, and shutdown on drop.
///!
///! - `new` returns `Err(PoolError::NoWorkers)` for a size of zero.
///! - `execute` returns `Err(PoolError::ShutDown)` once the pool no longer accepts
    work.
///! - Dropping the pool closes the queue and joins every worker, so a pool that goes
    out of scope still finishes the jobs it was given.

use std::fmt;
use std::sync::{Arc, Mutex, mpsc};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

/// Why the pool refused a request.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum PoolError {
    NoWorkers,
    ShutDown,
}

impl fmt::Display for PoolError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::NoWorkers => write!(f, "a thread pool needs at least one worker"),
            Self::ShutDown => write!(f, "the thread pool has shut down"),
        }
    }
}

impl std::error::Error for PoolError {}

pub struct ThreadPool {
```

```

// `None` after shutdown: dropping the sender is what tells workers to stop.
sender: Option<mpsc::Sender<Job>>,
workers: Vec<thread::JoinHandle<>>,
}

impl ThreadPool {
    /// Start `size` workers.
    pub fn new(size: usize) -> Result<Self, PoolError> {
        if size == 0 {
            return Err(PoolError::NoWorkers);
        }

        let (sender, receiver) = mpsc::channel::<Job>();
        let receiver = Arc::new(Mutex::new(receiver));

        let workers = (0..size)
            .map(|_| {
                let receiver = Arc::clone(&receiver);
                thread::spawn(move || worker_loop(&receiver))
            })
            .collect();

        Ok(Self {
            sender: Some(sender),
            workers,
        })
    }

    /// Queue a closure to run on the next free worker.
    pub fn execute<F>(&self, job: F) -> Result<>, PoolError>
    where
        F: FnOnce() + Send + 'static,
    {
        self.sender
            .as_ref()
            .ok_or(PoolError::ShutDown)?
            .send(Box::new(job))
            .map_err(|_| PoolError::ShutDown)
    }

    /// Stop accepting jobs, drain the queue, and wait for every worker.
    /// Calling it again does nothing.
    pub fn shutdown(&mut self) {
        drop(self.sender.take());
        for worker in self.workers.drain(..) {
            let _ = worker.join();
        }
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        self.shutdown();
    }
}

fn worker_loop(receiver: &Mutex<mpsc::Receiver<Job>>) {
    loop {
        // A poisoned lock means another worker panicked while holding it;
        // stop this worker instead of panicking a second time.
        let Ok(guard) = receiver.lock() else { return };
        let job = guard.recv();
        drop(guard);

        match job {
            Ok(job) => job(),
            Err(_) => return,
        }
    }
}

```

```

    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::atomic::{AtomicUsize, Ordering};

    #[test]
    fn zero_workers_is_an_error() {
        assert_eq!(ThreadPool::new(0).err(), Some(PoolError::NoWorkers));
    }

    #[test]
    fn dropping_the_pool_finishes_queued_jobs() {
        let counter = Arc::new(AtomicUsize::new(0));
        {
            let pool = ThreadPool::new(2).unwrap();
            for _ in 0..50 {
                let counter = Arc::clone(&counter);
                pool.execute(move || {
                    counter.fetch_add(1, Ordering::Relaxed);
                })
                .unwrap();
            }
        } // drop: close the queue and join
        assert_eq!(counter.load(Ordering::Relaxed), 50);
    }

    #[test]
    fn execute_after_shutdown_is_an_error() {
        let mut pool = ThreadPool::new(1).unwrap();
        pool.shutdown();
        assert_eq!(pool.execute(|| {}), Err(PoolError::ShutDown));
        pool.shutdown(); // idempotent
    }

    #[test]
    fn errors_render_a_message() {
        assert_eq!(
            PoolError::NoWorkers.to_string(),
            "a thread pool needs at least one worker"
        );
        assert_eq!(
            PoolError::ShutDown.to_string(),
            "the thread pool has shut down"
        );
    }
}

```

Errors are values. `PoolError` has two variants. `new` returns `Err(NoWorkers)` for a size of zero, and `execute` returns `Err(ShutDown)` when the pool no longer accepts work. The type implements `Display` and `std::error::Error`, so a caller can use `?` and print a readable message.

The sender is an Option. `shutdown` takes the sender out with `self.sender.take()` and drops it, then drains and joins the workers. Because the field is `None` afterwards, calling `shutdown` again does nothing, and `execute` reports `ShutDown` through `self.sender.as_ref().ok_or(PoolError::ShutDown)?`.

Dropping the pool shuts it down. `impl Drop` for `ThreadPool` calls `shutdown`. A pool that goes out of scope, including during an early return or a panic in the caller, still closes its queue and waits for the jobs it was given. The test `dropping_the_pool_finishes_queued_jobs` submits fifty jobs inside a block and checks the counter after the block ends.

The worker loop has no `unwrap`. `let Ok(guard) = receiver.lock() else { return };` ends the worker if the lock is poisoned instead of panicking again, and `drop(guard)` releases the lock explicitly before the job runs, which states the fix from version 2 in code rather than relying on the temporary rule.

Version 4: survive panics and report the outcome

```
/// Thread pool, version 4: survives panicking jobs and reports what happened.
///
/// - Workers are named threads created with `thread::Builder`, and a failure to
///   spawn one is returned as `PoolError::Spawn` after the already started workers
///   are shut down.
/// - Each job runs inside `catch_unwind`, so a panicking job no longer kills its
///   worker and silently shrinks the pool.
/// - `shutdown` consumes the pool and returns a `Report` with the number of jobs
///   that completed and the number that panicked.

use std::fmt;
use std::io;
use std::panic::{self, AssertUnwindSafe};
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::{Arc, Mutex, mpsc};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

/// Why the pool could not be created or refused a job.
#[derive(Debug)]
pub enum PoolError {
    NoWorkers,
    Spawn(io::Error),
    ShutDown,
}

impl fmt::Display for PoolError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::NoWorkers => write!(f, "a thread pool needs at least one worker"),
            Self::Spawn(error) => write!(f, "failed to spawn a worker thread: {error}"),
            Self::ShutDown => write!(f, "the thread pool has shut down"),
        }
    }
}

impl std::error::Error for PoolError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        match self {
            Self::Spawn(error) => Some(error),
            _ => None,
        }
    }
}
```

```

/// What the pool did over its lifetime.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Report {
    pub completed: usize,
    pub panicked: usize,
}

#[derive(Default)]
struct Counters {
    completed: AtomicUsize,
    panicked: AtomicUsize,
}

pub struct ThreadPool {
    sender: Option<mpsc::Sender<Job>>,
    workers: Vec<thread::JoinHandle<()>>,
    counters: Arc<Counters>,
}

impl ThreadPool {
    /// Start `size` named workers.
    pub fn new(size: usize) -> Result<Self, PoolError> {
        if size == 0 {
            return Err(PoolError::NoWorkers);
        }

        let (sender, receiver) = mpsc::channel::<Job>();
        let receiver = Arc::new(Mutex::new(receiver));
        let counters = Arc::new(Counters::default());

        let mut pool = Self {
            sender: Some(sender),
            workers: Vec::with_capacity(size),
            counters,
        };

        for id in 0..size {
            let receiver = Arc::clone(&receiver);
            let counters = Arc::clone(&pool.counters);
            let spawned = thread::Builder::new()
                .name(format!("pool-worker-{{id}}"))
                .spawn(move || worker_loop(&receiver, &counters));

            match spawned {
                Ok(handle) => pool.workers.push(handle),
                // Dropping `pool` here closes the queue and joins the workers
                // that did start, so none of them is leaked.
                Err(error) => return Err(PoolError::Spawn(error)),
            }
        }

        Ok(pool)
    }

    /// Queue a closure to run on the next free worker.
    pub fn execute<F>(&self, job: F) -> Result<(), PoolError>
    where
        F: FnOnce() + Send + 'static,
    {
        self.sender
            .as_ref()
            .ok_or(PoolError::ShutDown)?
            .send(Box::new(job))
            .map_err(|_| PoolError::ShutDown)
    }
}

```

```

/// Close the queue, wait for every queued job, and report the outcome.
pub fn shutdown(mut self) -> Report {
    self.close_and_join();
    Report {
        completed: self.counters.completed.load(Ordering::Relaxed),
        panicked: self.counters.panicked.load(Ordering::Relaxed),
    }
}

fn close_and_join(&mut self) {
    drop(self.sender.take());
    for worker in self.workers.drain(..) {
        // Jobs cannot unwind out of `worker_loop`, so a join error would mean
        // a bug in the loop itself; there is nothing further to clean up.
        let _ = worker.join();
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        self.close_and_join();
    }
}

fn worker_loop(receiver: &Mutex<mpsc::Receiver<Job>>, counters: &Counters) {
    loop {
        let Ok(guard) = receiver.lock() else { return };
        let job = guard.recv();
        drop(guard);

        let Ok(job) = job else { return };

        // `AssertUnwindSafe` is sound here: the job owns everything it touches,
        // and a panicking job's partial state is never observed by this loop.
        match panic::catch_unwind(AssertUnwindSafe(job)) {
            Ok(()) => counters.completed.fetch_add(1, Ordering::Relaxed),
            Err(_) => counters.panicked.fetch_add(1, Ordering::Relaxed),
        };
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::error::Error;
    use std::sync::mpsc::channel;
    use std::time::Duration;

    #[test]
    fn reports_completed_jobs() {
        let pool = ThreadPool::new(4).unwrap();
        for _ in 0..20 {
            pool.execute(|| {}).unwrap();
        }
        assert_eq!(
            pool.shutdown(),
            Report {
                completed: 20,
                panicked: 0
            }
        );
    }

    #[test]
    fn a_panicking_job_does_not_kill_its_worker() {
        // One worker: if the panic killed it, the second job would never run.
    }
}

```

```

    let pool = ThreadPool::new(1).unwrap();
    let (done_tx, done_rx) = channel();

    pool.execute(|| panic!("job failed on purpose")).unwrap();
    pool.execute(move || done_tx.send(()).unwrap()).unwrap();

    done_rx
        .recv_timeout(Duration::from_secs(5))
        .expect("the worker died with the panicking job");
    assert_eq!(
        pool.shutdown(),
        Report {
            completed: 1,
            panicked: 1
        }
    );
}

#[test]
fn workers_are_named() {
    let pool = ThreadPool::new(1).unwrap();
    let (name_tx, name_rx) = channel();
    pool.execute(move || {
        let name = thread::current().name().map(str::to_string);
        name_tx.send(name).unwrap();
    })
    .unwrap();
    assert_eq!(name_rx.recv().unwrap().as_deref(), Some("pool-worker-0"));
}

#[test]
fn zero_workers_is_an_error() {
    assert!(matches!(ThreadPool::new(0), Err(PoolError::NoWorkers)));
}

#[test]
fn spawn_errors_expose_their_source() {
    let error = PoolError::Spawn(io::Error::other("no threads left"));
    assert!(error.to_string().contains("no threads left"));
    assert!(error.source().is_some());
    assert!(PoolError::ShutDown.source().is_none());
}
}

```

A panicking job no longer kills its worker. The worker runs each job inside `panic::catch_unwind(AssertUnwindSafe(job))`. A panic unwinds only to that call, which returns `Err`, and the worker continues with the next job. `AssertUnwindSafe` tells the compiler that observing state after a panic is acceptable; here the job owns what it captured and the loop never looks at it. The test `a_panicking_job_does_not_kill_its_worker` uses a single worker, so the job after the panic can only run if that worker survived.

Outcomes are counted. A shared `Counters` struct holds two `AtomicUsize` values that the workers increment after each job. `shutdown(self)` consumes the pool, joins every worker, and returns a `Report { completed, panicked }`. Because `shutdown` takes the pool by value, calling `execute` after it is a compile error rather than a run-time `ShutDown`.

Workers are named, and spawn failures are errors. `thread::Builder::new().name(...)` names each worker `pool-worker-N`, which appears in panic messages and debuggers. `Builder::spawn` returns `io::Result`, unlike `thread::spawn`, which panics. When a spawn fails, `new` returns `Err(PoolError::Spawn(error))`. The partially built `pool` is dropped on that return, and its `Drop` implementation closes the queue and joins the workers that did start, so none of them leaks. `Error::source` exposes the underlying `io::Error`.

Intuition

Version 1 against version 2: eight jobs of 100 ms on four workers

```
version 1, lock held while the job runs

time (ms)  0    100   200   300   400   500   600   700   800
worker 0    [job][job]
worker 1           [job][job]
worker 2                [job][job]
worker 3                     [job][job]
           each worker waits for the lock the previous one holds    total 801 ms

version 2, lock released before the job runs

time (ms)  0    100   200
worker 0    [job][job]
worker 1    [job][job]
worker 2    [job][job]
worker 3    [job][job]
                                     total 201 ms
```

The exact assignment of jobs to workers varies between runs; the totals do not.

Version 4: one worker, a panicking job followed by a normal one

step	worker	counters
<code>execute(panicking job)</code>	receives it, <code>catch_unwind</code> returns <code>Err</code>	<code>panicked = 1</code>
<code>execute(send on channel)</code>	receives it, runs it, returns <code>Ok</code>	<code>completed = 1</code>
<code>shutdown()</code>	<code>recv</code> returns <code>Err</code> , loop ends, thread joined	<code>Report { completed: 1, panicked: 1 }</code>

Time and Space Complexity

Operation	Cost	Condition
new	$O(n)$ thread creations	n workers
execute	$O(1)$: one allocation for the box and one channel send	
per job	one lock acquisition and one <code>recv</code> on the worker side	version 2 onwards: the lock is not held while the job runs
shut-down	waits for every queued job, then $O(n)$ joins	
memory	one stack per worker, one box per queued job	the queue is unbounded

Limitations

The queue is unbounded. `mpsc::channel` never blocks the sender, so a producer faster than the workers grows the queue without limit. `mpsc::sync_channel(bound)`, or the bounded queue of chapter 46, makes `execute` wait instead.

Workers contend on one lock. Every worker takes the same `Mutex` to receive. For short jobs at high rates this lock becomes the bottleneck. A multi-consumer channel such as `crossbeam-channel`, or a work-stealing pool such as `rayon`, avoids it.

Jobs return nothing. A caller that needs a result must send it back itself, as the tests do with channels. A pool that returns a handle per job, similar to `JoinHandle<T>`, would carry the result and the panic.

`catch_unwind` does not catch everything. A job that aborts the process, overflows its stack, or panics while `panic = "abort"` is configured still ends the program. Version 4 protects the pool from ordinary panics only.

Shutdown waits for every job. None of the versions can cancel queued jobs or interrupt a running one. A pool that must stop quickly needs a separate stop flag that jobs check.

Version 1 is kept as written. Its serial behaviour is the most useful thing to learn from it, and the later versions exist to explain it.

Summary

- A thread pool is a channel of boxed `FnOnce() + Send + 'static` jobs and workers that share the receiver through `Arc<Mutex<Receiver<Job>>>`.
- Temporaries in a `while let` condition live for the whole loop body. Holding the `MutexGuard` there made version 1 run jobs one at a time; taking the job in a separate `let` statement made the same work four times faster.
- Returning `Result` from `new` and `execute`, and shutting down in `Drop`, turns panics into values and makes a dropped pool finish its work.
- `catch_unwind` keeps a panicking job from killing its worker, `thread::Builder` names workers and reports spawn failures, and a consuming `shutdown` can return what happened.
- A `Barrier` tests parallelism without timing, and a single-worker pool tests survival after a panic.

References

- The Rust Programming Language, [Final Project: Building a Multithreaded Web Server](#), which builds a similar pool and discusses the `while let` lock issue.
- The Rust Reference, [Temporary scopes](#).
- Standard library, `std::sync::mpsc`, `std::sync::Barrier`, and `std::thread::Builder`.
- Standard library, `std::panic::catch_unwind`.

Closing

References

How to read this list

Sources are grouped by kind, and each entry says where the claim it supports appears. Language and library claims cite the document that specifies the behaviour: the *Rust Reference* for the language, the standard library documentation for a function, and the *Rust Book* for a mechanism. Algorithm claims cite a textbook or the paper that introduced the algorithm. Quotations are attributed to the person who wrote them, with the work they appear in, and the list is in alphabetical order by surname.

Language and library documentation

- **The Rust Book.** The Rust Project Developers. <https://doc.rust-lang.org/book/>. Cited for monomorphisation (Chapter 1), enumerations (Chapter 7), boxes (Chapters 7, 14), `Rc` and `RefCell` (Chapter 16), shadowing and mutability (Chapter 25), and command-line arguments (Chapter 29).
- **The Rust Reference.** The Rust Project Developers. <https://doc.rust-lang.org/reference/>. Cited for names and scopes (Chapter 25).
- **The Rustonomicon.** The Rust Project Developers. <https://doc.rust-lang.org/nomicon/>. Cited for the rules an `unsafe` block relies on (Chapters 24, 33).
- **Rust API Guidelines.** The Rust Project Developers. <https://rust-lang.github.io/api-guidelines/>. Cited for documentation requirements and for the naming of conversion methods.
- **Standard library documentation.** The Rust Project Developers. <https://doc.rust-lang.org/std/>. Individual items are cited in each chapter's reference list, including `HashMap`, `VecDeque`, `BinaryHeap`, `Reverse`, `Rc`, `Weak`, `RefCell`, `Mutex`, `Condvar`, `PoisonError`, `Instant`, `Duration`, `Box`, `Option`, `JoinHandle`, `fs`, `OpenOptions`, `BufReader`, `Path`, `PathBuf`, `ErrorKind`, `String::as_mut_vec`, `thread::scope`, `black_box`, `sort_unstable_by_key`, `clear_poison`, and `try_reserve`.
- **Clippy lint documentation.** The Rust Project Developers. <https://rust-lang.github.io/rust-clippy/master/>. Cited for `new_without_default` (Chapter 15).
- **The `libc` crate.** <https://docs.rs/libc/>. Cited for `getpid` (Chapter 24).

Papers and books

- **Bellman, Richard.** *Dynamic Programming*. Princeton University Press, 1957. The origin of the method used in Chapter 12.
- **Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.** *Introduction to Algorithms*, 4th edition. MIT Press, 2022. Cited for binary search trees (Chapter 8), dynamic programming (Chapters 12, 13), breadth- first search (Chapter 23), priority queues (Chapter 32), and topological sort (Chapter 11).
- **Fredkin, Edward.** "Trie memory". *Communications of the ACM* 3(9), 1960, pages 490–499. The paper that introduced the structure in Chapter 10.
- **Hoare, C. A. R.** "Null References: The Billion Dollar Mistake". Presentation at QCon London, 2009. The source of the phrasing used in Chapter 12 for the choice between a sentinel and `Option`.
- **Kahn, Arthur B.** "Topological sorting of large networks". *Communications of the ACM* 5(11), 1962, pages 558–562. The algorithm in Chapter 11.
- **Knuth, Donald E.** "Structured Programming with go to Statements". *ACM Computing Surveys* 6(4), 1974, pages 261–301. The source of the observation about premature optimisation, including the qualification that summaries of it usually leave out.
- **Knuth, Donald E.** *The Art of Computer Programming*, Volume 3: *Sorting and Searching*, 2nd edition. Addison-Wesley, 1998. Cited for digital searching (Chapter 10).
- **Lamport, Leslie.** "Distribution". Letter, 1987; collected in *Distributed Systems*, 2nd edition, 1993. The source of the sentence about partial failure quoted in Chapter 22.
- **Patterson, David A.** "Latency lags bandwidth". *Communications of the ACM* 47(10), 2004, pages 71–75. The source of the phrasing quoted in Chapter 24.
- **Perlis, Alan J.** "Epigrams on Programming". *ACM SIGPLAN Notices* 17(9), 1982, pages 7–13.
- **Stroustrup, Bjarne.** "Foundations of C++". Keynote, *Proceedings of the 2012 European Conference on Object-Oriented Programming (ECOOP)*, 2012. The source of the two-part statement of the zero-overhead principle quoted at the opening of the preface.
- **Wirth, Niklaus.** "A Plea for Lean Software". *IEEE Computer* 28(2), 1995, pages 64–68.

Quotations in this book

Four sentences are quoted from other people, each in a place where the quotation states something the surrounding chapter then uses.

Where	Quotation	Source
Preface	The two-part statement of the zero-overhead principle	Stroustrup, <i>Foundations of C++</i> , 2012
Chapter 12	Null references as a mistake that has cost a great deal	Hoare, QCon London, 2009
Chapter 22	A distributed system as one in which a machine you did not know about can stop yours	Lamport, "Distribution", 1987
Chapter 24	"Latency lags bandwidth"	Patterson, <i>Communications of the ACM</i> , 2004

Each quotation is short, is attributed in the chapter where it appears, and is quoted from the work named here. Nothing attributed to anyone in this book is a paraphrase presented as a quotation.

Index

A

- **ApiError**, error enumeration carrying data, 19 (page 114)
- **Arc**, shared ownership across threads, 21 (page 125), 22 (page 131), 30 (page 185)
- **args**, the command-line boundary, 29 (page 172)
- **assert!**, refusal of an empty input as a design decision, 21 (page 125)
- **assertion in a destructor-free type**, 13 (page 77)
- **attach and detach**, index-linked list maintenance, 18 (page 106)
- **average, limits of**, a benchmark statistic that hides the distribution, 24 (page 145)

B

- **backtracking**, `result.push(path.clone())`, `path.pop()`, 13 (page 77)
- **BinaryHeap**, a max-heap, so pairs sort by the first field, 6 (page 39), 32 (page 202)
- **binary search**, half-open range and the sorted-half test, 9 (page 56)
- **binary search tree**, ordering invariant, three removal shapes, 8 (page 50)
- **binary tree**, algebraic data type with boxed children, 7 (page 44)
- **black_box**, keeping a measurement alive, 24 (page 145)
- **bounded buffer**, one mutex and two condition variables, 30 (page 185)
- **Box**, one ownership pointer, one allocation, 7 (page 44), 14 (page 82), 15 (page 86)
- **BFS**, flood fill with a queue, 23 (page 138)
- **borrow counter**, `RefCell` and the run-time check, 16 (page 91)

C

- **cache**, two designs, bounded and unbounded memory, 17 (page 100), 18 (page 106)
- **capacity hint**, `with_capacity` and its units, 1 (page 14), 2 (page 19), 13 (page 77)

- **channel**, `mpsc` for jobs and results, 21 (page 125)
- **checked_sub**, the fix for overflow at the target, 1 (page 14)
- **clone**, value copied out of a guard or a map, 16 (page 91), 17 (page 100)
- **closure**, taking a mutable reference as a parameter, 23 (page 138)
- **coin change**, dynamic program over amounts, 12 (page 73)
- **command-line arguments**, `env::args` and the first item, 29 (page 172)
- **condition variable**, a directed wake-up for each state change, 30 (page 185)
- **connected components**, counting them by flood fill, 23 (page 138)
- **copy**, `fs::copy` and what it preserves, 28 (page 168)
- **cost model**, instructions, memory, stack, compilation, 24 (page 145)
- **cycle detection**, a short topological order, 11 (page 65)

D

- **dead_code warning**, the unused second merge function, 5 (page 33), 24 (page 145)
- **Default**, `#[derive(Default)]` and `#[default]` on a variant, 3 (page 24), 8 (page 50), 10 (page 60)
- **destructor**, recursion in derived drop glue, 7 (page 44), 14 (page 82), 15 (page 86), 16 (page 91)
- **directory traversal**, `read_dir` and a recursive walk, 27 (page 163)
- **discriminant**, a `size_of` of one byte for a six-variant enum, 19 (page 114)
- **Display**, the trait a message method should implement, 19 (page 114)
- **drop**, releasing a sender to end a receive loop, 21 (page 125)
- **Duration**, a retry delay carried by an error variant, 19 (page 114)
- **dynamic dispatch**, a trait object and the vtable call, 21 (page 125)

E

- **enum**, states a caller cannot construct illegally, 7 (page 44), 8 (page 50), 19 (page 114)
- **epigraph**, attributed quotations in this book, references (page 18)
- **ErrorKind**, branching on a failure without parsing a message, 28 (page 168)
- **eviction**, least recently used, in two designs, 17 (page 100), 18 (page 106)

- **error type**, variants with data, and matching on them, 19 (page 114)

F

- **f64**, a rate that is neither validated nor reported, 20 (page 120); a median that cannot be an integer, 32 (page 202)
- **file I/O**, `fs::write`, `OpenOptions` and `BufReader`, 26 (page 158)
- **file operations**, `copy`, `rename` and `remove_file`, 28 (page 168)
- **flood fill**, marking on entry, 23 (page 138)
- **format!**, one allocation per call, 19 (page 114), 25 (page 151)

H

- **half-open range**, `[low, high)` as the search invariant, 9 (page 56)
- **hash map**, expected constant time and the assumption behind it, 1 (page 14), 6 (page 39), 10 (page 60)
- **heap**, `BinaryHeap` as a priority queue, 6 (page 39), 32 (page 202)
- **HashSet**, the visited set that a bitmap would replace, 23 (page 138)

I

- **idempotency**, a keyed store that runs an operation at most once, 34 (page 214)
- **in-degree**, the counter Kahn's algorithm waits on, 11 (page 65)
- **index-linked list**, nodes in a vector, links as integers, 18 (page 106)
- **Instant**, the clock a limiter trusts, 20 (page 120); the clock a measurement reads, 31 (page 195)
- **interior mutability**, mutation behind a shared reference, 16 (page 91)
- **invariant**, a sentence true before and after every operation, 3 (page 24), 9 (page 56), 13 (page 77), 32 (page 202)

J

- **JoinHandle::join**, a discarded panic, 21 (page 125), 22 (page 131)

K

- **Kahn's algorithm**, topological sort by in-degree, 11 (page 65)
- **key**, the field a full cache reads back before overwriting a slot, 18 (page 106)

L

- **lifetime**, 'a in the range query, and what it ties together, 8 (page 50)
- **linked list**, reversal with three bindings, 14 (page 82)
- **lock**, held across a clock read, released before the work, 20 (page 120), 21 (page 125), 30 (page 185)
- **LRU**, least recently used, 17 (page 100), 18 (page 106)

M

- **max-heap**, why the top-k result pops the largest pair, 6 (page 39)
- **median, running**, two heaps and a rebalancing rule, 32 (page 202)
- **mem::replace**, moving a value out of a place, 15 (page 86), 8 (page 50)
- **mem::swap**, exchanging two subtrees, 7 (page 44); two bytes, 33 (page 208)
- **Mutex**, protection, and the poison mark, 20 (page 120), 21 (page 125), 22 (page 131), 30 (page 185)
- **monomorphisation**, one copy of the code per type argument, 1 (page 14)
- **mutation**, writing to a binding rather than rebinding it, 25 (page 151)

N

- **new**, and the `Default` implementation Clippy expects, 15 (page 86)
- **None**, the end of a list, not a sentinel, 14 (page 82)

O

- **OpenOptions**, `create` and `append` flags, 26 (page 158)
- **Option**, absence in the type, 1 (page 14), 3 (page 24), 12 (page 73), 14 (page 82), 32 (page 202)
- **overhead**, the boundary cost of a system call, 24 (page 145)

- **overflow**, arithmetic that differs between debug and release, 1 (page 14), 9 (page 56), 13 (page 77)

P

- **panic**, unwinding while a guard is alive, 20 (page 120), 22 (page 131); reversal of a non-ASCII string, 33 (page 208)
- **parallel sum**, borrowed workers under a scope, 31 (page 195)
- **PartialEq**, the minimal bound on a grid cell, 23 (page 138)
- **Path and PathBuf**, borrowed and owned paths, 27 (page 163)
- **PoisonError**, the error a poisoned lock returns, 22 (page 131)
- **prefix tree**, see trie, 10 (page 60)
- **pruning**, a range query that skips whole subtrees, 8 (page 50)

R

- **Rc**, shared ownership on one thread, 16 (page 91)
- **read_dir**, one **Result** per directory entry, 27 (page 163)
- **Ref and Ref::map**, a guard projected to a field, 16 (page 91)
- **RefCell**, mutation with a run-time borrow check, 16 (page 91)
- **recursion**, stack depth proportional to input, 7 (page 44), 8 (page 50), 14 (page 82), 27 (page 163)
- **refill**, tokens computed from elapsed time, 20 (page 120)
- **rename**, an atomic replacement within one filesystem, 28 (page 168)
- **Result**, failure in the type, 19 (page 114), 22 (page 131)
- **Reverse**, flipping a heap's ordering, 32 (page 202)

S

- **Send and Sync**, what a single-threaded structure does not implement, 16 (page 91)
- **sentinel**, **usize::MAX** kept inside the table, 12 (page 73)
- **shadowing**, a new binding with an old name, 25 (page 151)
- **sliding window**, a start index that only moves forward, 4 (page 29)
- **sort_unstable_by_key**, ordering intervals by start, 5 (page 33)

- **stack**, frames per call, and the depth they reach, 7 (page 44), 15 (page 86), 27 (page 163)
- **state machine**, legal transitions in one `match`, 19 (page 114)
- **String**, an allocated message on an error path, 19 (page 114)
- **string reversal**, an in-place byte swap under an ASCII precondition, 33 (page 208)
- **subsets**, 2^n results and the clone per result, 13 (page 77)
- **syscall**, the user and kernel boundary, 24 (page 145)

T

- **thread::scope**, workers that borrow the input, 31 (page 195)
- **tie-break**, the ordering the top-k result depends on, 6 (page 39)
- **token bucket**, burst then rate, 20 (page 120)
- **topological sort**, an order in which edges point forward, 11 (page 65)
- **trace**, the state after each step, produced by running the code, 2 (page 19), 5 (page 33), 9 (page 56), 11 (page 65)
- **trait object**, `dyn` and what it costs, 21 (page 125)
- **try_unwrap**, moving a value out of an `Rc`, 16 (page 91)

U

- **unsafe**, the obligation a `SAFETY` comment states, 24 (page 145), 33 (page 208)
- **upgrade**, turning a weak pointer into a strong one, 16 (page 91)

V

- **Vec**, amortised growth and reuse of a slot, 18 (page 106)
- **VecDeque**, constant time at both ends, 11 (page 65), 17 (page 100), 30 (page 185)

W

- **warm-up**, why a benchmark runs the code before timing it, 24 (page 145)
- **Weak**, observing without owning, 16 (page 91)
- **worker pool**, bounded threads, ordered results, 21 (page 125)